C++面经八股

809 道分类面试题与速记答案

全文按主题分类整理，答案突出核心概念、适用场景和常见边界；带注释的伪代码用于解释关键原理、分支和边界，C++ 标准库名称均使用完整的 std:: 限定；参考文献列于文末。

目录

第 1 章 C++ 基础与面向对象 · · · · · **4**
1.1 类、接口与多态 · · · · · 4
1.2 对象模型、生命周期与类型 · · · · · 6
1.3 模板与初始化 · · · · · 9
1.4 指针、对象表示与调用接口 · · · · · 11
第 2 章 内存与资源管理 · · · · · **12**
2.1 所有权与 RAII · · · · · 12
2.2 分配策略与内存优化 · · · · · 13
2.3 原始存储、分配失败与字节操作 · · · · · 15
第 3 章 现代 C++ 与模板 · · · · · **16**
3.1 语言特性与类型系统 · · · · · 16
3.2 值类别、移动与泛型 · · · · · 18
3.3 Modules、协程与编译期计算 · · · · · 19
3.4 特殊成员函数控制 · · · · · 20
3.5 C++11 语言与模板补充 · · · · · 20
第 4 章 并发与线程安全 · · · · · **21**
4.1 同步原语与线程管理 · · · · · 21
4.2 并发任务组织 · · · · · 23
4.3 内存模型与无锁协作 · · · · · 23
第 5 章 性能与工程实践 · · · · · **25**
5.1 性能测量与基础优化 · · · · · 25
5.2 算法、缓存与 I/O · · · · · 26
5.3 链接期优化 · · · · · 27
第 6 章 编译、链接与构建 · · · · · **27**
6.1 头文件与项目组织 · · · · · 27
6.2 编译、链接与库 · · · · · 29
6.3 ODR 与 ABI · · · · · 30
6.4 头文件搜索与二进制装载 · · · · · 30
第 7 章 操作系统与 Linux · · · · · **31**
7.1 文件、权限与进程 · · · · · 31
7.2 I/O 多路复用 · · · · · 32
7.3 内存、进程间通信与并发模型 · · · · · 32
7.4 I/O 完成模型 · · · · · 34
7.5 进程、执行与启动链路 · · · · · 34
7.6 内核边界、调度与同步 · · · · · 34
7.7 地址转换与内存回收 · · · · · 36
7.8 文件系统、缓存与持久化 · · · · · 36
7.9 中断与 I/O 语义 · · · · · 36
第 8 章 计算机网络 · · · · · **37**
8.1 HTTP 与安全连接 · · · · · 37
8.2 TCP、UDP 与传输控制 · · · · · 38
8.3 名称解析与服务端网络 · · · · · 39
8.4 协议分层与应用交互 · · · · · 40
8.5 HTTP 缓存、会话与浏览器安全 · · · · · 41
8.6 TCP 状态与应用确认 · · · · · 42
8.7 网络故障与安全 · · · · · 43
8.8 网络层与链路层 · · · · · 43
第 9 章 数据库与缓存 · · · · · **44**
9.1 Redis 与缓存设计 · · · · · 44
9.2 MySQL 索引、事务与并发控制 · · · · · 45
9.3 热点与大键治理 · · · · · 47
9.4 Redis 与缓存设计 · · · · · 47
9.5 SQL 基础与数据建模 · · · · · 47
9.6 MySQL 引擎与存储 · · · · · 49
9.7 MySQL 索引补充 · · · · · 50
9.8 MySQL 锁与恢复 · · · · · 50
9.9 MySQL 查询与复制补充 · · · · · 51
9.10 Redis 内部结构与执行模型 · · · · · 51
9.11 Redis 事务与生命周期 · · · · · 52
9.12 Redis 复制、集群与缓存 · · · · · 52
第 10 章 系统设计与分布式 · · · · · **53**
10.1 分布式基础与数据结构 · · · · · 53
10.2 数据处理与在线服务设计 · · · · · 54
10.3 高并发、消息与容错 · · · · · 56
10.4 共识与消息一致性 · · · · · 58
10.5 分布式协调与服务治理 · · · · · 59
10.6 系统设计补充 · · · · · 59
10.7 消息队列模型与运行 · · · · · 60
10.8 RocketMQ、Kafka 与 RabbitMQ · · · · · 60
10.9 分布式理论与一致性 · · · · · 62
10.10 分布式组件与服务治理 · · · · · 62
10.11 分布式事务与一致性 · · · · · 62
10.12 高并发业务系统设计 · · · · · 63
第 11 章 自动驾驶与实时数据 · · · · · **63**
11.1 QoS 与点云处理 · · · · · 63
11.2 共享内存与时钟同步 · · · · · 64
11.3 实时数据流水线 · · · · · 64
第 12 章 C++ 进阶与工程 · · · · · **65**
12.1 内存模型与缓存一致性 · · · · · 65
12.2 模板、容器与线程安全 · · · · · 66
12.3 零拷贝、异常与并发问题 · · · · · 67

**第 13 章 数据结构与算法 · · ·

第 1 章 C++ 基础与面向对象

1.1 类、接口与多态

001 C++ 中如何使用友元函数（Friend Function）?

答案 友元函数不是类的成员，但经类内 friend 声明后，可以访问该类的私有和受保护成员。它常用于需要左右操作数对称的运算符，例如让 operator<< 同时访问流和对象；类本身仍须显式授权。友元会扩大访问范围，应限于紧密协作的少数接口，避免破坏封装。

```
#include <ostream>
class Money {
  friend std::ostream& operator<<(std::ostream&, const Money&); // 类内明确授权
  int cents_ = 0; // 其余代码不能直接访问
};
std::ostream& operator<<(std::ostream& os, const Money& m) { // 是非成员函数
  return os << m.cents_; // 左侧流和右侧对象都可自然参与调用
} // 只授权这个紧密协作接口；不要为方便广泛开放 private
```

002 C++ 中如何实现接口?

答案 C++ 没有 interface 关键字，通常用只定义公共操作的抽象基类表达运行时接口，纯虚函数声明可调用的行为，public 虚析构函数保证经基类指针删除派生对象时正确析构。实现类继承并用 override 实现接口。若只需编译期约束，也可用模板或 C++20 concept，不必引入虚调用。

```
#include <memory>
struct Shape { virtual ~Shape() = default; // 经接口删除派生对象时正确析构
  virtual double area() const = 0; // 接口规定行为，不能直接实例化
};
struct Circle final : Shape { explicit Circle(double r) : r(r) {} double r;
  double area() const override { return 3.14 * r * r; } };
std::unique_ptr<Shape> s = std::make_unique<Circle>(2.0);
auto a = s->area(); // 若无需运行时异构分派，可用模板/concept 避免虚调用
```

003 C++ 中如何实现运算符重载?

答案 运算符重载本质上是定义 operator 符号对应的函数，可做成员函数或非成员函数。非成员形式常适合对称二元运算，也便于左操作数类型不属于本类；流插入通常写成返回流引用的非成员函数。重载应保持运算符原有直觉，不能新增符号、改变优先级或操作数个数，点号、作用域解析和条件运算符也不能重载。

```
struct Vec2 { double x, y; Vec2& operator+=(Vec2 b) { x += b.x; y += b.y; return *this; } };
Vec2 operator+(Vec2 a, Vec2 b) { return a += b; } // 非成员使两个操作数对称
Vec2 p{1, 2}, q{3, 4};
Vec2 sum = p + q; // 保持“加法产生新值”的直觉
// 不改变运算符优先级或操作数个数；不能重载 ., :: 或 ?:
// 重载前先确认使用者会把它理解成普通向量加法
```

004 C++ 中的抽象类（Abstract Class）是什么?

答案 只要类中仍有至少一个纯虚函数未实现，它就是抽象类，不能直接创建对象，通常用来规定派生类必须提供的接口。派生类只有实现所有继承来的纯虚函数后才可实例化；可以继续保持抽象。若通过基类指针管理派生对象，基类析构函数通常应为 virtual，避免销毁时只运行基类析构。

```
struct Reader { virtual ~Reader() = default;
  virtual int read() = 0; // 未实现的纯虚函数令 Reader 抽象
};
struct FileReader : Reader { int read() override { return 1; } };
// Reader r; // 错误：抽象类不能直接构造
Reader* r = new FileReader;
delete r; // 虚析构保证派生析构也执行；实际代码优先用智能指针
```

005 C++ 中的模板特化（Template Specialization）是什么?

答案 全特化为模板参数给出一组完全确定的类型或值；类模板还可偏特化，以匹配例如指针类型或某类参数组合。函数模板没有偏特化规则，通常通过函数重载或约束表达类似选择。特化会改变某些类型的行为，优先考虑清晰的重载和 concept 约束，并检查特化声明可见性及重载歧义。

```
template<class T> struct Kind { static constexpr int value = 0; };
template<class T> struct Kind<T*> { static constexpr int value = 1; }; // 类模板偏特化
static_assert(Kind<int>::value == 0 && Kind<int*>::value == 1);
int score(int x) { return x; }
double score(double x) { return x; } // 函数模板“偏特化”用重载表达
// 特化改变匹配类型的行为；优先用清晰重载/concept，避免歧义与声明不可见
```

006 C++ 中的类型擦除（Type Erasure）是什么?

答案 类型擦除把具体类型隐藏在统一的运行时接口后面，使接口能接收多种类型，例如 std::function 保存不同可调用对象，std::any 保存满足要求的值。常见做法是用模板在编译期包装对象，再经虚函数或函数表间接调用。代价可能包括间接分派和动态分配；小对象优化是否触发由库实现决定。

```
#include <functional>
std::function<int(int)> operation;
operation = [](int x) { return x + 1; }; // 可替换为其他同签名闭包
operation = [](int x) { return x * 2; };
int answer = operation(3); // 统一运行时接口隐藏闭包具体类型
// 类型擦除通常带间接调用；目标过大时可能分配，SBO 由库实现决定
```

007 C++ 中的纯虚函数（Pure Virtual Function）是什么?

答案 纯虚函数在类内以 = 0 声明，表示基类不提供完整实现并使类成为抽象类。派生类需要覆盖所有未实现的纯虚函数才能成为具体类；即使提供纯虚函数的类外定义，它仍是纯虚函数，不能因此直接实例化。纯虚析构函数也必须提供定义，因为派生对象析构时仍会调用基类析构。

```
struct Interface { virtual ~Interface() = 0; }; // 纯虚析构仍使类抽象
Interface::~Interface() = default; // 仍必须给析构函数提供定义
struct Impl : Interface { /* 可继承析构 */ };
// Interface x; // 错误：纯虚析构也算纯虚函数
Interface* p = new Impl;
delete p; // 析构链会调用 Interface::~Interface()
```

008 C++ 中的继承有几种类型?

答案 按访问权限分，继承有 public、protected、private 三种；它们决定基类的 public/protected 成员在派生类中的可访问级别，不会让派生类直接访问基类 private 成员。public 继承表达可替代的子类型关系，通常用于多态接口；private 继承表达实现复用时应谨慎，也可考虑组合。

```
struct Service { virtual void run() = 0; virtual ~Service() = default; };
struct Worker : public Service { void run() override {} }; // public 表示可替代
struct Detail {};
class Wrapper { Detail detail_; }; // 实现复用常可用组合，避免私有继承耦合
// protected/private 继承会改变基类 public/protected 成员可见级别
// 三种继承都不会让派生类直接访问基类 private 成员
```

009 C++ 中的虚函数是什么?

答案 虚函数支持运行时多态：通过基类指针或引用调用时，执行对象动态类型中最终覆盖的函数；按值传递会发生对象切片，失去派生行为。派生声明用 override 可让编译器检查签名确实匹配。编译器常用虚表实现分派，但表结构和对象布局属于 ABI 约定，并非语言保证。

```
struct Base { virtual int id() const { return 0; } virtual ~Base() = default; };
struct Child : Base { int id() const override { return 1; } };
Child c; Base& ref = c;
int dynamic_result = ref.id(); // 按动态类型分派到 Child
Base sliced = c;
int sliced_result = sliced.id(); // 按值复制发生切片，结果来自 Base
// override 可检查签名；虚表是常见实现，不是标准规定的对象布局
```

010 C++ 中的虚拟继承是什么?

答案 菱形结构中，两个中间派生类若普通继承同一个基类，最下层对象会含有两份基类子对象，访问成员可能歧义。把中间继承声明为 virtual 后，最派生对象只包含一份共享虚基类子对象。代价是布局和初始化规则更复杂，虚基类由最派生类负责初始化，具体偏移依 ABI。

```
struct Root { explicit Root(int n) : n(n) {} int n; };
struct Left : virtual Root { Left() : Root(1) {} };
struct Right : virtual Root { Right() : Root(2) {} };
struct Join : Left, Right { Join() : Root(7), Left(), Right() {} };
Join j; // 只有一个共享 Root 子对象，值由最派生类 Join 初始化
// 普通继承会有两份 Root，访问成员可能歧义
// 虚基类布局/偏移依 ABI；虚继承也让构造顺序更复杂
```

011 C++ 中的赋值运算符重载如何实现？

答案 赋值运算符通常返回 *this 的 T&，这样 a = b = c 可以从右向左连续赋值。管理动态资源的类要正确处理自赋值和旧资源释放；copy-and-swap 可先构造副本，再无异常地交换状态，从而获得强异常保证。若支持移动赋值，应确保资源转移后源对象仍可安全析构，并按真实异常行为声明 noexcept。

```
#include <utility>
class Buffer { public:
  Buffer& operator=(const Buffer& other) {
    Buffer temp(other); // 先完整构造副本；若分配失败原对象不变
    swap(*this, temp); return *this; // 按引用返回支持 a = b = c
  }
  friend void swap(Buffer& a, Buffer& b) noexcept { a.swap_data(b); }
  void swap_data(Buffer&) noexcept; // 示意：仅交换资源句柄/状态
}; // 自赋值也安全；移动赋值另需让源对象可安全析构
```

012 C++ 和 C 有什么区别？

答案 C++ 保留许多 C 的底层和过程式能力，同时增加类、模板、异常、命名空间、重载以及更丰富的标准库，可支持面向对象、泛型和函数式风格。它并非 C 的严格超集：部分同名语法或类型规则不同，C 代码也未必能直接作为 C++ 编译。跨语言接口通常用稳定的 C ABI 边界。

```
// C ABI 常用于让 C 与 C++ 模块共享稳定边界
extern "C" int add_numbers(int a, int b) { return a + b; }
// C++ 调用方可使用类、模板和重载；C 调用方只见 C 风格函数签名
// C++ 保留许多 C 能力，但不是严格超集，源代码规则并不完全相同
// 跨语言边界避免直接暴露 std::string、模板类型或编译器特定类布局
// 资源所有权和错误码也要约定，避免异常穿过 C ABI
```

013 C++ 多态如何分类与实现？

答案 编译期多态包括函数重载、模板和 concept 约束，调用目标在编译时按类型和约束选择；运行期多态主要由虚函数实现，经基类引用或指针依据动态类型分派。前者通常便于内联但可能增加实例代码，后者支持异构对象集合但有间接调用成本。多态基类通常要有虚析构函数。

```
struct Base { virtual int value() const = 0; virtual ~Base() = default; };
struct One : Base { int value() const override { return 1; } };
template<class T> int static_call(const T& x) { return x.value(); } // 编译期绑定
int dynamic_call(const Base& x) { return x.value(); } // 运行期按动态类型分派
One x; int a = static_call(x), b = dynamic_call(x);
// 模板利于内联但增加实例代码；虚接口支持异构集合但有间接调用成本
```

1.2 对象模型、生命周期与类型

014 C++ 对象通常有哪些存储区？标准是否规定栈和堆？

答案 按标准语义，对象有自动、静态、线程和动态等存储期；常见操作系统还会把程序映像分成代码、全局数据、线程栈和动态分配区域。栈、堆是实现惯用称呼，标准不规定具体分区、方向或物理布局。判断对象何时销毁应看其存储期和所有权，而不是只看它位于哪个区域。

```
#include <memory>
int global = 0; // 静态存储期：整个程序期间存在
thread_local int per_thread = 0; // 每个线程各有一份
void f() { int local = 1; // 自动存储期，离开作用域销毁
  auto dynamic = std::make_unique<int>(2); // 动态对象由所有者托管
} // 判断生命周期看存储期/所有权；“栈、堆”只是常见实现布局说法
// 标准不规定对象必须落在哪个物理分区或栈的增长方向
```

015 C++ 的默认特殊成员函数有哪些？Rule of Three/Five 是什么？

答案 特殊成员函数包括默认构造、析构、拷贝构造/赋值和移动构造/赋值，编译器是否隐式声明或删除取决于类的成员及用户声明。Rule of Three/Five 提醒：手动管理资源时，析构、拷贝和移动语义要成套考虑。更好的常见做法是把资源交给容器和 RAII 成员，遵循 Rule of Zero。

```
#include <string>
#include <vector>
struct User { std::string name; std::vector<int> scores; }; // Rule of Zero
// vector/string 自己处理资源；编译器生成的复制/移动通常就是正确语义
struct Raw { int* p; ~Raw() { delete p; } }; // 自管裸资源需审视拷贝和移动
// Raw 若默认拷贝会双重释放；应禁拷贝并实现移动，或改用 RAII 成员
// Rule of Three/Five 是检查清单，优先通过 Rule of Zero 避免手写
```

016 const 在变量、指针、引用和成员函数中分别有什么作用?

答案 const 限制通过该表达式修改对象：const T* 表示不能经此指针改 T，T* const 表示指针本身不能改指向，const T& 提供只读访问。const 成员函数不能修改普通成员，但可修改声明为 mutable 的缓存等逻辑状态。const 对象不一定是编译期常量；需要常量表达式时使用 constexpr 并满足其求值条件。

```
int x = 1;
const int* pointee_readonly = &x; // 不能经此指针修改 x
int* const fixed_pointer = &x; // 指针不能改指向
const int& read_only_ref = x; // 只读别名
struct Cache { mutable int hits = 0; int get() const { ++hits; return 1; } };
// const 不等于编译期常量；constexpr 还要求表达式满足常量求值规则
```

017 explicit 关键字有何作用?

答案 explicit 用来阻止构造函数或转换函数被隐式转换链使用，尤其适合单参数构造，避免把一个普通值意外转成类对象；显式直接构造仍可调用它。C++11 起转换运算符也能声明 explicit，C++20 可用条件 explicit 按模板参数决定。引入它时要检查现有初始化和函数参数调用是否依赖隐式转换。

```
struct Port { explicit Port(int n) : value(n) {} int value; };
Port a{8080}; // 直接初始化可调用 explicit 构造
// Port b = 8080; // 错误：阻止意外的隐式转换
void connect(Port);
// connect(8080); // 错误；connect(Port{8080}) 意图清晰
// C++20 还可写 explicit(condition); 添加 explicit 时检查旧调用点
```

018 mutable 关键字是什么?

答案 mutable 允许 const 成员函数修改被它修饰的非静态成员，不改变对象对外可观察的逻辑值时很有用。例如延迟计算缓存、统计计数或互斥锁可以声明为 mutable。它不是线程安全工具：多个线程并发访问该成员仍须加锁或使用原子类型，也不能借它绕过对象本身的生命周期规则。

```
#include <mutex>
class Memo {
  mutable std::mutex m_; // const 查询仍可同步缓存
  mutable bool ready_ = false; mutable int cached_ = 0;
public: int value() const { std::lock_guard<std::mutex> lock(m_);
    if (!ready_) { cached_ = 42; ready_ = true; } return cached_; }
}; // mutable 适合逻辑上不改变结果的缓存/锁
// mutable 本身不提供同步；并发访问必须加锁或使用原子类型
```

019 sizeof 和 alignof 的作用是什么?

答案 sizeof(T) 给出 T 对象表示的字节数，数组结果包含所有元素而指针结果只包含指针本身；alignof(T) 给出类型所需的对齐。结构体成员之间和末尾可能有填充，因此成员顺序会影响大小。具体数值由实现和 ABI 决定，不能把本机 sizeof 当成跨平台序列化格式。

```
#include <iostream>
struct Record { char tag; double value; };
std::cout << sizeof(Record) << ' ' << alignof(Record); // 实际数值依 ABI
std::cout << ' ' << sizeof(char); // 单个成员类型大小
// 成员间/末尾可能填充以满足对齐；数组大小含所有元素
// 不要把本机 sizeof/布局直接当网络协议或跨平台文件格式
```

020 static 在 C++ 中有哪些常见用法?

答案 局部 static 对象具有静态存储期，在控制流首次经过定义时初始化；C++11 起此初始化由语言保证线程安全。命名空间作用域 static 通常赋予名字内部链接，类 static 数据成员则属于类而非某个实例。static 成员函数没有 this，不能直接访问非静态成员，也不能声明为 virtual 或带 cv/ref 限定；C++17 前非 inline 静态数据成员通常需在类外定义，C++17 inline static 可在类内定义。初始化完成后对共享状态的读写仍可能有数据竞争，需单独设计同步。

```
struct Registry { static int count; }; // 类级数据，不属于某个实例
int Registry::count = 0; // 非 inline 成员在一个源文件定义一次
int& shared_state() { static int state = 0; return state; }
// 首次经过时初始化由 C++11 保证线程安全
++shared_state(); // 初始化安全不代表之后的并发写安全
// 命名空间 static 通常内部链接；局部 static 生命周期贯穿程序
```

021 typedef 和 using 有何区别?

答案 typedef 与 using 都创建类型别名，不会产生新的类型。using Alias = Existing; 对复杂函数指针更清晰，也支持 template<class T> using Vec = std::vector<T> 这样的别名模板；typedef 不能直接定义别名模板。二者都只是命名类型，无法解决原类型本身的隐式转换或封装问题。

```
#include <vector>
typedef unsigned long LegacyId;
using Id = unsigned long; // 现代别名写法更易读
template<class T> using Vec = std::vector<T>; // 别名模板
Vec<int> values{1, 2};
// typedef 也能命名普通类型，但不能直接声明 template alias
// 两者都只是别名，不会创建新类型或阻止隐式转换
```

022 什么是 C++中的多重继承，如何避免菱形继承?

答案 多重继承让一个类同时获得多个基类接口或实现。若两个路径都普通继承同一祖先，最下层对象会保存两份祖先状态，访问需指明路径；对共享的共同基类采用虚继承可只保留一个子对象。虚继承会改变对象布局和构造责任，接口设计中应优先避免不必要的复杂菱形结构。

```
struct A { int id = 0; };
struct B : virtual A {};
struct C : virtual A {};
struct D : B, C {}; // 虚继承使最底层 D 共享一个 A
D d; d.id = 1;
// 普通继承会有 B::A 和 C::A 两个子对象，d.id 有歧义
// 若只为复用实现，组合通常比多重/虚继承更易维护
```

023 什么是内联函数（inline)?

答案 inline 不命令编译器一定展开调用；是否展开由优化器根据函数大小、调用上下文等决定。它的重要语言作用是允许满足 ODR 条件的定义出现在多个翻译单元，类内定义的成员函数通常隐式 inline。过度内联会扩大代码体积、损害指令缓存，因此性能判断应看基准和生成代码。

```
// 适合放在头文件供多个翻译单元包含
inline int clamp_low(int value) { return value < 0 ? 0 : value; }
int result = clamp_low(-1); // inline 不保证机器码展开
// inline 的语言作用包括允许符合 ODR 的相同定义跨翻译单元出现
// 编译器仍会根据优化和调用上下文决定是否内联；过度展开增大代码
// 大函数放头文件也会扩大每个包含者的编译工作
```

024 什么是对象切片？如何避免?

答案 把派生对象按值复制到基类对象时，只保留基类子对象，派生字段和动态行为信息被切掉，这叫对象切片。多态接口应接收基类引用或指针；异构对象容器可使用智能指针保存派生对象。若需要值语义，可用显式 clone 等机制定义多态复制，而非直接按基类值复制。

```
struct Shape { virtual double area() const { return 0; } virtual ~Shape() = default; };
struct Circle : Shape { double area() const override { return 12; } };
Circle c;
Shape copy = c; // 按值复制切掉 Circle 部分
Shape& view = c; // 引用保留动态对象
// API 用引用/指针传多态对象；需要多态复制时定义 clone() 明确语义
// 异构容器可保存 std::unique_ptr<Shape>
```

025 什么是构造函数和析构函数的顺序?

答案 构造时先构造虚基类，再按基类声明顺序构造直接基类，最后按成员声明顺序构造成员；析构顺序完全反向，而不是按初始化列表书写顺序。局部对象在作用域退出时析构，动态对象在 delete 时析构。经基类指针删除派生对象时，基类析构需为 virtual。

```
#include <string>
struct Part { Part(const char* n) : name(n) { log(name); } ~Part() { log("~" + name); } std::string name; };
struct Base { Base() { log("Base"); } virtual ~Base() { log("~Base"); } };
struct Child : Base { Part first{"first"}, second{"second"};
  Child() : second("second"), first("first") {} // 实际仍按声明顺序
  ~Child() { log("~Child"); }
}; // 构造基类→成员声明序；析构反向；虚基类先于直接基类构造
```

026 命名空间有什么作用？头文件中如何避免污染全局命名空间？

答案 命名空间把名字分组，降低不同库间的冲突；使用限定名如 lib::Type 或局部 using 声明可控制引入范围。头文件中的 using namespace 会把大量名字带入所有包含者的作用域，容易造成歧义，因此应避免。匿名命名空间适合在实现文件中隐藏仅本文件使用的名字。

```
namespace graphics { struct Color {}; }
void draw() {
  using graphics::Color; // 仅在本函数引入这个名字
  Color c;
}
// 头文件里不要写 using namespace graphics; 会污染所有包含者
// 直接使用 graphics::Color 最明确；匿名 namespace 可隐藏 .cpp 私有名字
```

027 如何实现 C++ 中的单例模式（Singleton）？

答案 常见单例把构造函数设为 private，删除拷贝和移动操作，并通过静态函数提供访问点。函数内局部 static 实现简单，C++11 起首次初始化线程安全；但之后共享状态的并发访问仍要同步。还应评估全局生命周期和退出析构顺序，测试中可注入接口替代单例依赖。

```
class Config { Config() = default; public:
  Config(const Config&) = delete; Config& operator=(const Config&) = delete;
  static Config& instance() { static Config one; return one; }
  int read() const { return 1; }
};
auto value = Config::instance().read();
// 局部 static 首次初始化线程安全；后续可变状态仍需同步
// 测试和析构顺序复杂时，优先依赖注入而非全局单例
```

028 引用（Reference）与指针（Pointer）有什么区别？

答案 引用声明时必须绑定对象，之后通常不能改绑，也没有空引用这一正常语义；指针可以为空或改指向，需要用 * 或 -> 间接访问。引用在实现中也可能占空间，不能依赖它一定是别名且零成本。API 中用引用表达必需对象，用指针表达可选对象，并另行明确所有权。

```
void use_required(int& x) { ++x; } // 引用表达必须存在的对象
void use_optional(int* x) { if (x) ++*x; } // 指针可为空且可改指向
int value = 0; int& alias = value; int* ptr = &value;
int other = 1; ptr = &other; // 可重新指向；alias = other 是赋值 value
// 引用声明时须绑定，通常不能重新绑定；两者都不自动表达所有权
// 返回借用引用/指针时，调用方必须确保目标仍存活
```

029 构造函数和析构函数分别负责什么？

答案 构造函数负责建立对象不变量并获取或初始化成员资源，析构函数负责释放对象拥有的资源；成员对象会自动按逆序析构。局部对象离开作用域时自动清理，动态对象由 delete 触发析构。若通过基类指针销毁派生对象，基类析构必须为 virtual，否则行为未定义。

```
#include <memory>
#include <string>
struct FileHandle { explicit FileHandle(int fd) : fd(fd) {} ~FileHandle() { close_fd(fd); }
  int fd; void close_fd(int); };
void process() { auto file = std::make_unique<FileHandle>(open_file());
  read_data(file->fd); // 正常返回或异常离开时都由析构清理
}
// 构造建立对象不变量/取得资源；析构释放自己拥有的资源
// 经基类指针删除派生对象时基类析构需 virtual
```

1.3 模板与初始化

030 模板（Template）是什么？

答案 模板以类型、值或模板参数描述一族函数或类，编译器在具体参数出现时实例化对应代码，因此可实现静态多态和通用容器。模板还支持编译期计算与类型选择。代价是诊断和构建可能更复杂，实例过多会增加目标文件体积；用 concept 或类型约束可让接口意图更清楚。

```
template<class T> T larger(T a, T b) { return a < b ? b : a; }
int i = larger(3, 5); double d = larger(1.0, 2.0);
// 调用时编译器实例化适合 T 的实现，形成静态多态
// 可用模板复用算法而不用运行期虚调用
// 用 concept/约束表达所需操作，减少不清楚的模板报错和过度实例化
// 同一模板参数会生成相应实例，构建体积也要留意
```

031 深拷贝与浅拷贝有什么区别?

答案 浅拷贝逐成员复制，若成员只是裸指针，两个对象会指向同一资源，可能互相修改、悬空或重复释放；深拷贝则为副本独立创建资源。若共享本来就是语义，应明确采用 shared_ptr 等共享所有权。自管资源的类要定义完整拷贝/移动策略，优先用 RAII 成员和 Rule of Zero。

```
#include <vector>
struct Shallow { int* p; }; // 默认拷贝只复制地址，两对象会别名
struct Deep { std::vector<int> values; }; // 容器复制时复制元素
int external = 1; Shallow a{&external}; Shallow b = a; // 两者指向同一 int
*b.p = 9; // 通过 b 修改会影响 a 所指的值
// 裸指针不说明是否拥有资源；不要让两个对象都负责 delete
Deep x{{1, 2}}, y = x; y.values[0] = 9; // x 不受影响
// 若资源应共享，明确用 std::shared_ptr；通常优先 Rule of Zero
```

032 类型转换（static_cast、dynamic_cast、const_cast、reinterpret_cast）有什么区别?

答案 static_cast 用于有明确语言语义的转换，如数值转换或已知层次中的上转型；dynamic_cast 对多态类型做运行时检查，下转失败时指针得 nullptr、引用抛 bad_cast。const_cast 只增删 cv 限定，修改原本 const 对象会触发未定义行为。reinterpret_cast 只适用于低层表示转换，不能保证跨平台含义。

```
#include <typeinfo>
struct Base { virtual ~Base() = default; }; struct Child : Base {};
Base* b = new Child;
Child* checked = dynamic_cast<Child*>(b); // 运行时检查；失败得 nullptr
int n = static_cast<int>(3.8); // 明确数值转换，可能丢信息
const Base* cb = b;
Base* writable = const_cast<Base*>(cb); // 原对象非 const，去限定后可写
// 若最初对象本来就是 const，经转换修改会是未定义行为
// reinterpret_cast 只用于受控低层表示；不保证跨平台意义
// dynamic_cast 引用下转失败会抛 std::bad_cast；用完释放 b
```

033 默认构造函数和拷贝构造函数分别是什么?

答案 默认构造函数是不提供必需实参即可构造对象的构造函数，可能由编译器隐式生成，也可显式声明为 default。拷贝构造函数以同类型对象初始化新对象，典型形式是 T(const T&)；它不同于对已存在对象执行的拷贝赋值。用户声明某些构造或移动操作会改变隐式生成规则，应检查是否被删除。

```
struct Item { Item() = default; Item(const Item&) = default; int value = 0; };
Item first; Item second(first); // 默认构造 vs 新对象拷贝构造
Item third = first; // 这里仍是拷贝构造，不是赋值
third = second; // 这是拷贝赋值，对象已存在
struct NeedsArg { explicit NeedsArg(int) {} };
// 编译器不一定生成默认构造；用户声明构造后通常需显式 default
```

034 C++ 中列表初始化和圆括号初始化有哪些区别?

答案 花括号列表初始化能拒绝窄化转换，例如把小数放进 int 会编译失败；对类类型，它会先尝试 initializer_list 构造函数，因此有时与圆括号选中不同的重载。圆括号是直接初始化，通常不做窄化检查。简单聚合类型还可按成员顺序用花括号初始化，适合显式表达字段值。〔S2〕

```
#include <vector>
std::vector<int> items{3, 4}; // initializer_list 构造：两个元素
std::vector<int> fill(3, 4); // 圆括号：三个值为 4 的元素
// 花括号还检查窄化，int x{3.5}; 会编译失败
struct Pair { int left; int right; };
Pair p{1, 2}; // 聚合按成员顺序初始化
// 类有 initializer_list 重载时，花括号会优先匹配它，可能改变重载选择
```

1.4 指针、对象表示与调用接口

035 对象指针、void* 和成员指针分别能表达什么？大小是否固定？

答案 对象指针表达某类对象的地址语义；解引用前必须满足非空、对齐及目标生命周期要求。void* 可暂存对象地址，但没有可解引用的对象类型，C++ 中转回 T* 通常需显式转换；不能把任意函数指针也当成可移植的 void*。成员数据指针和成员函数指针都须结合所属类对象使用：前者可写作 int Item::*，后者如 int (Item::*)() const，调用时分别用 .* 或 ->*。非静态成员函数中的 this 指向当前对象，const 成员函数中的 this 指向 const 对象；静态成员函数没有 this。它们与普通对象指针的表示和大小可能不同。常见 64 位 ABI 的普通对象指针为 8 字节，这不是语言对所有平台、所有指针类别的统一保证。malloc 返回原始存储，强转本身不负责完成任意 C++ 对象的构造。〔N1、N2〕

```
int value = 3;
void* storage = &value;
int* p = static_cast<int*>(storage); // 还原真实对象类型
struct Item {
  int score;
  int read_score() const { return this->score; } // 非静态成员中 this 指向当前对象
};
int Item::*member = &Item::score;
int (Item::*method)() const = &Item::read_score;
Item item{7};
int score = item.*member;
int copy = (item.*method)(); // 成员函数指针须绑定对象
```

036 如何区分函数指针、返回指针的函数和函数对象？

答案 int (*f)(int) 声明指向函数的指针；int* f(int) 则声明返回 int* 的函数，返回值还需明确所有权和有效期。函数对象是能用调用语法执行的对象，常通过重载 operator() 实现，可以同时保存配置或状态；Lambda 也是闭包对象。普通函数指针不保存每次调用的独立捕获状态，非捕获 Lambda 可转换成相容函数指针，带捕获 Lambda 通常不能。模板参数可接收不同可调用类型；若需统一签名并在运行期保存，可考虑 std::function 的类型擦除代价。〔N1、N3〕

```
int twice(int x) { return x * 2; }
int (*function_ptr)(int) = &twice;
struct Add { int bias; int operator()(int x) const { return x + bias; } };
Add add{3};
int result = add(function_ptr(2)); // 7，函数对象还保存 bias
```

037 sizeof(1 == 1) 为何在 C 与 C++ 中可能不同？

答案 C 的相等比较产生 int 类型的 0 或 1，因此 sizeof(1 == 1) 等于 sizeof(int)；C++ 的内建相等比较结果为 bool，因此等于 sizeof(bool)。常见平台分别得到 4 与 1，但标准没有要求 int 必为 4 字节或 bool 必为 1 字节，输出应按目标实现确认。sizeof 的单位是字节，sizeof(char) 恒为 1，而一个字节的位数由 CHAR_BIT 描述。面试时应先说明表达式结果的类型，再说明实现上的常见大小，不能把某台 64 位机器的结果当成所有平台的结论。〔N4、N5、N6〕

038 C++ 的 struct 与 class 默认规则有何差异？和 C 的 struct 如何区别？

答案 C++ 的 struct 与 class 都能定义成员函数、构造析构、模板成员及继承关系；主要语言差异是默认成员访问权限和默认继承权限：struct 默认为 public，class 默认为 private，显式写出后可获得相同规则。C 的 struct 没有这些 C++ 成员与继承机制，C 中声明对象常需 struct 标签名，除非另设 typedef。工程上常用 struct 表示公开的数据组合，class 表示维护不变量的抽象；这只是风格，不能据此推断对象一定是 POD、可以直接序列化或允许 memcpy。〔N7〕

039 union 如何共享存储？为何不能随意读取另一个成员？

答案 struct 的普通非静态成员各有自己的存储位置；union 的成员重叠使用存储，其大小需容纳最大成员并满足对齐。一般同一时刻只有一个活动成员，切换到含非平凡构造/析构的成员时还要显式管理生命周期。写入整数再直接读取浮点成员不是通用的可移植类型转换办法，公共初始序列等仅有特定例外。需要安全表达“多种类型选其一”时，优先使用 std::variant，让类型标签和析构逻辑一起管理。二进制协议应按字段编码，不能依赖本机 union 布局跨平台传输。〔N8〕

```
std::variant<int, std::string> value = 12;
value = std::string("ready"); // 自动切换活动类型与生命周期
if (const auto* s = std::get_if<std::string>(&value)) {
  use(*s); // 不把同一段字节强行解释为另一种活动对象
}
```

040 为什么没有虚构造函数？构造和析构期间的虚调用如何分派？

答案 构造函数负责建立对象及其基类、成员，调用它时目标构造类型已由初始化表达式确定，语言不允许把构造函数声明为 virtual。创建运行期决定的派生类型可用工厂函数返回基类智能指针；需要多态复制可用虚 clone。构造和析构函数中发生虚调用时，分派受当前正在构造或析构的类限制，不会借此调用尚未构造或已经析构的更派生部分。不要在基类构造期间调用纯虚接口期待派生实现，更不能把“vptr 还没分配”当作全部标准语义。〔N9、N10〕

041 重载、虚函数覆盖和派生类名字隐藏怎样区分？

答案 重载是在同一可见重载集中，用不同参数列表表达同名函数，编译器按实参和转换选择；只改返回类型不能形成重载。覆盖要求派生函数与基类虚函数满足对应的签名等规则，通过基类引用或指针才体现动态分派，用 override 可检查意图。隐藏发生在名字查找阶段：派生类声明某名字后，基类同名函数即使参数不同也可能不再进入候选；可用 using Base::f 引入基类重载。把非虚函数重新声明并不等于虚覆盖，默认参数的选择也不随虚分派动态改变。〔N10〕

```
struct Base { virtual void f(int); void f(double); };
struct Derived : Base {
  using Base::f; // 保留基类的另一个重载
  void f(int) override; // 由编译器检查真实覆盖
};
```

042 空类、含虚函数的类和多重继承对象的大小怎样判断？

答案 完整空类对象也需具有非零大小，以便数组中的不同元素具有不同地址；常见实现大小为 1，但空基类优化和 [[no_unique_address]] 成员允许特定子对象共享地址。普通成员、基类、对齐填充以及实现用于动态分派的字段共同决定非空类尺寸。常见 ABI 给多态对象放置 vptr，并在多重继承中使用多个基类视图、次级虚表及 this 调整；不能简单说所有对象都只有一个虚表指针或“有虚函数就固定 8 字节”。最终应结合目标 ABI、编译选项和布局工具验证。〔N11、N6〕

```
struct Empty {};
struct Poly { virtual ~Poly() = default; };
struct Value { int value; };
static_assert(sizeof(Empty) >= 1);
// sizeof(Poly)、sizeof(Value) 用目标 ABI 实测，不写跨平台固定值
```

第 2 章 内存与资源管理

2.1 所有权与 RAII

001 C++ 中如何使用智能指针？

答案 unique_ptr 表示单一所有者，移动可转移所有权且不允许默认拷贝；shared_ptr 在确需共同生命周期时共享控制块；weak_ptr 观察 shared_ptr 管理对象但不增加强引用。优先用 make_unique/make_shared 建立对象，把所有权写进类型。互相持有 shared_ptr 会形成环，应把非拥有方向改成 weak_ptr。

```
#include <memory>
struct Node { std::weak_ptr<Node> parent; std::shared_ptr<Node> child; };
auto root = std::make_shared<Node>(); // 多方确实共享生命周期才用 shared_ptr
auto child = std::make_shared<Node>(); root->child = child;
child->parent = root; // 反向观察用 weak_ptr，避免引用环导致永不释放
auto sole = std::make_unique<Node>(); // 单一所有者优先用 unique_ptr
// weak_ptr::lock() 成功时才临时取得共享所有权
```

002 C++ 中如何管理内存？

答案 先明确资源何时开始和结束，再让所有权由局部对象、标准容器或智能指针表达。自动存储期对象通常随作用域结束销毁，静态对象贯穿程序生命周期，动态存储应由 RAII 对象托管。标准规定的是存储期和分配语义，不要求实际一定有物理意义上的栈区或堆区。

```
#include <memory>
#include <vector>
int global = 0; // 静态存储期
thread_local int scratch = 0; // 每个线程一份
void work() { std::vector<int> local{1, 2}; // 自动对象，离开作用域析构
  auto heap = std::make_unique<int>(3); // 动态对象由局部所有者托管
} // 不直接依赖“栈/堆”说法；标准关心存储期与对象生命周期
// 分配细节由实现决定，确定所有权比猜物理区域更重要
```

003 C++ 中的内存对齐（Memory Alignment）是什么？

答案 对齐要求表示对象地址必须满足某个边界，alignof 可查询类型要求，alignas 可请求更严格的对齐。结构体成员间和末尾填充用于确保每个成员地址合规；过度对齐分配也需匹配分配接口。网络或文件格式应逐字段编码，不要直接写入本机结构体字节并假设布局一致。

```
#include <cstddef>
struct Plain { char c; double value; }; // 成员间可能出现填充
struct PackedOrder { double value; char c; }; // 顺序可能减少填充
struct alignas(64) Slot { int value; }; // 请求更严格对齐
static_assert(alignof(Slot) >= 64);
// 数值/布局仍依实现；过度对齐动态分配需配套正确分配接口
// 文件和网络协议逐字段编码，不直接写入结构体原始字节
```

004 C++ 中的垃圾回收与手动内存管理有什么区别?

答案 传统 C++ 常通过 RAII 和确定性析构及时释放对象资源，也可接入追踪式垃圾回收或用引用计数管理对象图。GC 依赖可达性回收内存，但文件、锁、事务等外部资源仍需显式控制释放时机；追踪 GC 还可能引入暂停。选择方案应看延迟、吞吐和资源种类，而非只比较是否手动 free。

```
#include <fstream>
#include <fstream>
void process() {
  std::ifstream file("input.txt"); // RAII 在作用域退出时关闭资源
  if (!file) return;
  use_file(file);
} // 即使异常栈展开，文件仍确定关闭
// 垃圾回收器管理可达内存不等于及时释放文件、锁或事务
// 追踪 GC 可能有暂停；资源释放时机应按延迟和资源类型设计
```

005 new/delete 与 malloc/free 有什么区别?

答案 new 表达式先取得合适存储，再调用构造函数；delete 表达式先析构对象再释放存储。malloc/free 只处理字节，不调用 C++ 构造和析构，不能与 new/delete 混搭；数组 new[] 要配对 delete[]。业务代码优先用容器和 make_unique，只有实现分配器时才直接管理原始存储。

```
#include <cstdlib>
Widget* p = new Widget(args...); // 分配存储并调用构造函数
delete p; // 调用析构后释放；必须成对使用 new/delete
void* raw = std::malloc(sizeof(Widget)); // 只取得字节，不构造 Widget
std::free(raw); // malloc/free 配对；不能 delete raw
// 数组 new[] 配 delete[]；业务代码优先容器/智能指针而非原始分配
```

006 什么是 RAII（资源获取即初始化)?

答案 RAII 把资源所有权绑定到对象：构造成功后对象持有资源，析构时释放。作用域正常退出和异常栈展开都会调用析构，因此可用 unique_ptr、lock_guard、文件流和容器避免手工遗漏清理。资源获取若可能失败，构造应保持异常安全，不要把未完整初始化的所有权暴露出去。

```
#include <fstream>
#include <mutex>
void write_log(std::mutex& m) {
  std::lock_guard<std::mutex> lock(m); // 获锁即建立资源所有权
  std::ofstream file("log.txt");
  file << "entry\n";
} // 异常或提前返回也会逆序析构：先关文件，再解锁
// 这种 RAII 清理比每条路径手写 unlock/close 更可靠
```

2.2 分配策略与内存优化

007 什么是内存池（Memory Pool)?

答案 内存池预留一批存储并按固定尺寸或大小类重复分配，减少通用分配器开销和外部碎片，适合高频、尺寸相近或生命周期集中对象。实现时必须分别处理存储分配与对象构造析构，还要考虑对齐、池耗尽、线程安全和释放策略。池不一定更快，需用真实分配分布做基准。

```
#include <vector>
#include <memory_resource>
std::pmr::unsynchronized_pool_resource pool; // 复用大小相近的分配块
{
  std::pmr::vector<Node> nodes{&pool}; // 元素仍需正常构造/析构
  nodes.emplace_back(); nodes.emplace_back();
} // 容器先销毁元素，再由 pool 回收存储
// pool 耗尽、对齐与线程安全策略都要定义；并先用真实负载测速度
```

008 如何减少 C++ 程序中的内存碎片?

答案 先用分配器统计或 profiler 判断是外部碎片、内部碎片还是峰值占用问题。尺寸稳定的对象可用大小类、对象池；生命周期相近的临时对象可用 arena/单调资源，整批释放。池化会增加保留内存，且需考虑对齐、线程竞争和池耗尽，不能只看分配速度就判定优化成功。

```
#include <vector>
#include <memory_resource>
std::pmr::monotonic_buffer_resource arena; // 一批临时对象统一分配
{
  std::pmr::vector<Node> frame{&arena};
  build_frame(frame); render(frame);
} // 容器先结束元素生命周期；arena 再一次性回收区块
// 同生命周期对象适合 arena；池化会多保留内存，不一定减少总占用
// 用 profiler 区分碎片、峰值占用和分配开销后再选大小类/池
```

009 如何通过 std::move 实现高效的内存管理?

答案 std::move 本质是把表达式转换成可绑定右值引用的值类别，不会复制或搬运任何字节。随后是否调用移动构造取决于目标类型是否提供可用移动操作；资源类可在移动时转交指针、文件句柄等。移动后的对象仍须有效且可析构，但其值通常未指定，应只做类型保证的操作或重新赋值。

```
#include <memory>
#include <utility>
auto source = std::make_unique<Buffer>(size);
auto destination = std::move(source); // 这里只转换值类别，不搬运字节
// unique_ptr 的移动构造接手句柄；source 现在为空
if (!source) source = std::make_unique<Buffer>(new_size); // 可重新赋值
// 普通移动后对象通常只保证有效可析构，具体值未指定
```

010 如何避免 C++ 中的内存泄漏?

答案 优先用自动对象、标准容器和 RAII 管理文件句柄、锁及动态资源，避免裸 new 后在多个返回路径手动 delete。unique_ptr 表达独占所有权，shared_ptr 只用于真正共享生命周期，weak_ptr 可打破引用环。AddressSanitizer/LeakSanitizer 等工具可检查泄漏和越界，但仍需覆盖异常及提前返回路径。

```
#include <memory>
std::unique_ptr<Widget> make_widget() { return std::make_unique<Widget>(); }
void run() {
  auto object = make_widget();
  if (condition()) return; // 提前返回也自动释放
  process(*object);
} // 异常展开也清理；共享所有权仅在确需时用 shared_ptr
// weak_ptr 可打破所有权环；测试构建可启用 Address/LeakSanitizer
```

011 什么是 placement new? 它和普通 new 有什么区别?

答案 placement new 在调用方给定的地址上开始对象生命周期并执行构造，不负责申请那块存储，因此不能对它直接 delete。调用方必须确保缓冲区足够大、满足对齐且生命周期覆盖对象；对象结束后显式调用析构，再按原来取得存储的方式释放。常用于对象池和底层容器实现。〔S2〕

```
#include <cstddef>
#include <new>
#include <memory>
alignas(T) std::byte storage[sizeof(T)]; // 足够大且满足 T 对齐
T* object = ::new (static_cast<void*>(storage)) T(args...); // 只开始生命周期
use(*object);
std::destroy_at(object); // 显式析构；placement new 不负责释放存储
// 仅在原始缓冲区生命周期结束/释放策略执行后回收 storage
```

2.3 原始存储、分配失败与字节操作

012 未初始化指针和悬空指针分别怎样产生？置空能解决全部问题吗？

答案 未初始化的自动存储期指针没有可供正常使用的确定指针值，俗称野指针；不要对它做解引用、比较等依赖其值的操作。悬空指针原先指向有效对象，但对象销毁、容器搬迁或其存储释放后已失去可用目标。把一个变量设为 nullptr 只修复这一个句柄，别处保存的别名不会一起更新。预防时用 RAII 表达所有者，用引用或裸指针表达借用，并限制借用跨越销毁、扩容和异步任务的边界；ASan 可以帮助发现执行路径中的释放后使用。〔N12〕

```
int* observer = nullptr;
{
  auto owner = std::make_unique<int>(42);
  observer = owner.get(); // 观察者不延长对象生命
} // owner 释放资源，observer 悬空
observer = nullptr; // 仅重置此变量；其他别名仍须单独处理
```

013 sizeof、strlen 与字符串拷贝各自需要哪些边界条件？

答案 sizeof 计算类型或对象表示的大小，数组保留数组类型时得到整个数组字节数，函数形参退化为指针后得到的就只是指针大小。strlen 从字符指针向后查找首个空字符，不包含结尾空字符，并要求此前始终是可读存储；它不反映缓冲区容量。strcpy 会连同结尾空字符一起复制，却不接收目标容量，目标不足或两区域重叠都会出错。strncpy 也不保证总有结尾空字符。业务代码优先用 std::string，固定缓冲区则显式校验长度与终止符，二进制数据不能靠 strlen 测长。〔N6、N13、N14〕

```
char buffer[16] = "abc";
static_assert(sizeof(buffer) == 16); // 容量，与有效文本长度不同
std::size_t length = std::strlen(buffer); // 3，不含结尾零
std::string safe_copy = buffer; // 所有权由字符串对象管理
```

014 memcpy 和 memcmp 为何不能普遍替代对象复制与相等比较？

答案 memcpy 按指定字节数复制对象表示，要求源目标有效且不重叠；重叠移动应使用 memmove。对满足标准条件的平凡可复制对象，字节复制可用于恢复其值，但不能用它复制含 std::string、虚表或资源所有权的任意对象，存储与生命周期条件也要满足。memcmp 只比较字节：结构体填充字节可能不同但字段值相同，浮点正负零和指针等也不适合据此推导业务相等。对象复制使用正确的拷贝/移动操作，值相等使用成员比较或合适的默认比较；网络与文件格式则另行序列化。〔N15、N16〕

```
struct Point { int x; int y; bool operator==(const Point&) const = default; };
Point a{1, 2}, b{1, 2};
bool equal = (a == b); // C++20，表达字段的值相等
// 不用 std::memcmp(&a, &b, sizeof(Point)) 推导一般对象相等
```

015 普通 new、nothrow new 与 malloc 分配失败时分别如何报告？

答案 普通 new 的标准分配路径在不能获得存储时抛出 std::bad_alloc，可能先按规则调用 new_handler；不能只在普通 new 后检查空指针期待处理分配失败。nothrow 分配形式用 nullptr 报告分配失败，但对象构造函数本身仍可能抛出异常。malloc 失败返回空指针，并按平台约定设置错误状态，不调用构造函数。零长度申请、过大数组长度和操作系统内存过量承诺还各有边界：获得非空地址并不必然意味着未来触碰所有页面都能成功。调用层应按接口语义恢复、降载或终止。〔N17、N18〕

```
try {
  auto buffer = std::make_unique<std::byte[]>(requested);
  use(buffer.get());
} catch (const std::bad_alloc&) {
  reportAllocationFailure(); // 真实业务中避免继续依赖新的大分配
}
```

016 glibc malloc 如何在空闲块、brk 与 mmap 之间分配？

答案 malloc 是用户态分配器接口，通常先尝试重用线程缓存、arena 及空闲块，不会为每次调用都进入内核。需要扩展后端存储时，glibc 可通过 brk 调整传统堆区，也可用匿名 mmap 建立映射；大分配更可能单独映射，但阈值可配置且会自适应，1 KB 与 1 MB 的区别不能固定概括成某个永恒分界。free 后内存可能保留在分配器中以供复用，RSS 不立即下降不等于泄漏。映射建立与物理页实际提交也有区别。分析时结合实现版本、分配大小分布、arena 数量和观测数据。〔N18〕

第 3 章 现代 C++ 与模板

3.1 语言特性与类型系统

001 C++ 中如何实现自定义的内存管理器?

答案 自定义分配器要把存储分配与对象构造分开设计：allocator 只提供满足大小和对齐的原始存储，容器再在其中构造元素。固定尺寸、高频对象可用预分配池和空闲链表；通用 allocator 还要满足标准库的传播、相等性和重绑定等要求。全局重载 new/delete 影响面很大，应先证明默认分配器是瓶颈。

```
#include <vector>
#include <memory_resource>
std::pmr::unsynchronized_pool_resource pool; // 面向相近尺寸的复用池
{
  std::pmr::vector<Node> nodes{&pool}; // 容器在原始存储中构造元素
  for (int i = 0; i < 100; ++i) nodes.emplace_back();
} // 先析构元素，再让资源池回收存储
// 通用 allocator 还需满足传播/相等性等要求; unsynchronized_pool_resource 不可并发共享
// 自定义全局 new 影响全程序; 先证明分配器是瓶颈
```

002 C++ 中的 auto 关键字是什么?

答案 auto 根据初始化表达式推导类型，使用值形式时通常像模板按值推导一样去除顶层 const 和引用；auto& 会保留引用，const auto 增加只读限定，auto&& 在适用场景下是转发引用。decltype(auto) 按 decltype 规则保留精确类型和值类别。遍历容器时用 auto& 可避免复制元素，需留意代理引用类型。

```
#include <string>
#include <vector>
std::vector<std::string> names{"Ada", "Lin"};
auto copy = names[0]; // 值推导去掉引用，复制字符串
const auto& view = names[1]; // 保留只读引用，不复制元素
auto&& forwarding = names[0]; // 在适当模板/推导语境中是转发引用
// decltype(auto) 按 decltype 规则保留精确引用和值类别
// vector<bool> 的元素访问可能是代理对象，auto& 不一定适用
```

003 C++ 中的 noexcept 关键字如何使用?

答案 noexcept 声明函数不向外传播异常；若异常逃出 noexcept 函数，运行时会调用 std::terminate。noexcept(expr) 可在编译期查询表达式是否不抛，条件异常规范常用于模板转发。移动操作只有确实不会抛时才标 noexcept，因为 vector 等容器在扩容时可能据此选择移动而非为异常安全回退到拷贝。

```
#include <type_traits>
struct MoveOnly { MoveOnly(MoveOnly&&) noexcept = default; };
static_assert(std::is_nothrow_move_constructible_v<MoveOnly>);
void never_fails() noexcept { /* 异常若逃逸会调用 std::terminate */ }
// vector 扩容时可能为强异常保证回退到拷贝
// 移动操作只有确实不抛才标 noexcept，不能作为优化装饰
// noexcept(expr) 还可在模板中查询表达式的异常属性
```

004 C++ 中的 typeid 是如何工作的?

答案 typeid 返回 std::type_info，可比较类型信息或用 name 查看实现相关名称。对普通表达式通常检查静态类型；对多态类的 glvalue 会求值并报告动态类型，因此相关表达式可能有副作用。它依赖 RTTI；对空指针解引用后应用 typeid 检查多态对象会抛 std::bad_typeid，不能把 name 当作跨平台稳定标识。

```
#include <typeinfo>
struct Base { virtual ~Base() = default; }; struct Derived : Base {};
Base* p = new Derived;
bool same = (typeid(*p) == typeid(Derived)); // 多态 glvalue 得到动态类型
const char* name = typeid(*p).name(); // 名称格式由实现决定
// 非多态表达式通常只报告静态类型; RTTI 需启用
// 不要解引用空多态指针再 typeid; 那会抛 std::bad_typeid
delete p;
```

005 C++ 中的模板元编程（Template Metaprogramming）是什么?

答案 模板元编程借助模板实例化、constexpr 和类型特征在编译期计算值或选择实现，例如按类型能力启用某个算法。它能把检查和分派前移到编译期，但复杂递归模板会让诊断和编译变慢。现代代码可用 constexpr 函数直接计算、用 concepts 表达约束，通常比深层模板技巧更易读。

```
#include <cstddef>
template<std::size_t N> struct Factorial {
  static constexpr std::size_t value = N * Factorial<N - 1>::value;
};
template<> struct Factorial<0> { static constexpr std::size_t value = 1; };
static_assert(Factorial<5>::value == 120); // 编译期实例化递归模板
constexpr std::size_t fact(std::size_t n) { return n < 2 ? 1 : n * fact(n - 1); }
// 常量表达式通常更清楚；深层 TMP 会增加编译时间和诊断难度
```

006 C++ 中的类型推导（Type Deduction）是什么?

答案 推导规则取决于上下文。auto 按近似模板按值规则推导，auto& 保留左值引用和 const；函数模板的 T、T&、T&& 参数也分别按值、左值引用和转发引用规则推导，转发引用会根据实参值类别折叠。decltype(auto) 使用 decltype 规则，可能得到引用，因此从局部变量返回时需检查是否悬空。

```
#include <utility>
int x = 1; const int cx = 2;
auto a = cx; // 按值推导为 int，顶层 const 被去掉
const auto& b = cx; // 保留 const 引用
template<class T> decltype(auto) identity(T&& value) { return std::forward<T>(value); }
// 模板 T&& 对左值折叠为左值引用，对右值保持右值引用
// 返回引用时要确认对象活得足够久，避免悬空
```

007 C++11 中的 Lambda 表达式是什么?

答案 Lambda 会生成一个匿名闭包类型，其 operator() 保存函数体和捕获状态；按值捕获保存副本，按引用捕获仍依赖原变量生命周期。C++14 支持初始化捕获，可移动捕获独占资源；C++20 可写模板形参列表。若闭包被异步保存或返回，不要引用捕获短生命周期局部变量。

```
#include <memory>
#include <utility>
#include <memory>
#include <utility>
auto make_task(std::unique_ptr<Job> job) {
  return [owned = std::move(job)] { owned->run(); }; // 初始化捕获转移所有权
}
int total = 0; auto add = [&total](int n) { total += n; }; // 依赖 total 生命周期
add(3);
// 闭包保存捕获状态；异步保存时避免引用短生命周期局部变量
// 按值捕获可保存副本，必要时用 mutable 改闭包自己的副本
```

008 C++11 引入的 nullptr 有什么用法?

答案 nullptr 是类型为 std::nullptr_t 的空指针字面量，可转换为任意对象指针或函数指针，但不会像整数 0 一样匹配普通整数重载。使用它能避免 NULL 在不同平台可能是整数宏而产生的重载歧义，例如 f(int) 与 f(char*) 并存时 f(nullptr) 明确选择指针语义。

```
#include <cstddef>
void open(int); void open(const char*);
open(nullptr); // 空指针语义，选择指针重载
open(0); // 整数 0 匹配 int，历史 NULL 宏也可能是整数
std::nullptr_t empty = nullptr;
// nullptr 可转换为对象/函数指针，不会误选整数重载
// 需要头文件 <cstddef> 才直接命名 std::nullptr_t
```

009 inline 函数与宏有什么区别?

答案 宏在预处理阶段做无类型文本替换，缺少作用域和类型检查，带副作用的参数还可能重复求值；inline 函数保留正常参数求值和类型检查，可调试也受作用域控制。inline 允许符合 ODR 的定义出现在多个翻译单元，但不保证机器码内联。宏仍可用于条件编译等预处理需求。

```
inline int square(int x) { return x * x; } // 类型检查，参数只求值一次
int i = 3; int value = square(i++); // i 只递增一次
#define SQUARE_BAD(x) ((x) * (x))
int j = 3;
// int wrong = SQUARE_BAD(j++); // 会重复递增且产生未定义行为
// inline 不保证生成机器码内联，但可满足多翻译单元 ODR 例外
// 宏仍适合条件编译；函数更安全、可调试、受作用域控制
```

3.2 值类别、移动与泛型

010 右值引用、移动语义、std::move 与 std::forward 的关系是什么？

答案 右值引用让类型可实现资源转移的移动构造和移动赋值；std::move 只是把表达式转换为右值类别，真正是否移动由重载决议决定。std::forward<T> 仅用于转发引用模板，按推导出的 T 保留实参原本是左值还是右值。移动后对象仍可析构和重新赋值，其他值通常未指定。

```
#include <string>
#include <utility>
void consume(std::string&& text) { store(std::move(text)); }
template<class F, class... Args>
decltype(auto) forward_call(F&& f, Args&&... args) {
  return std::forward<F>(f)(std::forward<Args>(args)...);
}
// std::move 只转换值类别，移动由目标类型的重载实现
// std::forward<T> 仅在转发引用模板里按 T 保留实参原值类别
```

011 如何使用 C++ 实现延迟计算（Lazy Evaluation）？

答案 延迟计算可把工作包装成 lambda 或函数对象，等调用方真正需要结果时再执行；对于可能重复读取的结果，可保存计算状态并缓存一次结果。std::optional 能区分尚无结果与已有结果，std::call_once 可协调单次初始化。要考虑线程安全、异常后是否重试，以及闭包引用的对象是否仍存活。

```
#include <mutex>
#include <mutex>
#include <optional>
class LazyValue { mutable std::optional<int> cache_; mutable std::once_flag once_;
public: int get() const { std::call_once(once_, [&] { cache_ = expensive_compute(); }); return *cache_; }
};
// 闭包/函数对象可推迟计算；optional 区分未计算和结果为 0
// call_once 成功初始化后只执行一次；异常时后续调用可重试
// 引用捕获对象仍须活得比延迟任务久
```

012 如何实现 C++ 中的类型萃取（Type Traits）？

答案 类型特征在编译期回答类型属性问题，标准库已有 is_integral、is_pointer、remove_reference 等，通常应优先复用。自定义特征可用主模板定义默认结果，再为特殊类型偏特化，并继承 true_type/false_type 提供编译期布尔值。C++17 的 void_t 可检测某个类型表达式是否有效；C++20 concept 更适合公开约束接口。

```
#include <type_traits>
#include <utility>
#include <vector>
template<class T> struct is_pointer_like : std::false_type {};
template<class T> struct is_pointer_like<T*> : std::true_type {}; // 偏特化
static_assert(is_pointer_like<int*>::value);
using ValueType = std::remove_cvref_t<const int&>; // 先复用标准特征
static_assert(std::is_same_v<ValueType, int>); // C++20 concept 可直接写约束
```

013 C++20 Concepts 与 SFINAE 有什么区别？

答案 Concepts 用 requires 直接写出类型必须满足的表达式或性质，并参与约束排序，错误信息通常更贴近调用点。SFINAE 是模板替换失败时把候选从重载集中移除，常通过 enable_if、检测惯用法实现，兼容旧标准且仍适合一些细粒度选择。两者规则不完全等价，迁移时要检查重载集合是否改变。〔S1，S2〕

```
#include <type_traits>
#include <concepts>
template<class T> requires std::integral<T>
T add(T a, T b) { return a + b; } // 约束直接说明所需性质
// 旧标准常用替换失败从候选集移除：
template<class T, std::enable_if_t<std::is_integral_v<T>, int> = 0>
T legacy_add(T a, T b) { return a + b; }
// concepts 错误信息通常更清楚，但重载排序规则不完全等同 SFINAE
```

014 C++20 Ranges 和 View 解决了什么问题?

答案 Ranges 让算法接受范围对象，减少 begin/end 成对传递；view 可把 filter、transform 等步骤惰性组合，遍历时才产生结果，避免中间容器。惰性意味着算法可能重复执行，且许多 view 不拥有元素。返回或保存 view 时要确认底层容器生命周期和迭代器有效性。〔S1，S2〕

```
#include <ranges>
#include <vector>
std::vector<int> values{1, 2, 3, 4};
auto odds = values | std::views::filter([](int n) { return n % 2; });
auto doubled = odds | std::views::transform([](int n) { return n * 2; });
for (int n : doubled) consume(n); // 迭代时才过滤/变换，无中间容器
// view 多数不拥有元素；保存它前要保证 values 和迭代器仍有效
```

015 std::span 是什么？它和 std::vector 有何区别?

答案 std::span 是连续内存的非拥有视图，保存地址与长度，适合作为函数参数统一接收数组、vector 或其他连续区间；vector 则拥有可增长的元素存储。span 不负责释放或延长数据生命周期。vector 扩容、被销毁或其他存储失效操作发生后，原 span 可能悬空。〔S2〕

```
#include <span>
#include <vector>
int sum(std::span<const int> values) { int total = 0; for (int x : values) total += x; return total; }
std::vector<int> owned{1, 2, 3};
int result = sum(owned); // span 借用 vector 的连续存储，不复制元素
// vector 拥有并可扩容；span 只保存地址和长度，不延长生命周期
// owned 被销毁或扩容后，不得继续用原 span
```

016 std::string_view 有什么优点和生命周期陷阱?

答案 string_view 以字符指针和长度提供只读视图，可用于解析函数参数而避免构造临时 string。它不拥有字符，data() 后面未必有空字符，不能不加判断就传给只接受 C 字符串的接口。若原 string 析构、修改导致重分配，或临时字符串生命周期结束，view 会失效。〔S2〕

```
#include <string>
#include <string_view>
#include <string_view>
int parse(std::string_view text) { return parse_digits(text); } // 无需复制字符串
std::string owner = "123"; std::string_view view = owner;
int n = parse(view); // 仅在 owner 存活且存储未失效时安全
std::string_view dangling() { std::string local = "bad"; return local; } // 错误：悬空
// view 不保证 data() 后有 NUL；传 C API 前要复制/保证终止
```

3.3 Modules、协程与编译期计算

017 C++20 Modules 和传统头文件包含有什么区别?

答案 传统头文件在预处理阶段把文本复制进每个翻译单元，因此宏会泄漏，重复解析也可能拖慢构建；模块通过 export/import 提供接口，普通宏状态不会像文本包含那样传播。模块接口需先编译成 BMI 等工件，构建系统必须安排依赖顺序。实际兼容性和构建收益取决于编译器、标准库和工程配置。〔S1，S2〕

```
// math.cppm: 模块接口单元
export module math;
export int add(int a, int b) { return a + b; }
// client.cpp: import math; 然后调用 add(1, 2)
// import 提供显式接口且宏不会像文本 include 那样泄漏
// BMI 需先构建；构建系统必须安排依赖，工具链支持仍需确认
```

018 C++20 协程的基本机制是什么？协程会自动创建线程吗?

答案 含 co_await、co_yield 或 co_return 的函数可按协程规则编译成可暂停和恢复的状态机。promise_type 定义返回对象与异常行为，awaiter 决定是否挂起以及如何恢复；调用方或调度器负责安排恢复时机。协程本身不创建线程，也不会自动提供 I/O、任务队列或并行调度。〔S1，S2〕

```
#include <coroutine>
struct Task { struct promise_type; std::coroutine_handle<promise_type> handle; };
Task fetch() { co_await wait_for_io(); co_return; } // 关键字让编译器生成可暂停状态机
// promise_type 定义返回对象、异常处理和初始/最终挂起行为
// awaiter 决定是否挂起及恢复方式；调度器/调用方负责何时 resume
// 协程本身不创建线程，也不自动并发或执行异步 I/O
```

019 std::expected 解决什么问题？它从哪个标准版本开始提供？

答案 std::expected<T,E>（C++23）保存成功值 T 或错误 E，适合把可预期、可恢复的失败显式放在返回类型中；调用者可检查 has_value，或用 value/error 访问对应分支。与 optional 相比它能携带错误原因，但并不会自动记录日志或替代所有异常。调用方仍应规定错误传播和处理策略。〔S2〕

```
#include <expected>
#include <string>
#include <string_view>
std::expected<int, std::string> parse(std::string_view text) { // C++23
  if (text.empty()) return std::unexpected(std::string("empty input"));
  return parse_integer(text);
}
auto result = parse("42");
if (result) use(*result); else log(result.error()); // 调用方显式处理错误
// expected 携带可恢复错误；不会自动日志，也不取代所有异常
```

020 constexpr、consteval 和 constinit 有什么区别？

答案 constexpr 可用于满足条件时的编译期求值，也可在运行时调用；consteval 是立即函数，调用必须在常量求值语境中完成。constinit 自 C++20 起要求静态或线程存储期变量经过常量初始化，防止动态初始化顺序问题，但变量仍可被修改。三者含义不同，不能互相替代。〔S2〕

```
constexpr int twice(int x) { return x * 2; } // 可编译期或运行期求值
consteval int fixed(int x) { return x * 2; } // 调用必须立即常量求值
constinit int startup = twice(4); // C++20 静态初始化保证，但变量可修改
int runtime = twice(read_input()); // constexpr 函数也可运行期调用
// constinit 不能替代 const；三者约束的是不同事情
// constinit 保证静态/线程存储期变量常量初始化，但不让对象自动 const
```

3.4 特殊成员函数控制

021 显式 = default 和 = delete 怎样控制类的特殊成员函数？

答案 = default 请求编译器按规则生成相应函数定义，函数能否最终可用仍取决于基类和成员；例如含不可拷贝成员时，默认化的拷贝操作也可能被定义为删除。= delete 明确禁止函数被使用，并且该函数仍参与重载决议，因此可精准拒绝某些参数类型，不只是把函数藏进 private。用户声明析构、拷贝或移动操作会影响其余操作的隐式生成，设计资源类时应整体核对 Rule of Five；业务类型优先由 RAII 成员实现 Rule of Zero，避免手写重复资源逻辑。〔N19、N20〕

```
struct Handle {
  Handle() = default;
  Handle(const Handle&) = delete;
  Handle& operator=(const Handle&) = delete;
  Handle(Handle&&) = default; // 仍须确认成员的移动语义正确
  Handle& operator=(Handle&&) = default;
};
```

3.5 C++11 语言与模板补充

022 C++11 的范围 for、enum class 和变长参数模板有何特点？

答案 C++11 范围 for 把范围遍历写成简洁循环，语义上依赖 begin/end；对大型元素使用 const auto& 可避免复制，遍历期间修改容器还要遵守迭代器失效规则。enum class 是有作用域的强类型枚举，枚举值不会隐式转成整数，可显式指定底层整数类型。变长参数模板用参数包表示零个或多个模板/函数参数，并通过展开把同一操作应用到各项；C++11 常用递归重载消费参数包，折叠表达式是 C++17 才加入。移动语义、智能指针、Lambda、auto、nullptr 等其他常见特性已在对应题目单独整理。〔N21、N22、N23〕

第 4 章 并发与线程安全

4.1 同步原语与线程管理

001 C++ 中如何使用 std::condition_variable 进行线程同步?

答案 条件变量不保存通知，必须与互斥锁保护的共享谓词一起使用。等待线程用 cv.wait(lock, [&]{ return ready; })，它会在等待时释放锁、被唤醒后重新加锁并循环检查，因而能处理虚假唤醒和通知早到。生产者先持锁更新 ready 或队列，再通知；队列/状态才是事实来源。

```
#include <condition_variable>
#include <mutex>
#include <thread>
std::mutex m; std::condition_variable cv; bool ready = false;
std::thread worker([&] { std::unique_lock<std::mutex> lock(m);
  cv.wait(lock, [&] { return ready; }); consume_data(); }); // wait 原子解锁并重查谓词
{ std::lock_guard<std::mutex> lock(m); ready = true; } // 状态才是事实来源
cv.notify_one(); worker.join(); // 线程退出前 join，避免悬空引用
```

002 C++ 中如何实现无锁队列?

答案 典型无锁队列用 CAS 比较并更新头尾指针或环形缓冲索引，失败时重读状态并重试。设计必须同时保证线性化正确、内存序足够、节点不会在其他线程仍读取时被释放，并处理 ABA。hazard pointer 或 epoch 回收可延迟释放。无锁不等于无等待或更快，优先用经过验证的实现并压力测试。

```
// 伪代码：Michael–Scott 队列；Node 含 atomic<Node*> next 和 value，先建 dummy
std::atomic<Node*> head{dummy}, tail{dummy};
void push(Value v) { Node* n = new Node(std::move(v));
  for (;;) { Node* t = tail.load(std::memory_order_acquire);
    Node* next = t->next.load(std::memory_order_acquire);
    if (t == tail.load() && !next && t->next.compare_exchange_weak(next, n)) break;
    if (next) tail.compare_exchange_weak(t, next); } // 帮助推进尾指针
}
// 省略 dequeue/内存回收：不能在其他线程可能读取时 delete；还须处理 ABA
```

003 C++ 中如何实现条件同步机制?

答案 先把要等待的条件表示为共享谓词，例如队列非空或可用槽位大于零，再用互斥锁保护谓词和数据，用条件变量等待。wait(lock,pred) 会在醒来后重查条件，避免虚假唤醒。计数型资源可用 C++20 semaphore；通知不是事件计数器，若每次事件都须保留，应把事件放进队列或计数状态。

```
#include <condition_variable>
std::mutex m; std::condition_variable cv; int permits = 0;
void wait_for_work() { std::unique_lock<std::mutex> lock(m);
  cv.wait(lock, [&] { return permits > 0; }); --permits; }
void signal_work() { { std::lock_guard<std::mutex> lock(m); ++permits; }
  cv.notify_one(); }
// wait 用谓词循环处理虚假唤醒；通知本身不保存事件
// 若要保留每次事件，队列/计数状态须记录它；多线程计数可用 semaphore
```

004 C++ 中如何避免死锁?

答案 死锁常由线程以相反顺序持有多把锁造成；可规定全局加锁顺序，或用 std::scoped_lock/std::lock 统一获取多把互斥量。缩小临界区，持锁时避免调用可能回调当前代码的未知函数。try_lock 和超时锁适合特定降级策略，但要定义失败后的恢复逻辑，不能只靠重试掩盖锁顺序问题。

```
#include <mutex>
std::mutex account, inventory;
void transfer() {
  std::scoped_lock lock(account, inventory); // 一次获取多锁，避免反序死锁
  update_account(); update_inventory(); // 只在临界区做必要工作
} // 异常或返回时 RAII 自动解锁
// 对外回调/未知代码可能反向获取锁；应避免持锁调用
// 也可规定所有路径按同一全局顺序加锁
```

005 C++ 中的 std::async 如何创建异步任务?

答案 std::async 以函数和参数启动异步共享状态并返回 future；若不指定策略，允许选择 launch::async 或 launch::deferred。async 策略可并发运行，deferred 则到 wait/get 时在调用线程执行。future 销毁等细节可能影响等待，且它不是可控线程池；要处理异常、取消需求和任务调度应选更明确的执行框架。

```
#include <future>
auto result = std::async(std::launch::async, [] { return compute(); });
// 指定 async 避免默认策略可能 deferred 到 get 时才在本线程执行
try { auto value = result.get(); use(value); }
catch (const std::exception& e) { report(e.what()); } // 异常经 future 传播
// get 只取结果一次；async 不是有界线程池，也不提供强制取消
```

006 C++ 中的 std::mutex 和 std::lock_guard 是如何工作的?

答案 std::mutex 通过 lock/unlock 保证同一临界区只有一个线程进入，竞争线程会等待；std::lock_guard 用 RAII 在构造时锁定、离开作用域时解锁，因此异常路径也能释放。多个互斥量需同时获取时可用 std::scoped_lock 避免常见的锁序死锁。互斥量须由成功获取它的线程释放。

```
#include <mutex>
std::mutex m; int balance = 0;
void deposit(int amount) {
  std::lock_guard<std::mutex> lock(m); // 构造时锁定
  balance += amount;
  if (should_throw()) throw Error{}; // 示例：异常也会退出作用域
} // 析构自动 unlock，避免手动 unlock 漏在异常路径
// 多把锁用 std::scoped_lock；只由成功获取锁的线程释放
```

007 C++ 中的原子操作（std::atomic）如何使用?

答案 std::atomic<T> 对指定对象上的读改写提供不可分割的原子操作，并用 memory_order 控制与普通内存的同步关系。它只保护这个原子对象，不会自动让余额和版本号等多字段更新成为一个事务。先使用默认顺序一致模型确保正确，再根据发布/获取关系谨慎放宽，并用 happens-before 推理。

```
#include <atomic>
std::atomic<int> requests{0};
requests.fetch_add(1, std::memory_order_relaxed); // 只需原子计数，不发布别的数据
std::atomic<bool> ready{false}; int payload = 0;
// 若用 ready 发布 payload，应使用 release store / acquire load
// atomic 只保护自身对象；余额与版本号等多字段不自动构成事务
// 正确性优先用默认 seq_cst，再按同步关系优化内存序
```

008 C++ 中的线程局部存储（Thread Local Storage）是什么?

答案 thread_local 为每个线程创建独立实例，适合线程本地缓存、错误状态或临时上下文，避免多个线程直接写同一个变量。它并不会同步线程间共享数据；线程退出时其对象通常会析构。线程局部对象若由动态库提供，析构和卸载行为还可能受平台运行时规则影响。

```
#include <string>
#include <string_view>
thread_local std::string scratch; // 每个线程都有独立字符串
void parse(std::string_view input) { scratch.assign(input); process(scratch); }
// 两线程调用时 scratch 不共享，避免为这份临时状态加锁
std::string shared; // 普通全局变量仍需同步保护
// 线程退出时会析构自己的 thread_local 对象
// TLS 不是跨线程通信方式；动态库卸载还需考虑平台运行时行为
```

009 C++ 中的线程（std::thread）如何创建和管理?

答案 构造 std::thread 时线程函数即可开始运行；对象销毁前必须 join 等待它结束，或 detach 让其独立运行，否则析构会调用 std::terminate。优先用具有清理保证的 RAII 或 C++20 jthread。按引用传递的参数和捕获对象必须活得比工作线程久，detach 尤其容易造成悬空访问。

```
#include <exception>
#include <thread>
std::thread worker([] { try { do_work(); } catch (...) { report_failure(); } });
// 线程函数不可让异常逃逸到线程入口，否则会 terminate
if (worker.joinable()) worker.join(); // 返回前确保工作结束
// 所有引用捕获的数据都必须比 worker 活得久
// detach 会让生命周期难以证明；优先 RAII 管理或 C++20 std::jthread
```

010 C++ 中的读写锁（std::shared_mutex）是什么？

答案 std::shared_mutex 允许多个线程同时持有共享锁读取，同一时刻写锁独占；shared_lock 用于读，unique_lock 用于写。它适合读多写少且读临界区足以覆盖加锁成本的场景。标准不保证对读写公平的具体策略，写者可能等待；不能持有读锁后再随意升级成写锁，应显式设计锁转换。

```
#include <shared_mutex>
std::shared_mutex data_mutex; Data data;
Data snapshot() { std::shared_lock<std::shared_mutex> lock(data_mutex); return data; }
void update(Data next) { std::unique_lock<std::shared_mutex> lock(data_mutex); data = std::move(next); }
// 多个 reader 可同时读，writer 独占；锁覆盖整个数据访问
// 写多时读写锁开销/等待可能不划算；标准不保证公平策略
// 不要假设能安全地从共享锁直接升级到独占锁
```

4.2 并发任务组织

011 如何使用 C++ 实现生产者-消费者模型？

答案 用互斥量保护任务队列，用两个条件谓词表示队列非空和未达到容量上限。生产者在锁内检查容量并入队，再通知消费者；消费者取走任务后通知生产者，执行任务要放在锁外。关闭时需唤醒所有等待者并让谓词包含停止状态，明确满队列是阻塞、拒绝还是丢弃。

```
#include <condition_variable>
#include <cstddef>
#include <mutex>
#include <queue>
#include <utility>
std::mutex m; std::condition_variable not_empty, not_full;
std::queue<Job> q; bool stopping = false; constexpr std::size_t capacity = 8;
void produce(Job job) { std::unique_lock<std::mutex> lk(m);
  not_full.wait(lk, [] { return q.size() < capacity || stopping; });
  if (stopping) return; q.push(std::move(job)); lk.unlock(); not_empty.notify_one(); }
void consume() { std::unique_lock<std::mutex> lk(m);
  not_empty.wait(lk, [] { return !q.empty() || stopping; });
  if (q.empty()) return; Job job = std::move(q.front()); q.pop(); lk.unlock();
  not_full.notify_one(); run(job); } // 锁外执行；shutdown 要唤醒等待者
```

012 如何实现线程池？

答案 线程池通常包含固定或可调数量的工作线程、有界任务队列、互斥量和条件变量；工作线程醒来后取出任务，在不持队列锁的情况下执行。接口需定义队列满时的拒绝/阻塞、任务异常如何回传、关闭时是否排空任务。析构或 shutdown 应设置停止状态、通知全部线程并 join，保证依赖资源仍存活。

```
// 线程池伪代码：队列由 mutex 保护，cv 表示“有任务或停止”
void worker_loop() { for (;;) { Task task;
  { std::unique_lock<std::mutex> lk(m);
    cv.wait(lk, [&] { return stopping || !tasks.empty(); });
    if (stopping && tasks.empty()) return;
    task = std::move(tasks.front()); tasks.pop(); }
  try { task(); } catch (...) { report_task_error(); } // 锁外执行
} }
void shutdown() { set_stopping(); cv.notify_all(); join_all(); }
// API 还需规定队列满、异常传回与“排空/丢弃”策略
```

4.3 内存模型与无锁协作

013 std::memory_order 中 relaxed、acquire/release 和 seq_cst 的差异是什么？

答案 relaxed 保证原子操作自身不可分割，但不建立与其他普通读写的同步。release store 与读到该值的 acquire load 可建立跨线程同步，从而让发布前的普通写对获取方可见；seq_cst 还要求顺序一致操作处在满足规则的单一全序中。选择内存序应先写出所需 happens-before 关系，再验证程序。〔S2〕

```
#include <atomic>
#include <thread>
int payload = 0; std::atomic<bool> ready{false};
std::thread producer([&] { payload = 42; ready.store(true, std::memory_order_release); });
while (!ready.load(std::memory_order_acquire)) {} // 读到 release 后同步
int value = payload; // producer 对 payload 的写 happens-before 此读
producer.join(); // join 管理生命周期；不能用 volatile 代替 atomic
```

014 std::jthread 和 std::thread 有什么不同？stop_token 能强制终止线程吗？

答案 std::thread 若析构时仍 joinable 会调用 terminate；std::jthread 析构时会请求停止，并在仍可 join 时等待线程结束。stop_token 是协作信号，函数需主动检查、使用可停止等待或注册回调；它不会强杀线程，也不会自动打断任意阻塞系统调用。线程必须保证停止请求后最终可退出。〔S2〕

```
#include <thread>
std::jthread worker([](std::stop_token stop) {
  while (!stop.stop_requested()) process_one(); // 合作式检查停止请求
});
worker.request_stop(); // 只发信号，不会强制杀掉线程
// jthread 析构会请求停止并 join；线程体必须能最终退出
// stop_token 不会自动中断任意阻塞调用，等待应使用可停止机制
```

015 std::latch 和 std::barrier 如何选择？

答案 latch 是一次性计数门闩，多个线程 count_down 后计数归零，等待者继续，之后不能重置。barrier 用于重复阶段：每一轮参与者到齐后进入下一阶段，可在阶段切换时执行 completion 函数。一次任务启动汇合适合 latch，图像分块处理等多轮同步阶段适合 barrier。〔S2〕

```
#include <latch>
std::latch start_line(4); // 一次性门闩，4 个参与者汇合后开放
void worker() { prepare(); start_line.count_down(); start_line.wait(); run_phase(); }
// 下一轮不可重置 latch；多轮阶段同步改用 std::barrier
// barrier 每轮参与者到齐后进入下一阶段，还可执行 completion 回调
// 启动后固定参与者数量的一次汇合选 latch；重复算法阶段选 barrier
```

016 C++20 atomic::wait/notify 与忙等轮询有什么区别？

答案 atomic::wait(old) 在原子值仍等于 old 时阻塞，值发生变化并收到通知后才重新检查；notify_one/all 只唤醒等待者，不代替状态更新。它相比忙等减少 CPU 空转，适合状态等待。仍要用循环检查应用谓词，且值变化后又回到 old 的 ABA 情况可能不会被观察到。〔S2〕

```
#include <atomic>
std::atomic<int> state{0};
void waiter() { while (state.load() == 0) state.wait(0); // 值未变时阻塞
  continue_work(); }
void signal() { state.store(1); state.notify_one(); } // 先更新状态，再唤醒
// notify 不是事件计数；谓词/值是状态来源，醒来后仍要重查
// 相比忙等减少 CPU 空转；若值快速变回旧值，可能看不到中间状态
```

017 数据竞争（data race）和竞态条件（race condition）有什么区别？

答案 数据竞争是 C++ 内存模型中的特定情形：不同线程对同一内存位置发生未排序的冲突访问，至少一个为非原子写，结果是未定义行为。竞态条件是更广的逻辑问题，例如两个线程各自原子地检查和扣库存，仍可能因检查与更新未组成整体操作而超卖。消除数据竞争不等于业务逻辑正确。〔S2〕

```
std::atomic<int> stock{1};
// 两次原子操作各自安全，但“检查再扣减”不是一个原子事务
if (stock.load() > 0) stock.fetch_sub(1); // 两线程都可能先读到 1，导致变负
// 用 CAS 循环，或把检查与更新放进同一把 mutex
// 数据竞争会导致 C++ 未定义行为；竞态条件还包括这种逻辑错误
// 修掉 data race 后仍要验证业务不变量
```

018 std::atomic_ref 有什么用途？使用时要注意什么？

答案 std::atomic_ref<T> 可在已有 T 对象上提供原子读改写，适合无法把数据结构成员声明为 atomic 的场景。目标对象地址须满足 atomic_ref<T>::required_alignment；atomic_ref 存续期间，对该对象的并发访问须符合原子访问约束，不能一边经 atomic_ref 写、一边普通非原子读。被引用对象也必须始终存活。〔S2〕

```
#include <atomic>
struct Counter { alignas(std::atomic_ref<int>::required_alignment) int value = 0; };
Counter counter;
std::atomic_ref<int> atomic_value(counter.value); // 在已有对象上原子操作
atomic_value.fetch_add(1, std::memory_order_relaxed);
// atomic_ref 存活期间，禁止对 value 并发做普通非原子读写
// Counter 和成员必须活到所有 atomic_ref 使用完；还需满足对齐要求
```

第 5 章 性能与工程实践

5.1 性能测量与基础优化

001 C++ 中如何提高程序的执行效率?

答案 先定义目标指标，如吞吐、尾延迟或内存占用，再用采样 profiler 和基准测试定位真实热点。通常优先降低算法复杂度、减少重复工作和不必要分配，再检查数据局部性与并行度。每次只改一个因素，在相同编译选项和代表性数据上复测，避免把微基准收益误认为端到端提升。

```
#include <chrono>
#include <chrono>
auto begin = std::chrono::steady_clock::now();
for (int i = 0; i < repetitions; ++i) run_representative_work();
auto elapsed = std::chrono::steady_clock::now() - begin;
report(elapsed); // 先记录正确性、吞吐/尾延迟/内存等目标指标
// 用 profiler 找热点，再一次只改一个因素并重复测量
// 固定编译选项和输入；微基准收益须用端到端负载验证
```

002 C++ 中如何避免不必要的内存拷贝?

答案 接口只读大对象时可用 const T& 或 span 借用，所有权需要转移时用值传递配合移动；从函数按值返回通常可由编译器消除拷贝，不必为避免复制而返回局部对象引用。emplace 只有在容器内直接构造确实省步骤时才可能有收益。先测量，并确保引用不会超过原对象生命周期。

```
#include <vector>
void inspect(const std::vector<Record>& records) { // 只读借用，不复制大容器
  for (const Record& record : records) inspect_one(record);
}
std::vector<Record> make_records() { std::vector<Record> result;
  fill(result); return result; // 返回局部值可做 NRVO，不要返回局部引用
}
void take_ownership(std::vector<Record> records) { consume(std::move(records)); }
// 借用用引用/span；真正转移所有权时用值加移动，并检查生命周期
```

003 C++ 中的 SIMD（单指令多数据）优化如何实现?

答案 SIMD 将同一条运算应用于一组独立数据，例如同时对多个像素做亮度变换。先采用连续、对齐友好的数据布局并查看优化报告，让编译器自动向量化；确有瓶颈时再用 intrinsic 或 SIMD 库。分支、尾部余数、内存带宽和数据对齐都会影响收益，需在目标 CPU 上测基准。

```
#include <cstddef>
// 连续数组、彼此独立的运算利于编译器自动向量化
for (std::size_t i = 0; i < count; ++i) output[i] = input[i] * scale + bias;
// 先确认无重叠/别名假设，查看编译器向量化报告
// 对齐、尾部元素、分支和内存带宽都会影响收益
// 手写 intrinsic 前，在目标 CPU 和代表性数据上比较吞吐
// SIMD 只适合彼此独立且能批量执行的工作
```

004 C++ 中的内联函数（inline）如何优化性能?

答案 inline 不要求编译器展开调用；编译器会根据优化级别、函数大小、调用频率和跨模块信息决定是否内联。内联可消除调用并暴露常量传播机会，但过多展开会增大代码、损害指令缓存。它还允许符合 ODR 的函数定义出现在多个翻译单元，性能效果应以生成代码和基准为准。

```
// 头文件中的定义可标 inline 以符合多翻译单元 ODR
inline int normalize(int x) { return x < 0 ? 0 : x; }
int result = normalize(input); // 编译器仍可决定保留普通调用
// 内联可省调用并暴露常量传播；过度展开增大代码和指令缓存压力
// 查看优化后汇编与基准，不要把 inline 当性能保证
// ODR 允许定义跨单元出现，与优化器是否展开是两回事
```

005 C++ 中的循环展开（Loop Unrolling）是什么?

答案 循环展开把多次迭代的工作写在同一循环体中，可能减少循环计数和分支开销，并暴露更多独立指令供处理器并行执行。但展开过多会增加代码体积、寄存器压力和指令缓存未命中，尾部处理也更复杂。现代编译器常能自动决定展开程度，手动优化前应检查汇编并实测。

```
#include <cstddef>
for (std::size_t i = 0; i + 3 < n; i += 4) { // 一次处理四项
  sum += values[i]; sum += values[i + 1];
  sum += values[i + 2]; sum += values[i + 3];
}
for (std::size_t i = n - n % 4; i < n; ++i) sum += values[i]; // 尾部
// 展开可能减少分支，但会增加代码体积/寄存器压力
// 现代编译器常能自动展开；先看汇编和真实基准
```

006 C++ 中的线程开销如何优化?

答案 短任务反复创建和销毁线程会产生调度与栈初始化开销，可用线程池、批处理或把工作合并成较粗粒度任务。还要测量锁竞争、上下文切换、线程数和任务队列延迟，避免并行度超过 CPU 或 I/O 能力。线程池会增加队列和关闭管理复杂度，按吞吐及尾延迟验证收益。

```
// 伪代码：启动时创建固定工作线程，后续任务复用线程
ThreadPool pool(worker_count);
for (Batch batch : split_into_coarse_batches(input))
  pool.submit([batch] { process(batch); });
pool.wait_idle(); // 完成后继续汇总；析构前先排空并 join
// 合并过小任务可降低调度/分配开销；队列、停止和异常需定义
// 线程太多会增加切换和争用，按吞吐与尾延迟测量
```

5.2 算法、缓存与 I/O

007 如何优化 C++ 中的算法复杂度?

答案 先声明分析口径：Big-O 描述渐近增长上界，面试中还要区分最坏、平均、期望和均摊复杂度，并分别计算辅助空间与输出空间。顺序代码通常由增长最快的主项决定；嵌套循环要按实际执行次数求和，不能只看语法层数；递归还要计入递归深度和调用栈。再结合输入规模与操作比例选算法和容器：unordered_map 查找平均期望 O(1) 但最坏可达 O(n)，map 为 O(log n) 且有序；list 只有在已持有位置迭代器时插删才便宜，查找仍是 O(n)。双指针、滑动窗口和二分需满足各自的数据条件，动态规划用于消除重复子问题；最后用复杂度和实测共同评估。

```
#include <algorithm>
#include <list>
#include <map>
#include <unordered_map>
std::unordered_map<Key, Value> by_key; // 平均期望 O(1)，最坏 O(n)
std::map<Key, Value> ordered; // 有序查找/插入 O(log n)
std::list<Value> chain; // 已有迭代器时插删便宜，按值查找仍 O(n)
// 二分需有序；双指针/滑窗需满足单调条件，DP 消除重复子问题
// 依输入规模和操作比例选结构，再用代表性负载验证
```

008 如何优化 C++ 的循环和内存访问?

答案 顺序遍历连续存储通常更利于缓存预取，指针追踪则可能造成随机内存访问。可根据访问模式选择 AoS 或 SoA、循环融合或分块，让工作集适合缓存；避免盲目重排会影响可读性和 ABI。用硬件计数器或 profiler 确认 cache miss、带宽或分支确实是瓶颈，再比较改动前后的目标负载。

```
#include <cstddef>
// 行主序矩阵按行扫描，可顺着连续地址访问
for (std::size_t r = 0; r < rows; ++r)
  for (std::size_t c = 0; c < cols; ++c) sum += matrix[r * cols + c];
// 分块让工作集更可能留在缓存；AoS 适合逐对象处理、SoA 适合批量单字段
// 循环融合可减少重复读写，也可能扩大工作集或降低可读性
// 用 cache-miss/带宽计数确认瓶颈，别为 ABI 贸然重排公开结构
```

009 如何优化 C++ 程序中的 I/O 性能?

答案 文本或文件 I/O 可通过批量读写和合适缓冲减少系统调用;C++ iostream 与 stdio 混用时,关闭 sync_with_stdio 后不要再假设两套缓冲顺序一致，交互输入还需按需 flush。setvbuf 只配置 C 流缓冲，并应在该流执行 I/O 前调用。std::async 只是调度任务，不会把阻塞文件调用变成真正异步 I/O。

```
#include <iostream>
#include <vector>
int main() {
  std::ios::sync_with_stdio(false); std::cin.tie(nullptr); // 所有 I/O 前调用
  std::vector<int> values; int x;
  while (std::cin >> x) values.push_back(x); // 批量缓冲输入
  std::cout << values.size() << '\n';
} // 关闭 sync 后勿假设与 printf 的缓冲顺序; 交互场景需 flush
// 异步调度不等于内核异步 I/O; 按平台选缓冲/批量接口
```

010 如何使用缓存优化提高 C++ 程序性能?

答案 缓存优化利用时间局部性（近期访问可能再次访问）和空间局部性（相邻数据可能一并使用）。连续访问、减少无用字段加载通常有利；AoS 适合一次处理单个完整对象，SoA 更适合对大量对象的单一字段批量运算。缓存行大小因处理器而异，伪共享与工作集大小需用硬件计数器和真实负载确认。

```
#include <cstddef>
#include <vector>
struct AoS { float x, y, z; };
std::vector<AoS> points(count); // 整个对象处理时 AoS 连续
std::vector<float> x(count), y(count), z(count); // 批量改 x 时 SoA 少加载无关字段
for (std::size_t i = 0; i < count; ++i) x[i] *= scale; // 空间局部性
// 重用近期数据利用时间局部性; 相邻数据利用空间局部性
// 缓存行和伪共享依硬件; 用工作集与硬件计数器验证
```

5.3 链接期优化

011 LTO 和 PGO 分别做什么? 可以一起使用吗?

答案 LTO 在链接阶段利用多个翻译单元的中间表示做跨单元优化；PGO 则用代表性运行负载采集的分支和调用频率指导优化。两者提供的信息不同，可组合使用，但会增加构建和采集步骤；训练负载偏离生产会误导优化。部署前应在目标机器和实际工作负载上比较吞吐、延迟和体积。〔S3〕

```
// PGO 先用代表性负载生成 profile，再用 profile 构建部署版本
// CXXFLAGS += -fprofile-generate
// run representative_workload
// CXXFLAGS += -fprofile-use -flto
// LTO 在链接时跨单元优化，PGO 按运行频率指导分支/调用优化
// 两者可组合；训练数据偏离生产会误导优化
// 比较实际延迟、吞吐和体积后再决定
```

第 6 章 编译、链接与构建

6.1 头文件与项目组织

001 #pragma once 和 #include guards 有什么区别?

答案 Include guard 是标准预处理器写法：定义唯一宏，后续重复包含时跳过文件内容；#pragma once 更简短，但属于编译器广泛支持的扩展，并非标准指令。两者目的都是避免同一翻译单元重复展开，并不解决两个不同文件意外使用相同 guard 名的问题。团队应采用一致且唯一的 guard 命名规范。

```
// 可移植的 include guard
#ifndef PROJECT_NET_SOCKET_HPP
#define PROJECT_NET_SOCKET_HPP
struct Socket { void close(); };
#endif
// #pragma once 更短且主流编译器支持，但属于实现扩展
// 二者都防止同一翻译单元重复展开; guard 宏必须在工程内唯一
// 它们不消除真正的循环依赖
```

002 C++ 中使用 -fPIC 编译选项有什么作用?

答案 -fPIC 让编译器生成适合装载到不同虚拟地址的代码，常用于 Linux 等平台的共享库；静态目标若之后可能被链接进共享库，也可能需要以 PIC 编译。其实现会通过平台相关寻址方式访问全局数据，可能有微小成本。具体选项和是否必需依目标平台、架构与链接器而异。

```
// 共享库里的目标文件通常需位置无关代码
// g++ -fPIC -c lib.cpp -o lib.o
// g++ -shared lib.o -o libexample.so
// -fPIC 使用平台相关寻址方式，适用于多地址装载
// 静态库以后若可能被链接进共享库，也可能要编成 PIC
// 是否需要/性能影响依架构和工具链，Windows 选项不同
```

003 C++ 中动态库的符号表（Symbol Table）是什么?

答案 动态符号表列出动态链接器需要查找的导出符号及相关绑定信息；普通完整符号表还可能包含调试或内部符号，二者用途不同。可通过默认隐藏可见性并显式标记公共 API 来减少 ABI 暴露，再用 readelf -Ws 或 nm -D 检查导出结果。符号导出变更可能影响运行时兼容性。

```
// 默认隐藏实现符号，只导出有意公开的 ABI
#if defined(__GNUC__)
__attribute__((visibility("default"))) void public_api();
#endif
// readelf -Ws libexample.so 查看 ELF 符号表
// nm -D libexample.so 只查看动态符号
// 普通符号表也可能包含调试/内部符号；导出集是 ABI 面的一部分
// visibility 属性是工具链扩展，跨平台需定义统一宏
```

004 C++ 中的 attribute((constructor)) 和 attribute((destructor)) 有何作用?

答案 __attribute__((constructor)) 和 destructor 是 GCC/Clang 等工具链支持的扩展，可让函数在程序常规入口前或退出阶段运行；它们不是标准 C++，跨平台行为和优先级应查对应编译器文档。此类初始化顺序不易表达依赖关系，也可能遇到动态库卸载问题。通常应使用静态局部对象、RAII 或显式启动/关闭接口。

```
#if defined(__GNUC__)
__attribute__((constructor)) static void before_main() { init_global(); }
__attribute__((destructor)) static void after_main() { shutdown_global(); }
#endif
// 这是 GCC/Clang 扩展，不是标准 C++，库卸载和顺序依平台
// 多个初始化器之间的依赖难表达，可能遇到静态初始化顺序问题
// 普通程序优先使用 RAII、函数内 static 或显式启动/停止接口
```

005 C++ 中的前向声明（Forward Declaration）是什么?

答案 前向声明只让编译器知道类型或函数名称存在，不提供类成员、布局或大小。声明指针、引用、函数参数和返回类型时经常足够；定义按值成员、继承、访问成员或实例化需要完整布局的操作时必须包含定义。把非必要 include 移到源文件可降低耦合和重编译范围。

```
// node.hpp：若只声明指针/引用，可避免包含完整定义
class Node;
void visit(const Node& node);
class Owner { Node* node_; }; // 指针成员允许不完整类型
// node.cpp 再 #include "node_definition.hpp" 访问 Node 成员
// 按值成员、继承、sizeof 或内联访问成员需要完整类型定义
```

006 C++ 多文件项目中，头文件与源文件如何组织?

答案 头文件放需要被调用方看到的声明、公共类型和模板定义；非模板实现通常放源文件，并在一个目标文件中定义一次。每个头文件应自包含并使用 guard；前向声明可减少依赖，但使用完整类型的位置必须 include 定义。模板若要在调用处实例化，通常需要可见定义，模块化项目也应明确接口依赖。

```
// widget.hpp：向调用方暴露声明和所需完整类型
#ifndef WIDGET_HPP
#define WIDGET_HPP
class Widget { public: void run(); };
#endif
// widget.cpp：#include "widget.hpp" 后定义 void Widget::run() { ... }
// 每个头文件应可自包含；非模板实现一般只在一个源文件定义
// 模板实现通常需在实例化点可见；前向声明可降低无关重编译
```

6.2 编译、链接与库

007 C++ 源文件如何经过编译生成可执行文件?

答案 典型工具链依次预处理宏和 include、编译语法语义并生成汇编、汇编成目标文件，最后由链接器解析符号、重定位并组合库生成可执行文件。实际工具可能把阶段封装在一次命令中。编译期错误多看源文件和头文件；未定义符号、重复定义多在链接阶段出现。启用 LTO 时链接器还可做跨单元优化。

```
// 逐阶段查看工具链输出（命令以 GCC/Clang 为例）
// c++ -E app.cpp -o app.ii     预处理宏与 include
// c++ -S app.ii -o app.s       编译为汇编
// c++ -c app.s -o app.o        汇编成目标文件
// c++ app.o lib.o -o app       链接符号、重定位并生成可执行文件
// 源/头错误看编译阶段；未定义/重复符号看链接阶段，LTO 会延伸优化
```

008 extern 在多文件程序中如何声明共享变量?

答案 在头文件写 extern int count; 这通常只是声明，不分配存储；在一个且仅一个源文件写 int count = 0; 才是定义，其他文件包含声明后可引用。C++17 起也可用 inline 变量在头文件定义共享实体。全局变量初始化顺序可能跨翻译单元不确定，应尽量封装并控制依赖。

```
// shared.hpp：仅声明，不分配定义
extern int shared_count;
// state.cpp：程序中恰好一个定义
int shared_count = 0;
// use.cpp：包含声明后可访问 shared_count
void increment() { ++shared_count; } // 多线程时仍需同步
// C++17 可用 inline int shared_count = 0; 放头文件共享一个实体
```

009 static 变量在头文件中的作用是什么?

答案 命名空间作用域 static 变量具有内部链接；如果定义在头文件，每个包含它的翻译单元都会得到自己的独立副本，可能导致状态不共享且增加存储。需要程序级共享对象时，可在头文件 extern 声明、单个源文件定义；C++17 也可使用 inline 变量，仍需考虑初始化顺序和并发访问。

```
// counter.hpp
static int counter = 0; // 每个包含该头的翻译单元各自一份
// a.cpp 和 b.cpp 各 include 后，修改的是不同对象
// 需要一个程序级共享实体时：extern 声明 + 单个源文件定义
// 或 C++17 inline int counter = 0; 允许头文件中的同一实体定义
// 全局共享仍需考虑初始化顺序和并发访问
```

010 头文件的依赖关系如何避免循环引用?

答案 循环引用通常是 A.h 包含 B.h、B.h 又包含 A.h，导致类型定义依赖纠缠。若一个类仅通过指针或引用提及另一类，可在头文件前向声明并把实际 include 放到源文件；成员按值存放、继承或内联实现访问成员时仍需完整定义。Include guard 防止重复展开，但不会消除真正的依赖循环。

```
// a.hpp：只通过引用提及 B，可前向声明
class B;
class A { public: void use(const B&); private: B* b_; };
// a.cpp 再 include b.hpp 并定义 A::use，因访问成员需要完整类型
// 如果 A 按值保存 B、继承 B 或内联访问 B 的字段，则前向声明不够
// include guard 防重复展开，不会消除真实依赖循环
```

011 如何定位常见的 C++ 链接错误?

答案 先看链接器报错中的符号和目标文件：undefined reference 常表示实现未参与链接、签名/命名空间不匹配或库缺失；multiple definition 表示非 inline 实体被重复定义；找不到库则核对搜索路径及平台前缀后缀。用 nm/readelf/dumpbin 查符号，并打开详细链接命令确认实际输入。

```
// undefined reference：实现源文件/库未链接，或声明签名、命名空间不一致
// nm -C app.o libpart.a | rg 'WantedSymbol' 检查符号是否存在
// g++ app.o libpart.a -o app   # 并确认静态库位于引用者之后
// multiple definition：查找重复定义，头文件函数考虑 inline
// 打开构建系统 verbose 命令核对实际输入文件、库路径和链接顺序
// 符号名存在但 ABI 不匹配时核对编译器/运行库/宏设置
```

012 如何调试链接错误?

答案 先分类错误：undefined reference 查定义是否编译入目标和签名是否完全匹配；multiple definition 查头文件中的非 inline 定义；找不到库查库路径、名称及链接顺序；ABI 错误查编译器、运行库和宏配置。用构建系统输出完整命令，再用 nm、dumpbin 或链接器 map 文件追踪符号来源。

```
// header.hpp：多翻译单元可见的相同函数定义要满足 inline 规则
inline int helper(int x) { return x + 1; }
// 若移除 inline 且 header 被 a.cpp、b.cpp 同时包含，会重复定义
// undefined reference 则相反：只有声明，定义没有进入链接输入
// 用 nm -C 查符号，构建 verbose 输出确认哪个 .o/.a 真正参与链接
// 链接成功也不能证明 ODR 一定满足
```

013 静态库和动态库的链接方式有什么区别?

答案 静态库通常是目标文件归档，链接时把被需要的目标代码复制进最终产物，部署较直接但更新一般要重新链接，体积和重复代码可能增加。动态库由加载器在运行时装入，多个进程可共享其映射且库可单独升级，但需正确部署搜索路径、依赖和兼容 ABI。两种方式的实际收益依平台及链接选项而异。

```
// 静态库：归档目标文件，所需代码通常复制进最终程序
// ar rcs libmath.a add.o
// c++ app.o -L. -lmath -o app
// 动态库：运行时加载，可共享映射/单独升级，但要部署依赖和搜索路径
// c++ -fPIC -shared add.cpp -o libmath.so
// c++ app.o -L. -lmath -Wl,-rpath,'$ORIGIN' -o app
// 部署便利、体积、ABI 兼容性和更新方式按平台权衡
```

6.3 ODR 与 ABI

014 C++ 的 ODR（单一定义规则）是什么？违反后一定会有链接错误吗?

答案 ODR 要求程序中需要定义的实体满足唯一性或特定例外规则；模板、类和 inline 函数可在多个翻译单元定义，但通常须遵守相同 token 序列及一致查找等条件。违规则可能是链接错误，也可能属于无需诊断的 ill-formed 程序，导致不同翻译单元行为不一致。链接成功不能证明满足 ODR。〔S2〕

```
// C++17 inline 变量允许同一头文件定义出现在多个翻译单元
inline constexpr int retry_limit = 4;
// 对应的定义必须满足 ODR 要求，例如 token 序列和名字查找一致
// 普通非 inline 函数/变量若定义多次通常会链接失败
// 某些 ODR 违规无需诊断，可能链接成功却让不同单元行为不一致
// 所以“链接通过”不是 ODR 正确性的证明
```

015 C++ ABI 是什么？它和源代码兼容有什么区别?

答案 ABI 描述已编译模块如何互通，例如符号修饰、调用约定、参数传递、对象布局、虚函数分派和异常处理。源代码兼容关注代码重新编译后能否通过；ABI 兼容关注旧二进制能否不重编译就与新库协作。改变公开类大小、虚函数顺序或编译器运行库常会破坏 ABI，需版本策略。〔S4〕

```
// 稳定 C ABI 示例：C/C++ 两侧共享不透明句柄，不暴露类布局
extern "C" { struct Engine;
  Engine* engine_create();
  void engine_destroy(Engine*);
  int engine_run(Engine*, const char* input);
}
// 源码兼容看重新编译后能否调用；ABI 兼容看旧二进制能否继续链接运行
// 改公开类大小、虚函数顺序、调用约定或运行库可能破坏 ABI
```

6.4 头文件搜索与二进制装载

016 头文件包含中的双引号和尖括号怎样影响搜索路径?

答案 二者都交给预处理器查找并展开文件，区别在于实现定义的搜索策略。以 GCC 为例，双引号形式先查当前被包含文件所在目录，再查 -iquote 路径及尖括号搜索链；尖括号通常查 -I、系统头文件等路径。具体顺序仍受编译器和命令行选项影响，不应简单等同于“用户文件”和“标准库文件”。包含同名文件时，应结合编译命令和预处理输出定位实际路径。选择形式不会自动解决循环依赖、重复定义或 ODR 问题，这些需另外设计声明和定义边界。〔N24〕

```
#include "project/config.hpp" // 常用于项目头文件
#include <vector> // 常用于库头文件
// GCC 可用 -E -v 查看预处理与搜索目录；实际以构建命令为准
```

017 Linux 进程启动或 dlopen 时，动态库如何映射并完成符号绑定？

答案 动态链接可执行文件启动时，内核读取 ELF 信息并把控制权交给指定的运行时装载器；装载器按依赖和搜索规则映射共享对象，处理必要重定位、符号解析及初始化，再进入程序。dlopen 可在运行中请求装载额外对象，dlsym 取得符号，dlclose 递减引用，但不保证所有情形都立即卸载。共享库的只读代码页可被多个进程共享，进程的可写数据仍有独立语义。库搜索路径、RUNPATH、符号版本和 RTLD_NOW/RTLD_LAZY 会影响行为；文件存在不代表依赖和 ABI 一定相容。〔N25〕

018 一次函数调用一定把所有参数压栈吗？栈帧通常保存什么？

答案 参数传递、返回值和寄存器保存由目标调用约定决定；例如常见 x86-64 System V ABI 优先用若干通用或向量寄存器传递合适的参数，剩余部分才使用栈，不能把 32 位教材的逐个压栈过程套到所有平台。栈帧可能含局部对象、溢出寄存器、保存的返回信息与对齐填充；内联、尾调用和省略帧指针会改变实际形态。调用者与被调用者分别保存哪些寄存器也是 ABI 的一部分。调试回溯还可依赖展开信息，优化后变量和源码帧未必一一存在。〔N26、N11〕

第 7 章 操作系统与 Linux

7.1 文件、权限与进程

001 Linux 文件权限 755 和 644 表示什么？

答案 755 按八进制拆成所有者 7、组 5、其他人 5，分别对应 rwx、r-x、r-x；644 则是 rw-、r--、r--。对普通文件，x 表示可执行；对目录，x 表示可搜索并穿越，缺少 r 时仍可能已知名称后访问条目。目录的写权限通常还要结合 x 才能创建或删除目录项。 设置权限时还要结合 umask；特殊位 setuid、setgid、sticky 会改变普通权限位的解释。

```
mode_t requested = 0755; // owner/group/other: rwx、r-x、r-x
create_file(path, requested); // 创建时内核按 requested & ~process_umask 过滤
chmod(path, requested); // chmod 修改既有文件模式，不受 umask 再过滤
// 普通文件的 x 表示可执行；目录的 x 表示可搜索/穿越
// 目录中删除目录项通常需目录同时有 w+x，和目标文件权限不同
// setuid/setgid/sticky 是额外语义位；设置时仍需按安全策略检查
```

002 Linux 文件系统中的 inode 保存什么？

答案 inode 记录权限、所有者、大小、时间戳、链接计数及数据块定位等元信息，文件名保存在目录项中，因此可有多个名字指向同一 inode。硬链接共享 inode 和内容，删除一个名字只减少链接计数；符号链接有自己的 inode，内容是目标路径，目标可跨文件系统且可能失效。 inode 编号只在同一文件系统内有意义；文件系统卸载后，挂载点名称也会影响路径解析。

```
struct Inode { Permission mode; Owner uid; Size bytes; BlockMap data; int links; };
// 目录项保存 name -> inode；文件名不是 inode 的一部分
link("a", "b"); // 两个名字共享 inode 和内容
unlink("a"); // 链接计数减一；另一个名字仍可访问
// 最后一个目录链接删除后，若仍有打开句柄，数据也要等关闭后回收
symlink("missing-or-any-path", "shortcut"); // 符号链接有自己的 inode，可悬空
// inode 编号只在本文件系统内有意义；路径解析还受挂载点影响
```

003 Linux 进程有哪些常见状态？阻塞态何时出现？

答案 Linux 常见可运行/运行、可中断睡眠、不可中断睡眠、停止或跟踪以及僵尸状态；ps 中的字母是状态的简写。等待磁盘 I/O 等不可立即完成的条件时进程睡眠，普通信号通常不能唤醒 D 态任务。退出后僵尸只保留退出码等信息，父进程 wait 系列调用回收它。 可用 ps 的 STAT 字段初步判断状态，持续 D 态需结合内核栈和 I/O 设备排查。

```
state = inspect_with_ps_STAT(pid);
if (state == "R") schedule_or_running(); // 可运行/运行
if (state == "S") wait_interruptible_event(); // 可被普通信号打断的睡眠
if (state == "D") inspect_kernel_stack_and_io(); // 常见于不可中断 I/O 等待
if (state == "T") inspect_job_control_or_debugger();
if (state == "Z") parent.waitpid(pid); // 僵尸只留退出信息，等父进程回收
// 持续 D 态查设备/I/O；普通信号通常不能立即唤醒 D 态任务。
```

004 fork() 与 exec() 分别做什么?

答案 fork 创建逻辑上独立的子进程，父进程得到子 PID、子进程得到 0，失败返回 -1；通常先通过写时复制共享物理页。exec 系列在当前进程中装入新程序映像，PID 保持不变，成功不会返回，失败才返回错误。典型启动流程是 fork 后在子进程 exec、父进程 wait。 需要子进程退出码时父进程应 wait，长期不回收会积累僵尸条目。

```
pid = fork();
if (pid == 0) { // 子进程：逻辑独立，物理页通常先通过写时复制共享
    execve(program, argv, envp); // 成功后替换进程映像、PID 不变且不返回
    report_exec_error(errno); _exit(127); // 只有失败才走到这里
} else if (pid > 0) {
    waitpid(pid, &status, 0); // 回收子进程并读取退出状态，避免僵尸
} else { report_fork_error(errno); }
```

7.2 I/O 多路复用

005 select、poll、epoll 有什么区别?

答案 select 每次需传入并扫描 fd 集合，且常受 FD_SETSIZE 限制；poll 去掉固定位图限制，但仍要遍历监视列表。epoll 在内核保存兴趣集和就绪列表，等待时主要取回就绪项，适合大量长连接。连接少或多数连接都活跃时差异未必明显，ET 模式还要求非阻塞并读写到 EAGAIN。 例如连接较少或大部分都活跃时，扫描成本可能不再是主要瓶颈。

```
// select：每次重建 fd_set，并扫描受 FD_SETSIZE 限制的描述符。
select_wait_and_scan(all_fds);
// poll：没有固定 fd_set 位图上限，但等待返回仍要遍历监视列表。
poll_wait_and_scan(all_fds);
// epoll：内核维护兴趣集与就绪队列，wait 主要取回有事件的 fd。
auto ready = epoll_wait(epfd); for (auto event : ready) dispatch(event);
// 连接少/多数都活跃时差别可能不明显；先按实测瓶颈选择。
```

006 epoll 的 LT 与 ET 模式有什么区别?

答案 LT 是电平触发：只要 socket 仍可读或可写，后续等待仍会报告，程序漏处理后通常还会再收到通知。ET 侧重状态从未就绪变为就绪的边沿，应用要使用非阻塞 I/O 并循环处理到 EAGAIN，否则缓冲区残留数据可能没有新边沿。ET 减少重复通知的机会，但处理逻辑更严格，不保证吞吐必然更高。 LT 适合先保证正确性，压测确认通知开销确实明显后再考虑 ET。

```
// ET：注册前把 fd 设为非阻塞；就绪后循环读取直至 EAGAIN
for (;;) { auto n = ::read(fd, buffer, sizeof(buffer));
    if (n > 0) { consume(buffer, n); continue; }
    if (n == 0) { close(fd); break; } // EOF 后关闭并退出，不能继续读已关闭 fd
    if (errno == EINTR) continue; // 信号中断但无数据，继续尝试
    if (errno == EAGAIN || errno == EWOULDBLOCK) break; // 当前缓冲读空
    handle_error(errno); break;
} // ET 漏读到 EAGAIN 后不一定再有边沿；LT 通常会继续报告可读
```

7.3 内存、进程间通信与并发模型

007 什么是缺页异常?

答案 缺页异常发生在 CPU 访问虚拟地址而页表无法满足映射或权限要求时，它是处理器交给内核的同步异常。若是合法的按需分配、文件映射或换入页面，内核补齐页表后可重试原指令；越权或无效地址通常向进程报告 SIGSEGV/SIGBUS。少量缺页正常，持续主缺页和换页则可能说明内存压力或访问局部性差。内存驻留缺页常无需磁盘 I/O，文件换入的主缺页则延迟可能更高。

```
address = load_or_store(virtual_address);
if (!page_table_has_mapping(address) || !permission_allows(address, operation)) {
    trap_to_kernel_page_fault(address); // CPU 同步异常，交给内核判因
    if (is_valid_demand_page(address)) { map_or_load_page(); retry_instruction(); }
    else signal_process(SIGSEGV_or_SIGBUS); // 越权/无效地址不应盲目重试
}
// 匿名页分配、驻留页权限处理可无需磁盘；主缺页调入文件/交换页更慢。
// 少量缺页常见，持续主缺页要排查内存压力与访问局部性。
```

008 什么是虚拟内存？mmap 做了什么？

答案 虚拟内存让进程使用独立、连续的虚拟地址空间，页表再把页面映射到物理内存、交换区或文件，隔离和按需加载由此实现。mmap 建立文件或匿名映射，首次访问时可能才分配/调入页面；MAP_SHARED 可共享修改，MAP_PRIVATE 通常以写时复制隔离修改。映射不等于数据已读入 RAM，也需处理文件截断等边界。

```
void* p = mmap(nullptr, length, PROT_READ | PROT_WRITE, MAP_SHARED, fd, offset);
if (p == MAP_FAILED) return report_mapping_error(errno); // 先检查失败
// offset 需满足页对齐等平台要求；返回映射不代表页已驻留 RAM
write_bytes(p); // MAP_SHARED 与文件映射共享修改，落盘时机另行管理
void* q = mmap(..., MAP_PRIVATE, ...); // 写入通常触发写时复制隔离
// 文件截断后访问超出新 EOF 的页可能收到 SIGBUS
munmap(p, length); // 映射长度与释放范围要匹配
```

009 如何排查进程 CPU 使用率突然升高？

答案 先用 top/ps 确认进程 CPU，再查看线程级占用并区分用户态和系统态。Linux 可用 mpstat -P ALL 1 看各核的 usr/sys/iowait，再用 pidstat -u 1 找进程；JVM 可用 top -H -p PID 找高 CPU 线程，把线程十进制 TID 转成十六进制后与 jstack 的 nid 对照，也可用 Arthas thread 查看栈。随后用 perf top/record 或采样 profiler 取得热点，结合请求量、日志和最近变更判断忙循环、锁争用、系统调用或算法退化。优化后在相同负载下比较吞吐和延迟，避免只看瞬时百分比。

```
top_or_ps(pid); // 先确认进程 CPU，再找高占用线程并识别用户态/系统态
for (tid : hot_threads(pid)) { // ps -L / top -H 等工具得到线程 ID
    profile_stack(tid); // perf top/record 或采样 profiler 找热点栈
    map_tid_decimal_to_hex_for_logs(tid); // 日志/内核工具的 ID 进制可能不同
}
correlate_with_request_rate_logs_locks_syscalls_and_recent_changes();
optimize_confirmed_hot_path();
benchmark_same_load_and_compare_throughput_and_tail_latency(); // 不凭瞬时 CPU 百分比下结论
```

010 如何用 GDB 定位崩溃？

答案 core dump 是进程崩溃时保存的现场快照，可包含线程寄存器、栈、映射内存和信号等信息；具体内容受系统配置、权限和文件大小限制。确保 core 与匹配的可执行文件、共享库和调试符号对应，再执行 gdb ./程序 core 并查看 bt、线程列表及崩溃线程栈。GDB 按符号和展开信息重建调用帧；优化后参数或局部变量可能已被优化掉，不能假设所有变量都在栈中或可恢复。围绕故障指令检查参数、对象生命周期和相邻内存，必要时切换线程并用 frame、print 查看变量。ASan/UBSan 或最小复现可帮助确认越界、空指针、释放后使用；栈顶位置往往只是症状，不一定是根因。

```
enable_core_dump_and_reproduce();
gdb("./program", "core");
thread_apply_all("bt"); // 先看所有线程与崩溃线程栈，不把栈顶直接当根因
select_thread(crashing_thread); frame(fault_frame); print(arguments_and_nearby_state);
check_binary_matches_core_and_debug_symbols(); // 符号不匹配会误导栈和变量
rebuild_with_asan_ubsan_or_minimal_reproducer(); // 定位越界、空指针、释放后使用
```

011 常见的进程间通信方式有哪些？

答案 同机可用匿名/命名管道、消息队列、共享内存、信号量和信号；跨进程或跨主机也可用 Unix 域或 TCP/UDP socket。共享内存减少数据复制，需另配锁、原子量或进程间同步；管道和队列自带传递/排队语义但有容量限制。选型要看数据量、是否跨机器、背压与故障恢复要求。例如同机大块数据选共享内存，小型父子进程流式通信可优先考虑管道。

```
if (large_payload && same_host) { shared_memory(); add_process_shared_mutex_or_atomic(); }
else if (small_parent_child_stream) { pipe(); respect_pipe_capacity_and_backpressure(); }
else if (same_host && socket_protocol_needed) unix_domain_socket();
else { tcp_or_udp_socket(); } // 跨主机需要考虑网络故障与协议
// 消息队列适合带排队语义的离散消息；信号只传少量通知信息。
// IPC 选型同时看数据量、主机范围、同步/容量、背压和故障恢复。
```

012 进程、线程和协程有什么区别？

答案 进程有独立虚拟地址空间和资源边界，崩溃隔离较好但创建与切换开销通常较大；同进程线程共享堆和文件描述符，由内核调度，需要同步共享状态。协程由运行时在用户态按挂起点切换，适合大量等待 I/O 的任务，但阻塞调用可能卡住承载线程。协程能否并行取决于运行时是否把它们分派到多个内核线程。 例如大量网络等待适合轻量协程，独立隔离和不同故障域则更适合进程。

```
Process p = isolated_address_space_and_resources(); // 故障隔离较好，创建/切换成本通常较高
Thread t = shared_heap_and_file_descriptors(p); // 内核调度；共享状态需加锁/原子保护
Coroutine c = runtime_switches_at_await_points(); // 用户态轻量切换，适合大量 I/O 等待
if (coroutine_calls_blocking_io()) block_carrier_thread(); // 可能连带卡住其他协程
if (runtime_schedules_on_multiple_kernel_threads()) coroutine_work_can_run_in_parallel();
// 选型按隔离、共享、调度成本与阻塞行为，不按“协程一定并行”判断。
```

013 Linux 中 fork 后的 Copy-on-Write（写时复制）是怎样工作的?

答案 fork 后父子拥有逻辑独立的页表映射，但 Linux 通常先把只读映射指向同一物理页并标记为写时复制。任一方写页面时触发缺页，内核分配新页并复制内容，写入方改用副本；未被修改的页继续共享。这样 fork+exec 避免复制大量数据，但写入密集时仍会付出页复制和页表建立成本。大量写入会把节省下来的初始复制成本转化为按页发生的复制，因此要按写入比例评估收益。〔S5〕

```
fork(); // 父子逻辑上各自拥有地址空间，物理只读映射可暂时共享
mark_shared_pages_read_only_and_cow();
if (parent_or_child_writes(page)) {
    page_fault(); // 写入方触发内核异常
    private_copy = allocate_and_copy(page); remap_writer(private_copy);
}
// 未写页面继续共享；fork 后立即 exec 常省去大部分数据复制。
// 若大量页被写，仍要复制页面并建立页表，成本按写入比例发生。
```

7.4 I/O 完成模型

014 Reactor 和 Proactor 两种 I/O 模式有什么区别?

答案 Reactor 通知“某 fd 现在可读/可写”，应用收到后自行调用非阻塞 read/write；Proactor 则先提交读写请求，完成后通知应用并提供结果。Linux epoll 本身是就绪通知，因此常用作 Reactor；异步库可以封装出完成回调式接口。比较时看应用实际收到的是就绪还是操作完成，不要仅凭 API 的回调形式判断内核模型。〔S5，S6〕

```
// Reactor：就绪通知只说明“可以尝试”，应用仍执行非阻塞读写。
for (event : epoll_wait(epfd)) { if (event.readable) read_until_eagain(event.fd); }
// Proactor：先提交 I/O 请求，完成时通知，回调拿到结果/字节数。
submit_async_read(fd, buffer, on_completion_result);
// 判断模式看通知表示 ready 还是 operation completed，而不是看 API 有无 callback。
// Linux epoll 是就绪通知；异步库可在上层包装成完成回调式接口。
```

015 epoll 和 io_uring 的 I/O 模型有什么不同?

答案 epoll 注册 fd 并等待就绪事件，收到可读通知后仍由应用调用 read；它报告的是“现在可能做得了”，并非读取结果。io_uring 通过共享提交队列提交读写等请求、从完成队列取操作结果，可减少逐次系统调用和数据复制。可用操作、内核支持和安全限制随版本与文件类型而异，部署要检查目标内核并按实际负载基准测试。〔S5〕

```
epoll_add(fd, READABLE); // 内核记录兴趣；返回就绪后仍由应用调用 read
if (epoll_wait_reports(fd)) n = ::read(fd, buffer, size);
submit_io_uring_sqe(READ, fd, buffer, size); // 提交队列发布读写操作请求
cqe = wait_io_uring_cqe(); // 完成队列带回操作结果，而不只是“可能可读”
// io_uring 可减少逐次系统调用/复制，但受内核版本、文件类型和安全限制影响。
// 先核对目标环境支持情况，再按真实负载基准测试。
```

7.5 进程、执行与启动链路

016 一个用户态程序从 execve 到 main 经历哪些步骤?

答案 以 Linux 的动态 ELF 程序为例，shell 或其他进程先调用 execve；内核检查文件格式和权限，建立新的地址空间并映射可加载段，若 ELF 指定解释器则把控制权交给动态装载器。ELF 的 entry point 指向运行时入口或动态解释器，不是 main；运行时设置初始栈、环境与参数，C 运行时的 _start 再完成初始化并调用 main。装载器按依赖映射共享对象、处理重定位和初始化。静态链接、PIE、懒绑定和不同 ABI 会改变细节；“文件能执行”不等于所有依赖或运行时架构都兼容。〔N27、N25、N26〕

017 操作系统从固件到第一个用户进程如何启动?

答案 具体流程取决于平台；以常见 Linux 机器为例，固件完成基本硬件初始化并选择引导项，bootloader 读取内核、命令行和可能的 initramfs，按启动协议把控制权交给内核。内核建立页表、内存管理、驱动和调度器，挂载或切换到根文件系统，再启动 PID 1，由它继续挂载服务、建立设备和用户空间。每一步都要校验架构、启动介质、参数和根文件系统；引导成功不代表服务依赖已就绪。〔N28〕

7.6 内核边界、调度与同步

018 用户态与内核态如何切换，系统调用为什么不同于普通函数调用?

答案 用户态代码受权限约束，不能任意访问内核地址或直接操作受保护硬件；内核态执行调度、内存、文件和设备等受控服务。普通函数调用通常只在当前权限和地址空间内转移控制；系统调用通过架构规定的入口保存必要上下文、进入内核、检查参数权限，完成服务后返回。设备中断和同步异常也可能使 CPU 进入内核。调用 read 等库函数往往会触发系统调用，但库包装和内核入口不是一回事，有些接口可通过 vDSO 等机制完成。进入内核并不等于一定切换到另一个线程，系统调用开销与任务上下文切换开销要分开分析。〔N29〕

019 并发与并行怎样区分，进程和线程上下文切换主要保存什么？

答案 并发表示多个任务在一段时间内交错推进，并行表示它们在同一时刻执行；单核也能通过调度实现并发。切换执行任务时，内核须保存和恢复程序计数器、栈指针、寄存器及调度相关状态，具体布局依架构实现，部分信息保存在任务结构或内核栈中。不同地址空间的任务切换还涉及页表上下文及地址转换缓存管理；同进程线程可共享地址空间，通常省去这部分成本，但线程仍有独立栈和寄存器状态。TLB 标记、缓存局部性、CPU 迁移和实际工作集都会影响开销，不能断言任意线程切换总比任意进程切换快，也不需要复制整个堆来完成切换。〔N30、N31、N32〕

020 多线程程序调用 fork 后，为什么子进程容易卡在锁上？

答案 fork 后子进程只保留调用 fork 的那个线程，地址空间中的其他线程却不会一同存在；互斥量等对象的状态仍可能来自父进程快照。若某把锁原由已消失的线程持有，子进程再获取它可能永远等待。POSIX 因而限制多线程 fork 后、exec 前的子进程路径，只调用异步信号安全的函数；printf、复杂分配和一般库调用不能随意使用。pthread_atfork 可协调已知锁，但不容易覆盖第三方库的全部内部状态。启动新程序时宜保持子路径极小，或在符合需求时使用 posix_spawn 等接口，不能指望父进程后来解锁会自动修复子进程的锁副本。〔N33〕

021 信号、信号量和互斥量分别表达什么，为什么不能混用？

答案 信号是向进程或线程报告事件的异步通知机制，例如终止请求或子进程状态变化；处理函数运行上下文受限，只能使用允许的异步信号安全操作，普通信号也不保证按次数排队。信号量维护可用资源计数，等待操作扣减、发布操作增加，适合多个槽位或生产消费协作；它通常没有互斥量那样的所有者约束。互斥量用于保护临界区及共享不变量，通常应由成功加锁的线程释放。把信号量设为 1 可限制并发，却不意味着拥有互斥量的全部语义；选择时还须明确是线程内同步、跨进程同步，还是仅通知一个事件。〔N34、N35〕

022 常见 CPU 调度策略怎样取舍，Linux 是否一直使用同一种公平调度器？

答案 先来先服务容易受长任务阻塞，短作业优先可降低平均等待但需要估计任务长度；时间片轮转改善交互响应，优先级和多级反馈队列可区别紧急程度与运行行为。任何策略都要兼顾吞吐、响应、公平和切换成本，优先级过强可能使低优先任务长期得不到 CPU。Linux 暴露普通、FIFO、RR、Deadline 等不同调度类别，内核实现也持续演变：公平调度曾长期以 CFS 为核心，较新内核逐步转向 EEVDF，不能把某个版本的红黑树细节说成永久规则。实时策略还需限制运行预算并检查权限，避免普通任务被饿死。〔N31、N36〕

023 自旋、阻塞、悲观和乐观并发控制应怎样选择？

答案 自旋锁竞争时持续检查状态，省去睡眠唤醒但占用 CPU，适合预期很短且持有者能继续运行的临界区；若持锁者被调度走或线程过量，自旋会浪费算力。阻塞锁让等待者睡眠，适合较长等待，代价是调度开销。悲观控制先取得互斥或读写权限，再检查修改状态；乐观控制先计算，提交时用版本或 CAS 检查冲突并决定重试。乐观并不等于“完全没有同步”，冲突频繁或副作用不能撤销时可能更差。读写锁、递归锁等也有各自边界；Linux 内核普通自旋锁在 PREEMPT_RT 下的实现语义还会变化，须区分用户态接口与内核锁。〔N37、N38〕

024 活锁与饥饿为什么可能在没有死锁时出现？

答案 死锁中的参与者相互等待，整体无法继续；活锁中的参与者仍执行重试、回退或让步，却始终没有完成有效工作；饥饿则是某个任务长期得不到 CPU、锁或其他资源，其他任务仍能前进。例如两个线程按同样节奏不断释放并重取两把锁，可能形成活锁；持续优先服务高优先任务可能使低优先任务饥饿。治理方式不同：死锁可通过统一锁序或避免持有等待处理，活锁可引入有界且带随机性的退避和明确仲裁，饥饿可考虑公平排队、优先级老化或资源配额。仅证明没有锁环不能保证每个任务最终完成。〔N31、N38〕

025 银行家算法怎样判断资源分配安全，不安全是否等于已经死锁？

答案 银行家算法假定每个任务预先声明最大资源需求，并维护 Available、Allocation 和 Need=Max−Allocation。收到申请时先检查它不超过剩余需求和当前可用资源，再试分配，搜索是否存在一个顺序，使每个任务都能用当时可用资源满足剩余需求、完成并释放已有分配。能覆盖所有任务即安全，可正式批准；否则回滚并等待。安全性检查是在保证最坏声明需求下仍可完成，不安全只表示无法给出这种保证，并非此刻一定已经死锁。它依赖事先已知需求和可回收资源，因此更适合作为资源分配模型，不能当作任意业务系统可直接套用的通用锁方案。〔N39〕

```
work = Available.copy()
finished = set()
while 存在 i 不在 finished 且 Need[i] 各维均 <= work:
    work += Allocation[i]  # 只模拟释放已有分配，不修改真实资源
    finished.add(i)
safe = len(finished) == 任务数  # 试分配后不安全就回滚并等待
```

7.7 地址转换与内存回收

026 页表、多级页表、分段和 TLB 怎样参与地址转换?

答案 分页把虚拟地址拆成虚页号与页内偏移，页表记录虚页到物理页框的映射及权限，物理地址由页框和偏移组成。多级页表按虚页号的不同位逐层索引，稀疏地址空间可不建立未使用分支，减少单级大表浪费；代价是未缓存的转换可能需要多次访存。TLB 缓存近期转换，未命中时可由硬件或软件查页表，不代表页面一定不在内存，也不必然触发缺页。固定大小分页便于分配并避免外部碎片，但最后一页或分配粒度可能产生内部碎片；分段按代码、数据等逻辑段描述基址、界限和权限，大小可变且可能产生外部碎片。有些系统把段再分页以组合两者。具体页级数、页大小与转换机制依架构配置。〔N32〕

027 内存紧张时怎样回收页面，页面置换与 OOM 有什么关系?

答案 系统可先回收可重建的文件缓存，对脏页安排回写，符合条件的匿名页也可换出到交换空间；回收不足时，分配可能阻塞、失败，或在特定内存约束下触发 OOM 处理。是否有空闲内存不能只看 free 数字，还要看可回收内存、工作集和 cgroup 限制。经典置换算法包括 FIFO、LRU、Clock、LFU 和理想化的 OPT：LFU 按历史访问频次保留页面，可能长期留住过去的热点而忽略近期工作集变化；OPT 依赖未来访问，主要作理论比较。实际内核常用近似及多代管理，不能说每次都按精确 LRU。工作集超出可用内存可造成频繁换页和抖动，应从减少保留数据、限制并发或扩容等方面处理，避免反复重启掩盖根因。〔N40〕

7.8 文件系统、缓存与持久化

028 文件描述符与 inode 的关系是什么，删除文件为何可能不释放空间?

答案 文件描述符是进程内的整数索引，指向内核的打开文件描述；打开文件描述保存偏移、状态标志等，并关联 inode 等文件系统对象。多个 fd 可经 dup 或 fork 共享同一个打开文件描述，也可对同一 inode 独立 open 而拥有不同偏移。unlink 删除目录项并减少链接计数，不会强制使已经打开的 fd 失效；只有没有硬链接且不再被打开等引用持有时，文件数据才具备最终回收条件。因此日志虽已删除，进程继续持有它仍可能占空间。排查可检查 /proc/<pid>/fd 或 lsof 的已删除文件，先让应用正确关闭或轮转，不能仅重复 rm。〔N41、N42〕

029 Page Cache 如何支持 read 与 mmap，mmap 为什么不保证一定更快?

答案 Page Cache 缓存文件页，普通缓冲文件读取通常先查缓存，未命中再从存储装入，然后把数据复制到用户缓冲区。mmap 建立文件页到进程虚拟地址的映射，访问时可能触发缺页并取得相应缓存页，减少显式 read 和部分复制；但仍有页表管理、缺页、同步和存储访问成本。顺序扫描、随机访问、工作集与并发方式不同，谁更快应实测，不能把 mmap 等同于把整个文件预读到物理内存。共享写映射要按协议同步，文件被截断还可能导致 SIGBUS；直接 I/O 等路径也未必遵循普通 Page Cache 读取模式。〔N43、N44〕

030 write 成功与 fsync 成功的持久化保证有何差别?

答案 普通缓冲 I/O 的 write 返回成功通常表示对应字节已被内核接收，不代表数据已进入可承受掉电的存储；还须正确处理短写和稍后报告的错误。fsync 请求同步文件数据及必要元信息，fdatasync 可减少与取回数据无关的元信息同步。若采用“写临时文件再 rename 替换”的发布流程，要分别处理文件内容及目录项持久化，通常还需同步包含目录。具体保证受文件系统、挂载方式、设备缓存与硬件诚实性影响；数据库的提交还要结合日志刷盘策略。不能把用户态 fflush、内核写缓存、fsync 和远端业务确认当作同一个完成点。〔N45〕

7.9 中断与 I/O 语义

031 硬件中断、同步异常、软中断和工作队列各适合承担什么工作?

答案 硬件中断让设备报告异步事件；普通外部 IRQ 通常可由内核屏蔽，NMI 不受普通屏蔽控制，常用于机器检查或 watchdog 等紧急观察，不意味着用户进程必然崩溃。处理器按架构保存必要状态并进入相应入口，内核确认来源、完成紧急处理，再返回或安排后续工作；系统调用和缺页等同步异常也能进入内核，但并非外设 IRQ。Linux 硬中断上下文通常应短小，不能像普通线程一样任意睡眠；延后处理可使用软中断等机制，软中断即使由 ksoftirqd 承载也不能直接按普通可睡眠工作编写。普通工作队列由工作线程执行，适合可能阻塞的延后任务。线程化中断及 PREEMPT_RT 会改变部分执行细节，所以“上半部/下半部”的概念需与具体内核配置对应。〔N46、N47、N38〕

032 阻塞、非阻塞、同步与异步 I/O 怎样区分，就绪后为什么仍要检查返回值?

答案 阻塞与非阻塞关注调用在无法立刻完成时是否等待：非阻塞 read 可返回 EAGAIN，而阻塞调用可能等待数据。同步与异步关注操作如何完成及交付结果。经典 I/O 模型通常包括阻塞、非阻塞、I/O 多路复用、信号驱动和异步 I/O；前四种最终仍由应用发起 read/recv 取走数据，信号驱动只是先通知就绪，完成通知才表示已提交的异步操作已经取得结果。epoll 报告可读后，数据可能已被其他执行者读走，也可能对应 EOF 或错误，所以事件循环应使用非阻塞 fd，逐次处理实际返回值；read 为 0 表示流读取结束，EINTR、EAGAIN 和真实错误须区别处理。就绪只提供进行操作的时机，不保证一次调用正好得到完整消息。〔N48、N49〕

第 8 章 计算机网络

8.1 HTTP 与安全连接

001 HTTP GET 与 POST 有什么区别?

答案 GET 的核心语义是请求目标资源的表示，通常应安全且幂等，因此适合查询；POST 请求目标资源按其语义处理所附内容，常用于创建或触发动作。把参数放 URL 是常见做法，不是判定方法或安全性的标准；URL 可能进入日志和历史记录。是否缓存取决于方法、响应头和缓存规则，POST 也不是天然不可缓存。 安全方法也应避免服务端把 GET 用作不可重复的资金扣减等写操作。

```
request = HTTP_GET(resource_uri); // 获取表示；语义应安全且幂等，适合查询
request = HTTP_POST(collection_uri, payload); // 按目标资源语义处理内容，常用于创建/动作
// 不能仅凭“参数在 URL 还是 body”判定方法语义或安全性。
// GET 写操作若扣款，重试/预取会造成副作用，设计本身就不安全。
response_cache = cache_by_method_and_headers(request, response); // 按规则决定
// URL 可能进入浏览器历史和日志；敏感数据需正确认证/授权与传输保护。
```

002 HTTP/1.1 常见状态码如何分类?

答案 1xx 是临时信息响应，2xx 表示请求被成功接收、理解或处理，如 200、201、204；3xx 要求重定向或使用缓存结果。301 表示永久移动，302 表示临时移动；历史客户端对 POST 的处理可能转成 GET，若需保留方法和请求体应考虑 307/308。4xx 表示请求端问题，如 400、401、403、404、429；5xx 表示服务器处理失败或网关异常，如 500、502、503、504。502 表示网关/代理收到上游无效响应，504 表示等待上游超时。重试要看状态语义，429/503 可结合 Retry-After，普通 4xx 通常不应盲目重试。

```
switch (response.status) {
case 100: case 103: interim_information_only(); break; // 1xx 不是最终结果
case 200: success_with_representation(); break; case 201: resource_created(); break;
case 204: success_without_body(); break; // 2xx 成功类
case 301: follow_redirect_or_cache_by_rules(); break; // 3xx
case 401: authenticate_request(); break; case 403: deny_authorized_identity(); break;
case 404: resource_absent(); break; case 429: wait_retry_after_with_backoff(); break;
case 503: retry_if_safe_and_retry_after_allows(); break; // 5xx
}
// 4xx 通常先修正请求，不盲目重试；仍须考虑幂等性和截止时间
```

003 HTTP/HTTPS、TLS 和证书分别解决什么问题?

答案 HTTPS 是 HTTP 在 TLS 安全通道上的使用方式；TLS 握手协商算法与密钥并认证服务端，记录层保护后续数据机密性和完整性。证书含公钥及身份信息，由证书链连接到客户端信任的 CA，客户端还要检查域名、有效期和用途。握手常用非对称密码认证/协商，再用对称密钥高效保护数据；TLS 本身不保证应用权限正确。

```
client = connect_tls(server_name);
cert = peer_certificate_chain();
verify_chain_to_trusted_CA(cert); verify_hostname(cert, server_name);
verify_expiry_and_usage(cert); // 证书提供公钥/身份链，客户端仍须检查用途和域名
keys = tls_handshake_negotiate_and_authenticate(cert); // 非对称机制用于认证/协商
secure_channel = derive_symmetric_keys(keys); // 后续记录层保护机密性与完整性
send_http_over(secure_channel); // TLS 不替代应用自身的权限检查
```

004 TCP Keep-Alive 与应用层心跳有什么区别?

答案 TCP Keep-Alive 是操作系统针对空闲 TCP 连接发送探测报文，按系统参数等待并重试，以发现已失联的对端；默认探测间隔可能很长，各平台可配置。应用心跳由协议主动发送，能检测应用进程卡死、会话过期或业务层失效，周期可按 SLA 设定。二者都只提供存活线索，调用仍须有截止时间、退避重试及幂等保护。

```
tcp_keepalive_enable(socket); // 空闲探测由 OS 参数控制，默认间隔可能很长
schedule_application_heartbeat(session, sla_period); // 检查进程卡死/会话失效
if (heartbeat_deadline_missed()) mark_suspect_then_reconnect_with_backoff();
set_request_deadline(deadline); // 连接探测成功不代表某次业务调用会完成
retry_only_if_operation_is_idempotent_or_has_request_id();
// 代理、线程阻塞和网络分区都可能影响探测；两者只是存活线索
// 应按 SLA 配探测间隔/重试，而不是依赖操作系统默认 keepalive
```

8.2 TCP、UDP 与传输控制

005 TCP 三次握手和四次挥手如何进行？为什么需要 TIME_WAIT?

答案 客户端发送 SYN，服务端以 SYN+ACK 确认并提供自己的初始序号，客户端 ACK 后连接建立，双方确认双向通信并同步序号。关闭时两个发送方向可独立终止，所以常见 FIN、ACK、FIN、ACK 四段。主动关闭方进入 TIME_WAIT 一段时间，以便重传最后 ACK 并让旧连接报文从网络消退；它有助于端口复用安全，不是单纯浪费连接。

```
client.send(SYN, seq=c); server.reply(SYN | ACK, ack=c+1, seq=s);
client.send(ACK, ack=s+1); // 确认双方初始序号与双向可达后建立连接
// 关闭方向独立：FIN -> ACK -> FIN -> ACK，常见四段挥手
active_closer.send(FIN); peer.ack_then_later_send(FIN); active_closer.send(ACK);
active_closer.enter(TIME_WAIT); // 主动关闭方等待，必要时重传最后 ACK
wait_2MSL_then_release_tuple(); // 让旧报文消退，避免新连接误收旧段
// TIME_WAIT 占用有成本，但也保护端口复用和连接正确性
```

006 TCP 与 UDP 有什么区别?

答案 TCP 提供连接内有序、可靠的字节流，并实现重传、流量控制与拥塞控制；应用必须自行定义消息帧。UDP 保留数据报边界，但协议不保证交付、顺序或去重，也没有内建拥塞控制。DNS 查询常用 UDP 并按需转 TCP，实时媒体可容忍部分丢包；选择应看时延、可靠性及应用层能否处理丢包。

```
tcp.send(bytes); // TCP 不保留 send/recv 边界，提供有序可靠字节流
receive_with_length_prefix_delimiter_or_fixed_frame();
// TCP 用 ACK/重传、流量控制和拥塞控制；可靠性会带来重传/队头等待。
udp.send_datagram(bytes); // 保留数据报边界，但协议不承诺交付/顺序/去重
if (dns_query || loss_tolerant_realtime_media) choose_udp_when_app_semantics_fit();
// DNS 需要时也会转 TCP；UDP 应用需自行考虑丢包与拥塞，不能假设可靠。
```

007 TCP 如何实现可靠传输?

答案 TCP 用序列号标记字节位置、ACK 确认已收到的数据，校验和检测传输损坏；缺失数据会由重传定时器或重复 ACK 触发重传。接收窗口限制发送方未被接收端接受的数据量，拥塞控制再根据网络状况调速。它交付有序字节流而非报文边界，所以一次 send 对应几次 recv 没有保证。 连接建立后的 MSS、接收缓存和网络路径变化也会影响可用吞吐。

```
#include <algorithm>
segment.seq = first_byte_sequence; segment.checksum = checksum(header, payload);
receiver.accept_in_order_bytes(); receiver.send_cumulative_ack(next_expected_seq);
if (timer_expired || duplicate_acks_indicate_gap) retransmit_missing_range();
limit_in_flight_by(std::min(receiver_window_rwnd, congestion_window_cwnd));
// 序列号/累计 ACK 支持有序交付；校验和发现传输损坏
// rwnd 保护接收缓存，cwnd 保护网络路径；二者较小者限制在途字节
// TCP 向应用交付字节流，无消息边界；MSS、缓存和路径也影响吞吐
```

008 TCP 流量控制与拥塞控制有什么区别?

答案 流量控制保护接收端：接收方通告窗口 rwnd，发送方不能超过接收缓冲区可容纳的数据。拥塞控制保护路径中的网络：发送方根据丢包、时延或 ECN 等信号调整拥塞窗口 cwnd。实际在途数据通常受 rwnd 与 cwnd 中较小者约束；前者慢可能是消费者读取不及时，后者小则可能是链路拥塞。接收窗口降到零时，发送方会按协议探测窗口何时重新开放。

```
#include <algorithm>
rwnd = receiver_advertised_free_buffer(); // 接收端流控，保护消费/缓存能力
cwnd = sender_congestion_controller_estimate(); // 网络拥塞控制，保护传输路径
in_flight_limit = std::min(rwnd, cwnd);
if (rwnd == 0) periodically_probe_window_update(); // 对端读出数据后可重新开放
if (cwnd_is_small_or_loss_detected()) reduce_send_rate_and_recover();
// rwnd 小先查应用读取/缓冲；cwnd 小查丢包、时延、ECN 等路径反馈
// 窗口探测不是立即发送任意数据，发送方仍须遵守协议窗口规则
```

009 TCP 滑动窗口是什么？窗口会向左移动吗?

答案 发送窗口表示可发送但尚未确认的序列号区间，接收窗口反映接收端可接收空间，二者共同限制在途字节。ACK 确认推进左边界，窗口通告决定右边界，因此可见窗口宽度收缩或停滞；序列号环绕和旧数据重传需依 TCP 规则处理。调试时区分“窗口整体位置”与“窗口大小”，不能把窗口缩小误解为已发送数据可被撤销。 ACK 可能累计确认多个连续字节，窗口更新也可独立于新数据到达。

```
send_window = [oldest_unacked_seq, right_edge_allowed_by_peer_and_cwnd);
if (ack_covers_contiguous_bytes) move_left_edge_forward(ack.next_expected_seq);
if (peer_updates_receive_window) recompute_right_edge(peer.rwnd);
// ACK 可累计确认多个连续字节，窗口更新也可不伴随新数据。
if (visible_width_shrinks) do_not_retract_already_sent_bytes();
// 区分窗口位置和窗口宽度；序列号环绕/重传依 TCP 规则处理。
```

010 什么是 MTU？IP 包超过路径 MTU 会怎样？

答案 MTU 是一段链路能承载的最大网络层包尺寸；路径 MTU 由端到端路径中最小的链路 MTU 限制。IPv4 在允许时可由路由器分片，设置 DF 后则需要 ICMP 错误协助发现；IPv6 路由器不分片，源端依据 ICMPv6 调整。TCP MSS 约束单段 TCP 负载，需扣除 IP/TCP 头部，路径配置异常会造成黑洞和重传。

```
path_mtu = minimum_link_mtu_on_route(); // 端到端路径上的最小链路 MTU
mss = path_mtu - ip_header_bytes - tcp_header_bytes; // TCP 负载要扣除头部
if (ipv4 && fragmentation_allowed) router_may_fragment_packet();
if (ipv4_df || ipv6) source_uses_icmp_feedback_to_adjust_packet_size();
// IPv6 路由器不分片，需由源端依据 ICMPv6 做 PMTU 发现
if (icmp_blocked_or_misconfigured()) detect_path_mtu_black_hole_and_retransmissions();
// TCP MSS 影响分段负载；隧道封装等还会进一步减小可用 MTU
```

011 什么是 TCP 粘包/拆包？如何解决？

答案 TCP 只提供连续字节，不保留发送调用的分界；短时间内多次 send 可能被一次 recv 读到，单次 send 也可能分多次到达。协议应定义消息边界：固定长度、分隔符、长度前缀或自描述编码都可行。解析器要缓存半包、循环处理多帧，并限制声明长度、校验格式，防止恶意超长帧耗尽内存。常见长度前缀帧还需限制最大帧大小，并支持一次缓冲中解析多帧。

```
buffer.append(socket.read_available()); // TCP 是字节流，一次 send 不对应一次 recv
while (buffer.size() >= header_bytes) {
    length = decode_big_endian_length(buffer);
    if (length > MAX_FRAME || length > SIZE_MAX - header_bytes) close_as_protocol_error();
    if (buffer.size() < header_bytes + length) break; // 半包留待后续读取
    parse_and_validate_frame(buffer.subspan(header_bytes, length));
    buffer.erase_prefix(header_bytes + length); // 同一缓冲可能含多帧
}
// 定长、分隔符或长度前缀都能定界；校验格式并限制长度防内存耗尽
// 解析循环只消费完整帧，不能把残留半帧误判为错误
```

8.3 名称解析与服务端网络

012 浏览器输入 URL 到页面显示大致经历什么？

答案 浏览器先解析 URL 和策略，可能命中缓存或 Service Worker；未命中时通过系统解析器获取地址，并建立 TCP/TLS 或 QUIC 连接。随后发送 HTTP 请求并接收响应，解析 HTML 构建 DOM、解析 CSS 构建样式信息，再计算布局、绘制合成，同时执行脚本并请求图片等子资源。DNS、连接和资源缓存、重定向、预加载都会改变顺序与等待时间。

```
url = parse_and_normalize(input_url); if (service_worker_or_cache_hit(url)) render_cached();
ip = resolve_dns_via_browser_os_or_recursive_cache(url.host); // 未命中才向上查询
connection = connect_tcp_tls_or_quic(ip, url); response = send_http(connection, url);
dom = parse_html(response); styles = parse_css_and_compute_style(dom);
layout(dom, styles); paint_and_composite();
// 脚本可能在过程中执行并请求子资源；缓存、重定向和预加载会改变顺序。
// DNS/连接/资源耗时分别影响等待，不是单一固定串行流水线。
```

013 Linux listen(backlog) 参数控制什么？SYN 队列和 accept 队列有什么区别？

答案 Linux 上 listen(fd, backlog) 的 backlog 控制握手完成、等待 accept 的连接队列容量；尚未完成握手的 SYN 请求属于另一队列，受 tcp_max_syn_backlog 等机制影响。请求值会受系统 somaxconn 上限约束，队列满时客户端可能重试或失败。排查连接积压还要看应用 accept 速度、半连接压力和内核参数，其他操作系统语义可能不同。〔S5〕

```
#include <algorithm>
listen(fd, requested_backlog); // Linux 主要限制已完成握手、等待 accept 的队列
syn_queue_capacity = tcp_max_syn_backlog; // 半连接请求属于另一队列/机制
actual_accept_limit = std::min(requested_backlog, somaxconn);
while (accept_queue_growing()) measure_application_accept_rate_and_worker_latency();
// 队列满时客户端可能重试或失败；检查半连接压力、内核参数和 accept 速度
// tcp_max_syn_backlog/somaxconn 的实际行为依 Linux 版本和配置验证
// backlog 不能代替及时 accept、连接超时和容量规划
```

014 TCP 的 Nagle 算法和 Delayed ACK 分别做什么？为什么会增加请求响应延迟？

答案 Nagle 算法减少小 TCP 段：已有未确认数据时，新的小写入通常先暂存，等 ACK 或攒到足够数据再发。Delayed ACK 则让接收端短暂等待，尝试合并 ACK 或与反向数据一起确认；两者组合可能让小请求和小响应多等一个确认周期。交互协议可测试 TCP_NODELAY，但批量发送、拥塞和包开销也应一起衡量。〔S7〕

```
write_small_request(); // 有未确认数据时，Nagle 可能暂存后续小段
receiver_may_delay_ack_briefly(); // 接收端尝试合并 ACK 或搭载反向数据
// 两者叠加时，小请求/响应可能多等一个确认周期
if (interactive_latency_is_measured_bottleneck) setsockopt(TCP_NODELAY, true);
// 关闭 Nagle 可降低等待，但会增加小包数量和网络开销
// Delayed ACK 时机依实现/流量；测量真实 RTT 和请求大小再调整
// 批量吞吐、拥塞和服务器负载也应一起比较
```

015 HTTP/2 和 HTTP/3 在传输层和多路复用上有什么区别？

答案 HTTP/2 将多个请求流复用到一条 TCP 连接并使用 HPACK 压缩头部，但 TCP 丢包时，后续字节即使属于其他流也要等待重传数据。HTTP/3 将 HTTP 帧映射到 QUIC 流，QUIC 在 UDP 上实现加密、可靠传输和按流复用；一个流丢包通常不阻塞其他流的数据交付。共享拥塞控制、连接层丢包仍会影响整体吞吐，因此不是消除所有队头阻塞。〔S7〕

```
http2.multiplex_streams_over_one_tcp_connection(); http2.compress_headers_with_HPACK();
if (tcp_segment_lost) later_tcp_bytes_wait_for_retransmission(); // 跨流字节队头阻塞
http3.map_http_frames_to_quic_streams_over_udp();
if (quic_stream_A_lost) stream_B_can_deliver_independent_bytes();
// QUIC 仍有共享拥塞控制和连接层丢包影响，不是消除所有阻塞。
// HTTP/3 通常还整合 TLS 加密；性能依网络与实现。
```

016 一次典型 DNS 解析会经过哪些角色？

答案 客户端通常先查浏览器、系统或本地缓存，再向配置的递归解析器发起查询；缓存命中即可返回。未命中时递归解析器沿 DNS 委派关系查询根、顶级域及权威服务器，可能从缓存或转发器获得答案，并按 TTL 缓存。DoH/DoT 改变客户端到解析器的传输方式，但不会改变权威数据层级；实际请求未必逐级访问所有服务器。〔S8〕

```
if (browser_cache.contains(name)) return browser_cache[name];
if (os_or_recursive_resolver_cache.contains(name)) return cached_answer;
answer = recursive_resolver.query_root_then_tld_then_authoritative(name);
recursive_resolver.cache(answer, answer.ttl); // 未命中才按委派关系逐层查询
// 实际递归器可能命中转发器/缓存，不一定访问每一级权威服务器。
// DoH/DoT 改变客户端到解析器的传输保护，不改变 DNS 权威层级。
```

017 SO_REUSEADDR 和 SO_REUSEPORT 有什么区别？

答案 在 Linux 上，SO_REUSEADDR 主要影响地址复用和重启后绑定等场景，不能简单视作多个进程共同监听同一端口的开关。SO_REUSEPORT 允许满足条件的多个 socket 在 bind 前启用该选项并绑定相同地址端口，内核可把连接分散给监听者；权限及地址匹配有具体规则。Windows、BSD 与 Linux 的行为存在差别，依赖该选项时应查目标平台文档并实测。〔S5〕

```
set_option(SO_REUSEADDR); // Linux 常用于重启绑定等规则，不代表任意多进程共享监听
if (multiple_listeners && each_socket_sets_before_bind(SO_REUSEPORT)) {
    bind_same_eligible_address_and_port(); kernel_distributes_connections();
}
// 需满足权限、地址匹配等条件；不同 Linux/BSD/Windows 语义有差异。
verify_target_os_docs_and_test(); // 不把同名选项当跨平台等价保证
```

8.4 协议分层与应用交互

018 OSI 与 TCP/IP 如何对应，socket 和常用协议处于什么位置？

答案 OSI 将通信分为物理、链路、网络、传输、会话、表示、应用七层；常用 TCP/IP 模型合并为链路、网际、传输和应用四层。发送端可理解为逐层封装：HTTP 等应用数据交给 TCP/UDP 加端口信息，再由 IP 加源/目的地址，最后由链路层封装为面向下一跳的 MAC 帧和比特流；接收端逐层解封装。MAC 地址用于本地链路转发，经过路由器时链路帧会更新，不能当作端到端地址。socket 是操作系统提供的通信接口，不能与某个应用协议画等号。Nginx 作为 HTTP 反向代理处理应用层消息，stream 模块也能代理传输层连接。修改端口不会改变协议本质。〔N50、N51〕

019 HTTP/1.1 长连接如何区分报文，并在什么情况下关闭？

答案 HTTP/1.1 请求通常由请求行、首部、空行和可选消息体组成；响应以状态行开始。长连接让多个请求复用连接，本身不提供消息边界。接收方按方法、状态和消息长度规则解析：HEAD 响应等无消息体，其他情况依据有效的 Transfer-Encoding、Content-Length 或关闭连接等条件定界，不能简单按一次 recv 判断。含冲突长度字段的报文应严格拒绝或按规范处理，避免前后端解析不一致。持久连接也会因 Connection: close、空闲超时、错误或进程退出而结束。HTTP 复用连接与 TCP 的空闲探测用途不同。〔N52〕

020 HEAD、PUT、DELETE、PATCH 与 OPTIONS 分别表达什么操作?

答案 HEAD 与 GET 的响应语义相近，但不返回响应消息体，适合获取资源元信息。PUT 通常要求用提交的表示创建或替换指定资源；DELETE 请求解除目标资源与其当前功能的关联，二者按 HTTP 语义均幂等，但幂等不代表每次返回码相同。PATCH 表达部分修改，是否幂等取决于补丁格式和业务约定。OPTIONS 查询目标支持的通信选项，也用于某些 CORS 预检。方法名称不能替代权限校验或自动保证实现正确；服务端仍须约束副作用、验证输入，并为重试设计一致的业务语义。〔N51〕

021 HTTP 断点续传如何使用 Range，并保证拼接的是同一份资源?

答案 客户端保存已经接收的字节范围，再发送 Range: bytes=起点- 请求剩余内容；支持范围请求的服务端可返回 206，并用 Content-Range 说明该片段在完整表示中的位置。超出有效范围可能返回 416，服务端也可以忽略 Range 而返回完整的 200 响应，因此客户端必须检查状态和头部。若下载期间资源变化，直接拼接会损坏文件，可配合 If-Range 和强 ETag 等验证器，仅在表示未变化时续传，否则重新获取。字节范围针对选定表示，压缩编码或版本变化也要纳入校验；断点续传并非 TCP 自动记住上次下载位置。〔N51〕

022 DNS 通常用什么端口和传输协议，何时会使用 TCP?

答案 传统 DNS 查询使用端口 53；多数普通查询以 UDP 为起点，解析器和权威服务器也必须支持 TCP。区域传送通常用 TCP；UDP 响应被截断（TC 置位）、超出可安全传送的大小或其他实现策略要求时，解析器可改用 TCP 重试。EDNS 扩展允许协商更大的 UDP 响应，但大报文会遇到 IP 分片丢失和路径 MTU 问题，所以不能简单说 DNS 只用 UDP 或只有超过 512 字节才用 TCP。DoT、DoH 及 DoQ 也使用各自的安全传输端口或封装；题目若只问传统 DNS，答案应注明 53 端口的语境。〔N56〕

023 已经使用 HTTP，为什么系统还会引入 RPC 框架?

答案 HTTP 规定请求响应等通信语义，RPC 规定如何把远端能力表达成服务、方法和参数；两者不在同一个抽象层，也可以组合，例如 gRPC 通常通过 HTTP/2 承载调用。RPC 框架往往配套接口描述、代码生成、序列化、流式调用、截止时间、状态传播和拦截器，减少团队重复实现通信约定。选择应比较跨语言契约、调试方式、浏览器兼容、性能和服务治理需求，而不是认为 HTTP 一定慢或 RPC 必然使用某种传输。远程调用仍可能超时、部分失败或重复执行，不能因调用外观像本地函数就忽略失败语义。〔N62〕

024 HTTP 长连接与 WebSocket 的连接复用和双向通信有何差别?

答案 HTTP 长连接只是让多次 HTTP 交互复用底层连接，消息仍遵循对应版本的请求响应或流语义；并不自动变成可任意推送的自定义双向通道。经典 WebSocket 通过 HTTP/1.1 握手升级，随后以独立帧协议在连接上双向传输消息，可携带文本、二进制、控制帧并维护消息边界，适合聊天和实时协作。WebSocket 不会自动提供业务级 ACK、离线消息、重连补发或权限续期，应用仍须设计这些行为；代理空闲超时和慢客户端背压也要处理。选择时还可考虑 SSE 等单向推送方式。〔N63〕

025 正向代理与反向代理分别代表谁，TLS 终止后需要注意什么?

答案 正向代理通常由客户端选择，代客户端访问外部资源，可承担出口控制、缓存或隧道转发；反向代理位于服务入口，代表一组后端接收请求，可承担路由、负载均衡、缓存和 TLS 终止。二者区别在部署角色和通信对象，不由是否修改报文或使用哪一端口决定。若代理终止 TLS，客户端到代理与代理到源站就是两段独立连接，后段是否加密及如何认证须另行配置。后端获取原始地址、协议等信息时，只可信任受控代理写入的转发头，并清理客户端伪造值；代理头不是天然可信的身份凭证。〔N51〕

026 CDN 怎样加速内容访问，缓存键和回源有哪些边界?

答案 CDN 把请求导向较合适的边缘节点，节点命中可复用的缓存表示时直接响应，降低源站负载和长距离传输；未命中则按规则回源并决定是否缓存。常见调度从 DNS 开始：权威解析可用 CNAME 指向 CDN/GSLB，由调度服务结合递归解析器带来的客户端地址信息、运营商、URL、节点负载和健康状态返回边缘 IP；递归解析器会按 TTL 缓存结果，所以切换节点并非立刻对所有用户生效。CDN 还可能复用连接、优化跨地域链路，但不保证每个动态请求都更快。缓存键及转发策略要与资源语义一致，谨慎处理查询参数、Cookie、鉴权头和压缩差异，避免缓存污染或把个人数据跨用户共享。发布更新可用内容哈希版本名、较短 TTL 或失效刷新策略；源站故障时能否提供旧内容取决于配置与一致性要求。CDN 加速也不能代替源站鉴权。〔N65、N53〕

8.5 HTTP 缓存、会话与浏览器安全

027 HTTP 新鲜缓存与条件验证如何配合，no-cache 和 no-store 有何差别?

答案 缓存先判断已有响应能否直接复用：max-age 等新鲜度约束尚满足时，通常无需回源；过期或要求验证时，携带 If-None-Match/ETag 或 If-Modified-Since/Last-Modified，请服务端判断表示是否变化。未变化可用 304 响应继续使用本地内容，否则接收新表示。no-cache 允许存储，但复用前须成功验证；no-store 要求不存储该次请求或响应，不能等价成“每次都返回 200”。还要区分 private 与共享缓存、考虑 Vary 和鉴权内容，避免把用户 A 的响应提供给用户 B。缓存命中、是否过期与连接是否复用是不同维度。〔N53〕

028 hosts 文件怎样影响名称解析，如何区分本地配置与 DNS 劫持?

答案 hosts 保存本机主机名到地址的静态映射；使用系统解析器的程序通常按名称服务配置查询它和 DNS，顺序由平台及配置决定。dig 等直接查询 DNS 的工具可能绕过 hosts，所以它的结果与应用实际解析结果不一定相同。定位解析异常时应比较 hosts、解析器配置、缓存、递归器返回和权威记录，并确认浏览器是否使用独立的安全 DNS。DNS 劫持可能来自篡改解析配置、恶意递归器或伪造记录。DNSSEC 验证可保护已签名数据的真实性与完整性；DoH/DoT 保护传输，不能保证恶意解析器给出的答案正确。〔N54、N55〕

029 HTTP 无状态与 Cookie、Session、Token 登录态是什么关系?

答案 HTTP 无状态指协议语义不要求服务器记住前一次请求才能理解本次请求，不代表应用不能维护会话。Cookie 是浏览器按域、路径等规则保存并附带的数据；Session 通常把会话状态放在服务端，客户端只保存随机会话标识。Token 是代表身份或授权的凭证形式，既可是不透明随机串，也可采用自包含格式。会话标识可以装在 Cookie 或显式请求头中，携带 Cookie 并不会改变 HTTP 协议本身的无状态属性。禁用 Cookie 后可设计其他传递方式，但把凭证放 URL 容易进入历史、日志和引用来源，需谨慎处理。〔N57、N51〕

030 Cookie 与 localStorage 如何选择，登录凭证需要哪些浏览器侧保护?

答案 Cookie 可由浏览器按作用范围自动随请求发送，适合会话标识，但自动携带也要求防范 CSRF；localStorage 按源提供脚本可读的持久键值存储，不会自动成为 HTTP 请求头，常用于非敏感界面偏好。HttpOnly 限制脚本读取 Cookie，Secure 限制经安全通道发送，SameSite 约束跨站携带；它们解决的问题不同，并非开启一个属性就全面安全。JWT 放 localStorage 时可被成功执行的同源恶意脚本读取；放 HttpOnly Cookie 时则要设计 CSRF 防护。HttpOnly 也不能阻止 XSS 在页面内代用户发请求，应同时治理脚本注入。〔N58、N59〕

031 JWT 为什么便于跨实例校验，它的结构与撤销边界是什么?

答案 常见签名型 JWT 由头部、声明载荷和签名三段编码组成；声明可包含签发者、受众、主体和有效期。实例持有受信公钥或约定密钥便可验证，不必为每次认证查询同一会话表，便于多实例部署；但会话系统也可通过共享存储扩展。签名不等于加密，普通 JWT 载荷可读，不应直接装入秘密。验证时须限制算法并检查签名、iss、aud、exp 等预期条件。纯离线验证的令牌泄露后通常无法立即撤销，需要短有效期、刷新令牌轮换、撤销列表或版本检查等机制；这些机制会重新引入状态和运维成本。〔N60、N61〕

032 跨源访问为什么受限，CORS 预检能否代替身份认证?

答案 浏览器以协议、主机和端口组成源，并用同源策略限制脚本读取跨源资源。CORS 让服务器通过允许来源、方法、头部等响应字段声明哪些跨源脚本可访问结果；部分请求会先发送 OPTIONS 预检，但满足条件的简单请求可直接发出。携带凭证时，允许来源不能简单用通配符代替指定来源，并须按规范允许凭证。CORS 是浏览器执行的读取控制，不是服务端访问控制：非浏览器客户端可以直接发请求，有些跨源写请求也已到达服务端。服务端仍要独立认证、授权和防范 CSRF，不能仅信任 Origin 或预检成功。〔N64〕

033 XSS 与 CSRF 分别怎样利用浏览器身份，防护重点为何不同?

答案 XSS 让不可信输入变成受信页面中的脚本执行，可读取脚本可见数据或代用户操作；应按 HTML、属性、URL、脚本等输出上下文编码，使用安全 DOM 接口，对允许的富文本做可信净化，并以 CSP 等措施加强约束。CSRF 则诱导浏览器向目标站发送带有现有凭证的操作请求，攻击者不一定能读取响应或执行目标源脚本。服务端可用不可预测的 CSRF token、合适的 SameSite Cookie、Origin 等校验，并让敏感操作避开 GET。二者都与身份凭证有关，但仅设置 HttpOnly 或 CORS 无法完整解决；成功的 XSS 还可能绕过某些 CSRF 防护。〔N75、N76〕

8.6 TCP 状态与应用确认

034 如何识别一条 TCP 连接，TCP 头部字段分别解决什么问题?

答案 在相同协议和网络命名空间中，通常用源 IP、源端口、目的 IP、目的端口组成四元组识别 TCP 连接；监听 socket 与已经建立的连接不是同一个连接实例。TCP 首部含端口、序号、确认号、首部长度、控制位、窗口、校验和等，选项还能协商 MSS、窗口缩放、时间戳和 SACK 能力。序号与确认号描述字节流位置，窗口描述接收能力，SYN/FIN 等控制位驱动连接状态；最小首部为 20 字节，带选项会更长。理解字段要与具体方向和状态结合，抓到一个 ACK 并不等价于某次业务操作已经成功。〔N66〕

035 握手报文丢失或重复时 TCP 怎样恢复，accept 参与握手吗?

答案 客户端发送 SYN 后等待 SYN+ACK，超时会按实现策略重传；服务端收到 SYN 后通常进入 SYN-RECEIVED，发出 SYN+ACK 并等待确认。若最终 ACK 丢失，客户端可已进入 ESTABLISHED，而服务端仍重传 SYN+ACK；客户端对重传再次确认，或者后续携带合适 ACK 的数据推动服务端完成握手。重复 SYN 不应创建无限多条相同连接，协议按四元组、序号和当前状态处理。Linux accept 通常从握手完成的队列取出连接并返回新 fd，不是由应用调用它才发送第三次握手；使用 TCP Fast Open 等机制时还需看具体扩展。〔N66、N67〕

036 CLOSE_WAIT 和 FIN_WAIT_2 积压通常说明什么，半关闭还能做什么?

答案 TCP 两个方向可独立关闭。某端收到 FIN 后确认它并进入 CLOSE_WAIT，表示对端不再发送，但本地应用尚未关闭自己的发送方向；大量长期 CLOSE_WAIT 通常要检查异常路径是否遗漏 close、任务是否一直阻塞。主动方 FIN 被确认后进入 FIN_WAIT_2，等待对端也关闭，等待多久取决于应用生命周期与内核策略。shutdown 写方向后仍可接收对端剩余数据，不能再发送应用数据。若首个 FIN 丢失，接收方可能仍处 ESTABLISHED，由发送方的重传恢复。排查应定位 socket 所属进程、调用栈和超时流程，不能只调整 TIME_WAIT 参数。〔N70、N66〕

037 send 返回成功与 TCP ACK 分别能证明什么?

答案 流式 socket 的 send 返回正数通常表示相应字节已经被本地发送路径接受，返回值还可能小于请求长度；它并不保证对端已经收到，更不保证对端业务处理完成。连接在返回后仍可能断开，错误也可能稍后才被观察到。TCP ACK 只确认对端传输层按序接收了相应字节，应用可能尚未读取或在处理前崩溃。业务成功必须由应用协议定义确认点，例如持久化完成后返回结果。若请求超时而结果未知，应依赖请求 ID、幂等约束或状态查询收敛，不能把 send 成功或一次 ACK 当成资金、订单操作已经提交。〔N74〕

8.7 网络故障与安全

038 DDoS 与 SYN Flood 会耗尽哪些资源，应怎样分层缓解?

答案 DDoS 通过多个来源消耗带宽、连接状态或应用处理能力，正常流量因此无法获得服务；常见方式包括 UDP/NTP 反射放大、SYN Flood 消耗未完成握手资源、TLS/SSL 握手耗尽计算资源，以及 DNS 或 HTTP 请求洪泛。应先监测流量、半连接数量、入口错误率与应用耗时，区分链路饱和和业务热点。网络侧可配合运营商或清洗服务、边缘分发与合理过滤，主机侧审慎使用 SYN cookies 和队列参数，应用侧设置有界并发、超时、鉴权和公平限流。单纯增大队列或扩机器不保证解决问题，源站地址绕过和误伤正常用户也要评估；防护容量与恢复流程应提前验证。〔N68、N69〕

039 服务已经启动但请求失败或很慢，应怎样按阶段定位?

答案 先记录请求的目标地址、时间、错误和复现条件，区分 DNS、连接、TLS、首字节与响应体阶段。核对实际解析地址、路由、防火墙和代理配置，再用 ss 查看监听地址、端口和连接状态，避免服务只监听回环地址或错误网络命名空间。可用 curl 的分阶段计时、证书检查和经过授权的抓包判断故障停在哪里，并对照服务日志、请求 ID、队列、线程池及依赖耗时。TCP 连接成功只说明到该端口可建立传输，不能证明鉴权或业务正常；超时也未必是网络丢包，应把客户端与服务端时间线关联。〔N70、N74〕

```
curl -sS -o /dev/null \
  -w "dns=%{time_namelookup} connect=%{time_connect} tls=%{time_appconnect} first=%{time_starttransfer}
total=%{time_total}\n" \
  https://example.com/
# 指标为从请求开始的累计时间，按差值分析各阶段；不要记录鉴权头。
```

8.8 网络层与链路层

040 IP 与 MAC 地址如何协作，访问外网时 ARP 解析谁的地址?

答案 IP 地址用于网络层寻址和路由，MAC 地址用于具体链路上的帧交付；同一个 IP 包逐跳转发时，链路帧通常会被重新封装，源、目的 MAC 也随跳点变化。主机先查路由，判断目的地址是否在直连链路：同网段可解析目的主机的 MAC，跨网段通常解析下一跳网关的 MAC，而不是远端服务器的 MAC。ARP 在 IPv4 以太网场景中把下一跳 IPv4 地址映射为链路地址，并缓存结果；IPv6 使用邻居发现等机制。MAC 不是跨互联网查找服务器的全局路径标识，单凭 MAC 也不能提供可信身份认证。〔N71、N50〕

041 ping 与 traceroute 怎样工作，ping 不通为什么不代表 HTTP 不通?

答案 ping 通常发送 ICMP Echo Request，并根据 Echo Reply 观察往返时间和丢包；它测试的是对应 ICMP 交换，不会验证目标端口、TLS 或应用是否可用。traceroute 逐步增加 IP 生存时间，使中间路由器在到期时返回 ICMP Time Exceeded，由此推测路径；探测可使用 UDP、ICMP 或 TCP，返回路径也未必相同。防火墙限速或丢弃 ICMP 时，ping 可以失败而 TCP/HTTP 正常；反过来 ping 成功也可能遇到端口未监听或应用错误。某跳不回应探测不等于该跳不转发业务流量，应结合实际协议验证。〔N72〕

042 多个内网设备怎样通过 NAT/NAPT 共用一个公网地址?

答案 基本 NAT 在地址域之间改写 IP 地址；常见的 NAPT 还改写传输层端口，用不同外部端口区分多个内部连接。网关维护映射状态，出站包被改写为公网地址及映射端口，返回包再依据状态还原到内部主机。映射会过期，外部主机通常不能凭一个公网地址直接找到任意内网服务，入站访问需明确的映射、转发或其他连通机制。NAT 会影响端到端寻址，携带地址的应用协议、点对点连接和故障切换可能需要特别处理；它也不等同于防火墙策略，安全边界仍应由明确的访问控制决定。〔N73〕

第 9 章 数据库与缓存

9.1 Redis 与缓存设计

001 MVCC 是什么?

答案 MVCC 为同一行保留带事务可见性信息的多个版本，读事务按快照选择自己可见的版本，从而让普通读尽量不阻塞写。写操作仍要取得锁或检测冲突，因此 MVCC 不等于“没有锁”。长事务会拖住旧版本清理、扩大 undo/history，因此应控制事务时长并观察历史版本积压。 例如一致性读可看到事务开始时的快照，而当前读通常还要加锁协调。

```
T1.begin(); snapshot = T1.capture_visible_versions();
T2.update(row); T2.commit();
T1.read_consistent(row, snapshot); // 按可见性信息沿版本链选快照版本
T1.current_read_for_update(row); // 当前读可能加锁，写仍需冲突检测/锁
T1.commit(); vacuum_or_purge_old_versions_when_no_reader_needs_them();
// 长事务拖住 undo/history 清理；MVCC 减少普通读写冲突，不等于没有锁。
```

002 Redis RDB 与 AOF 持久化有什么区别?

答案 RDB 在某个时点生成紧凑快照，通常启动恢复快、适合备份，但故障时可能丢失上次快照后的写入。AOF 记录可重放的写命令，数据恢复点可更近，代价是文件和重放时间可能较大；每秒 fsync 与每次 fsync 的耐久性和延迟不同。两者可并用，先按可接受丢失窗口和恢复时间目标测试配置。 RDB 后台快照会利用 fork 和写时复制，写密集时仍可能增加内存占用。

```
periodically_fork_snapshot_to_RDB(); // 紧凑，恢复通常快；只含某时点，可能丢其后写入
append_write_commands_to_AOF(); // 恢复点可近，需重放且文件可能更大
configure_fsync("everysec"); // 延迟/耐久折中；always 更耐久也可能更慢
if (background_snapshot_writes) account_for_COW_memory_amplification();
// 监控快照年龄、AOF 重写、恢复耗时；按 RPO/RTO 实测，而非只看格式。
enable_both_if_backup_restore_speed_and_a_finer_RPO_are_both_required(); // 同时运行仍要监控资源
```

003 Redis 内存满时如何处理?

答案 达到 maxmemory 时 Redis 按配置的策略淘汰键；noeviction 会对需要额外内存的写操作返回错误，读仍可能正常。volatile 策略只从设过过期时间的键中选择，若没有此类键也可能无法淘汰；allkeys 策略可从所有键选。预留复制、AOF 缓冲和碎片等内存，并监控淘汰次数、命令错误与延迟。

```
if (used_memory_reaches_maxmemory) {
    if (policy == noeviction) return write_error_but_reads_may_work();
    candidates = policy_is_volatile ? keys_with_ttl : all_keys;
    if (candidates.empty()) return eviction_cannot_free_key();
    evict_by_configured_policy(candidates); // LRU/LFU/TTL 等按配置近似选择
}
reserve_memory_for_replication_AOF_buffers_and_fragmentation();
monitor(evicted_keys, rejected_writes, latency);
```

004 Redis 分布式锁需要注意什么?

答案 获取锁可用 SET key 唯一令牌 NX PX ttl 原子完成，释放时用 Lua 仅在令牌仍匹配时删除，防止过期后误删他人锁。任务执行时间可能超过租约，续租也会遇到暂停或网络分区，因此锁过期后旧持有者仍可能继续写。若资源要求严格互斥，应让下游检查单调递增的 fencing token 或用数据库条件更新兜底。

```
token = cryptographically_random_unique_owner_token();
if (SET(lock_key, token, NX, PX=lease_ms) != OK) return LockBusy;
fencing = allocate_monotonically_increasing_fencing_token(); // 每次获取严格递增
// 业务执行可能超过租期；长暂停/分区后旧持有者仍可能继续运行
resource.write_if_fencing_token_is_newer(fencing, update); // 下游拒绝旧 token
release_with_lua_if_get_equals_token_then_del(); // 避免误删后来 owner 的锁
// token 比对只保护锁删除；严格互斥还需受保护资源校验 fencing/DB 条件更新
// 锁租约、续租超时和业务截止时间需一致设计
```

005 Redis 常见数据结构及适用场景是什么?

答案 String 可存简单值、计数器和缓存；Hash 适合对象字段，List 可做两端队列，Set 做去重，ZSet 支持按分数排序和取 Top N。Stream 提供带 ID 的追加消息流及消费组，Bitmap、HyperLogLog、GEO 分别支持位状态、近似基数和地理查询。选择结构时要看命令复杂度、集合大小、持久化及数据是否需要精确。

```
SET(key, value); INCR(counter); // String: 缓存值/计数
HSET(user, field, value); // Hash: 字段对象; List 可做两端队列
SADD(tags, tag); // Set 去重; ZADD(rank, score, member); // ZSet 排序/Top N
XADD(stream, id, fields); // Stream 带 ID 消息流/消费组
SETBIT(flags, offset, 1); HLL_ADD(unique_visitors, item); GEOADD(locations, lon, lat, id);
// Bitmap / HyperLogLog / GEO 分别用于位状态、近似基数、地理查询。
// 比较命令复杂度、集合规模、精确性、持久化与恢复需求再选。
```

9.2 MySQL 索引、事务与并发控制

006 为什么 MySQL InnoDB 索引常用 B+ 树?

答案 B+ 树内部节点只保存分隔键和子指针，扇出高，树高通常较低；叶子按键有序，范围扫描能顺序访问相邻页。InnoDB 聚簇索引叶子存整行，二级索引叶子保存主键值，因此二级索引回表可能增加随机访问。相比哈希索引，B+ 树还能支持排序、前缀和范围查询，代价是维护索引会增加写入与存储开销。InnoDB 页内存放有序键和记录引用，分裂页也会带来写放大。

```
page = read_index_page(); // InnoDB 页内按序保存键和记录引用
while (!leaf_page(page)) page = page.child_for(key); // 高扇出，降低树高/随机 I/O
record = page.find_in_leaf(key); // 叶子有序，范围查询可顺序扫描相邻叶
// 聚簇索引叶子包含整行；二级索引叶子保存主键，必要时回表。
insert_new_key_may_split_page_and_write_more_pages(); // 写入/空间开销
// 哈希查找不天然支持排序/范围；B+ 树维护成本换范围能力。
```

007 什么情况下索引可能无法有效使用?

答案 函数包裹索引列、隐式类型转换、复合索引跳过左侧列、前缀通配符等可能让条件无法按预期做索引查找；低选择性时优化器也可能判断全表扫描更便宜。统计信息过期或返回大量列会改变计划。先看 EXPLAIN 的访问类型、估计行数和实际执行情况，再用代表性数据验证改写，不能机械地要求每个查询都走索引。复合索引中等值列和范围列的排列会决定后续列还能否用于定位。

```
EXPLAIN ANALYZE SELECT ... WHERE indexed_column = ?; // 看真实行数/耗时
// 函数包列、隐式类型转换、跳过复合索引左前缀、LIKE '%x' 可能妨碍查找。
// 等值列通常放在范围列前；范围列之后的索引列定位能力有限。
if (predicate_has_low_selectivity || returns_many_rows) optimizer_may_scan_table();
refresh_statistics_if_stale(); rewrite_predicate_without_changing_semantics();
// 用代表性数据验证新计划；全表扫描有时确实更便宜。
```

008 如何分析并优化一条慢 SQL?

答案 先从慢查询日志定位高频或高耗时语句，用 EXPLAIN/ANALYZE 看估算与实际行数、连接顺序、回表和排序临时表。检查过滤条件、联合索引顺序、返回列、分页方式及 N+1 调用，减少扫描和往返。改动后用真实数据分布压测读写两侧，并检查锁等待、索引维护成本和执行计划回归。EXPLAIN 的估算值不一定代表实际成本，热点参数和冷数据都应纳入基准。

```
plan = EXPLAIN_ANALYZE(query); // 先区分估算与实际扫描行数、索引、排序和回表
profile_slow_stages(); check_lock_waits_and_io();
rewrite_query_or_add_composite_covering_index_if_measurements_justify();
run_on_representative_data(); compare_rows_latency_and_write_cost();
// 避免只靠 EXPLAIN 估算或盲目添加索引；索引会增加存储和写放大。
// 检查参数分布、统计信息、返回列与 LIMIT 是否改变计划。
```

009 如何防止 SQL 注入?

答案 预编译或参数化查询把用户值作为数据绑定，避免输入被解析成 SQL 语法；例如 `WHERE id = ?` 不把字符串拼进 SQL。表名、列名等通常不能作为普通参数绑定，应通过固定白名单映射选择。输入校验与最小数据库权限是纵深防御，转义字符串不能替代参数化。 参数绑定也能避免不同输入导致语句结构变化，便于数据库复用执行计划。

```
query = db.prepare("SELECT id FROM users WHERE name = ?");
query.bind(0, user_input); // 把用户值作为数据绑定，不拼进 SQL 语法
query.execute(); // 数据库按参数类型处理，语句结构保持固定
// 表名/列名/排序方向通常不能用值占位符绑定
column = whitelist_map[user_selected_sort]; // 用固定映射选择标识符
execute("SELECT id FROM users ORDER BY " + column); // 仅拼接可信白名单项
// 参数化不替代权限校验、最小数据库权限和错误处理
```

010 数据库事务的 ACID 是什么?

答案 原子性保证事务作为整体提交或回滚；一致性表示提交前后都满足业务约束；隔离性规定并发事务互相可见的效果；持久性保证提交后能经受规定范围内的故障恢复。数据库通过 undo/redo、锁或 MVCC、日志刷盘等机制提供这些性质。实际持久性受刷盘参数、复制确认和存储设备保证影响，ACID 不会自动修复业务规则错误。

```
BEGIN TRANSACTION;
// Atomicity: 全部提交或回滚; Consistency: 约束/业务不变量保持
UPDATE account SET balance = balance - amount WHERE id = from_id;
UPDATE account SET balance = balance + amount WHERE id = to_id;
// Isolation: 并发事务按隔离级别协调可见性; Durability: 提交后依刷盘策略持久
if (any_update_failed) ROLLBACK(); else COMMIT();
// ACID 不替业务定义正确性；检查余额/行数并理解持久化配置。
```

011 数据库读写分离有哪些一致性风险?

答案 主库提交后从库可能尚未应用日志，因此立刻读从库会看不到刚写的数据；故障切换也可能丢掉未复制或未确认的提交。可对写后读粘主库、等待复制位点追平，或给客户端明确最终一致读取语义。还需定义超时重试的幂等性及切主时读写路由，不能把“复制正常”当作零延迟保证。需要强读后写语义时，可在应用会话内暂时固定读主库并设置最长等待时间。

```
primary.commit(write); replica.apply_async(write); // 复制存在延迟
read_from_replica(); // 可能读到旧值，读己之写不一定成立
if (must_read_own_write) read_from_primary_or_wait_until_replica_lsn >= commit_lsn;
if (failover_occurs) verify_acknowledged_write_was_replicated();
// 还需处理复制中断、切换和读一致性目标；异步复制可能丢最近确认写。
// 按业务在延迟、可用性、读写扩展间取舍。
```

012 数据库隔离级别与常见并发异常是什么?

答案 标准隔离级别从读未提交、读已提交、可重复读到可串行化，对并发异常的限制逐步加强，但实现方式和代价因数据库而异。脏读是读到未提交数据，不可重复读是同一行再次读取变值，幻读是同一范围再次查询出现新行。MySQL InnoDB 的 MVCC 与间隙/next-key 锁会影响具体表现，回答时应说明数据库、语句类型和隔离级别。

```
T1.begin(isolation_level);
value = T1.read(row); T2.update_and_commit(row);
// Read Uncommitted 可见脏读；Read Committed 每条语句新快照，可能不可重复读。
// Repeatable Read 通常保持事务快照；Serializable 会更严格地冲突/串行化。
T1.read_again(row); // 幻读、不可重复读依数据库实现/当前读和范围锁而异
// 用真实事务并发复现并检查锁/执行计划，不把隔离级别标签当实现细节。
```

013 缓存穿透、击穿、雪崩分别是什么?

答案 穿透是大量请求查询本不存在的数据，可在入口校验、缓存短 TTL 空值或用 Bloom Filter 过滤；过滤器有假阳性，不能据此断言一定存在。击穿指单个热点键失效后并发回源，可互斥重建或逻辑过期并限制并发。雪崩是大量键同时失效或缓存集群故障，应错开 TTL、限流、降级并保护数据库。 重建热点值时要限制并发数，避免互斥粒度过大导致请求集中排队。

```
value = cache.get(key);
if (value == MISS) { // 穿透：键不存在；负缓存可短暂记住空结果
    if (bloom.definitely_absent(key)) return NotFound; // Bloom 有假阳性，不会证明存在
    if (try_singleflight_lock(key)) rebuild_from_database_with_ttl_jitter();
    else serve_stale_or_wait_with_deadline(); // 单热点失效时避免并发回源
}
if (many_keys_expire_together || cache_cluster_down) rate_limit_and_degrade();
// 雪崩用 TTL 抖动/限流保护 DB；重建锁粒度要小并限制等待
// 对 Bloom 命中仍要查真实存储，过滤器不能代替数据源
```

014 InnoDB 的 undo log、redo log 和 MySQL binlog 分别有什么作用?

答案 undo log 保存回滚所需的旧值信息，也供一致性读沿版本链构造快照；redo log 记录 InnoDB 页变更相关信息，用于崩溃恢复；binlog 是 MySQL Server 层的逻辑变更日志，用于复制和时间点恢复。三者服务的层次、格式和生命周期不同，不能互相替代。长事务可能延迟 undo 清理，redo/binlog 的持久保证还取决于刷盘配置。〔S9〕

```
old_value = undo_log.append_for_rollback_and_snapshot_visibility();
redo_log.append_page_change_for_crash_recovery();
binlog.append_logical_server_change_for_replication_and_point_in_time_restore();
// undo 服务回滚/MVCC 版本链；redo 服务 InnoDB 崩溃恢复；binlog 属 Server 层。
// 日志格式、生命周期和持久化配置不同，不能相互替代。
if (long_transaction) monitor_undo_history_cleanup_lag();
```

015 MySQL 为什么要协调 InnoDB redo log 和 binlog 的两阶段提交?

答案 一个提交要让 InnoDB 的引擎日志状态与 Server 层 binlog 一致，否则恢复和复制可能看到不同事务集合。MySQL 使用内部两阶段提交：引擎先准备，服务器写入并按配置持久化 binlog，再通知引擎提交；崩溃恢复据 prepare 状态与 binlog 记录协调完成提交或回滚。刷盘策略决定掉电时的耐久窗口，逻辑协调本身不等于硬件已持久化。〔S9〕

```
engine.prepare(txn); // 引擎先 prepare，崩溃后可被协调
server.write_binlog(txn); flush_binlog_according_to_sync_policy();
engine.commit(txn); // Server 确认日志后通知引擎提交
// 崩溃恢复查看 prepare 状态与 binlog 记录，协调提交或回滚
// 保持 InnoDB redo 与 Server binlog 的事务集合一致，服务恢复/复制
// 刷盘频率影响掉电耐久窗口；两阶段协调不等于硬件已经持久化
// 长事务/日志积压也会影响恢复与复制延迟
```

016 InnoDB 的 gap lock 和 next-key lock 是什么?

答案 Record lock 锁定索引记录；gap lock 锁住索引记录之间的间隙，阻止其他事务向该范围插入；next-key lock 是记录锁加其前方间隙锁。InnoDB 在某些隔离级别的范围锁定读中用 next-key 锁抑制幻读，但实际范围受扫描索引和查询条件影响。完整唯一键等值查找通常只需记录锁，调试应结合执行计划与锁等待信息。〔S9〕

```
SELECT * FROM records WHERE key BETWEEN low AND high FOR UPDATE;
// Record lock 锁住索引记录；gap lock 锁住记录间隙、阻止范围插入
// Next-key lock = 记录锁 + 其前方间隙锁，可在部分范围读防幻行
INSERT INTO records(key) VALUES (inside_locked_gap); // 可能等待/冲突
// 完整唯一键等值查询常只需记录锁，范围扫描会涉及更多索引项
// 实际锁取决于隔离级别、索引/执行计划和查询谓词，查看锁等待验证
```

9.3 热点与大键治理

017 Redis 热 Key 和大 Key 有什么区别？如何排查?

答案 热 Key 的核心问题是请求集中，可能让单分片 CPU、网络或串行命令成为瓶颈；大 Key 则是单键拥有大量元素或大负载，整键读取、删除和复制可能耗时并放大内存峰值。可用访问采样/监控定位热键，按只读场景评估本地缓存或副本分流；大集合应拆分并分页操作。拆分会增加聚合与一致性复杂度，先确认键访问模式再调整。〔S10〕

```
sample_key_access_frequency_and_per_shard_cpu_network(); // 热 Key：请求集中
measure_key_bytes_element_count_and_whole_key_operation_latency(); // 大 Key：单键操作重
if (hot_and_read_only) add_bounded_local_cache_or_read_replicas();
if (large_collection) split_by_bucket_and_page_operations();
// 大 Key 的整键读/删/复制可能阻塞分片并放大内存峰值
// 热 Key 拆读流量要考虑一致性；大 Key 拆分增加聚合和更新复杂度
// 调整后继续监控分片 CPU、网络、延迟和内存峰值
```

9.4 Redis 与缓存设计

018 Redis 的进程、事件循环、持久化与集群边界是什么?

答案 Redis 服务端通常在一个主执行路径中顺序处理键空间命令，以事件循环和 I/O 多路复用管理连接；网络 I/O、异步释放等工作可由线程辅助，持久化与复制是独立的可靠性边界。RDB/AOF 影响重启恢复和丢失窗口，副本与 Sentinel 处理可用性，Cluster 按哈希槽分片。客户端、单机实例、副本组和集群的路由语义不同，热键、长命令、巨大返回值和异步复制仍可能造成延迟或丢失；不能把“内存数据库”当作自动持久或强一致。〔N77〕

9.5 SQL 基础与数据建模

019 关系型数据库与 NoSQL 应怎样按数据访问方式选型?

答案 关系型数据库以表、键、约束和 SQL 为核心，适合多实体关联、复杂查询以及需要事务维护不变量的业务。NoSQL 是文档、键值、列族、图等模型的统称，适合围绕某类访问路径组织数据；它不等于“没有模式”或“没有事务”。选型应先明确主要查询、数据规模、延迟、跨记录原子性及一致性目标，再评估索引、分片和运维成本。订单核心数据可以放关系库，查询视图或缓存采用另一模型；避免仅因预期高并发就先放弃约束，也不要假设任何 NoSQL 都能无代价水平扩展。〔N78〕

020 第一、第二、第三范式分别在消除什么数据依赖?

答案 第一范式要求关系中的属性值符合选定的原子域，避免在同一列塞入需要独立查询的重复组。第二范式在第一范式上消除非主属性对候选键的部分依赖，尤其要检查复合键只取一部分就能决定的字段。第三范式继续消除非主属性对候选键的传递依赖，例如订单中直接重复保存客户地址可能产生更新异常。规范化的目标是减少插入、修改、删除异常，并不意味着表越拆越好；在明确读取模式、同步机制和一致性代价后，可以有意识地反规范化保存冗余读模型。〔N79〕

021 内连接、外连接和反连接各适合什么查询?

答案 内连接只保留满足连接条件的组合；左外连接保留左表全部行，匹配不到的右表列补 NULL，右外连接则保留右表全部行。MySQL 没有原生 FULL OUTER JOIN，可用左连接与右连接的结果 UNION ALL，并在右连接部分用左表主键 IS NULL 只取未匹配的右表行来模拟。查“选了课程 02 但没选 01”的学生，可用 EXISTS 配合 NOT EXISTS，避免连接后的多行放大；展示所有学生的选课情况则从学生表 LEFT JOIN 选课表。外连接的右表过滤放在 WHERE 还是 ON 会改变结果：WHERE right.col = 值会排除补 NULL 的行。连接前应确认一对一或一对多基数、外键和索引，不能依靠随手加 DISTINCT 掩盖错误的连接条件。〔N80、N81〕

```
SELECT s.stu_id
FROM student AS s
WHERE EXISTS (SELECT 1 FROM score AS c
              WHERE c.stu_id=s.stu_id AND c.subject_id='02')
  AND NOT EXISTS (SELECT 1 FROM score AS c
                  WHERE c.stu_id=s.stu_id AND c.subject_id='01');
-- 半连接/反连接表达存在性，不会因同一学生多条成绩而重复输出。
```

022 MySQL 字符串列的长度、字节数和 TEXT 上限怎样区分?

答案 CHAR(M) 与 VARCHAR(M) 的 M 表示字符数；实际占用还取决于字符集，例如 utf8mb4 的一个字符最多需要四字节。CHAR 偏向固定长度值，VARCHAR 保存可变长内容并有长度开销，比较和尾部空格行为还受类型及排序规则影响。TEXT 也有上限，普通 TEXT 最多约 64 KiB 字节，另有更小或更大的系列，不能认为能无限保存；VARCHAR 还受整行大小约束。应以字符语义设计长度，用 CHAR_LENGTH 看字符数、LENGTH 看字节数，结合索引限制、行格式及实际数据选择类型。〔N82、N83〕

023 INT 的显示宽度会改变取值范围或存储空间吗?

答案 INT(1) 和 INT(10) 的历史括号值是显示宽度，不把整数限制为一位或十位，也不改变 INT 的四字节存储及有符号范围。不要把它与 VARCHAR 的长度参数混淆。整数显示宽度在 MySQL 8.0 中已经弃用，旧版本配合 ZEROFILL 的显示补零也不是数值精度控制。需要业务范围时应选 TINYINT、SMALLINT、INT、BIGINT，明确是否允许负数，并用约束及应用校验限制有效值。若编号必须显示前导零，它通常是格式化字符串，输出层处理比依赖整数列的显示属性更明确。〔N84〕

024 金额列为什么通常使用 DECIMAL 或整数最小货币单位?

答案 FLOAT 和 DOUBLE 按二进制近似表示数值，许多十进制小数无法精确表达，累加、比较与舍入可能出现意外结果。DECIMAL(p,s) 保存精确十进制值，p 是总有效位数，s 是小数位数，适合金额及按十进制规则结算的数量；另一方案是用整数保存分、厘等约定单位。类型本身不能替业务决定税率精度、换汇时机及舍入方向，运算中也要避免先转为浮点再写回。设计时明确最大金额、币种和小数位，边界输入要验证溢出与舍入结果，跨服务传输保持相同的单位和精度约定。〔N85〕

025 数据库应如何统一保存 IPv4 和 IPv6 地址?

答案 可以用 VARCHAR 保存规范文本，调试直观，但占空间且同一 IPv6 地址存在多种等价写法；需要紧凑存储和精确相等比较时，可用 VARBINARY(16) 保存 INET6_ATON 的结果，用 INET6_NTOA 转回文本。该函数处理 IPv4 时会返回四字节结果，因此应预先约定是否统一映射为十六字节及如何区分地址族，不能混用两套归一化规则。IPv4 也可用无符号整数，但不适合直接承接 IPv6。若要做网段匹配、范围检索或黑名单，应明确网络字节序、前缀长度与边界计算，不把展示字符串的字典序当作地址数值顺序。〔N86〕

026 外键约束怎样维护引用完整性，有哪些使用代价?

答案 外键让子表中的引用必须匹配父表键，或按允许的 NULL 语义为空；父表更新和删除时可选择拒绝、级联或置 NULL 等动作。它把引用完整性落实在数据库中，能阻止绕过应用写入的孤儿记录，但不替代事务中的其他业务条件。InnoDB 对外键和被引用键有索引要求，级联操作还可能扩大锁定范围与一次写入的工作量。跨库分片或跨服务时通常无法直接保留同样的约束，需要事件、补偿及对账。是否采用外键应依据数据所有权和运维需求，而不能笼统地认为外键一定拖慢所有查询。〔N87〕

027 IN 和 EXISTS 的语义差异是什么，怎样避免 NOT IN 的空值陷阱?

答案 IN 判断一个值是否属于候选集合，EXISTS 只判断子查询是否产生至少一行，子查询 SELECT 的具体字段通常不影响存在性。相关子查询可以表达逐行关联，但数据库也可能将 IN 或 EXISTS 改写成半连接等计划，因此不能只凭“外表大用某个写法”判断性能。尤其要注意 NOT IN：候选集合只要包含 NULL，就可能使原本以为成立的判断变成 UNKNOWN，最终被 WHERE 过滤；NOT EXISTS 通常更直接表达不存在关联行。先保证 NULL 语义与连接键正确，再用 EXPLAIN 和实际数据评估计划、索引及执行次数。〔N81〕

028 SQL 常见标量函数应怎样使用才不改变业务语义?

答案 字符串处理可用 CONCAT、SUBSTRING，数值处理有 ROUND、ABS，时间处理有日期提取和区间运算，条件表达式常用 CASE、COALESCE。选择函数时要确认 NULL 如何传播、字符串按字符还是字节计数、舍入规则和会话时区，避免把展示格式参与数值计算。对索引列直接套普通函数可能妨碍常规索引范围定位，例如把时间列格式化后再比较日期，通常可改成明确的半开时间区间；存在函数索引或生成列时另行评估。应将字段类型、参数类型和边界样本一起验证，不靠函数名字推断隐式转换或空值处理。〔N88〕

029 COUNT 和 NULL 的组合会怎样影响统计结果?

答案 NULL 表示缺失或未知，空字符串则是一个存在的字符串值，两者在比较、约束及聚合中的语义不同。COUNT(*) 统计结果集行数，COUNT(1) 对每一行计算非空常量，语义上同样统计行数；COUNT(col) 只数该表达式非 NULL 的行，COUNT(DISTINCT col) 还要去重。测试空值要用 IS NULL，而不是 col = NULL，因为后者通常得到 UNKNOWN。LEFT JOIN 后统计子表记录时，应数非空的子表键，不能直接 COUNT(*) 把无匹配的父行也计为一条。对 InnoDB，计数还受事务可见性影响，不存在适用于所有事务的即时精确总行数。〔N89〕

030 DELETE、TRUNCATE 和 DROP 的事务及对象语义有何区别?

答案 DELETE 删除符合条件的行，保留表结构；对 InnoDB 可在事务中回滚，但实际日志、锁和执行时间随删除范围增长。TRUNCATE 清空整张表并重建相关存储，属于 DDL，通常会隐式提交，不能像普通事务 DELETE 那样回滚，还会重置自增计数并受外键引用限制。DROP 则删除表对象及其数据，后续语句不能继续把它当作原表使用。不能仅按“哪个快”决定命令：先确认是否保留结构、是否需按条件删、引用关系、备份恢复和权限，再评估分批清理方案。一次误执行的 DDL 不能靠应用层重试恢复。〔N90〕

031 SELECT 的逻辑处理顺序与实际执行计划有什么不同?

答案 理解查询可按 FROM/JOIN、WHERE、GROUP BY、HAVING、窗口计算、SELECT/DISTINCT、ORDER BY、LIMIT 这样的逻辑阶段推理：WHERE 过滤分组前的行，HAVING 过滤分组后的结果，窗口函数通常在分组之后保留每行输出。这个顺序解释了为何很多位置不能直接使用 SELECT 中定义的别名，也解释了外连接过滤的位置为何影响结果。MySQL 对别名等有自己的语法扩展，应按具体规则判断。优化器可在保持语义的前提下推过滤、重排内连接或改写子查询，所以逻辑顺序不是逐步执行的物理流水线；真实访问路径应查看执行计划。〔N91、N92〕

032 按学生总分取第 5 到第 10 名，怎样定义并列与分页?

答案 先按学生聚合 SUM(score)，再给聚合结果计算排名，最后过滤排名区间；不能先对成绩明细做 LIMIT 再汇总。ROW_NUMBER 为每行分配不同序号，若必须恰好取六名学生，可在总分后加学生 ID 作为稳定次序。RANK 遇到并列时共享名次且后续跳号，DENSE_RANK 则不跳号，适合按业务定义处理并列。窗口函数不能在同层 WHERE 中直接过滤结果，可放到 CTE 或子查询中。需要说明成绩为 NULL、同课程多条成绩及补考记录怎样计入，排名规则的一致性比仅给出一段可执行 SQL 更重要。〔N92〕

```
WITH total AS (
  SELECT stu_id, SUM(score) AS total_score
  FROM student_score GROUP BY stu_id
), ranked AS (
  SELECT total.*, ROW_NUMBER() OVER
    (ORDER BY total_score DESC, stu_id) AS rn FROM total
)
SELECT stu_id, total_score FROM ranked
WHERE rn BETWEEN 5 AND 10 ORDER BY rn;
-- 此处按稳定序号选六行；业务按并列名次时改用 RANK/DENSE_RANK。
```

9.6 MySQL 引擎与存储

033 一条查询或更新怎样经过 MySQL Server 与 InnoDB?

答案 客户端连接后先完成认证和权限相关检查，Server 解析 SQL，优化器选择访问路径和连接顺序，执行器按计划调用存储引擎接口。InnoDB 通过 Buffer Pool 访问数据页，按隔离级别使用版本链或锁；更新还会产生用于回滚与恢复的日志，提交时协调引擎和 binlog。权限、元数据锁以及锁等待可能出现在不同阶段，慢查询不一定是查找算法慢。MySQL 8.0 已移除旧的查询缓存，不能把过时执行流程直接套用。定位时将连接耗时、执行计划、页命中、事务等待与日志刷盘分别观测，不能仅根据一次总耗时归因。〔N93、N94、N95〕

034 InnoDB 与 MyISAM 在事务、索引和并发上怎样取舍?

答案 InnoDB 提供事务、崩溃恢复、MVCC、行级锁与外键，是现代 MySQL 的默认通用引擎；其聚簇索引组织整行数据，二级索引保存主键，主键宽度也会放大辅助索引。MyISAM 不提供同等的事务及恢复语义，数据与索引文件分离，写入主要依赖表级锁，更适合少数明确的兼容场景，不能仅因简单 COUNT(*) 较快就迁移业务表。InnoDB 的行锁不代表任何语句只锁单行，范围、索引及隔离级别仍决定锁范围。引擎选型应一起评估故障恢复、一致性、并发读写和备份方案，而不只比较单次查询耗时。〔N93〕

035 MySQL 表结构和数据文件的布局为什么必须区分版本?

答案 MySQL 5.7 常见 .frm 保存表定义，启用独立表空间的 InnoDB 表以 .ibd 保存数据及索引；这套描述不能直接当作 MySQL 8.x 的文件规范。8.0 起采用事务型数据字典，移除了 .frm 等旧元数据文件。InnoDB 还可使用系统、通用、独立、undo 和临时等表空间，是否一表一个 .ibd 取决于表空间配置及建表方式，不能只凭目录中某个扩展名判断全部数据归属。排查前先确认版本、datadir、引擎和表空间，备份迁移使用受支持的流程；单独复制正在运行中的一个表文件，不能自动保证元数据和事务状态一致。〔N96、N97〕

036 Buffer Pool 怎样减少磁盘访问，脏页又在何时落盘?

答案 Buffer Pool 是 InnoDB 缓存数据页和索引页的主要内存区域。读取先查缓存，未命中才从存储加载；修改通常先发生在内存页中并把页面标为脏页，redo 先行保证之后可恢复，再由后台刷脏等流程把页面写回磁盘，所以事务提交不必逐一把所有改过的数据页写盘。它还维护页状态和替换队列，扫描大量冷数据可能冲击热点，需观察命中率、脏页比例及淘汰等待。容量并非越大越好，应给连接、排序、日志、操作系统和其他进程留空间；仅看高命中率也不能排除锁等待、慢存储或低效扫描。〔N94〕

9.7 MySQL 索引补充

037 InnoDB 主键为何偏好短且稳定，自增值为什么会有空洞?

答案 主键决定聚簇索引的组织顺序，二级索引也携带主键，因此短、稳定、具有局部递增性的键通常能降低空间、页分裂与随机写入。随机 UUID 较宽且插入位置分散，但是否采用仍要考虑离线生成、隐私及跨库唯一性；有序 UUID 或业务号也要结合具体编码验证。AUTO_INCREMENT 保证分配规则，不保证提交后的编号连续：事务回滚、失败插入或批量预分配都可能留下空洞，不能用相邻 ID 之差推断删除数量。数据库主键与对外业务编号可以分离，要求严格连续的财务流水需专门协议，不能依赖普通自增列。〔N98、N97〕

038 覆盖索引与索引条件下推分别减少哪一类工作?

答案 覆盖索引指查询所需的列都能从所用索引取得，无须为了补齐这些列再读取聚簇记录；它由查询投影和索引内容共同决定。索引条件下推则把可用索引列判断的部分条件交给存储引擎，先在索引记录上过滤，再对通过者读取完整记录，从而减少不必要回表。二者可以共同发生，但不是同一概念。复合索引后面的列即使不能继续缩小定位范围，也可能仍用于过滤或覆盖，不能一概叫作“完全失效”。应结合 EXPLAIN 中的访问路径和 Extra 信息检查，索引越宽也会增加写入与存储成本。〔N99〕

039 字符串前缀索引怎样节省空间，有哪些功能限制?

答案 前缀索引只索引字符串的前若干字符或字节，具体计量取决于字符串类型；常用于较长文本以控制索引大小和维护开销。前缀越短通常冲突越多，查询仍需核对完整值，必须用真实数据比较前缀区分度，而不是任意选一个长度。它不能天然替代完整列索引的全部能力，例如仅凭前缀不足以证明完整列相等或覆盖完整列输出，排序和分组是否可利用也要看执行条件。TEXT/BLOB 创建普通索引通常需明确前缀长度，同时受引擎与字符集的索引字节上限约束。唯一前缀索引限制的是前缀唯一，可能拒绝完整值不同但前缀相同的数据。〔N100〕

9.8 MySQL 锁与恢复

040 数据库发生死锁后应用应怎样恢复，如何降低再发生概率?

答案 死锁是事务之间形成循环等待，增加锁等待时间不能消除这个环。InnoDB 通常会检测死锁并选择事务回滚，应用需识别相应错误、回滚并有限次重试完整事务，而不是只重放最后一条 SQL 导致部分业务状态遗漏。应收集事务语句、执行计划与死锁日志，统一访问多行和多表的顺序，缩短事务并建立合适索引，以减少锁范围和持有时间。关闭死锁检测时还需理解超时回滚策略。锁等待超时与死锁并不相同，默认情况下它们对语句及整个事务的回滚范围可能不同，错误处理必须主动收敛事务状态。〔N101〕

041 为什么 ALTER TABLE 可能让后续普通查询也排队?

答案 元数据锁保护表结构在语句或事务使用期间保持有效；已有事务持有兼容的元数据锁时，要求更强锁的 ALTER TABLE 可能等待。新的查询又可能排在待授予的排他元数据锁后面，形成看似“一个改表拖住整张表”的队列，源头往往是未结束的长事务。排查应观察 performance_schema.metadata_locks、会话及事务状态，识别等待者与真正阻塞者；只杀等待的查询未必解决源头。上线 DDL 前设置合理等待策略、控制长事务并验证 Online DDL 支持范围，不能把 LOCK=NONE 理解成绝不会申请任何元数据锁。〔N102〕

042 innodb_flush_log_at_trx_commit 的三个值如何影响故障丢失窗口?

答案 值为 1 时，提交要求相应 redo 写出并同步持久存储；值为 2 时提交写到文件层，但刷盘通常由后台周期进行；值为 0 时提交不要求立即完成写出及刷盘。日志先经过内存缓冲和文件缓存，进程崩溃、操作系统崩溃与掉电的影响因此不同。后台默认周期常是一秒，但调度、存储和配置会影响实际窗口，不能声称 0 或 2 严格最多丢一秒。若启用 binlog，还要配合 sync_binlog 和存储设备的真实耐久保证评估完整事务。组提交能合并刷盘工作，并不意味着可以忽略耐久参数或备份恢复演练。〔N103、N95〕

043 Redo 空间将耗尽时，为何 checkpoint 和刷脏会限制写入？

答案 Redo 是有容量边界的循环日志，不能覆盖崩溃恢复仍需要的记录。Checkpoint 表示此前相应页面修改已具备足够的落盘进度，恢复可以从较新的位置开始，旧日志空间才可复用。若写入产生 redo 太快而脏页刷盘跟不上，检查点推进不足，系统会加强刷脏甚至限制前台写入，表现为延迟升高。应检查 redo 容量、checkpoint age、存储吞吐及脏页压力，不能仅靠无限增加日志容量掩盖持续过载。WAL 将提交关键路径从随机写多个数据页转向日志顺序写，但页面最终仍须落盘，并没有消除数据页 I/O。〔N104〕

044 组提交怎样提高日志刷盘效率，代价是什么？

答案 多个并发事务可以共享一次日志同步操作，减少每个事务独占 fsync 所造成的 I/O 和等待开销。Server 的 binlog 提交流水线可把相关工作分阶段批量处理，并与 InnoDB 提交协调；它提升吞吐的关键是摊薄耐久操作成本，不是让多个业务事务合成同一个可回滚事务。适当等待更多提交者可能形成更大批次，但也会增加单事务延迟，低并发场景未必受益。评估时同时看吞吐、提交尾延迟、存储延迟和持久化参数，不能通过关闭刷盘获得的性能数字推断组提交自身效果。〔N95、N103〕

045 InnoDB doublewrite 为什么不能简单地由 redo 替代？

答案 数据库页较大，而底层写入可能在崩溃时只完成一部分，留下撕裂页；redo 描述页变更，并不意味着能安全地在任意损坏页上直接重放。Doublewrite 先把待刷页写到受管理的双写区域并保证相应持久性，再写目标数据文件，恢复时可用完好的页副本修复受影响页，然后应用 redo。它解决页写完整性，与事务提交的日志持久性互补。现代 MySQL 的实现和存储优化随版本演进，应按版本及设备保证配置；不能仅因为 SSD 较快就推断支持数据库页的完整原子写，从而随意关闭保护。〔N105〕

9.9 MySQL 查询与复制补充

046 深分页为什么常变慢，如何用游标方式继续读取？

答案 LIMIT offset,count 往往仍要访问并丢弃前 offset 条符合条件的结果，偏移越大扫描和回表成本越高，并发插入还会让按页号翻页发生重复或跳行。若业务支持“下一页”，可按稳定且唯一的排序键记录上一页末尾位置，用范围条件继续扫描，例如按创建时间和 ID 的组合游标。需要任意页跳转时，可评估覆盖索引先取主键再回表、限制最大页深或预计算结果，但这些方案不会自动消除偏移扫描。游标应包含全部排序列并处理同值和方向，过滤条件及索引顺序也要匹配，不能只凭 ID 大小替代业务的实际排序。〔N106、N91〕

047 MySQL 复制、半同步与 GTID 分别解决什么问题？

答案 典型复制中，源端记录 binlog，副本接收写入 relay log，再由应用线程执行；接收和应用是不同进度，因此收到日志不代表查询已经可见。异步复制提交无需等待副本，半同步则按配置等待副本的日志接收确认，能缩小部分丢失窗口，但不等于等待副本完成应用，也不自动提供无损切主或线性一致读。GTID 为事务提供全局身份，可帮助识别已执行集合并简化重新定位复制起点；它不是物理文件偏移，也不会消除复制延迟。应把确认条件、超时退化、候选副本和业务读后写策略一同设计。〔N107、N108、N109〕

048 分库分表怎样选择分片键并规划扩容？

答案 分表把数据拆到多个表，分库进一步分散到多个数据库实例，主要解决单节点容量、写入和故障影响范围，代价是跨片查询、关联、唯一性与事务更复杂。分片键应兼顾主要路由条件、分布均匀性和增长模式；若按买家 ID 分片但只拿订单主键查详情，就需要路由映射、冗余分片字段或把分片信息编码到 ID，否则只能广播。分页要定义全局排序：浅页可让每片返回 offset+limit 条再归并，代价随 offset 和分片数上升；连续翻页优先使用稳定复合排序键作游标。任意深页可考虑单独排序/搜索索引，并评估一致性与维护成本。扩容还要有迁移批次、增量追平、路由版本、校验和回滚机制。全局聚合、对账及跨片事务应在拆分前定义清楚。〔N110〕

9.10 Redis 内部结构与执行模型

049 Redis 的大规模有序集合为什么使用跳表配合字典？

答案 以 Redis 7.2 的典型实现为例，较大的 ZSet 使用字典支持按成员快速定位，再用跳表按分数及同分成员顺序维护有序关系；较小的集合可采用紧凑编码。跳表由多层有序链构成，节点层高按概率选择，形成稀疏的上层索引，平均查找、插入、删除约为 O(log n)，从已定位位置继续范围遍历则按输出量推进。它避免每次插入移动整个数组，也便于实现排名所需的跨度统计。选择跳表主要契合内存中的更新与范围操作，不意味着它在磁盘页访问、空间或任何负载下都比 B+ 树更好；编码阈值和结构应按具体 Redis 版本核对。〔N111〕

050 listpack 等紧凑编码如何节省内存，为什么不适合无限增大？

答案 紧凑编码把多个小元素连续存放，减少指针、分配器和对象头开销，并改善访问局部性；小 Hash、ZSet 及 List 的某些内部节点会使用这类结构，但使用条件随版本与配置变化。旧 ziplist 的前项长度编码可能引起级联扩展，listpack 使用不同的记录长度设计，避免该类连锁更新。连续布局仍有代价：按位置查找常需顺序遍历，插入或扩容也可能移动一段内容，因此不能将“小对象省内存”推广为任意规模都更快。分析实际对象先查 OBJECT ENCODING、元素数量和最大元素长度，再结合命令复杂度判断是否会转换编码。〔N112〕

051 Redis 字典扩容为什么要渐进迁移，迁移期间怎样读写？

答案 大字典若一次把全部桶搬到新表，会长时间占用处理请求的线程。渐进 rehash 在一段过渡期保留旧表和新表，把迁移工作分摊到后续操作及后台推进步骤，直到旧表清空后再完成切换。过渡期间查找需要按规则检查两个表，新插入通常进入新表，删除和更新也必须处理记录可能位于哪一边，不能只查新表就返回不存在。迁移粒度可以降低单次停顿，但碰撞链、负载、内存和执行节奏仍会影响延迟；“渐进”不等于零额外开销。这里指 Redis 字典的实现，不应与业务把一个 Redis 分片迁到另一个节点的扩容流程混为一谈。〔N113〕

052 SDS 与 C 风格字符串相比怎样支持二进制值和高效增长？

答案 SDS 在数据前维护长度与容量等元信息，长度不靠扫描到第一个零字节计算，因此可以保存包含零字节的二进制内容；末尾保留终止符便于兼容部分 C 接口，但逻辑长度仍以元信息为准。追加时可检查剩余容量、预分配并更新长度，减少反复求长和某些重复分配。以 Redis 7.2 的 String 对象为例，合适的整数可能用 int 编码，小字符串可用 embstr 紧凑布局，较大或修改后的对象可采用 raw；这些是对象编码，不能等同于对外数据类型。阈值及转换受版本实现影响，应用应依据命令语义而不是内存布局使用 Redis。〔N114〕

053 Redis 的高吞吐来自哪些因素，多线程负责什么？

答案 典型 Redis 以内存数据和合适的数据结构减少存储等待，用事件循环及 I/O 多路复用同时管理连接，并在主要命令执行路径上串行访问键空间，降低常见锁竞争；网络往返、序列化和命令复杂度依然有成本。现代 Redis 会在网络 I/O、异步释放、后台任务等环节使用线程，RDB/AOF 任务还可能涉及子进程，所以“整个 Redis 永远只有一个线程”并不准确。增加 I/O 线程也不意味着任意慢命令可并行执行，长脚本、大范围操作和巨型返回值仍可能拖住其他请求。性能结论应明确版本、持久化配置、命令和数据大小。〔N115、N116〕

9.11 Redis 事务与生命周期

054 MULTI/EXEC、WATCH、Lua 与 Pipeline 在原子性上怎样区分？

答案 MULTI 把命令排队，EXEC 执行时命令组不会被其他客户端命令插入；但执行期某条命令出错不会自动回滚此前成功的命令。WATCH 在被监视键变化后使 EXEC 放弃执行，用于先读后改的乐观并发控制，失败需要重新读取并重试。Lua 脚本将条件判断与写入放在服务端一次执行，但运行期错误也不提供数据库式自动回滚，长脚本会阻塞其他工作。Pipeline 只是批量发送以减少往返，可以与事务结合，本身不保证隔离或原子性。多条命令修改库存等共享状态时，必须选明确的原子方案并处理失败语义，而不能依赖连接发送顺序。〔N117、N118〕

055 键过期与内存淘汰有什么区别，更新值会保留 TTL 吗？

答案 过期由每个键的失效时间驱动，内存淘汰则在内存压力下按 maxmemory 策略选择牺牲对象，未到期键也可能被淘汰。Redis 通常结合访问时检查和主动扫描清理过期键，逻辑失效不要求在同一时刻立即释放所有物理内存；过期键不能因为尚未清理就正常作为有效值返回。修改值是否保留 TTL 要按命令判断：INCR、HSET 等原地修改通常保留，普通 SET 覆盖值会清除原 TTL，需显式使用相应过期选项或 KEEPTTL。不能笼统声称“写入就续期”或“任何更新都不影响 TTL”，缓存续期策略应被明确编码。〔N119〕

9.12 Redis 复制、集群与缓存

056 Redis 全量复制和部分重同步怎样衔接，复制能保证强一致吗？

答案 连接正常时主节点向副本发送复制流，并跟踪复制标识和偏移。断线重连后，若复制历史及积压缓冲仍能覆盖缺失区间，可按偏移进行部分重同步；否则需全量同步快照，再补齐生成快照期间的新写入。积压缓冲容量和断线时长会影响是否频繁触发昂贵全量同步。默认复制是异步的，主节点已确认的写入在切换时仍可能丢失；WAIT 等确认机制可提高某些故障下的数据安全性，却不能自动把整个集群变为线性一致系统。应分别设计备份、持久化、复制确认和切换约束，并明确允许的丢失及读旧窗口。〔N120〕

057 Sentinel 怎样判定故障并切主，脑裂风险为何仍然存在？

答案 Sentinel 负责监控主从、通知客户端及协调故障转移，并不替代数据分片。单个 Sentinel 怀疑主节点不可达属于主观判断，达到配置的故障认定条件后再由 Sentinel 多数派授权执行转移；候选副本会按可用性、优先级和复制进度等规则选择，不能只理解为随意选一台。网络分区中旧主可能仍接受部分客户端写入，而另一侧已提升新主，异步复制使这些写入存在丢失风险。可用 min-replicas-to-write 等约束降低部分风险，但会牺牲可用性，也不是严格一致性证明。客户端必须支持发现新主、连接重建和幂等重试。〔N121〕

058 Redis Cluster 如何按槽路由，MOVED、ASK 和 CROSSSLOT 表示什么？

答案 Redis Cluster 把键映射到 16384 个哈希槽，由不同主节点负责，副本提供相应故障转移能力；客户端缓存槽到节点的映射，而不是每次随机询问全部节点。MOVED 表示应更新路由，迁槽期间 ASK 表示该次请求临时转向另一节点并按协议发送 ASKING，不能当成永久映射。涉及多个键的某些命令或脚本要求同槽，否则报 CROSSSLOT；hash tag 可让大括号中的共同部分参与槽计算，把相关键放在一起。这样也可能制造单槽热点，不能为方便将所有键放到同一个 tag。分片扩展容量与吞吐，但并不自动解决热键、跨槽原子操作和异步复制丢失窗口。〔N122〕

059 本地缓存和共享 Redis 缓存怎样组合而不过度牺牲一致性？

答案 本地缓存省去网络往返，适合小而热、允许短暂陈旧的数据，但每个进程各存一份，更新传播、容量和冷启动都要处理；共享 Redis 让实例复用缓存，并增加网络和集群可用性依赖。二级缓存可先查本地再查 Redis，未命中再回源，但每一层都应规定 TTL、失效通知和降级行为，否则旧值会多层滞留。Redis 客户端缓存跟踪可辅助发出失效信息，连接中断时仍须按协议清理或重新建立可信状态。权限、余额等要求强实时判断的数据不宜仅依据本地副本，命中率提升也必须与陈旧读容忍度、数据库保护和内存成本共同评估。〔N123、N124〕

060 为什么生产环境遍历键通常用 SCAN，游标返回意味着什么？

答案 KEYS 一次扫描并返回匹配键，大键空间会长时间占用执行线程且产生大响应；SCAN 用游标把工作拆成多次调用，降低单次阻塞，但它不是稳定快照。完整迭代通常从零游标开始，直到服务端再次返回零，COUNT 只是工作量提示，某一批返回空集合不代表遍历结束。同一个键可能重复出现，调用者应保证处理幂等；全程持续存在的元素有相应返回保证，而迭代过程中新增或删除的元素不能依赖精确覆盖。Cluster 还需遍历目标分片并考虑迁槽。扫描后删除或修改时应分批限速，避免把扫描的压力转移成大批同步写。〔N125〕

061 Cache Aside 中为什么先提交数据库再删缓存，仍有哪些竞态？

答案 数据库是事实源时，Cache Aside 由应用显式读缓存、未命中读库并回填，写入后通常提交数据库再删缓存；Read/Write Through 由缓存层代管读写，Write Back 先确认缓存并延迟落库，性能较高但故障时有数据丢失风险。这个顺序仍不是强一致事务：旧读请求可能先读到旧数据库值，在写事务提交并删缓存后才把旧值回填；删除失败也会让旧缓存继续存在。Singleflight 只合并同一键的并发 miss，不提供跨请求一致性。异步失效事件最好作为删除信号后回查数据库最新值；若消息携带新值，则需用原子版本比较拒绝较旧事件，避免乱序把缓存回滚。可结合 TTL、可靠重试、版本校验及按键串行重建减少窗口，不能把延迟双删当成证明。〔N124〕

062 Redis Stream 消费者宕机后，怎样找回未确认消息？

答案 使用消费组读取时，已投递但未确认的记录进入待处理条目列表，消费者成功完成业务后调用 XACK；ACK 只是移出待处理状态，不自动删除流中的消息。消费者宕机后，可检查 XPENDING，按空闲时间用 XCLAIM 或 XAUTOCLAIM 把消息移交给存活消费者，并让业务按消息 ID 幂等执行。不能只用新消息读取模式，就假设历史待处理项会自动重放。流裁剪、消息删除、组状态和重试上限需协调，过早删除可能使待处理记录失去内容；持久化与复制也决定故障后的数据保留边界，因此 Stream 不会免除可靠投递和对账设计。〔N126〕

063 Redis 突然变慢，应怎样区分服务端、网络与客户端排队？

答案 先把延迟拆成客户端连接池等待、网络往返、服务端排队和命令执行，结合慢日志、延迟监控、INFO、CPU、网络及系统指标定位；慢日志主要记录命令执行，不能覆盖所有网络和排队时间。检查复杂度较高的命令、长 Lua、大键与巨大返回值，再看 fork、写时复制、AOF 同步、内存碎片、换页或 CPU 争用。不要在故障时反复运行全量 KEYS 或输出全部数据加重负载。修复按证据选择拆键、限流、异步释放、批次大小和容量调整，并用相同负载比较尾延迟及错误率，而不是仅凭“内存数据库应该很快”排除服务端瓶颈。〔N116〕

第 10 章 系统设计与分布式

10.1 分布式基础与数据结构

001 CAP 理论如何用于分布式系统选型？

答案 CAP 讨论发生网络分区时的一致性与可用性：分区两侧无法可靠通信时，系统不能同时保证线性一致读写和所有请求都成功返回。CP 方案可能拒绝少数派写入，AP 方案可继续响应但允许暂时不一致；这是故障期间的选择，不是正常网络下永远只能二选一。先定义可用性、一致性语义和分区场景，再决定数据操作的策略。 AP 侧仍可按业务收敛数据，CP 侧也能在分区恢复后重新提供服务。

```
if (network_partition) {
    if (require_linearizable_reads_and_writes) reject_or_wait_on_minority(); // CP
    else if (require_every_partition_to_respond) accept_writes_and_reconcile_later(); // AP
}
// CAP 约束故障分区期间的保证，不意味着正常网络下永远只能选 C 或 A。
define_consistency_semantics_availability_slo_and_partition_cases_first();
// AP 需要冲突解决/收敛；CP 多数派恢复后可继续服务。
```

002 什么是 Bloom Filter？如何选择参数？

答案 Bloom Filter 用 m 位数组和 k 个哈希标记集合元素；查询时只要有一位为 0 就能确定不存在，全为 1 则只是可能存在。标准结构在正确使用下无假阴性但有假阳性，也不支持直接删除。预估 n 个元素和误判率 p 后可用 m≈-n ln(p)/(ln2)^2 求位数、k≈(m/n)ln2 求哈希数，并为超容量留余量。当实测误判率偏高时，可扩容过滤器或做分层过滤，而不是把命中当真。

```
#include <cmath>
#include <cstddef>
#include <cstdint>
#include <vector>
// n > 0, 0 < p < 1; 容量超预估会提高误判率
std::size_t m = static_cast<std::size_t>(std::ceil(-n * std::log(p) / std::pow(std::log(2.0), 2)));
std::size_t k = static_cast<std::size_t>(std::round((static_cast<double>(m) / n) * std::log(2.0)));
if (k == 0) k = 1; // 至少使用一个哈希位置
std::vector<std::uint8_t> bits((m + 7) / 8, 0);
for (auto h : k_hashes(key)) set_bit(bits, h % m); // 插入只置位
for (auto h : k_hashes(query)) if (!test_bit(bits, h % m)) return DefinitelyAbsent;
return MaybePresent; // 无假阴性、有假阳性；命中仍查真实存储
// 标准结构不能单键删除；监测误判率后扩容/分层
```

003 什么是一致性哈希？

答案 一致性哈希将键和节点映射到环上，键顺时针归属后继节点；增删节点主要影响相邻区间，减少全量搬迁。每个物理节点设置多个虚拟节点可平衡区间大小，但节点异构时应按权重分配。它只解决映射与部分迁移，热点键、故障复制和数据再均衡仍需单独设计。虚拟节点数和权重会影响负载方差，迁移时还需做限速与双读切换。重新映射后应渐进搬数据，并在迁移窗口设置读写转发或版本校验。

```
for (Node node : nodes) for (int v = 0; v < virtual_nodes(node); ++v)
    ring[hash(node.id, v)] = node; // 虚拟节点帮助均衡区间，也可按容量加权
auto it = ring.lower_bound(hash(key)); if (it == ring.end()) it = ring.begin();
owner = it->second; // 键映射到顺时针后继，增删节点主要移动相邻区间
// 变更节点后限速迁移，可双读/转发并校验版本后切流
// 一致性哈希不解决热点、复制和故障恢复；虚拟节点数需按负载调优
// 监控分片负载方差，不能只凭“节点均匀”假设均衡
```

004 分布式 ID 生成器要满足什么要求？

答案 分布式 ID 通常要求唯一、可用、吞吐足够，并按业务决定是否要大致有序、紧凑或不可预测。数据库号段可在事务里原子预留一段 [start,end)，服务在本地缓存使用，并以双 Buffer 后台预取下一段；进程崩溃可能留下未用 ID 空洞，号段耗尽或预取失败要明确定义等待、拒绝或降级策略。分片键还应与常见查询路由相配：若按买家 ID 分片却只拿订单 ID 查详情，需要路由映射、冗余分片字段或把分片信息编码进主键，否则会广播查询。Snowflake 类 ID 把时间、节点号和序列组合，必须确保节点号不冲突并处理时钟回拨和单毫秒序列耗尽。UUID 易生成且分布式方便，但较长、按时间排序能力有限；严格全局单调通常要以协调和可用性为代价。

```
id = database_allocated_range(); // 简单；需分配服务、缓存段和故障恢复
id = UUID_random(); // 易分布式生成但较长，通常不按生成时间顺序聚集
id = (timestamp_ms << (node_bits + seq_bits)) | (node_id << seq_bits) | sequence;
if (node_id_conflicts) reject_configuration();
if (clock_moves_backwards || sequence_exhausted) wait_or_reject_or_use_new_epoch();
// 明确唯一/大致有序/紧凑/不可预测等要求；严格全局单调需协调，降低可用性。
```

10.2 数据处理与在线服务设计

005 如何从超大文件中找 Top K？

答案 数据可装入内存时用大小 K 的最小堆：前 K 个先入堆，之后只替换堆顶更小的候选，时间 O(N log K)、空间 O(K)。超大文件可按稳定哈希分桶，保证相同键落在同桶，再逐桶求候选并归并；若 Top K 是频率统计，还要先明确是否需全局聚合。近似算法可省内存，但应给出误差和置信边界。局部各取 K 个候选再做堆归并可得到全局 K 个，但需保证分桶可完备划分。

```
std::priority_queue<int, std::vector<int>, std::greater<int>> top_k; // 最小堆
for (value : stream) { if (top_k.size() < K) top_k.push(value);
    else if (value > top_k.top()) { top_k.pop(); top_k.push(value); } }
// 内存可容纳时 O(N log K)、O(K)；堆顶是候选里最小项。
// 超大文件先稳定哈希分桶相同 key，再逐桶统计/取候选并归并。
// 频率 Top K 要定义跨桶全局计数；近似算法应给误差/置信度。
```

006 如何处理分布式事务？TCC 与 Saga 有何区别?

答案 分布式事务方案要根据一致性、可用性、锁持有时间和业务补偿能力取舍。XA/2PC 通过协调者准备后统一提交，但同步阻塞和协调节点故障会影响可用性；Seata AT 通过数据源代理和 undo log 降低业务侵入，但受数据库、SQL 和版本支持边界约束。TCC 把一次业务拆为 Try 预留、Confirm 确认和 Cancel 释放，能显式控制资源但要求各阶段幂等，并处理空回滚、悬挂和超时重试。Saga 将多个本地事务串成流程，失败时执行补偿动作；补偿是新的业务操作，不一定能精确还原，且中间结果可能被其他事务看到。能避免跨服务强事务时优先本地事务加 Outbox，按隔离和补偿要求选择。

```
// TCC: Try 预留；各阶段按 transaction_id 幂等，并记录事务状态
if (!inventory.try_reserve(txn_id)) return Failed;
if (payment.try_charge(txn_id)) inventory.confirm(txn_id);
else { inventory.cancel_if_reserved(txn_id); payment.cancel_if_charged(txn_id); }
// 空回滚：Try 未发生也可能收到 Cancel；悬挂：迟到 Try 不能复活已取消事务
// Saga 则串本地事务，失败执行逆序业务补偿；补偿不保证精确还原
// 能避免跨服务强事务时，用本地事务 + Outbox 降低协调复杂度
```

007 如何实现业务幂等?

答案 为每个逻辑请求生成唯一业务键，并在数据库唯一约束或原子条件更新下记录处理状态和结果；同键重试时返回已保存结果或正在处理状态。仅先查询再写入有并发竞态，两请求可能同时通过检查。Kafka PID/sequence 或 Broker 去重能减少生产重试造成的重复，但通常受 producer、topic/partition 等协议范围限制；ACK 丢失仍会引发重投，不能替代消费者按全局业务 ID 原子幂等。MySQL 可用 INSERT ... ON DUPLICATE KEY UPDATE；8.0.19 起可给待插入行取别名，VALUES(col) 引用方式从 8.0.20 起弃用。不要把 INSERT IGNORE 当作只忽略重复键，它也可能把截断等数据问题降为 warning；唯一索引允许多行包含 NULL，若缺值也须视为冲突应声明 NOT NULL。还需规定幂等键作用域、过期时间、失败重试和同键不同参数的冲突行为。

```
BEGIN;
INSERT INTO request_dedup(request_id, status, parameter_hash)
VALUES (?, 'processing', ?); // 唯一键原子抢占，避免“先查再写”竞态
-- duplicate key 时读取已保存状态/结果，不重复执行副作用。
UPDATE orders SET status = 'created' WHERE id = ?;
UPDATE request_dedup SET status = 'done', result = ? WHERE request_id = ?;
COMMIT;
-- 同 key 不同参数应拒绝；处理中超时需恢复/重试，键作用域与 TTL 要明确
```

008 如何检索 TB 级日志中的关键词?

答案 先把 TB 级日志按时间、服务、租户或分区缩小扫描范围；全文检索可把文本分析成 term，倒排索引中的 term dictionary 定位词项，postings 列表记录匹配文档 ID，必要时还含词频和位置，再合并/求交候选列表并评分。Term Index 用于导航词典，Stored Fields 按文档 ID 取回原始字段，Doc Values 以适合排序和聚合的布局读取字段。Lucene/Elasticsearch 的 segment 通常不可变，新写入先形成新 segment，后台 merge 合并并回收已删除文档，因此还要考虑合并期间的磁盘与 CPU 成本。压缩和列式存储可降低 I/O，但解压、索引维护也耗资源；为防查询拖垮集群，应限制时间范围、并发、返回量和敏感字段访问。

```
partitions = narrow_by_time_service_tenant_and_region(query);
if (frequent_full_text_search) candidates = inverted_index.lookup(keyword);
else candidates = parallel_scan_compressed_partitions(partitions, keyword);
// 列式/压缩可减少 I/O，但有解压和索引维护成本。
apply_time_range_concurrency_result_limit_and_field_acl();
// 防止无范围查询拖垮集群；对敏感字段做访问控制和审计。
```

009 如何用 Bloom Filter 判断海量键是否可能存在?

答案 对每个键做多个哈希并置位，查询时若任何位为 0 就可以判定不存在；位全为 1 时只能说可能存在，因为其他键可能置过这些位。插入量和目标误判率决定位数组大小与哈希个数，超出容量后误判率增加。标准 Bloom Filter 不支持删除，需删除时可考虑计数变体并权衡额外内存。 Bloom Filter 适合在访问昂贵存储前做快速负向过滤，正向命中仍要查真实数据。

```
for (key : indexed_keys) bloom.insert(hash_positions(key));
if (!bloom.maybe_contains(query)) return NotFound; // 任意一位为 0 可确定不存在
record = primary_storage.lookup(query); // 位全为 1 也只是可能存在
if (measured_false_positive_rate > target) resize_or_layer_filter();
// 正确构造下标准 Bloom 无假阴性，但有假阳性，命中仍查真实数据
// 标准 Bloom 不支持删除；Counting Bloom 支持删除但内存更高
// n 和目标误判率 p 决定 m/k；超容量后需扩容或分层
```

010 如何设计 O(1) 的 LRU 缓存?

答案 哈希表保存 key 到双向链表节点的映射，链表头代表最近使用、尾部代表最久未用；命中时摘下节点移到头部，淘汰时删尾节点。插入、查找和删除平均 O(1)，容量满时先移除尾节点并清理哈希项。并发实现需保护两种结构的一致性，分片锁会改变全局严格 LRU 语义。 若每次命中都更新链表头，高并发下会形成写竞争，可按分片近似维护。

```
std::list<std::pair<Key, Value>> lru; // 头最近，尾最久未用
std::unordered_map<Key, decltype(lru.begin())> index;
if (index.contains(key)) { lru.splice(lru.begin(), lru, index[key]); return index[key]->second; }
if (lru.size() == capacity) { index.erase(lru.back().first); lru.pop_back(); }
lru.emplace_front(key, value); index[key] = lru.begin(); // 查/移/淘汰平均 O(1)
// 并发需保护 map 与 list 原子一致；分片锁会变为局部/近似 LRU。
```

011 如何设计分布式计数器?

答案 先明确计数是否必须精确、读可否延迟，以及更新是否允许丢失或重复；高写入时可按键分片累加，再在读取或后台聚合时汇总。每个分片用原子更新或可靠消息，并以事件 ID 去重，才能避免重试重复加一。还要定义热点分片、持久化恢复、重置周期和跨分片快照一致性。 热点计数可拆成多个随机后缀分桶，读取时求和并接受短暂汇总延迟。

```
shard = hash(request_id) % shard_count; // 分散热点写入
shards[shard].apply_once(request_id, +1); // 单分片原子去重并递增，重试不重复计数
if (read_total) sum = aggregate_all_shards(); // 汇总可能有短暂延迟
persist_shard_checkpoints_and_dedup_ids(); // 恢复时既不漏计也不重计
// 明确计数是否精确、读延迟与跨分片快照一致性
// 热点可拆多个随机后缀分桶；扩缩分片需迁移去重状态/计数
```

012 如何设计实时排行榜?

答案 维护成员分数和可排序索引，Redis ZSet 可用 ZADD 更新分数、ZREVRANK 查询名次，适合常见单分片排名。高吞吐时按用户或榜单分区写入，再周期性汇总各分区候选；全局精确排名需要额外协调。设计前要定同分规则、窗口周期、迟到更新、缓存持久化和读写延迟目标。 榜单规模和查询窗口会影响排序结构选择，滑动窗口榜单还要淘汰旧分数。

```
ZADD(board, score, user_id); // 分数与排序索引一起更新
rank = ZREVRANK(board, user_id); top = ZREVRANGE(board, 0, K - 1);
// 先定义同分排序、迟到更新、窗口过期和读延迟目标
if (single_shard_capacity_exceeded) write_to_partitioned_boards_then_merge_candidates();
// 分区候选汇总可扩展；全局精确 rank 仍需协调
if (sliding_window_board) remove_expired_scores_or_events();
// 还要考虑持久化/重启恢复、排行榜规模和查询频率
```

10.3 高并发、消息与容错

013 如何设计高并发秒杀系统?

答案 静态商品信息由 CDN 承载，入口先鉴权、限流和排队，库存预热后通过原子条件扣减避免超卖。成功请求再异步下单、用消息队列削峰，消费者按请求 ID 幂等处理；数据库更新仍用库存条件和订单唯一约束兜底。需压测热点键与队列积压，并提供失败回滚、超时订单释放和库存对账。 令牌校验和业务资格检查应尽量在入口前完成，减少无效请求争抢库存。

```
authenticate_and_check_eligibility_before_hot_inventory(); // 过滤无效流量
if (!rate_limiter.try_acquire(user)) enqueue_or_reject();
if (atomic_decrement_if_stock_positive(sku, request_id)) publish_order_event();
// 成功后异步削峰；消费者按 request_id 幂等，订单键有唯一约束。
if (order_times_out) release_reserved_stock_idempotently();
// DB 条件更新兜底防超卖；压测热键/积压并做库存对账。
```

014 常见负载均衡算法有哪些?

答案 轮询简单，适合实例能力近似的无状态流量；加权轮询可按容量分配，最少连接适用于请求时长差异明显的场景。一条 HTTP/2 等复用连接可能同时承载多个在途请求，因此连接数未必代表实际负载；可观察活跃请求数和延迟，但请求成本不同仍会造成估计误差。一致性哈希有利于会话或缓存局部性，随机/延迟感知策略可减轻热点。负载均衡还要结合健康检查、连接池、重试放大和跨区成本，避免把故障实例继续选中。健康检查应区分进程存活与业务就绪，避免恢复中实例过早接收大流量。

```
if (instances_have_similar_capacity) choose_round_robin();
else if (capacity_weights_known) choose_weighted_round_robin();
else if (request_durations_vary) choose_least_connections();
if (session_or_cache_locality_matters) choose_consistent_hash();
exclude_if_unhealthy_or_not_ready(); // 活着不等于业务就绪
// 同时控制重试放大、连接池和跨区成本；策略按实际延迟/负载验证。
```

015 微服务相对单体架构有哪些权衡?

答案 单体部署简单、进程内调用快、事务边界清楚，适合早期团队快速交付；微服务让团队可按边界独立发布、扩缩容，但引入网络失败、接口治理、分布式数据和观测成本。拆分应基于领域与团队所有权，而非按技术层拆很多小服务。需要配套超时、有限重试、熔断、版本兼容和故障定位能力。 服务拆分还会影响数据一致性和本地调试，拆分前先定义清晰的服务边界。

```
if (team_is_small_and_domain_boundaries_unclear) start_with_modular_monolith();
// 单体进程内调用快、事务清楚、部署简单。
if (service_boundary_owned_by_team && independent_release_or_scale_needed) split_service();
// 微服务引入网络超时、部分失败、数据一致性、观测/发布治理成本。
add_deadlines_bounded_retries_circuit_breakers_and_version_compatibility();
// 按领域与团队所有权拆分，避免按技术层切成大量小服务。
```

016 消息队列如何尽量避免消息丢失?

答案 生产端等待 Broker 确认并启用合适的持久化/副本策略，避免消息只留在客户端缓冲或单个易失节点。消费者应先完成可恢复的业务处理，再提交位点或 ACK；否则先确认后崩溃会丢业务。可在业务消息内维护 producer 与 partition 维度的递增序号，消费侧检查序号缺口来发现丢失；多个生产者要分别维护序列，不能把不同来源的编号混为一条连续流。至少一次投递会重发，所以消费者按消息 ID 幂等、设置重试和死信队列，并监控积压与复制状态。确认和 ACK 的先后关系决定故障时是可能重复还是可能丢失，需按目标选取。

```
producer.send(message); await broker_durable_replica_ack(); // 持久化确认后再报成功
consumer_result = apply_business_idempotently(message.id, message.payload);
if (consumer_result.durable) consumer.ack_or_commit_offset(message);
// 先 ACK 再持久处理会丢业务；先处理后 ACK 崩溃会重放
retry_with_limit_and_dead_letter_after_policy(); monitor_lag_and_replica_health();
// 按消息 ID 幂等，监控复制健康/积压；Broker ACK 语义要核对配置
// 保证边界还依赖副本数、刷盘策略和故障类型
```

017 限流常用算法有哪些?

答案 固定窗口以简单计数实现，但窗口切换边界可能让短时间请求量接近双倍；滑动日志或计数窗口更平滑但状态更多。令牌桶按速率补充令牌，允许桶容量范围内的突发；漏桶以较稳定速率放行。部署可在网关、独立限流服务或进程内完成：中心化便于共享额度但增加网络依赖，进程内延迟低但多副本总额度需协调。共享令牌桶的读-改-写必须原子执行，热键可按用户/资源分片并定义跨片额度。规则应带版本并可靠传播；限流服务故障时按业务风险选择 fail-open 或 fail-closed，关键写路径通常不能默默放行。

```
#include <algorithm>
// 固定窗口易实现，但边界前后短时间内可接近两倍限额
if (fixed_window_count(key, now) < limit) allow();
// 令牌桶按 rate 补充，最多 burst_capacity，允许短突发
tokens = std::min(capacity, tokens + elapsed * rate);
if (tokens >= 1) { tokens -= 1; allow(); }
// 滑动日志更精确但状态多；漏桶适合较匀速处理后台任务
// 分布式还要定义原子更新、公平性和状态丢失时的降级
```

018 高可用系统通常从哪些方面设计?

答案 高可用从消除单点开始：跨进程/机器冗余、健康检查、故障切换和可恢复的数据备份。先定义服务可用性 SLO 与停机预算，冗余、超时、重试和降级才有可验收目标；例如全年 99.9% 可用性约允许 8.76 小时停机，99.99% 约 52.56 分钟。调用链配置有限超时、带抖动退避的重试、熔断与降级，避免局部故障扩散；对关键依赖明确 RTO/RPO。监控告警、故障演练和回滚验证机制是否真能在目标时间内恢复。备份只有通过定期恢复演练验证，才能证明满足恢复时间和数据点目标。

```
replicate_across_processes_and_failure_domains(); health_check_readiness_and_fail_over();
call_dependency_with_deadline(); retry_with_bounded_jittered_backoff();
if (failure_rate_high) circuit_break_and_serve_degraded_result();
backup_data(); periodically_restore_test_and_measure_RTO_RPO();
// RTO 是恢复所需时间，RPO 是可接受的数据丢失窗口
// 监控告警、回滚和故障演练验证冗余可恢复，而非只看副本数
// 备份需定期实测恢复，才能证明目标满足
```

019 高并发 IM 或消息推送服务如何拆分?

答案 连接网关维护长连接和心跳，认证服务管理身份，路由层定位用户所在网关，消息服务负责持久化、投递、ACK 和离线存储。按会话或用户分片并通过消息队列解耦突发流量，客户端重连后按序号补拉未确认消息。评论类写入还要在发布前做权限与内容审核，可同步拦截明显风险、异步复核边界内容。直播广播若用简单 Pub/Sub 让每台实时节点处理所有房间消息，会浪费带宽和 CPU；可按 liveVideoId 分区，并把房间路由到负责节点。还要处理多端在线、重复投递、消息顺序、离线 TTL 和连接状态过期。网关故障后客户端通常要指数退避重连，避免整群同时冲击认证和路由服务。

```
gateway.accept_long_connection(); authenticate_and_heartbeat();
route = presence_service.locate_user_gateway(user_id);
message_store.persist_with_sequence_and_ttl(message); queue.deliver(route, message);
if (client_ack(message.id)) mark_delivered_idempotently();
if (reconnect) backoff_with_jitter(); fetch_messages_after(last_acked_sequence);
// 设计多端、离线存储、重复投递、顺序和路由状态过期语义。
```

020 高并发日志系统要考虑哪些模块?

答案 日志先经过异步有界队列，由后台批量格式化并写文件或远端，降低业务线程的 I/O 等待。队列满时要明确阻塞、丢低级别日志或采样策略；配置轮转、刷盘和进程退出时的有序 flush。日志记录请求 ID、时间和线程信息，但应避免敏感数据，并监测队列深度和丢弃数。 日志队列采用有界容量，才能在异常高峰时限制内存增长并形成可观测背压。

```
if (pending.size() == capacity) apply_block_drop_low_level_or_sample_policy(record);
else pending.push(record); // 业务线程只排入有界异步队列
background_worker.batch_format_and_write_file_or_remote();
rotate_logs_and_flush_by_durability_policy();
on_shutdown_flush_then_join_worker();
// 记录 request ID/时间/线程但脱敏；监控队列深度、丢弃量和写入错误。
```

021 高并发网络服务器通常如何设计?

答案 事件循环用非阻塞 socket 处理连接建立、读写和定时器，工作线程池执行计算密集或可能阻塞的业务，避免卡住 I/O 线程。连接数、待处理任务、缓冲区和单请求执行时间都要设上限，并实现背压、超时和优雅关闭。通过压测与 profiling 找到瓶颈后再决定线程数、进程模型和 CPU 亲和性。 事件循环不应执行同步磁盘访问或无界计算，否则一个任务会阻塞同循环其他连接。

```
while (running) { auto ready = poller.wait(deadline);
    for (event : ready) handle_nonblocking_socket(event); // I/O 线程保持轻量
    submit_cpu_or_blocking_work_to_bounded_pool(); // 防止磁盘/业务阻塞事件循环
    apply_backpressure_to_connection_and_task_queues(); expire_slow_requests(); }
// 限连接、缓冲与执行时长；优雅关闭时停止接入、排空任务并回收线程。
// 压测/profiling 后再调线程数、进程模型与 CPU 亲和性。
```

10.4 共识与消息一致性

022 Raft 共识算法由哪些核心部分组成？多数派有什么作用?

答案 Raft 将复制状态机共识分解为领导者选举、日志复制和安全性规则；领导者接收命令并复制日志，通常复制到多数派后条目才能提交并应用。多数派相交使新领导者能继承已提交条目；网络分区中失去多数派的一侧不能提交新写入。多数派提供安全提交条件，不等于每个节点时刻拥有相同最新状态。 日志提交受任期和前序日志匹配规则约束，不能只凭副本数就认定任意条目已提交。〔S11〕

```
leader = elect_by_term_and_timeout(); // 领导者接收命令
leader.append_log(entry); replicate_to_followers(entry);
if (replicated_to_majority && raft_term_and_prefix_rules_hold) commit(entry);
apply_committed_entries_to_state_machine();
// 多数派交集让新领导者能继承已提交日志；少数派不能提交新写。
// 多数不代表每个节点总有最新状态，提交还受任期/日志匹配规则约束。
```

023 Transactional Outbox 如何解决数据库更新与发消息之间的双写问题?

答案 业务更新与待发事件写入同一数据库事务，避免业务表提交成功但进程在发消息前崩溃的双写窗口。后台轮询或 CDC 读取 Outbox 并投递消息，发送失败可重试，因此同一事件可能被重复发送。消费者需按事件 ID 幂等处理；还要处理 Outbox 清理、发布顺序、积压监控和数据库到 Broker 的延迟。〔S13〕

```
BEGIN;
UPDATE orders SET status = 'paid' WHERE id = ?;
INSERT INTO outbox(event_id, topic, payload) VALUES (?, 'OrderPaid', ?);
COMMIT; -- 业务状态与事件原子提交，避免 DB 成功但进程发消息前崩溃
publisher.read_unpublished_rows_and_send(); mark_published_after_broker_ack();
-- 发送失败会重试，消费者按 event_id 幂等；监控积压、顺序和清理。
```

024 消息系统里的“恰好一次”语义通常有什么边界？

答案 最多一次可能丢消息但不重投，至少一次通过重试提高送达机会但可能重复，消费者需要幂等；“恰好一次”要说明是生产者写入日志、Broker 内消费，还是连外部数据库副作用也包括。Kafka 幂等生产者可避免协议范围内的重试重复记录，事务可把多分区输出和消费位点原子提交，并由 read_committed 消费已提交数据。若业务还写外部数据库，仍需事务性 Sink、Outbox 或幂等键，不能由 Kafka 单独保证端到端只执行一次。〔S12〕

```
producer.enable_idempotence(); // 去除生产重试带来的同分区重复记录
begin_broker_transaction(); write_output_records(); commit_consumed_offsets_atomically();
consumer.read_committed(); // 只读已提交事务数据
// 若还写外部数据库，Kafka 事务不能把 DB 副作用纳入同一原子提交。
use_transactional_sink_or_outbox_or_idempotency_key_for_external_effect();
// 先说清“一次”覆盖生产、Broker 内消费，还是端到端业务副作用。
```

025 分布式系统中的熔断器是什么？Closed、Open 和 Half-Open 状态如何工作？

答案 熔断器统计依赖调用结果：Closed 正常转发并累计失败率，达到阈值后进入 Open，快速拒绝请求以给下游恢复空间。阈值应结合下游 SLA、历史错误率和延迟分位数设定，并要求在统计窗口持续超限以过滤单次尖峰。冷却后进入 Half-Open，只放少量探测请求；成功后逐步恢复流量，失败则重新打开，避免冷却结束一次性放开造成反复过载。熔断不是重试：重试增加调用次数，熔断在故障持续时减少调用。降级应按业务价值选择明确兜底，例如保留核心读取、暂停非核心写，或缓存命中时继续服务而 miss 时跳过数据库慢路径；不能把未完成的写伪造成成功。〔S15〕

```
switch (state) { // 每次请求只执行当前状态的一条路径
case Closed: forward_call_and_record_result();
    if (failure_rate() > threshold) state = Open; break;
case Open: fail_fast_or_degrade();
    if (now >= cooldown_end) state = HalfOpen; break;
case HalfOpen:
    if (allow_limited_probe()) state = probe_succeeds() ? Closed : Open;
    break;
}
// Open 降低持续故障时的下游调用；Half-Open 限量探测恢复情况
// 熔断不是重试；失败可延长冷却，并配合超时、降级和指标告警
```

10.5 分布式协调与服务治理

026 怎样实现可重入的业务锁，并处理持有者失联？

答案 可重入锁要按锁名保存全局唯一的持有者标识和重入计数：空闲时设置 owner 并置一，同一 owner 再入则递增，释放时只有 owner 可递减，归零才释放。数据库可用固定锁记录和短事务 SELECT FOR UPDATE 串行修改；Redis 可用 Hash 与 Lua 原子判断。数据库事务结束释放的是行锁，持久记录表示的业务锁仍存在，两者不可混淆。还需租约、续租、故障恢复和下游 fencing 校验，避免失联的旧持有者恢复后继续写入。ZooKeeper/etcd 等协调服务也可构建锁，但会话过期后的旧客户端同样不能凭本地“仍持锁”状态绕过资源校验。〔N127、N128、N129〕

10.6 系统设计补充

027 流量增加十倍时，怎样估算瓶颈并验证扩容方案？

答案 不能给单 Redis 或单 MySQL 一个脱离条件的固定 QPS。先确定请求分布、读写比例、命中率、数据大小、连接数、持久化和延迟目标，再用接近生产的数据压测，找出 CPU、网络、锁、磁盘和下游哪个先饱和。用吞吐与响应时间估计同时在途请求，保留故障与突发余量；流量过载时先做准入、限流、降级和有界排队，避免无界线程或重试把系统压垮。随后针对瓶颈选择缓存、批处理、只读副本、分片或异步化，逐层验证扩容后新瓶颈及尾延迟。容量规划应包含切换、积压恢复和成本，不能只看短时成功吞吐。〔N130〕

028 跨服务接口超时如何沿调用链排查，而不把所有问题归因于网络？

答案 先为请求生成 trace/span 标识，记录入口排队、DNS/连接、序列化、服务处理、下游调用和响应写回的耗时及状态；把客户端 deadline 与各子调用预算关联起来。再按时间线比较成功、超时、重试和取消请求，核对 CPU、锁、线程池、连接池、数据库等待、消息积压及网络错误。一个客户端超时不说明服务端没有执行，重试可能令副作用执行多次，因此写操作要结合幂等键和状态查询。指标适合发现总体和尾延迟趋势，日志提供单事件细节，trace 串起跨服务因果，三者须统一采样和脱敏。修复应先止血限流或降级，再根据证据调整资源、依赖和超时，不用单纯延长 timeout 掩盖瓶颈。〔N148、N146、N130〕

10.7 消息队列模型与运行

029 消息队列如何实现异步解耦，自研时至少需要哪些状态？

答案 生产者把工作或事件交给中间层，消费者按自己的进度处理，可在一定范围内隔离故障、削平突发并支持发布订阅；同一个消费组内分工与不同订阅组各收一份是两种不同语义。队列只能暂存峰值，长期生产速率超过消费能力仍会耗尽容量。自研持久队列至少需要定义消息标识、主题或队列、存储与恢复、确认及重投、消费进度、保留清理和配额，再决定复制及分区方式。引入 MQ 也增加端到端延迟、重复投递和运维复杂度，异步处理未必适合要求立即得到确定结果的接口；先定义失败语义，才能谈解耦收益。〔N131、N132、N133〕

030 Kafka、RocketMQ 与 RabbitMQ 应按哪些需求比较？

答案 Kafka 以可保留、可重放的分区日志及流处理生态见长；RocketMQ 提供面向业务消息的队列、顺序、定时和事务消息等能力；RabbitMQ 的交换机路由、消费确认和工作队列模型适合灵活任务分发。它们都能用于多个场景，不能把选型简化成“某个一定更快”。应确认吞吐及尾延迟、消息大小、保留周期、路由和重放、顺序范围、消费组规模、事务边界、部署成本与团队经验，再按目标版本压测。所谓可靠性还取决于复制、持久化、确认和业务幂等配置；产品支持事务不等于可以原子覆盖任意外部数据库副作用。〔N133、N131、N132〕

031 如何按订单维持消息顺序，而不把整个系统串行化？

答案 先界定需要顺序的业务键，例如同一订单，而不是追求全局消息有序。将同键消息路由到同一分区、队列或受支持的消息组，生产端保持同键发送关系，消费端按该范围串行处理并在成功后推进进度；不同键仍可并行。Kafka 的分区顺序和 RocketMQ 的 FIFO 消息组都有各自范围及前提，不能把跨分区到达时间当成业务顺序。重试、切换和多生产者可能制造重复、迟到或断序，因此业务状态机还应校验版本、事件序号与合法迁移；遇到毒消息要明确等待、隔离还是补偿，跳过消息可能破坏后续因果关系。〔N133、N134〕

032 消息积压时怎样定位生产、消费或依赖瓶颈？

答案 先观察积压条数、最老消息年龄、生产与处理速率、失败重试、消费者存活及分区分布，区分突发峰值、某个热分区和整体消费能力不足。Kafka 的 lag 应结合提交进度与处理耗时解释，不能只看队列总长度就断言业务停顿。消费者慢可能来自下游数据库、外部接口、串行锁或频繁重平衡，盲目加线程会放大压力；扩容也受分区数和顺序约束。短期可限产、隔离失败消息并保护关键依赖，长期优化批处理和瓶颈资源。恢复后验证去重、业务延迟和保留周期，不能直接丢弃积压来让监控归零。〔N135、N136〕

033 消费失败应怎样退避重试，并让死信可以恢复处理？

答案 把可重试的临时故障与永久的格式或业务错误区分，使用有次数及总时长上限的退避重试，记录消息 ID、失败原因和尝试次数，避免立即反复重投形成热循环。达到上限后转到死信队列或隔离存储，告警并提供经过修复的审计回放流程；消费者仍要幂等。RabbitMQ 可通过拒绝且不重新入队、TTL 或队列限制等机制触发死信交换机路由，死信不是“消息自动安全永久保存”的保证。TTL 配合死信可构建某些延迟处理，但顺序、到期精度及重新发布安全性需按队列类型和版本确认，关键状态最终还要对账。〔N136、N137〕

034 延迟消息关闭未支付订单时，怎样避免误关已付款订单？

答案 延迟任务可用数据库定期扫描，简单且易恢复但会增加轮询和延迟；进程内 DelayQueue 或时间轮开销低、精度好，却需处理进程崩溃后任务丢失和重建。Redis ZSet 可按到期时间拉取并通过租约领取，但要有持久化、续租、重试和扫表恢复；Key 过期通知不可靠，不能作为唯一触发源。MQ 延迟消息便于异步扩展，TTL+死信队列可组成补偿链路，但同队列混放不同 TTL 可能造成队头阻塞。无论选哪种，延迟消息只负责在目标时间附近触发检查，消费者须验证订单状态与截止时间并用条件更新原子迁移，支付回调和关单共享状态机，库存释放按订单键幂等；同时保留数据库扫描对账。RocketMQ 不同大版本的定时能力、精度与范围不同，不能视作硬实时定时器。〔N138、N127〕

10.8 RocketMQ、Kafka 与 RabbitMQ

035 RocketMQ 的半消息与事务回查如何连接本地事务和消息可见性？

答案 生产者先发送暂不向消费者投递的半消息，Broker 保存成功后执行业务本地事务，再按本地结果通知提交或回滚；提交才使消息具备投递条件。若结果通知丢失或状态未知，Broker 可回查生产者，生产者须根据可靠保存的业务事务状态回答，而不是凭内存变量猜测。这个协议帮助源端本地事务与发消息最终达成一致，不把消费者数据库自动纳入同一个原子事务。事务状态记录、回查服务可用性、重试及超时限制必须设计完整，消费者仍需幂等和失败恢复；不能把半消息机制宣传为任意多服务的强隔离事务。〔N139〕

036 RocketMQ 的 CommitLog、ConsumeQueue 和索引各承担什么角色？

答案 以经典 RocketMQ 存储为例，CommitLog 顺序保存完整消息，多个主题与队列可共享该日志；ConsumeQueue 为逻辑队列建立紧凑条目，指向 CommitLog 位置并包含必要的长度、过滤等信息，便于按消费位点读取；IndexFile 支持按消息键等条件定位，不是普通顺序消费的唯一入口。Kafka 则围绕各分区的日志段及相关索引组织数据，两者不能只按“都有 Broker”就认为布局相同。消费索引的重建、文件清理和复制模式会影响恢复及可用性，应按部署版本检查；RocketMQ 5.x 的代理与 Controller 等组件也不能完全套用旧版主从图。〔N140、N131、N133〕

037 RocketMQ NameServer 故障后，现有连接与新路由分别会怎样？

答案 NameServer 提供 Broker 注册与主题路由发现，客户端据此定位消息队列和 Broker，实际消息传输不必逐条经过 NameServer。经典架构可部署多个相对独立的 NameServer，Broker 向它们注册，客户端配置多个地址并缓存路由。短时全部不可用时，已有缓存且 Broker 未变化的请求可能继续工作，但新客户端启动、路由刷新、Broker 变化后的访问会受影响，所以“挂了完全没事”与“所有消息立刻中断”都过于绝对。应分别监控注册、路由获取和真实发送消费，设计客户端重试及 Broker 故障策略，并按版本区分 NameServer、Controller 与 Proxy 的职责。〔N140、N131〕

038 消费组如何决定消息分工，增加消费者何时不再提高吞吐？

答案 在经典 Kafka 消费组的分区分配模型中，一个分区同一时刻分配给组内一个消费者，不同组分别维护消费进度；十个分区的主题，即使组内有更多消费者，多余实例也可能空闲。一个消费者可以处理多个分区，但把分区内任务任意并行会破坏完成顺序及位点提交安全。RocketMQ 的集群消费同样强调组内分工，经典客户端的广播消费则让每个实例接收各自副本，其 API 支持须按客户端版本确认。增加实例前要判断分区、热键和下游容量限制，不能把消费线程数、客户端实例数和独立业务订阅组数混为一谈。〔N133、N131〕

039 Kafka 怎样通过分区日志、批处理和拉取消费提高吞吐？

答案 主题由多个分区构成，分区内按 offset 追加日志，消费者主动拉取并管理读取位置；已消费的数据可按保留策略继续存在，便于其他组和重放使用，而不是 ACK 后立刻从所有存储删除。顺序追加、批量请求、压缩、页缓存及合适路径下的零拷贝减少每条消息的磁盘寻道、网络往返和用户态复制成本。Kafka 可分为存储/协调一侧的 Broker 与控制器，以及生产、消费和计算集成一侧；Kafka Connect 连接外部数据源，Kafka Streams 处理流与 KStream/KTable 状态。控制面由 ZooKeeper 迁移到 KRaft，实际可用模式取决于 Kafka 部署版本，升级时应按版本文档核对。分区可并行扩展，也带来顺序范围、文件数量与重平衡成本。高吞吐不等于每条消息都低延迟，批量积累会引入等待，随机历史读取、大消息和副本同步也可能改变性能，应按目标场景测量。〔N133〕

040 Kafka 位点为什么应在业务成功后推进，异步处理怎样避免跳过消息？

答案 提交位点通常表示下一条应读取的位置，提前提交后若进程崩溃，尚未完成的业务可能在重启后被跳过；业务成功后再提交则可能重复投递，需要幂等处理。自动提交依据客户端拉取和周期策略，不能自然证明异步任务已完成。对同一分区并行处理时，应维护连续完成水位：后面的记录完成也不能越过前面尚未成功的记录提交，否则重启会遗漏前项。暂停分区、限制在途数量并处理重平衡交接，防止内存与任务无界增长。若要把消费位点与 Kafka 输出原子提交，可使用其事务能力；外部数据库副作用仍须独立协调。〔N141、N133〕

041 acks=all 和 ISR 怎样影响 Kafka 写入安全与可用性？

答案 ISR 是满足同步要求的副本集合，分区 leader 根据复制进度维护它。acks=all 要求达到相应 ISR 的确认条件，但 ISR 可能已经缩小，不能仅看到 all 就推断至少有多个安全副本；应配合足够副本因子、min.insync.replicas、合理的 leader 选举和持久化运维策略。ISR 不足时拒绝写入会牺牲部分可用性以保住约定的副本要求。还要区分生产者超时与真正未写入：超时可能发生在 Broker 已接受后，重试须使用幂等生产者及业务去重。某些版本的副本选举机制持续演进，安全结论应依据实际版本、配置及允许的故障模型，而非口号。〔N133〕

042 Kafka 重平衡为什么会引起延迟和重复，怎样减少无效重分配？

答案 组成员加入、离开或订阅分区变化时，协调流程会重新安排分区所有权；不同协议与分配策略影响暂停范围，不能把所有版本都等同于全组长期停顿。消费者必须在交接时处理在途任务和已完成位点，提交尚未完成的记录会漏处理，未提交则可能重复。处理过慢、长时间不调用 poll、频繁重启和不稳定网络都可能反复触发重平衡。应限制每批及在途处理量，正确设置存活、轮询和会话参数，按支持情况使用稳定成员身份及更平滑的协议。优化目标是有界交接和正确恢复，不能只把超时设得极大来隐藏失联消费者。〔N133、N141〕

043 RabbitMQ 如何通过交换机、绑定和队列决定消息去向？

答案 AMQP 是协议家族，RabbitMQ 是实现相关协议及扩展的消息服务器；一个连接可承载多个 channel，vhost 提供逻辑命名与权限边界。生产者通常把消息发给 exchange，交换机依据绑定和 routing key 路由到队列，再由消费者获取。direct 按绑定键精确匹配，topic 按层次模式匹配，fanout 向所有绑定队列广播，headers 按头部条件匹配；一个队列上的多个消费者通常竞争分工，而复制到多个队列才形成多个独立订阅。路由成功不等于业务处理成功，拓扑、持久化、确认和权限需分别设计；未匹配任何队列的消息也需要显式处理策略。〔N132〕

044 RabbitMQ 的 Confirm、Return 和消费者 ACK 分别确认哪一步？

答案 Publisher Confirm 告诉生产者 Broker 对发布动作的接受结果，其安全范围受队列类型、持久化和复制配置影响；它不表示消费者已经完成业务。mandatory 发布无法路由到队列时可触发 basic.return，让生产者知道路由失败，消息也可能在交换机处理后获得 confirm，因此必须分别处理。消费者 ACK 表示应用已完成这次投递，Broker 才能释放相应未确认状态；业务提交前 ACK 可能导致漏处理，提交后 ACK 丢失则可能重投。Confirm 与消费者 delivery tag 属于不同方向和 channel 的状态，不能混用。可靠发送应记录待确认结果并幂等重试，而不是仅在 TCP 写入成功时认定消息已落地。〔N142〕

045 RabbitMQ prefetch 怎样限制在途消息，为什么太大会恶化公平性？

答案 在手动确认场景中，prefetch 限制消费者或 channel 可保留的未确认投递数，具体作用范围要按 basic.qos 设置和 RabbitMQ 扩展理解。它相当于消费端在途窗口：过小可能让工作线程等网络，过大则把大量消息集中到慢消费者，占用内存、拖长故障恢复和重投时间，也可能让其他消费者缺少工作。数值为零通常表示不设此限制，而不是完全不取消息。设置应结合并发处理数、单条耗时、大小和下游容量，并用吞吐与尾延迟逐步调整。确认必须对应已完成业务，不能为了腾窗口提前 ACK，把背压变成无法恢复的应用内积压。〔N143、N142〕

046 RabbitMQ 加入集群为何不等于每个队列都具备多副本？

答案 集群解决节点间协作及部分元数据共享，队列消息是否复制取决于具体队列类型；不能只看到多个节点就假设任意单节点损坏后所有消息仍存在。Quorum Queue 使用 Raft 为一个队列建立复制组，通过多数派协调日志与领导者，适合需要明确复制耐久性的任务队列。失去多数派时会限制可用性，队列数、副本数和跨节点网络都会影响成本。应按当前支持的队列类型规划副本分布、确认和恢复演练，不把旧版镜像经典队列的方案当成所有新版本的默认实践。多副本也不替代业务 ACK、幂等和备份，它们保护的是不同故障环节。〔N144〕

10.9 分布式理论与一致性

047 强一致、最终一致和读写一致性分别承诺什么？

答案 “一致性”必须说明观察者和操作语义。线性一致性要求每个操作看起来在调用与返回之间某一时刻生效，并尊重真实时间顺序；顺序一致性只要求所有操作存在一个符合各线程程序顺序的全局顺序；最终一致性允许暂时读到旧值，但在停止更新且网络恢复后副本趋于同一状态。BASE 概括另一种分布式取舍：Basically Available（基本可用）、Soft State（软状态，复制期间状态可能变化）和 Eventually Consistent（最终一致，系统恢复并停止更新后副本收敛）。BASE 不等于业务可以随意出错；中间状态应可观测，失败应有可靠重试、冲突处理或补偿。读己之写、单调读和会话一致性是较弱但有用的应用保证；应把延迟、冲突合并与故障时可用性写进接口契约，别把数据库事务 ACID 中的 C 直接等同于分布式线性一致性。〔N145〕

10.10 分布式组件与服务治理

048 RPC 调用链至少应设计哪些协议和失败语义？

答案 RPC 把远程调用封装成接口，但网络会引入超时、重复、连接中断、版本不兼容和部分成功。客户端可由动态代理把方法调用编码为请求；协议帧应包含版本、类型、请求 ID 和长度，并限制最大帧长。I/O 线程负责收发与解帧，再把业务工作交给有界线程池，队列满时按策略拒绝或背压，不能无限堆积；响应携带请求 ID，由客户端映射回等待中的 Future。还需定义序列化兼容、认证、截止时间、取消、错误码、重试资格和幂等键，并通过 SPI 等机制扩展序列化或负载策略。gRPC 的 deadline 会向下游传播；重试只能针对明确的瞬时错误并设有预算与退避。客户端超时不能断言服务端未执行，写请求应通过业务状态查询或幂等键收敛结果。〔N62、N146〕

049 ZooKeeper 的 ZAB、临时节点和 Watch 怎样支持协调？

答案 ZooKeeper 通过集群中的领导者和多数派协议对写请求排序并复制日志，ZAB 的核心目标是让已提交更新按一致顺序传播；写入成功需要满足相应多数派条件，失去多数派时宁可停止写入也不应伪造进度。临时节点绑定会话，客户端断开并在会话超时后节点消失，常用于服务注册或锁；watch 是一次性变更通知，收到后需重新读取状态并重新注册，不能把通知当成永不丢失的队列。用它做选主、注册发现或锁时要处理会话抖动、羊群效应、重复回调和旧客户端恢复；业务资源仍应验证 fencing/版本，不能只相信本地持有锁标志。〔N147、N129〕

050 服务注册发现怎样面对注册中心不可用和过期实例？

答案 实例启动后注册地址、版本、能力和健康租约，客户端通过订阅或轮询获得可用实例列表；心跳或会话过期清理失联实例。注册中心短暂不可用时，客户端可使用带时间戳的缓存路由继续调用，但必须设置最大陈旧时间和失败降级，不能无限信任已经下线的地址。路由选取还要结合负载、区域、熔断状态和连接复用；发布期间旧版本与新版本要共存并可回滚。健康检查只能证明探针时刻可达，不能保证业务依赖正常，调用端仍要有超时和错误隔离。采用 ZooKeeper、DNS、控制面或服务网格时，分别核对一致性、传播延迟、权限、证书与数据面断联行为。〔N147、N148〕

10.11 分布式事务与一致性

051 Seata AT 和 TCC、Saga 的补偿边界如何比较？

答案 Seata AT 通过代理本地事务、记录回滚日志与全局锁协调常见关系库操作，回滚阶段依据镜像恢复数据；它要求参与者支持事务和合适的 SQL，锁冲突、DDL、长事务及外部副作用并不会自动变成可回滚。TCC 把预留、确认、取消显式交给业务，需处理幂等、空回滚、悬挂和超时；Saga 通过补偿动作串联长流程，补偿可能被观察到且不一定精确逆转。分布式事务框架不能替代本地约束、Outbox、幂等和对账，选择要按隔离级别、业务可补偿性、延迟及参与者类型评估。〔N149〕

052 Snowflake 类 ID 的位段设计怎样处理时钟回拨与序列耗尽?

答案 通常把时间戳、节点标识和同一时间片内的序列号拼成整数，以获得近似按生成时间排序的唯一 ID。节点标识必须由可靠分配机制保证不冲突；同一毫秒生成数超过序列容量时，可等待下一时间片或返回限流错误。检测到时钟回拨时不能继续盲增时间戳，否则可能与已发 ID 冲突；可等待时钟追平、切换逻辑纪元或暂时拒绝，策略要和业务可用性取舍。时间回拨、节点重启、数据中心漂移和位宽升级都应在测试中验证。它不保证跨节点严格全局顺序，也不自动隐藏业务规模，若 ID 暴露给外部还需考虑可枚举性。〔N98〕

10.12 高并发业务系统设计

053 短链服务怎样生成唯一编码并支撑高频跳转?

答案 写入时先验证目标 URL、权限、过期策略和自定义别名，再分配短码；是否去重必须按租户、活动和有效期定义，不能无条件全局合并相同长链，创建重试可用固定 request ID 保证幂等。可用数据库序列或分布式 ID 编码为 Base62；随机码必须有唯一约束并在冲突时有限重试。按短码建立索引并把映射写入可靠存储，读请求先查缓存再回源；缓存 TTL 不应超过业务过期时间，禁用或修改目标后要失效缓存。过期短码通常不回收，复用可能让旧链接或长期缓存的 301 指向新目标；只有能证明所有旧链接、客户端缓存和业务生命周期都已失效时才考虑回收。点击统计需过滤爬虫和短信预览，短码应避免可枚举并只允许 HTTP(S) 等受控协议，配合恶意链接审核、下架和滥用监控。规模增长后可按短码路由分片。〔N110、N98、N124〕

054 点赞系统怎样保证用户状态幂等并承受热点写入?

答案 把“设置为已点赞/取消点赞”建模为显式状态接口，并用用户 ID 与内容 ID 的唯一约束保证一个用户最多一条关系；相同请求重试返回同一结果，避免用不可幂等的 toggle 让网络重试反转状态。关系表是可恢复的事实数据，点赞总数可作为派生计数。Redis 状态更新与发送 MQ 之间存在崩溃窗口，不能简单地先写 Redis 再发消息；可在同一 Lua 脚本中原子更新状态并写入 Redis Stream，或用 MySQL 事务 Outbox 产生可重放事件。每个用户-内容状态带递增版本，乱序事件只应用更高版本，消费端在事务里仅当状态真实变化时调整计数。热点键可拆分计数桶后汇总，并提供周期性对账与修复；读侧分别定义本人状态与总数允许的陈旧时间。〔N110、N124、N133〕

第 11 章 自动驾驶与实时数据

11.1 QoS 与点云处理

001 ROS 2 DDS 的 QoS 如何按数据类型选择?

答案 ROS 2 QoS 由 History/Depth、Reliability、Durability、Deadline 等策略组合而成，发布者和订阅者的配置还必须兼容。高频相机或点云通常更看重最新帧，可选 BestEffort 和较浅队列，避免排队传旧数据；关键控制命令可考虑 Reliable，但重试延迟和资源占用也要纳入时限分析。最终选择应在目标 DDS 实现、网络和负载下验证。

```
camera_qos = {BestEffort, KeepLast, depth=1, deadline=frame_period}; // 保新帧
control_qos = {Reliable, KeepLast, bounded_depth}; // 关键指令要可靠但受时限约束
if (!publisher_qos_compatible_with(subscriber_qos)) no_samples_are_delivered();
// History/Depth、Reliability、Durability、Deadline 策略须兼容
// 队列太深会积压旧图像；Reliable 重试也会增加时延和资源占用
// 在目标 DDS、网络和负载下测延迟/丢帧，再按数据重要性定配置
```

002 体素网格降采样如何实现?

答案 把每个点的坐标按体素边长量化为三维整数索引，以索引为哈希键累计坐标和计数，最后输出每格质心，可显著减少点数。扫描 n 个点平均 O(n)，内存与非空体素数相关；稀疏数据适合哈希表，规则小范围也可用数组。体素越大降采样越强，但边缘、细小障碍和定位精度会损失。质心并非原始点，若后续算法依赖表面边缘，可改为保留代表点或自适应叶尺寸。

```
#include <cmath>
#include <unordered_map>
std::unordered_map<Cell, SumAndCount, CellHash> voxels;
for (const Point& p : cloud) {
    Cell c{std::floor(p.x / size), std::floor(p.y / size), std::floor(p.z / size)};
    voxels[c].add(p); // 相同体素累计坐标和计数，floor 也正确处理负坐标
}
for (const auto& [cell, aggregate] : voxels) output.push_back(aggregate.centroid());
// 平均 O(n)，内存随非空体素数增长；体素越大细节/边缘精度损失越明显
// 质心不是原始点；依赖表面边缘时可保留代表点或自适应叶尺寸
```

11.2 共享内存与时钟同步

003 共享内存传递大帧如何管理所有权?

答案 生产者把帧写入共享内存池，向消费者传递槽位索引、长度、版本和时间戳等轻量句柄，避免跨进程复制大图像。通过引用计数、所有权转移或租约状态确保消费者读完前槽位不会被复用，并用原子状态避免读写竞争。进程异常退出时要有超时回收机制，且映射地址不能假设各进程相同。还应验证缓存对齐、像素步长和 NUMA 放置，避免零拷贝后仍发生隐式复制。

```
slot = shared_pool.acquire(); producer.write_frame(slot); // 跨进程只传句柄而非复制大帧
publish({slot.index, slot.generation, length, timestamp, stride});
consumer.map_own_address_and_validate_layout(handle); consumer.retain(slot);
if (consumer_done) release_reference(slot); // 引用归零/所有权归还前不能复用
if (owner_crashed_or_lease_expired) reclaim_after_safe_recovery();
// 共享槽位状态需原子同步；检查步长、缓存对齐、NUMA 和隐式复制。
```

004 多传感器如何做时间同步?

答案 最可靠的同步是传感器共用硬件触发或 PPS；分布式设备可用 PTP/gPTP 校准时钟，采集端尽量记录硬件时间戳而非软件收到时间。算法融合时统一时钟域，并估计固定偏移与随时间变化的漂移。还应明确图像曝光中点、雷达扫描周期等时间定义，启动时对齐一次无法消除长期漂移。融合时通常按时间戳做插值或最近邻匹配，并设置最大允许时间差。

```
capture_hardware_timestamp(sensor); // 比应用收到回调时间更接近实际采集
synchronize_clocks_with_hardware_trigger_or_PPS_or_PTP();
estimate_clock_offset_and_drift_over_time();
normalize_to_common_clock_domain(); // 明确曝光中点/扫描周期等时间定义
frame = nearest_or_interpolate(sensor_a, sensor_b, max_allowed_skew);
// 只启动时校一次不能消除长期漂移；融合需设最大时间差。
```

11.3 实时数据流水线

005 实时传感器数据流水线如何避免阻塞和积压?

答案 把采集、校时、预处理、融合/推理和输出拆成独立阶段，各阶段用有界队列隔离速度波动。队列满时按数据类型规定丢最旧帧、丢最新帧或背压；实时感知常需优先保持新鲜度，不能无限积压旧帧。记录每阶段时间戳并监控端到端 P95/P99 延迟、丢帧和队列深度，设置过期数据截止时间。对控制链路可为每帧设置截止时间，超时结果直接作废而不进入下游队列。

```
capture -> bounded_queue -> timestamp_and_preprocess -> fusion_or_inference -> output;
if (queue.full()) drop_oldest_or_newest_or_apply_backpressure_by_data_type();
if (now - frame.timestamp > deadline) discard_expired_frame();
record_each_stage_timestamp_and_queue_depth();
alert_on_p95_p99_latency_drops_and_backlog();
// 实时感知常优先新鲜度，不能无限排旧帧；策略按链路安全要求设定。
```

006 工业相机图像采集与处理系统如何设计?

答案 采集线程专注从相机 SDK 取缓冲区并记录硬件时间戳，处理线程通过有界队列分阶段做去马赛克、校正或检测。大帧可用共享池/引用句柄减少复制，同时校验像素格式、步长、尺寸和缓冲生命周期。负载超限时应明确丢弃过期帧策略，并监控丢帧率、处理尾延迟、CPU/GPU 和内存占用。 缓冲区不足时记录设备端丢帧计数，才能区分采集丢失与处理来不及。

```
frame = camera_sdk.acquire_buffer(); timestamp = frame.hardware_timestamp();
if (!bounded_queue.try_push({frame_handle, timestamp, format, stride, dimensions})) count_drop();
worker_demosaic_rectify_and_detect(frame_handle); // 句柄复用可减少大帧复制
validate_pixel_format_stride_dimensions_and_buffer_lifetime();
if (processing_is_late) drop_expired_frame_and_record_stage_latency();
// 分开采集/处理线程，避免耗时算法阻塞 SDK 取帧
// 记录设备端丢帧计数，区分采集丢失与处理积压
// 监控 CPU/GPU、内存、丢帧和 P99；队列有界避免无界缓存
```

007 生产者永不阻塞的帧环形队列如何设计?

答案 使用固定槽位帧池保存大帧，生产者只发布槽位序号和代际号，避免在采集路径等待消费者或复制图像。单消费者可用序号协议区分新旧数据，覆盖前需确认读者不会再访问；多消费者还需分别跟踪读位置或用引用计数安全回收。若采用丢旧帧保持实时性，消费者应能检测序号跳跃并统计丢帧。无锁实现需用适当内存序发布槽位内容，并避免序号回绕造成 ABA 误判。

```
slot = fixed_pool.acquire_without_wait(); if (!slot) return DropAndCount;
write_frame(slot.buffer); slot.sequence = next_sequence.fetch_add(1);
slot.state.store(Ready, std::memory_order_release); // 内容写完再发布 ready
consumer.load_state(std::memory_order_acquire); verify_generation_and_sequence();
if (reader_finished) release_slot(); // 槽位回收前确认无消费者仍持有
// 固定槽避免大帧复制，生产者不等待；消费者靠序号发现覆盖/丢帧
// 多消费者要追踪每个读位置或引用数；序号回绕需防 ABA
// 超时回收需确认进程已不再访问映射，不能仅因租约过期就覆盖
```

第 12 章 C++ 进阶与工程

12.1 内存模型与缓存一致性

001 C++ 中 CPU 缓存一致性和 false sharing 是什么?

答案 缓存一致性协议协调不同 CPU 核心对同一缓存行的读写副本，使写入最终保持一致。false sharing 指线程改写不同变量，但变量恰好落在同一缓存行，导致缓存行在核心间反复转移，性能下降而非数据竞争。可将热点计数器分离或按平台硬件间隔对齐，并用 profiler 验证是否值得增加内存。

```
#include <atomic>
#include <cstdint>
#include <thread>
struct alignas(64) PaddedCounter { std::atomic<std::uint64_t> value{0}; };
PaddedCounter counters[2];
std::jthread a([&] { ++counters[0].value; });
std::jthread b([&] { ++counters[1].value; });
// 逻辑独立但同缓存行会反复转移，形成 false sharing 性能损失
// 64 是平台假设；对齐会增加内存，应先用 profiler/硬件常量验证
```

002 C++ 中 volatile 与 std::atomic 有什么区别?

答案 volatile 限制某些对 volatile 对象访问的优化，常用于平台定义的内存映射寄存器；它不保证原子性、线程间顺序或可见性。std::atomic 为并发访问提供原子操作及内存序，适合线程共享状态，但不能自动保护多个字段组成的不变量。普通共享变量有未同步冲突访问会构成数据竞争。

```
#include <atomic>
#include <cstdint>
volatile std::uint32_t* device_register = platform_mapped_register();
*device_register = command; // volatile 用于平台定义的设备访问语义
std::atomic<bool> ready{false}; // 线程共享状态需要原子同步
ready.store(true, std::memory_order_release);
if (ready.load(std::memory_order_acquire)) consume_published_data();
// volatile 不保证原子性、线程可见性或 happens-before；atomic 也不保护多字段
```

003 C++ 内存模型中的 happens-before 是什么?

答案 happens-before 是 C++ 内存模型用来定义操作顺序和可见性的关系：同一线程的先后求值形成 sequenced-before，线程间通过互斥锁、release/acquire 等同步操作建立跨线程关系。发布线程先写数据再 release，读取相应发布值的 acquire 线程可见这些写入。缺少关系的冲突非原子访问会产生数据竞争。

```
#include <atomic>
#include <thread>
int payload = 0; std::atomic<bool> published{false};
std::thread producer([&] { payload = 42; published.store(true, std::memory_order_release); });
while (!published.load(std::memory_order_acquire)) {} // 观察到发布值
int value = payload; // 写入 happens-before 此读取
producer.join(); // acquire/release 同步普通数据访问；join 回收线程
```

12.2 模板、容器与线程安全

004 SFINAE 是什么?

答案 SFINAE 是 Substitution Failure Is Not An Error：在模板候选的替换阶段，特定替换失败会让该候选从重载集中移除，而不是立即使程序报错。它常用于检测成员或限制模板参数，但错误信息可能难读。C++20 concepts 可将条件直接写成 requires 约束；仍需理解替换失败与函数体实例化失败的区别。

```
#include <type_traits>
#include <utility>
#include <vector>
template<class T, class = void> struct has_size : std::false_type {};
template<class T> struct has_size<T, std::void_t<decltype(std::declval<T>().size())>> : std::true_type {};
static_assert(has_size<std::vector<int>>::value); // 无 size() 的类型选主模板
// 替换失败才会从候选集中移除；函数体实例化失败不自动算 SFINAE
// C++20 requires/concepts 更直观表达约束，但旧代码仍会遇到 SFINAE
```

005 STL 容器的线程安全边界是什么?

答案 标准库容器允许符合规则的并发 const 操作；除 vector<bool> 这类特殊打包表示外，对不同元素的某些并发修改也可不互相构成数据竞争。但插入、删除、扩容等会改变容器结构的操作，与遍历或访问通常需外部同步。多个线程写同一元素、或读写同一元素冲突时也要加锁或改用原子设计。

```
#include <thread>
#include <vector>
std::vector<int> values{1, 2, 3, 4};
std::jthread a([&] { values[0] = 10; });
std::jthread b([&] { values[1] = 20; }); // 修改不同普通元素
// 两个线程不应同时 push_back/resize 与遍历：结构和迭代器会改变
// 写同一元素需同步；vector<bool> 位打包有特殊风险
// 并发 const 访问也不能与未同步结构修改同时进行
```

006 shared_ptr 是线程安全的吗?

答案 多个不同 shared_ptr 实例若共享一个控制块，其引用计数增减可并发进行；这不代表同一个 shared_ptr 变量可被多个线程同时读写。对同一实例并发赋值要加锁，或在支持的标准版本使用 atomic<shared_ptr>。无论如何，所指对象内部状态的线程安全仍由程序自行保证。

```
#include <memory>
#include <thread>
std::shared_ptr<Widget> shared = std::make_shared<Widget>();
std::shared_ptr<Widget> left = shared, right = shared;
std::jthread a([copy = left] { use(copy); }); // 独立 shared_ptr 实例
std::jthread b([copy = right] { use(copy); }); // 共享控制块计数可并发更新
// 同一个 shared_ptr 变量若同时读写仍需同步；Widget 内部也要自行保护
```

007 std::function 与模板回调有什么取舍?

答案 std::function 通过类型擦除提供固定签名，适合回调需跨模块保存不同闭包类型的接口；会有间接调用，某些目标可能触发动态分配，但具体优化依实现。模板回调保留具体类型，更容易内联且可编译期检查，但会增加实例和接口暴露。需要运行时异构存储时选 function，局部泛型算法常用模板。

```
#include <functional>
#include <utility>
std::function<void(Event)> callback = [state](Event e) { state.handle(e); };
registry.add(callback); // 固定签名，可运行时存不同闭包类型
// 类型擦除带间接调用；目标可能分配，SBO 依具体实现
auto templated_call = [](auto&& f, auto&& value) {
  std::forward<decltype(f)>(f)(
      std::forward<decltype(value)>(value));
};
templated_call([](Event e) { handle(e); }, event); // 具体类型利于内联
// 异构长期存储用 function；局部泛型算法通常用模板
```

008 std::vector 扩容和迭代器失效有什么影响?

答案 vector 容量不足时通常申请新缓冲区并迁移元素，扩容会使其元素指针、引用和迭代器失效；尾部插入平均摊还 O(1)，单次扩容仍为 O(n)。已知大致规模可 reserve 减少重分配。erase 会移动后续元素，因此被删位置及其后的迭代器/引用失效，遍历删除应使用返回的新迭代器。

```
#include <vector>
std::vector<Item> items;
items.reserve(100); // 已知规模时预留，降低扩容次数
items.push_back(first);
auto it = items.begin();
items.push_back(next); // 若容量不足扩容，全部元素迭代器/指针/引用失效
// reserve 只降低失效概率；erase 也会使删除点及之后句柄失效
```

12.3 零拷贝、异常与并发问题

009 如何实现零拷贝？零拷贝是否意味着完全不复制?

答案 零拷贝通常表示减少某段数据路径上的 memcpy，例如内核 sendfile、mmap、共享内存或设备 DMA；它不一定代表数据没有搬动或复制。系统仍可能复制页表、增加引用计数、进行格式转换或由设备搬运数据。要确认具体路径，并正确管理缓冲区所有权、生命周期、同步和背压。

```
#include <cstddef>
#include <span>
#include <vector>
void checksum(std::span<const std::byte> bytes) { consume(bytes); }
std::vector<std::byte> buffer = read_once(); checksum(buffer); // 借用缓冲，不复制字节
// mmap/sendfile/共享内存可减少特定路径 memcpy，但页表/DMA 仍有成本
// 仍须保证 buffer 生命周期、所有权移交、并发访问和背压
```

010 异常安全的三个常见保证是什么?

答案 不抛出保证表示操作不会让异常逃出；强保证表示失败时程序状态不变；基本保证表示对象仍有效且资源无泄漏，但状态可改变。RAII 管理资源，先构造临时结果再提交可帮助实现强保证，copy-and-swap 是一种例子。noexcept 是可观察承诺，异常逃逸会 terminate，不能为优化而随意标记。

```
class Value { public:
  Value& operator=(const Value& other) {
    Value temp(other); // 强保证：复制失败时当前状态不变
    swap(*this, temp); return *this; // ADL 选本类的非抛出 friend swap
  }
  friend void swap(Value&, Value&) noexcept;
};
// 基本保证：对象有效且无泄漏；不抛出保证：异常不外传
// noexcept 只有操作真实不抛时才能承诺
```

011 无锁队列中的 ABA 问题是什么?

答案 CAS 读取到 A 后，其他线程可能先把状态改成 B，再改回 A，比较时仍相等，算法误以为状态没有变化；无锁栈中节点地址复用是典型情况。带版本计数的指针可检测部分 ABA，hazard pointer 或 epoch 回收避免节点过早释放。标记位宽和回绕也需考虑，且必须一并证明内存回收安全。

```
#include <atomic>
#include <cstdint>
struct TaggedHead { Node* ptr; std::uint64_t version; };
std::atomic<TaggedHead> head; // CAS 同比地址和版本，部分检测 ABA
bool replace(TaggedHead old, Node* next) {
  TaggedHead desired{next, old.version + 1};
  return head.compare_exchange_strong(old, desired);
}
// 仍需 hazard pointer/epoch 确保 Node 未提前释放；版本会回绕且未必 lock-free
```

012 移动构造为什么常标记 noexcept?

答案 vector 扩容要在移动元素途中遇到异常时维持异常保证；若移动构造可能抛且元素可拷贝，库实现可选择拷贝以保留强保证。noexcept 移动允许实现安全地移动，降低回退拷贝的需要。只有所有移动步骤确实不会抛时才能标记 noexcept，否则异常逃出会导致 terminate。

```
#include <utility>
#include <vector>
struct Item { std::vector<int> data;
  Item(Item&& other) noexcept : data(std::move(other.data)) {} // 移动确实不抛
};
std::vector<Item> items; items.reserve(100); // 降低扩容搬移次数
// move 可能抛而 copy 可用时，vector 为强保证可能选择 copy
// noexcept 错误标注会让逃逸异常调用 terminate
```

013 结构体成员重排为什么可能节省内存?

答案 成员地址必须满足各自对齐要求，编译器会在成员间和对象末尾插入填充；把高对齐成员放在前面有时能减少空隙，例如先放 8 字节成员再放 1 字节成员。重排会改变 sizeof、成员偏移和 ABI，也可能破坏外部二进制协议。对持久化或网络数据应显式序列化，不依赖内存布局。

```
#include <cstddef>
struct Before { char tag; double value; int count; };
struct After { double value; int count; char tag; };
std::size_t a = sizeof(Before), b = sizeof(After); // 只比较当前 ABI
// 成员对齐会插入填充；将高对齐成员前置有时能减少空隙
// 重排会改变成员偏移/sizeof 并破坏 ABI
// 网络或持久化数据应显式序列化，不能依赖本机内存布局
```

014 迭代器失效通常如何处理?

答案 迭代器失效规则按容器和具体操作区分：vector 扩容会使全部失效，插入/删除会影响相应位置之后的元素；list 插入一般不影响其他迭代器，删除只使指向被删节点者失效；unordered 容器 rehash 会使迭代器失效。读取文档并在 erase 中接收返回迭代器，避免继续使用失效句柄。

```
#include <vector>
for (auto it = values.begin(); it != values.end(); ) {
  if (should_remove(*it)) it = values.erase(it); else ++it;
}
// erase 返回下一个有效迭代器，删除后不要再用旧 it
// vector 扩容使所有迭代器失效；list 插入通常不影响其他节点迭代器
// unordered_map rehash 会使迭代器失效；具体规则按容器和操作查文档
```

第 13 章 数据结构与算法

13.1 基础动态规划与数组问题

001 0/1 背包问题（0/1 Knapsack）

答案 令 dp[c] 表示当前已处理物品在容量不超过 c 时的最大价值，初值为 0。每件重量 w、价值 v 的物品按 c 从 W 递减到 w 更新，避免本轮结果重复使用该物品；若正序更新就变成完全背包。时间 O(nW)、空间 O(W)，重量为零时要按题目约束避免无限取用。 物品重量大于当前容量时自然跳过，初始化为零也适用于允许不选任何物品的情形。

```
// dp[c] 只由已经处理的物品决定，容量倒序是 0/1 约束的关键。
// dp[c]: 已处理物品在容量不超过 c 时的最大价值；允许不选物品
std::vector<long long> dp(W + 1, 0);
for (const auto& item : items) {
    for (int c = W; c >= item.weight; --c) // 倒序保证本轮不会重复用同件物品
        dp[c] = std::max(dp[c], dp[c - item.weight] + item.value);
}
return dp[W]; // O(nW) 时间、O(W) 空间；重量为 0 时需按约束处理
```

002 K 个一组翻转链表的思路是什么?

答案 先从组首向后数满 K 个节点；不足 K 个时直接返回，保留尾段原样。对整组用 prev/current/next 反转指针，随后把前一组尾接到新组首，并把旧组首作为新组尾连接剩余链表。虚拟头节点可统一处理首组，迭代法时间 O(n)、额外空间 O(1)。 每段反转前后都要保存连接点，避免链表被截断或下一组失去入口。

```
// prev/cur/next 维护组内反转；组外连接点必须先保存。
// dummy 统一处理首组；不足 k 个的尾段不反转
Node dummy{0, head}; Node* before = &dummy;
while (true) { Node* end = before; for (int i = 0; i < k && end; ++i) end = end->next;
    if (!end) break; Node* oldFirst = before->next; Node* prev = end->next; Node* cur = oldFirst;
    while (cur != end->next) { Node* next = cur->next; cur->next = prev; prev = cur; cur = next; }
    before->next = end; before = oldFirst;
}
return dummy.next; // 每个节点只改指针，O(n) 时间、O(1) 额外空间
```

003 三数之和如何去重并求解?

答案 先排序，再固定 i，并用左右指针在剩余区间找和为目标的两数；和小于目标左指针右移，大于则右指针左移。找到后记录三元组并跳过重复的 i、left、right 值，避免重复答案。时间 $O(n^2)$，求和前转成 64 位以防溢出，结果中三数可按题目要求排序输出。 排序后若当前固定值已大于目标且全为非负数，可提前结束枚举。

```
// 固定首数后双指针单调逼近目标，排序后同值结果要去重。
std::sort(a.begin(), a.end());
for (int i = 0; i + 2 < a.size(); ++i) { if (i && a[i] == a[i - 1]) continue;
    int l = i + 1, r = a.size() - 1;
    while (l < r) { long long sum = (long long)a[i] + a[l] + a[r];
        if (sum < target) ++l; else if (sum > target) --r;
        else { out.emplace_back(a[i], a[l], a[r]); int x = a[l], y = a[r];
            while (l < r && a[l] == x) ++l;
            while (l < r && a[r] == y) --r;
        }
    }
} // O(n^2)；求和先扩为 64 位，避免整数溢出
```

004 不同路径数（Unique Paths）

答案 状态 dp[i][j] 表示到达格子 (i,j) 的路径数，最后一步只能从上方或左方进入，因此两者相加。首行和首列没有障碍时只有一条路径，遇障碍则当前状态为 0。滚动一维数组按行更新，时间 O(mn)、空间 O(n)；若网格有障碍，转移同样适用。 路径数量增长很快，网格较大时应使用足够宽的整数类型。

```
// 一维 dp[c] 在本行代表左侧，尚未覆盖时代表上方。
// dp[c]: 当前行到列 c 的路径数；障碍格状态为 0
std::vector<long long> dp(n, 0);
for (int r = 0; r < m; ++r) for (int c = 0; c < n; ++c) {
    if (grid[r][c] == '#') dp[c] = 0;
    else if (r == 0 && c == 0) dp[c] = 1;
    else dp[c] += c ? dp[c - 1] : 0; // 旧 dp[c] 是上方，已更新 dp[c-1] 是左方
}
return dp[n - 1]; // O(mn) 时间、O(n) 空间；路径数大时选宽整数
```

005 买卖股票的最佳时机（Best Time to Buy and Sell Stock）

答案 只允许一次买入和卖出时，遍历价格并维护此前最低买入价，当前价减最低价就是以当天卖出能获得的最大收益。先计算收益再更新最低价可避免同一天买卖的歧义；若题目允许同日按规则处理即可。时间 O(n)、空间 O(1)，所有价格单调下跌时返回 0。最低价只从此前日期更新，避免把未来价格误当成买入价。例如价格先跌后涨时会在后续卖出点更新利润，反向顺序则保持零收益。

```
// lowest 始终是当前卖出日之前的最低价格。
if (prices.empty()) return 0;
int lowest = prices[0], best = 0;
for (int p : prices) {
    best = std::max(best, p - lowest); // 先卖出计算，再把当天纳入未来买入价
    lowest = std::min(lowest, p);
}
return best; // 一次遍历、O(1) 空间；下跌行情返回 0
```

13.2 树与图的遍历

006 二分图检测（Bipartite Graph Detection）

答案 从每个未染色节点启动 BFS/DFS，给起点染一种颜色，并把每条边的另一端染成相反颜色；遇到同色相邻点就不是二分图。遍历所有连通分量，孤立点可单独构成一侧；自环会立即冲突。时间 O(V+E)、空间 O(V)，常用于判断二分匹配建模是否成立。 用邻接表可在线性规模上完成染色，邻接矩阵会多出 $O(V^2)$ 扫描。

```
// 相邻点必须异色；每个连通分量都要独立启动染色。
std::vector<int> color(n, -1); std::queue<int> q;
for (int start = 0; start < n; ++start) if (color[start] < 0) { color[start] = 0; q.push(start);
    while (!q.empty()) { int u = q.front(); q.pop();
        for (int v : graph[u]) {
            if (color[v] < 0) { color[v] = color[u] ^ 1; q.push(v); }
            else if (color[v] == color[u]) return false; // 自环也会冲突
        }
    }
}
return true; // 每个分量染色通过即为二分图，O(V+E) 时间、O(V) 空间
```

007 二叉搜索树的第 k 大元素

答案 BST 中序遍历是升序，反向中序按右、根、左访问即为降序；访问计数到 k 时即可停止，避免遍历整棵树。时间为 O(h+k) 左右、递归栈 O(h)，极端偏斜树仍可能到 O(n)。若 k 越界或树可能含重复值，应按题目约定处理。 若树节点不可变，也可维护大小为 k 的最小堆，但需 O(k) 空间。

```
// 右根左顺序形成降序访问，计数到 k 后可提前终止。
// 反向中序为降序；假设 k 从 1 开始且 1 <= k <= 节点数
void visit(Node* p) { if (!p || found) return; visit(p->right);
    if (!found && --k == 0) { answer = p->value; found = true; }
    visit(p->left);
}
visit(root); // O(h+k) 量级的提前停止遍历，递归栈 O(h)
// 若 k 越界由 found=false 表示未找到；极斜树可改显式栈
```

008 二叉树的前序、中序、后序遍历如何实现?

答案 递归遍历分别按根左右、左根右、左右根访问；迭代前序用栈先压右再压左，中序用栈一路走到最左，后序可用双栈或修改访问标记。层序遍历用队列逐层取节点。所有遍历时间 O(n)，深度优先递归/栈空间 O(h)，退化树要留意递归栈溢出。后序单栈实现需记录上次访问的右子树，防止根节点重复处理。若只需要某一种遍历，避免同时构造所有序列可减少不必要的内存分配。

```
// 递归位置决定根左右三种访问次序，空节点是递归边界。
void preorder(Node* p) { if (!p) return; out.push_back(p->value); preorder(p->left); preorder(p->right); }
void inorder(Node* p) { if (!p) return; inorder(p->left); out.push_back(p->value); inorder(p->right); }
void postorder(Node* p) { if (!p) return; postorder(p->left); postorder(p->right); out.push_back(p->value); }
// 分别按根左右、左根右、左右根访问；每次遍历 O(n)，递归栈 O(h)
// 极深退化树可用显式栈避免递归深度过大
```

009 二叉树的层次遍历

答案 把根节点入队，每轮先记录当前队列长度作为本层节点数，再恰好弹出这些节点并将左右孩子入队，就能按层输出。队列保证先访问较浅节点，空树直接返回空结果。时间 O(n)，最坏队列空间 O(w)，w 为最大层宽；需要每层分组时按轮次切片。入队时标记访问状态，尤其对图层次遍历可避免同节点重复入队。该队列在宽而浅的树上可能占用 O(n) 空间，深树则通常明显更省。

```
// 每轮固定队列长度，保证本层节点与新入队孩子不混淆。
if (!root) return levels; // 空树返回空列表
std::queue<Node*> q; q.push(root);
while (!q.empty()) { int width = q.size(); std::vector<int> level;
    while (width-- > 0) { Node* p = q.front(); q.pop(); level.push_back(p->value);
        if (p->left) q.push(p->left); if (p->right) q.push(p->right); }
    levels.push_back(std::move(level)); // 本轮固定 width，孩子留到下一层
}
// 每节点入队一次：O(n) 时间，队列最多 O(w) 个节点
```

010 二叉树的最大深度

答案 定义深度为根到最远叶子的节点数，空树深度为 0，则节点深度是 1 加左右子树深度的较大值。DFS 每个节点只访问一次，时间 O(n)、栈空间 O(h)。若题目把路径长度定义为边数，返回值会比节点数少 1，需先确认约定。 对极深输入可改为显式栈，避免递归深度超过语言运行环境限制。

```
// 深度按节点数定义，空子树返回 0。
int depth(Node* p) {
    if (!p) return 0; // 按节点数计深度，空树为 0
    int left = depth(p->left), right = depth(p->right);
    return 1 + std::max(left, right);
}
// 每节点访问一次，O(n) 时间、O(h) 递归空间；按边计时定义会不同
```

011 二叉树的最大路径和

答案 对每个节点递归求左右子树能向上贡献的最大单边和，负贡献截为 0，因为路径可不选该侧。用 left+right+node 更新全局最大值，返回父节点时只能选一侧延伸，避免分叉路径。时间 O(n)、空间 O(h)，全负树要用负无穷初始化全局答案，不能默认为 0。 路径可以不经过根节点，因此答案与返回父节点的单边贡献要分开维护。

```
// best 是路径可在当前节点拐弯的答案，返回值只能向父节点单边延伸。
long long gain(Node* p) { if (!p) return 0;
    long long left = std::max(0LL, gain(p->left));
    long long right = std::max(0LL, gain(p->right));
    best = std::max(best, p->value + left + right); // 以 p 为拐点的完整路径
    return p->value + std::max(left, right); // 向父节点只能延伸一侧
}
long long best = std::numeric_limits<long long>::lowest(); gain(root);
// 全负树不能把 best 初始化成 0；时间 O(n)、栈 O(h)
```

012 二叉树的最小深度

答案 最小深度必须从根走到叶子，叶子定义为左右孩子都为空；节点只有一个孩子时不能把空子树深度当作答案。递归可在单侧为空时沿非空侧继续，否则取较小深度加一。时间 O(n)、栈空间 O(h)，也可 BFS 首次遇到叶子就返回。BFS 按深度递增扩展，通常比 DFS 更早找到最近叶子并提前返回。例如根只有左孩子时，答案应沿左侧继续计算，而不能把空右侧算成深度 1。

```
// 只有左右孩子均空才是叶子；BFS 按深度递增弹出。
if (!root) return 0;
std::queue<std::pair<Node*, int>> q; q.push({root, 1});
while (!q.empty()) { auto [p, d] = q.front(); q.pop();
    if (!p->left && !p->right) return d; // BFS 首个叶子就是最近叶子
    if (p->left) q.push({p->left, d + 1}); if (p->right) q.push({p->right, d + 1});
}
// 只有一个孩子时必须沿非空孩子走，不能把空子树当叶子；O(n) 时间
```

013 二叉树的镜像

答案 在每个节点交换左右指针，再递归处理交换后的两个子树，所有节点各访问一次，时间 O(n)、空间 O(h)。迭代实现可把节点压入栈/队列并逐个交换，额外空间取决于树宽或深度。原树若仍被其他引用共享，原地修改会同时改变这些引用看到的结构。 该操作会修改原树；需要保留输入时应先深拷贝或构造新树。

```
// 交换左右孩子后递归，原地镜像会修改输入树。
Node* invert(Node* p) { if (!p) return nullptr;
    std::swap(p->left, p->right); // 先交换，再递归处理两侧
    invert(p->left); invert(p->right); return p;
}
// 每个节点访问一次，O(n) 时间、O(h) 递归栈；原地修改，需保留原树时先深拷贝
// 极深树可换显式栈，避免递归栈深度受限
```

014 从前序和中序遍历构造二叉树

答案 前序首元素是根，在中序中找到根的位置即可确定左子树长度，再切分前序剩余部分递归构造左右子树。用哈希表保存中序值到下标，可将总时间降为 O(n)、空间 O(n)。通常要求节点值唯一；有重复值时遍历序列可能无法唯一确定原树。递归参数使用左右边界能避免子数组切片带来的额外复制成本。建树前应校验两种遍历长度相同且元素集合一致，避免非法输入越界。

```
// 前序当前首项给根，中序根位置给左子树长度。
// pos[value] 为中序下标；唯一值，区间均为左闭右开
Node* build(int pl, int il, int ir) { if (il == ir) return nullptr;
    int root = pre[pl], k = pos[root], leftSize = k - il;
    Node* left = build(pl + 1, il, k);
    Node* right = build(pl + 1 + leftSize, k + 1, ir);
    return new Node(root, left, right);
}
// 预建哈希表后 O(n) 时间、O(n) 空间；先校验两遍历元素一致
```

015 从后序和中序遍历构造二叉树

答案 后序末元素是根，在中序中找到该值后，左侧为左子树，右侧为右子树；按左右子树大小从后序序列切片递归构造。用下标范围而非频繁复制子数组，并用哈希表定位中序位置，可达 O(n) 时间和 O(n) 辅助空间。节点值需唯一才能直接确定分割位置。 也可倒序消费后序数组，先构造根和右子树，再构造左子树。 递归切分时后序区间的根在最后，子树长度仍由中序中的分界位置确定。

```
// 后序当前末项给根，中序根位置分割左右子树。
// pos[value] 为中序下标；后序末项为根，节点值唯一
Node* build(int pl, int pr, int il, int ir) { if (il == ir) return nullptr;
    int root = post[pr - 1], k = pos[root], leftSize = k - il;
    Node* left = build(pl, pl + leftSize, il, k);
    Node* right = build(pl + leftSize, pr - 1, k + 1, ir);
    return new Node(root, left, right);
}
// 用区间避免复制子数组；哈希定位后 O(n) 时间、O(n) 空间
```

13.3 排序、查找与有序结构

016 冒泡排序（Bubble Sort）

答案 每轮从左到右比较相邻元素，将逆序对交换，最大元素逐步冒到未排序区间末尾；某一轮无交换时可提前结束。排序稳定且原地，最好 O(n)，平均和最坏 $O(n^2)$。它适合解释排序思想或很小的数据量，不适合大数组。对逆序数组每轮都会发生大量交换，数据规模稍大时应换高效排序。 若输入几乎有序，提前停止标志会让算法接近一次扫描完成。

```
// 每轮把最大值沉到未排序区末端，无交换表示已有序。
for (int end = n - 1; end > 0; --end) { bool changed = false;
    for (int i = 0; i < end; ++i) if (a[i] > a[i + 1]) { std::swap(a[i], a[i + 1]); changed = true; }
    if (!changed) break; // 已有序时提前结束
}
// 相等元素不交换所以稳定；原地，最好 O(n)，平均/最坏 O(n^2)
```

017 判断一个二叉树是否为二叉搜索树（BST）

答案 递归检查每个节点是否落在祖先给出的合法开闭区间内，左子树上界收紧为当前值，右子树下界收紧为当前值。这样可发现仅比较父子节点会漏掉的远端违例。时间 O(n)、空间 O(h)；重复键是否允许需明确采用严格还是非严格边界。中序遍历也能检验结果是否严格递增，但处理极值边界时区间法更直接。允许重复键时，重复值必须始终被分配到约定的一侧并贯穿所有递归边界。

```
// 祖先上下界同时约束节点，检查远端子树是否越界。
bool valid(Node* p, long long low, long long high) {
    if (!p) return true;
    if (p->value <= low || p->value >= high) return false; // 严格 BST 边界
    return valid(p->left, low, p->value) && valid(p->right, p->value, high);
}
bool isBst = valid(root, std::numeric_limits<long long>::lowest(), std::numeric_limits<long long>::max());
// 若允许重复键，需按约定改开闭边界；O(n) 时间、O(h) 栈
```

018 判断一个二叉树是否是平衡二叉树

答案 后序遍历一次计算子树高度，若任一子树已不平衡或左右高度差超过 1，就返回特殊值（如 -1）向上传播。该方法每个节点处理一次，时间 O(n)、栈空间 O(h)；逐节点重新计算高度会在偏斜树上退化到 $O(n^2)$。空树通常视为平衡。返回高度而非只返回真假，能把 “是否平衡” 和高度计算合并为一次遍历。平衡判断的高度差是每个节点的局部约束，仅比较整树左右高度不够。

```
// 后序返回高度，-1 作为失衡哨兵向上短路。
int heightOrBad(Node* p) { if (!p) return 0;
    int left = heightOrBad(p->left); if (left < 0) return -1;
    int right = heightOrBad(p->right); if (right < 0) return -1;
    if (std::abs(left - right) > 1) return -1; // -1 表示子树已失衡
    return 1 + std::max(left, right);
}
bool balanced = heightOrBad(root) >= 0;
// 后序一次算高度，O(n) 时间、O(h) 栈；空树平衡
```

019 反转单链表的核心指针操作是什么?

答案 用 prev 保存前驱、cur 遍历当前节点，先保存 cur->next，再把 cur->next 指向 prev，最后推进两指针。循环结束后 prev 是新头节点，不能在改指针前丢失原后继。时间 O(n)、额外空间 O(1)，虚拟头结点不是必须的。 若要保留原链表，可新建节点复制；这会把额外空间增至 O(n)。

```
// 反转前保存 next，避免改写链接后丢掉未处理后缀。
Node* previous = nullptr; Node* current = head;
while (current) {
    Node* next = current->next; // 改链前先保存后继
    current->next = previous; previous = current; current = next;
}
head = previous; // 原尾成为新头
// 原地 O(n) 时间、O(1) 额外空间；空链表也返回 nullptr
```

020 和为 K 的子数组个数如何计算?

答案 遍历数组维护前缀和 p；以当前位置为终点的和为 K 的子数组数量，等于之前出现过 p-K 的前缀和频次。先令频次表中 0 出现一次以涵盖从下标 0 开始的区间，然后计答案、再记录当前 p。平均 O(n) 时间、O(n) 空间，前缀和及答案都可能需要 64 位。 该方法适用于包含负数的数组，滑动窗口在负数场景通常不成立。

```
// prefix-k 的旧频次就是以当前位置结尾的合法子数组数。
std::unordered_map<long long, long long> seen{{0, 1}};
long long prefix = 0, count = 0;
for (int x : a) { prefix += x;
    count += seen[prefix - k]; // 先查旧前缀，涵盖负数与从下标 0 开始的段
    ++seen[prefix];
}
return count; // 平均 O(n) 时间、O(n) 空间；和与答案使用 64 位
```

021 图中的环检测（Cycle Detection）

答案 无向图 DFS 时，遇到已访问且不是当前节点父边的邻点即有环；多重图需按边 ID 区分父边，否则平行边可能漏判。对有向图用白/灰/黑三色，访问到灰色节点表示回边和环。时间 O(V+E)，要遍历所有分量，不能只从一个起点检查。无向图自环也是长度为一的环，简单图与多重图的边定义要统一。DFS 颜色状态能在线性时间检测有向环，若只要拓扑序也可复用该结果。

```
// 有向图灰色回边与无向图非父返边的判环条件不同。
// 有向图看灰色回边；无向图忽略父边，平行边用 edgeId 区分。
bool directedCycle(int u) { state[u] = 1;
    for (int v : graph[u]) if (state[v] == 1 || (state[v] == 0 && directedCycle(v))) return true;
    state[u] = 2; return false; }
bool undirectedCycle(int u, int parentEdge) { seen[u] = true;
    for (auto [v, edgeId] : graph[u]) if (edgeId != parentEdge && (seen[v] || undirectedCycle(v, edgeId))) return true;
    return false; }
bool hasCycle(bool directed) {
    if (directed) { for (int u = 0; u < n; ++u) if (state[u] == 0 && directedCycle(u)) return true; return false; }
    std::fill(seen.begin(), seen.end(), false);
    for (int u = 0; u < n; ++u) if (!seen[u] && undirectedCycle(u, -1)) return true;
    return false;
} // 遍历全部连通分量，邻接表 O(V+E)
```

022 图的广度优先搜索（BFS）

答案 BFS 将起点入队并按层弹出，每发现未访问邻居就标记后入队，标记应在入队时做以免重复排队。无权图中首次发现节点的路径边数最少，因此可同时保存 parent 重建路径。邻接表实现时间 O(V+E)、空间 O(V)，边权非一致时不能直接用 BFS 求最短路。 保存每个节点首次入队的父节点，可按逆序回溯出一条最短路径。

```
// 首次入队即固定无权最短距离，并可记录 parent 重建路径。
std::queue<int> q; distance[start] = 0; q.push(start);
while (!q.empty()) { int u = q.front(); q.pop(); order.push_back(u);
    for (int v : graph[u]) if (distance[v] == -1) {
        distance[v] = distance[u] + 1; parent[v] = u; q.push(v); // 入队即标记
    }
}
// 无权图首次发现即为最少边数；邻接表 O(V+E)，带权图不适用
```

023 图的强连通分量（Strongly Connected Components）

答案 Kosaraju 第一次 DFS 记录节点完成时间顺序，再反转所有边，并按完成时间从大到小在反图做 DFS；每次新遍历得到一个强连通分量。每个节点和边被扫描有限次，时间 O(V+E)，需要保存反图和访问状态，空间 O(V+E)。大图递归 DFS 可能栈溢出，可改显式栈或用 Tarjan。 有向图反转边不会改变强连通分量集合，但决定第二次 DFS 的搜索方向。

```
// 第一遍完成次序决定第二遍在反图搜索的 SCC 边界。
void finishDfs(int u) { seen[u] = true; for (int v : graph[u]) if (!seen[v]) finishDfs(v); order.push_back(u); }
void collect(int u) { seen[u] = true; component.push_back(u); for (int v : reverseGraph[u]) if (!seen[v]) collect(v); }
for (int u = 0; u < n; ++u) if (!seen[u]) finishDfs(u);
std::fill(seen.begin(), seen.end(), false);
for (auto it = order.rbegin(); it != order.rend(); ++it) if (!seen[*it]) {
    component.clear(); collect(*it); groups.push_back(component);
}
// 反图保存边方向反转；总 O(V+E) 时间、O(V+E) 空间
```

024 图的深度优先搜索（DFS）

答案 从未访问节点开始沿邻边深入，递归返回时回溯；visited 必须在递归前标记，避免环导致无限访问。递归写法简洁，显式栈则适合避免深图调用栈限制；两者时间 O(V+E)、空间 O(V)。若图不连通，外层仍需依次启动未访问节点。有向图若要识别回边，需用三色状态区分“已完成”和“当前路径”。 无向图遍历还需忽略已访问的父边，否则会把树边误判为环。

```
// visited 必须在递归前设置，并从未访问节点覆盖所有分量。
void dfs(int u) { seen[u] = true; order.push_back(u); // 首次发现即标记，防止环中重复递归
    for (int v : graph[u]) if (!seen[v]) dfs(v);
}
for (int u = 0; u < n; ++u) if (!seen[u]) dfs(u); // 覆盖非连通分量
// 邻接表 O(V+E) 时间、O(V) 访问标记；递归深度最坏 O(V)
```

025 图的连通性检测

答案 无向图从任意顶点做 BFS/DFS，访问数等于顶点数则连通；邻接表下时间 O(V+E)。空图是否视为连通要按题意定义，单点图通常连通。若题目要求有向图强连通，单次从一个点可达全体并不足够，应使用 Kosaraju、Tarjan 或等价判定。 邻接表扫描全部连通分量仍是 O(V+E)，不会重复处理每条边很多次。

```
// 连通性由从起点可达的节点数与顶点总数比较。
if (n == 0) return true; // 空图是否连通按题意定义
std::queue<int> q; seen[0] = true; q.push(0);
while (!q.empty()) { int u = q.front(); q.pop();
    for (int v : graph[u]) if (!seen[v]) { seen[v] = true; q.push(v); }
}
bool connected = std::all_of(seen.begin(), seen.end(), [](bool x) { return x; });
// 无向图邻接表 O(V+E)；有向图强连通不能只做一次可达遍历
```

026 在旋转排序数组中查找元素

答案 每轮判断中点左侧还是右侧包含旋转断点，再检查目标是否落在有序半区，保留可能区间。无重复元素时每轮缩半，时间 O(log n)、空间 O(1)；有重复值会让端点与中点相等而无法判断，此时需收缩边界，最坏 O(n)。边界比较要避免遗漏端点。若只要任意命中值，可在找到目标时立即结束，不必继续找边界。推荐维护闭区间时用 while (left <= right)，并统一以 mid±1 收缩。

```
// 每轮先识别一侧有序区间，只保留目标可能所在半区。
int left = 0, right = static_cast<int>(a.size()) - 1;
while (left <= right) { int mid = left + (right - left) / 2;
    if (a[mid] == target) return mid;
    if (a[left] <= a[mid]) { if (a[left] <= target && target < a[mid]) right = mid - 1; else left = mid + 1; }
    else { if (a[mid] < target && target <= a[right]) left = mid + 1; else right = mid - 1; }
}
return -1; // 无重复时 O(log n)；有重复且端点相等需收缩，最坏 O(n)
```

027 基数排序（Radix Sort）

答案 基数排序按数字位或字符位多轮排序，每轮使用稳定排序，低位优先时才能保留较低位次序。若每位用计数排序，时间 O(d(n+k))，空间 O(n+k)，d 为位数、k 为每位值域。负数需做符号映射或分开处理，不能直接假定所有整数都是非负定长。 字符串键可逐字符做基数排序，但排序方向和字符编码顺序须先定义。

```
// 低位到高位逐位稳定排序，稳定性传递低位顺序。
// 假设键均为非负整数；每一位必须稳定排序
std::vector<int> out(a.size());
for (long long exp = 1; maxValue / exp > 0; exp *= 10) {
    std::array<int, 10> count{}; for (int x : a) ++count[(x / exp) % 10];
    for (int d = 1; d < 10; ++d) count[d] += count[d - 1];
    for (auto it = a.rbegin(); it != a.rend(); ++it) out[--count[(*it / exp) % 10]] = *it;
    std::swap(a, out); // exp 从低位到高位，稳定性保留低位次序
}
// d 位、基数 10：O(d(n+10)) 时间、O(n+10) 空间；负数需另处理
```

028 堆排序（Heap Sort）

答案 先将数组建成大顶堆，再反复交换堆顶最大值与当前末尾，缩小堆范围并向下调整根节点。建堆可在线性 O(n) 完成，整体 O(n log n)，原地且不稳定。若要升序输出使用大顶堆；处理空数组和单元素无需特殊搬移。堆顶和末尾交换后只需向下调整根，因为左右子堆仍满足堆性质。 每次下滤时选择较大的孩子与父节点比较，直到局部堆序恢复。

```
// 大顶堆根是当前最大值，交换后缩小有效堆区间。
void siftDown(std::vector<int>& a, int root, int size) {
    while (2 * root + 1 < size) { int child = 2 * root + 1;
        if (child + 1 < size && a[child] < a[child + 1]) ++child;
        if (a[root] >= a[child]) break; std::swap(a[root], a[child]); root = child;
    }
}
for (int i = n / 2 - 1; i >= 0; --i) siftDown(a, i, n); // O(n) 建大顶堆
for (int end = n - 1; end > 0; --end) { std::swap(a[0], a[end]); siftDown(a, 0, end); }
// 原地升序、O(n log n)，不稳定；每轮只需修复根
```

029 如何使用 C++ 实现二分查找树?

答案 BST 查找时比较当前键并选择左右子树，插入在对应空位置建立节点；删除要区分叶节点、单孩子和双孩子，可用后继/前驱替换。操作复杂度 O(h)，未平衡时 h 可达 n；需要有界复杂度优先用 std::map 等平衡容器。自写实现还要定义重复键策略和节点内存所有权。 搜索递归版易实现，但深度退化树会造成栈深和性能问题。

```
// BST 每一步按键比较收缩到一条搜索路径；高度未必平衡。
Node* find(Node* p, int key) { while (p && p->key != key) p = key < p->key ? p->left : p->right; return p; }
Node* insert(Node* p, int key) { if (!p) return new Node{key};
    if (key < p->key) p->left = insert(p->left, key); else if (key > p->key) p->right = insert(p->right, key);
    return p; // 重复键按策略忽略
}
Node* erase(Node* p, int key) { if (!p) return nullptr;
    if (key < p->key) p->left = erase(p->left, key); else if (key > p->key) p->right = erase(p->right, key);
    else if (!p->left || !p->right) { Node* child = p->left ? p->left : p->right; delete p; return child; }
    else { Node* s = p->right; while (s->left) s = s->left; p->key = s->key; p->right = erase(p->right, s->key); }
    return p;
}
// 查找/插入/删除 O(h)，双子树删除用右子树最小后继；需明确内存释放策略
```

030 如何使用 STL 实现有序容器?

答案 set/map 保存唯一键，multiset/multimap 允许重复键；关联容器按比较器维持顺序，便于 lower_bound、范围查询和有序遍历。查找、插入、删除通常 O(log n)，不保证连续存储。自定义键要提供满足严格弱序的比较器，否则容器行为不符合预期。 需要按插入顺序遍历时应选 unordered 容器之外的顺序容器，map 也不保插入序。

```
// 按比较器顺序组织键，std::set/std::map 与 multi 版本决定重复键行为。
std::set<int> ordered{4, 1, 3}; ordered.insert(2);
auto firstAtLeast = ordered.lower_bound(3); // 有序范围查询 O(log n)
std::map<std::string, int> score; score["Ada"] = 10;
for (const auto& [name, value] : score) consume(name, value); // 按键排序遍历
std::multiset<int> allowDuplicates; allowDuplicates.insert(5); allowDuplicates.insert(5);
// std::set/std::map 键唯一，multi 版本允许重复；查找插入删除 O(log n)，非连续存储
```

031 如何使用 std::map 和 std::unordered_map?

答案 std::map 按比较器有序，查找、插入和删除 O(log n)，支持范围查询；std::unordered_map 按哈希分桶，平均 O(1)，冲突严重时最坏 O(n)，遍历顺序不稳定。unordered_map 扩容会使迭代器失效，引用/指针规则不同于迭代器。按是否需要排序、稳定复杂度和可哈希键选择。

```
// 是否需要范围有序遍历决定 std::map 与 std::unordered_map 的选择。
std::map<std::string, int> ordered; ordered["key"] = 1;
if (auto it = ordered.find("key"); it != ordered.end()) use(it->second); // 有序，O(log n)
std::unordered_map<std::string, int> hashed; ++hashed["key"];
auto found = hashed.find("missing"); if (found == hashed.end()) handleMissing();
// 哈希表平均 O(1)、最坏 O(n)，遍历顺序不稳定；std::map 支持范围查询
// std::unordered_map 扩容会使迭代器失效，选择取决于顺序和复杂度要求
```

032 如何使用 std::vector 和 std::array?

答案 vector<T> 运行时长度可变，元素连续，适合动态数组和传给要求连续内存的接口；扩容会分配新存储并移动元素。array<T,N> 长度 N 编译期固定，通常直接内嵌存储且没有动态分配。二者随机访问 O(1)，都提供连续元素；还需考虑对象大小、生命周期和是否允许调整长度。 array 的 N 可为零；vector 还可显式 reserve 预留容量以减少扩容次数。

```
// std::vector 长度可变、std::array 长度编译期固定，两者元素连续。
std::vector<int> dynamic; dynamic.reserve(100); dynamic.push_back(7); // 运行期可变且连续
std::array<int, 3> fixed{{1, 2, 3}}; // 长度编译期固定，也连续
int first = dynamic[0], second = fixed.at(1); // 随机访问 O(1)
// std::vector 扩容可能搬迁元素；reserve 可减少扩容次数
// std::array 不动态分配长度；二者都可提供连续元素数据
```

13.4 链表、字符串与基础数据结构

033 如何删除链表倒数第 N 个节点?

答案 设置虚拟头节点，快指针先从 dummy 向前走 n 步，再让快慢指针一起走到快指针到尾部；此时慢指针后继就是待删节点。删除后继并返回 dummy.next，统一处理删除原头节点。时间 O(n)、空间 O(1)，先验证 n 在链表长度范围内可避免空指针。

```
// 快慢指针保持 n 个节点距离，慢指针停在待删节点前。
if (n <= 0) return head; // 非法 n 按接口约定处理
Node dummy{0, head}; Node* fast = &dummy; Node* slow = &dummy;
for (int i = 0; i < n && fast; ++i) fast = fast->next;
if (!fast) return head; // n 大于链表长度时不删除
while (fast->next) { fast = fast->next; slow = slow->next; }
slow->next = slow->next->next; return dummy.next;
// 单趟 O(n) 时间、O(1) 空间；dummy 统一覆盖删除头节点
```

034 如何判断一个字符串是回文?

答案 左右指针分别从字符串两端向中间移动，按题目规则跳过标点或空格后比较对应字符，不等即可判非回文。忽略大小写时先统一大小写或按规则比较，时间 O(n)、额外空间 O(1)。C++ 的 std::string 按字节存储，若输入为 UTF-8 且需按 Unicode 字符判断，必须先解码。 若比较的字节包含组合字符，视觉上相同的文本也可能有不同编码序列。

```
// 左右端字符按题目规范化后逐对比较，跳过规则需一致。
bool palindrome(const std::string& s) { int left = 0, right = static_cast<int>(s.size()) - 1;
    while (left < right) {
        while (left < right && ignored(s[left])) ++left; // 若题目忽略标点则跳过
        while (left < right && ignored(s[right])) --right;
        if (lower(s[left++]) != lower(s[right--])) return false;
    }
    return true;
}
// O(n) 时间、O(1) 额外空间；UTF-8 应先按 Unicode 字符解码
```

035 如何判断两个链表是否相交?

答案 链表相交指节点地址相同，而不是节点值相同；分别遍历到尾部后，令指针切换到另一条链表头，最终会在交点或同时到空处相遇。时间 O(m+n)、空间 O(1)，无需预先求长度，空链表也自然适用。若题意是值相同，则那是不同问题。 两链表长度差由“换头后走完两段”自动抵消，无需额外长度计算。交点之后两条链表共享同一后缀，所以尾节点地址不同可以先排除相交。

```
// 换到另一链表后抵消两表长度差，按节点地址判断交点。
Node* a = headA; Node* b = headB;
while (a != b) { a = a ? a->next : headB; b = b ? b->next : headA; }
return a; // 同时到 nullptr 表示不相交
// 相交按节点地址相同判定，不是值相等；换头抵消长度差
// O(m+n) 时间、O(1) 空间，空链表自然返回 nullptr
```

036 如何判断二叉树是否对称?

答案 递归比较一对镜像节点：二者同时为空才匹配，只有一边为空或值不同则失败；继续比较左节点的左孩子与右节点的右孩子，以及相反方向。时间 O(n)、递归栈 O(h)，也可把节点对压入队列迭代。空树通常对称，重复值不会影响结构判断。 用节点值相等不够，还必须比较两侧结构是否按镜像方向对应。 迭代法每次把两个待比较子树成对入队，能避免极深树的递归风险。

```
// 每次比较一对镜像位置，结构和节点值都必须对应。
bool mirror(Node* a, Node* b) {
    if (!a || !b) return a == b; // 必须同时为空
    return a->value == b->value && mirror(a->left, b->right) && mirror(a->right, b->left);
}
bool symmetric = !root || mirror(root->left, root->right);
// 同时比较值与镜像结构；每节点至多访问一次，O(n) 时间、O(h) 栈
```

037 如何判断链表是否有环并找到入环点?

答案 快慢指针从头出发，慢走一步、快走两步；若相遇则有环，若快指针或其 next 为空则无环。相遇后把一个指针放回头节点，两者每次各走一步，第二次相遇点是环入口。证明依赖相遇时两者在环上的距离关系；时间 O(n)、空间 O(1)。若只判断有无环，可在首次快慢相遇时返回，不需要第二阶段。有环时快慢指针必相遇，因为相对速度为每轮一个环内节点。

```
// 首次相遇用于确认有环，第二阶段交点才是入环位置。
Node* slow = head; Node* fast = head;
do { if (!fast || !fast->next) return nullptr; slow = slow->next; fast = fast->next->next; } while (slow != fast);
Node* fromHead = head;
while (fromHead != slow) { fromHead = fromHead->next; slow = slow->next; }
return fromHead; // 两指针再次相遇即环入口
// 有环时快慢必相遇；时间 O(n)、额外空间 O(1)
```

038 如何原地合并两个有序数组?

答案 第一个数组预留了 m+n 个位置时，从两数组有效区间尾部比较，将较大值写入第一个数组末尾并向前移动。尾部填充不会覆盖尚未比较的数据；当第二数组耗尽时第一个数组剩余内容已在正确位置。时间 O(m+n)、额外空间 O(1)，前提是第一个数组容量足够。 尾部写入过程中若第一数组已无有效元素，直接复制第二数组剩余部分。

```
// 从尾部写入避免覆盖 a 尚未处理的前缀。
int i = m - 1, j = n - 1, write = m + n - 1;
while (j >= 0) { // 从后向前写，不覆盖 a 中尚未读取的有效元素
    if (i >= 0 && a[i] > b[j]) a[write--] = a[i--];
    else a[write--] = b[j--];
}
// b 用尽时 a 剩余前缀原位有序；O(m+n) 时间、O(1) 额外空间
```

039 如何合并两个有序链表?

答案 用虚拟头节点逐次比较两个链表当前值，把较小节点接到结果尾部并推进对应指针；一方耗尽后直接接上另一方剩余部分。复用原节点时额外空间 O(1)、时间 O(m+n)，相等时固定优先取左侧可保持稳定次序。若不能修改输入节点，则需分配新节点。迭代时每轮至少消耗一个节点，保证链表长度有限时算法终止。合并完成后结果总节点数为 m+n，不应额外复制或重复链接已消费节点。

```
// 每次摘取较小头节点，尾指针始终指向结果末端。
Node dummy{}; Node* tail = &dummy;
while (a && b) { Node*& pick = a->value <= b->value ? a : b;
    tail->next = pick; pick = pick->next; tail = tail->next; // 复用节点，左侧优先保持稳定
}
tail->next = a ? a : b; return dummy.next;
// 一方耗尽后直接接剩余链；O(m+n) 时间、O(1) 额外空间
```

040 如何合并重叠区间?

答案 先按左端点排序，遍历时维护当前合并区间；下一段与当前段重叠或相接时扩展右边界，否则输出当前段并开始新区间。相接是否算重叠取决于端点是闭区间还是开区间。排序耗时 O(n log n)，额外空间视排序实现和输出是否计入而定。 例如闭区间 [1,3] 与 [3,5] 是否合并，取决于题目是否把端点相接算重叠。

```
// 已排序区间只需比较当前新区间与结果末尾的右边界。
std::sort(intervals.begin(), intervals.end(), [](const auto& a, const auto& b) { return a.start < b.start; });
std::vector<Interval> merged;
for (const auto& x : intervals) {
    if (merged.empty() || merged.back().end < x.start) merged.push_back(x);
    else merged.back().end = std::max(merged.back().end, x.end); // 合并闭区间重叠/相接
}
// 空输入返回空结果；排序 O(n log n)，扫描 O(n)
// 开区间是否合并端点相接要按题意调整比较符
```

041 如何在 C++ 中实现并查集?

答案 初始化每个元素的父节点为自身，find 沿父指针找代表元并做路径压缩；union 比较代表元，再按秩或集合大小挂接较小树。路径压缩和按秩合并结合后，m 次操作摊还 O(mα(n))，可高效维护只增不删的连通关系。删除边或动态连通性需要其他数据结构。 并查集只支持合并而不支持高效拆分，不能直接用于任意删除关系。

```
// 代表元标识集合；路径压缩与按大小合并控制树高。
int find(int x) { return parent[x] == x ? x : parent[x] = find(parent[x]); } // 路径压缩
bool unite(int a, int b) { a = find(a); b = find(b); if (a == b) return false;
    if (size[a] < size[b]) std::swap(a, b); // 小树挂大树
    parent[b] = a; size[a] += size[b]; return true;
}
// 初始化 parent[i]=i、size[i]=1；合并+压缩摊还 O(α(n))
// 适合只合并不拆分的动态连通关系
```

042 如何实现 KMP 算法?

答案 KMP 先计算模式串每个前缀的最长相等真前后缀长度；文本失配时将模式位置回退到该长度，而文本指针不回退。这样利用已匹配信息避免重复比较，预处理 O(m)、扫描 O(n)，总计 O(n+m)。空模式和字符比较规则需按接口定义处理。前缀数组的定义有多种写法，构造和失配回退必须保持同一套下标约定。面试中可以手算模式串 `ababaca` 的前缀表，解释失配时为何不回退文本指针。

```
// pi 记录可复用的前缀长度，失配时文本下标不回退。
// pi[i] 是 pattern[0..i] 的最长相等真前后缀长度
if (pattern.empty()) return {0}; // 若接口要求所有边界位置，在此改为返回 0..text.size()
std::vector<int> pi(pattern.size());
for (int i = 1, j = 0; i < pattern.size(); ++i) {
    while (j && pattern[i] != pattern[j]) j = pi[j - 1];
    if (pattern[i] == pattern[j]) ++j; pi[i] = j;
}
for (int i = 0, j = 0; i < text.size(); ++i) {
    while (j && text[i] != pattern[j]) j = pi[j - 1];
    if (text[i] == pattern[j]) ++j;
    if (j == pattern.size()) { matches.push_back(i - j + 1); j = pi[j - 1]; }
}
// 空模式需单独定义；预处理 O(m)、扫描 O(n)，文本指针不回退
```

043 如何实现一个链表?

答案 单链表节点通常含值和 next，双链表再含 prev；插入删除要分别处理头、尾、空表和单节点边界。查找节点 O(n)，已知节点位置改指针可 O(1)，但拥有节点的代码必须负责释放且避免重复释放。现代 C++ 一般优先用 std::list 或容器，只有明确需要自定义链式结构时才手动管理。 C++ 可用 unique_ptr 表达单向所有权，避免手写析构遗漏节点。

```
// head 独占整条单链表，移动 std::unique_ptr 转移所有权。
struct Node { int value; std::unique_ptr<Node> next; };
std::unique_ptr<Node> head;
void pushFront(int x) { auto node = std::make_unique<Node>(); node->value = x;
    node->next = std::move(head); head = std::move(node);
}
void popFront() { if (head) head = std::move(head->next); }
// 单链表头插/头删 O(1)，std::unique_ptr 自动释放整条所有权链
// 查找仍为 O(n)；双链表还需维护 prev 与头尾边界
```

044 如何实现动态规划?

答案 先把状态定义成能回答目标的量，例如 dp[i] 表示前 i 个元素的最优值，再从最后一步推导转移与边界。依据依赖关系确定循环顺序，滚动数组压缩空间前要确认旧状态不会再被需要。最后用小输入手算，并检查每个合法方案是否都被状态转移覆盖。 例如零钱兑换按金额递增计算，而 0/1 背包按容量递减以限制物品使用次数。

```
// dp[x] 代表金额 x 的最优值，状态按依赖从小到大计算。
// dp[x]：组成金额 x 所需的最少硬币数；硬币面值必须为正
const int inf = amount + 1; std::vector<int> dp(amount + 1, inf); dp[0] = 0;
for (int x = 1; x <= amount; ++x) for (int coin : coins) {
    if (coin <= x && dp[x - coin] != inf) dp[x] = std::min(dp[x], dp[x - coin] + 1);
}
return dp[amount] == inf ? -1 : dp[amount];
// 先定义状态/边界，再按依赖从小到大计算；O(amount * coins) 时间
```

045 如何实现哈希表?

答案 哈希表用哈希函数将键映射到桶，碰撞时可用链地址或开放寻址继续查找；相等的键必须产生相同哈希值。负载因子过高时扩容重哈希，删除操作在开放寻址中需采用墓碑等策略避免搜索链断裂。平均查找 O(1)，恶意碰撞或不佳分布下可退化 O(n)。 自定义键还需同步实现相等比较与哈希函数，二者必须保持一致。 对开放寻址表，装载因子和探测序列共同影响聚簇与查找长度。

```
// 桶下标由哈希函数决定，桶内相等比较解决碰撞。
// 链地址法：哈希相同的键进入同桶，再用相等比较区分
if (capacity == 0) capacity = 1; std::vector<std::vector<Entry>> buckets(capacity);
auto bucket = [&](const Key& key) { return std::hash<Key>{}(key) % buckets.size(); };
Entry* find(const Key& key) { for (Entry& e : buckets[bucket(key)]) if (equal(e.key, key)) return &e; return nullptr; }
void insert(Key key, Value value) { auto& chain = buckets[bucket(key)];
    for (Entry& e : chain) if (equal(e.key, key)) { e.value = value; return; }
    chain.push_back({std::move(key), std::move(value)}); ++size;
    if (static_cast<double>(size) / buckets.size() > maxLoad) rehash();
}
// rehash 需重算所有桶下标；开放寻址删除要用墓碑避免探测链断开
// 平均查找 O(1)，恶意碰撞或不佳分布下可退化 O(n)
```

13.5 常见结构与搜索算法

046 如何实现图的 DFS 和 BFS?

答案 邻接表保存每个顶点的邻居，DFS 用递归/显式栈深入，BFS 用队列逐层访问；两者均在首次发现时标记 visited。DFS 适合连通分量与回溯，BFS 在无权图中能求最少边数路径。遍历所有节点和边的时间 O(V+E)，递归深度可能达到 V。 无权最短路选 BFS；带权非负图应改用 Dijkstra 而不是 DFS。

```
// DFS 深入回溯，BFS 按层扩展并在无权图求最短边数。
void dfs(int u) { seen[u] = true; for (int v : graph[u]) if (!seen[v]) dfs(v); }
for (int u = 0; u < n; ++u) if (!seen[u]) dfs(u); // 覆盖所有分量
std::fill(seen.begin(), seen.end(), false); // 重新初始化后从 start 做 BFS
std::queue<int> q; seen[start] = true; distance[start] = 0; q.push(start);
while (!q.empty()) { int u = q.front(); q.pop();
    for (int v : graph[u]) if (!seen[v]) { seen[v] = true; distance[v] = distance[u] + 1; q.push(v); }
}
// 首次发现时标记；DFS 用于深入/回溯，BFS 求无权最少边数
// 邻接表两者 O(V+E)；带权非负图用 Dijkstra
```

047 如何实现堆排序?

答案 大顶堆可用数组表示，子节点下标由父下标计算；从最后一个非叶节点开始向下调整可在线性时间建堆。之后将堆顶与末尾交换并缩小有效区间，逐步输出升序结果。总时间 O(n log n)、额外空间 O(1)，堆排序不稳定。数组从零开始时左右子节点索引为 2*i+1 和 2*i+2。 自底向上 heapify 每个节点的工作量随层高递减，所以建堆不是 O(n log n)。

048 如何实现快速排序和归并排序?

答案 快速排序选 pivot 分区后递归，平均 O(n log n)、最坏 $O(n^2)$，通常原地且不稳定；随机 pivot 或三数取中可降低常见输入退化概率。归并排序递归拆半再合并，时间稳定 O(n log n)，数组实现通常需 O(n) 临时空间且稳定。选择时看内存限制、稳定性和最坏时间要求。 对重复 pivot 键，三路划分能把相等区间一次处理完，减少递归规模。

049 如何实现栈和队列?

答案 栈按后进先出，用 vector/deque 支持 push、pop、top；队列按先进先出，用 deque 或固定数组环形缓冲区支持入队出队。动态容器常见端操作均摊 O(1)，环形队列需用计数或空槽规则区分满与空。多线程环境还需同步，普通容器本身不保证并发安全。 STL 的 std::stack 和 std::queue 是容器适配器，可用 deque 作为底层容器。

```
// 栈操作尾端、队列操作两端，读取或弹出前要检查空状态。
std::stack<int> lifo; lifo.push(x); int top = lifo.top(); lifo.pop();
std::queue<int> fifo; fifo.push(x); int front = fifo.front(); fifo.pop();
// 栈后进先出，队列先进先出；先检查 empty 再读 top/front 或 pop
// std::stack/std::queue 是容器适配器，端操作通常 O(1)
// 固定数组队列可用 head/tail 环绕，并用计数区分满和空
```

050 如何找到二叉树的最低公共祖先（LCA)?

答案 后序递归在目标节点命中时返回自身，空子树返回空；若左右结果都非空，当前节点就是最低公共祖先，否则返回那一侧的非空结果。时间 O(n)、空间 O(h)，适用于普通二叉树。若节点不存在时也要求返回空，需额外统计两目标是否均被找到；BST 可利用键值顺序优化。 若树是二叉搜索树，可比较两个目标值与根值并只递归一个子树。

```
// 左右递归都返回目标时当前节点是分叉最低点。
Node* lca(Node* p, Node* a, Node* b) { if (!p || p == a || p == b) return p;
    Node* left = lca(p->left, a, b); Node* right = lca(p->right, a, b);
    if (left && right) return p; // 两侧各找到一个，当前节点是最低公共祖先
    return left ? left : right;
}
// 普通二叉树 O(n) 时间、O(h) 栈；若节点可能不存在需另计数确认两者均找到
```

051 如何求二叉树右视图?

答案 层序遍历时记录每层最后弹出的节点，即为从右侧看到的值；也可 DFS 按右、左顺序先访问，并在首次到达某深度时记录节点。时间 O(n)，队列法空间 O(w)，DFS 栈 O(h)。树为空时结果为空，DFS 记录深度时要确保数组按层扩展。 若用 DFS，必须先访问右孩子再访问左孩子，才能保证同层首次出现的是最右节点。

```
// 队列中固定的一层宽度内，最后一个弹出的节点是右视值。
if (!root) return view;
std::queue<Node*> q; q.push(root);
while (!q.empty()) { int width = q.size();
    for (int i = 0; i < width; ++i) { Node* p = q.front(); q.pop();
        if (i == width - 1) view.push_back(p->value); // 每层最后弹出者
        if (p->left) q.push(p->left); if (p->right) q.push(p->right);
    }
}
// O(n) 时间、队列 O(w) 空间；DFS 也可按右、左顺序首次记层
```

052 如何生成全排列或子集?

答案 回溯维护 path，在每个选择点尝试尚未使用的元素；全排列用 visited 或交换控制，子集则在每层先记录当前 path 再继续扩展。若输入有重复值，先排序并跳过“同层前一个相同且未使用”的元素。时间至少与输出总长度成正比，排列规模可达 n!，需要考虑结果内存。若要求字典序输出，可在排序后按固定候选顺序扩展，并按输出规模估算内存。

```
// path 是当前选择；回溯返回时必须撤销本层选择。
void subsets(int start) { out.push_back(path); // 每个递归状态本身就是一个子集
    for (int i = start; i < a.size(); ++i) {
        if (i > start && a[i] == a[i - 1]) continue; // 排序后跳过同层重复值
        path.push_back(a[i]); subsets(i + 1); path.pop_back();
    }
}
void permutations() { if (path.size() == a.size()) { out.push_back(path); return; }
    for (int i = 0; i < a.size(); ++i) if (!used[i]) {
        if (i && a[i] == a[i - 1] && !used[i - 1]) continue;
        used[i] = true; path.push_back(a[i]); permutations(); path.pop_back(); used[i] = false;
    }
}
// 输入先排序以去重；输出规模可达 2^n 或 n!，结果可能占主要内存
```

053 如何用两个栈实现队列?

答案 入队时把元素压入输入栈；出队或查看队首时，只有输出栈为空才把输入栈全部弹出并压入输出栈，然后从输出栈顶取值。一次倒序转换后后续出队无需搬动，每个元素最多进出两个栈各一次，因此操作均摊 O(1)。若输出栈非空时新元素仍留在输入栈即可保持 FIFO。

```
// 只有输出栈空才倒序搬运，之后其栈顶即队首。
std::stack<int> input, output;
void push(int x) { input.push(x); } // 新元素暂放输入栈
int front() { if (output.empty()) while (!input.empty()) { output.push(input.top()); input.pop(); }
    if (output.empty()) throw EmptyQueue{}; return output.top();
}
void pop() { (void)front(); output.pop(); }
// 仅当输出栈空才搬运，保持 FIFO；每元素搬运一次，均摊 O(1)
```

054 如何用单调栈求柱状图最大矩形?

答案 维护柱高递增的下标栈，遇到低于栈顶的高度就弹出；弹出柱子的右边界为当前下标，左边界为新栈顶之后一格，面积为高度乘宽度。末尾加入高度 0 的哨兵可统一结算所有柱子。时间 O(n)、空间 O(n)，相同高度的入栈/弹出规则要保持边界计算一致。 柱高含重复值时，统一用严格递增或非递减栈即可，但宽度边界需配套。

```
// 单调栈保存尚未找到右侧更矮柱子的下标。
for (int i = 0; i <= heights.size(); ++i) { // i==n 视作高度 0 的哨兵
    int current = i == heights.size() ? 0 : heights[i];
    while (!st.empty() && heights[st.back()] > current) {
        int h = heights[st.back()]; st.pop_back();
        int left = st.empty() ? -1 : st.back();
        best = std::max(best, h * (i - left - 1)); // 左右第一个更矮柱间的宽度
    }
    if (i < heights.size()) st.push_back(i);
}
// 每个下标入栈出栈一次，O(n) 时间、O(n) 空间；空输入答案 0
```

055 如何用哈希表解决数组查重和两数之和?

答案 查重时遍历并把已见值放入哈希集合，集合已有该值即可报告重复；两数之和则先查 target-x，再保存 x 与下标，确保不会把当前元素重复使用。平均时间 O(n)、空间 O(n)，若需返回所有索引或唯一数对，必须明确重复元素与答案去重规则。补数运算要考虑整数溢出。 若数组元素可重复，两数之和应在查找补数之后记录当前值，避免自配对。

```
// 两数之和先查补数再登记当前值，避免元素自配对。
std::unordered_set<int> seen;
for (int x : a) if (!seen.insert(x).second) duplicates.push_back(x); // 查重
std::unordered_map<int, int> index;
for (int i = 0; i < a.size(); ++i) { auto it = index.find(target - a[i]);
    if (it != index.end()) return std::pair<int, int>{it->second, i}; // 先查再存，避免自配对
    index[a[i]] = i;
}
// 平均 O(n) 时间、O(n) 空间；补数运算也要防溢出
```

056 如何用队列实现栈?

答案 使用一个队列时，每次入栈先把新元素入队，再把队列前面的 n-1 个元素依次移到队尾，让新元素位于队首；出栈直接弹队首。这样 push O(n)、pop/top O(1)，队列只需保存元素。若采用双队列交换，可让入栈 O(1) 而出栈 O(n)，按操作频率选择。

```
// 旋转旧队列元素到尾部，让新入队值位于队首模拟栈顶。
std::queue<int> q;
void push(int x) { q.push(x); for (int i = 1; i < q.size(); ++i) { q.push(q.front()); q.pop(); } }
int top() { return q.front(); }
void pop() { q.pop(); }
// 每次 push 后轮转旧元素，使新元素在队首：push O(n)，top/pop O(1)
// 空栈操作前需检查；双队列可改为 push O(1)、pop O(n)
```

057 如何解决背包问题?

答案 先明确每件物品能否重复、重量是否整数以及目标是最大价值还是方案数。0/1 背包容量倒序避免重复取当前物品，完全背包正序允许同一物品再次参与本轮转移；多重背包需拆分数量或用单调队列优化。基础动态规划时间 O(nW)、空间 O(W)，W 很大时要考虑值域或稀疏状态优化。 若返回具体选取方案而非最优值，空间压缩后还需保留决策或回溯记录。

```
// 0/1 倒序阻止重复取用；完全背包正序允许本轮重用。
// 0/1 背包：容量倒序，同件物品本轮不能再次使用
for (const auto& item : items) for (int c = W; c >= item.weight; --c)
    dp[c] = std::max(dp[c], dp[c - item.weight] + item.value);
// 完全背包：容量正序，让当前物品的本轮新状态继续参与转移
for (const auto& item : reusable) for (int c = item.weight; c <= W; ++c)
    dp[c] = std::max(dp[c], dp[c - item.weight] + item.value);
// 基础 O(nW) 时间、O(W) 空间；多重背包可二进制拆分数量
```

058 如何计算二叉树路径总和?

答案 根到叶是否存在目标和，可在 DFS 中传递剩余目标，只有到叶子且剩余值匹配才成功，负数节点也能正确处理。若统计任意起止向下路径数，用前缀和频次：当前累和 s 时累加此前 s-target 的次数，并在返回时回溯计数。需区分路径方向和起止范围，时间 O(n)、额外空间 O(h) 或路径前缀表。 回溯时移除当前前缀和计数，避免兄弟子树把彼此路径错误地组合。

```
// remain 只在从根向下路径累减，且仅叶节点可结束路径。
bool hasPath(Node* p, long long remain) { if (!p) return false;
    remain -= p->value;
    if (!p->left && !p->right) return remain == 0; // 只有叶子才能结束路径
    return hasPath(p->left, remain) || hasPath(p->right, remain);
}
// 根到叶路径，负数也适用；O(n) 时间、O(h) 栈
// 若统计任意向下路径，改用前缀和频次并在回溯时撤销计数
```

059 如何设计支持 O(1) push、pop、top 和 getMin 的最小栈?

答案 主栈保存元素，辅助栈在每次压入时记录当前最小值；弹出主栈时同步弹出辅助栈，getMin 即读辅助栈顶。相等最小值也应重复入栈，避免弹出一个后丢失剩余最小值；也可每项存值和当前最小值。操作 O(1)，额外空间 O(n)。辅助栈若只在新值小于当前最小时入栈，弹出时需同步判断是否等于最小值。 对空栈调用 top、pop 或 getMin 应规定异常、断言或错误返回行为。

```
// 每个栈项保存截至当前位置的最小值，弹出时自动恢复旧最小值。
std::stack<std::pair<int, int>> values; // 每项保存（值，到此为止的最小值）
void push(int x) { int m = values.empty() ? x : std::min(x, values.top().second); values.emplace(x, m); }
void pop() { if (!values.empty()) values.pop(); }
int top() const { return values.top().first; }
int getMin() const { return values.top().second; }
// push/pop/top/getMin 均 O(1)，空间 O(n)；空栈行为需定义
```

060 将二叉树转换为链表（Flatten Binary Tree to Linked List）

答案 若不要求 O(1) 额外空间，可做逆前序递归（右、左、根），用 prev 将节点串成右链，时间 O(n)、调用栈 O(h)。若必须原地 O(1) 空间，沿树遍历：找到当前左子树最右节点，把原右子树接到其后，再将左子树移到右侧并清空 left。展开顺序应为根、左、右；不要把递归调用误称为常数空间。迭代前序加显式栈也正确，但栈最坏 O(h)；常数空间版本需谨慎检查重连指针。

```
// 把左子树移到右侧前，先把原右子树接到左子树最右端。
Node* current = root;
while (current) {
    if (current->left) { Node* rightmost = current->left;
        while (rightmost->right) rightmost = rightmost->right;
        rightmost->right = current->right; // 接上原右子树
        current->right = current->left; current->left = nullptr;
    }
    current = current->right;
}
// 原地前序展开，额外空间 O(1)、时间最坏 O(n²) 找右边界
// 逆前序递归更简洁但需 O(h) 调用栈
```

061 岛屿数量问题（Number of Islands）

答案 逐格扫描网格，遇到未访问陆地就把岛屿数加一，并用 DFS/BFS 将上下左右相连的陆地标记为已访问。原地把 1 改成 0 可省 visited 数组，但若输入不可修改则需单独标记。每个格子至多处理一次，时间 O(MN)，最坏队列或递归栈 O(MN)；对角线是否连接按题目定义。 若递归实现的地图很大，BFS 或显式栈能避免调用栈深度受限。

```
// 每片陆地首次发现就标记，避免相邻格重复扩展。
void flood(int r, int c) { if (r < 0 || r == rows || c < 0 || c == cols || grid[r][c] != '1') return;
    grid[r][c] = '0'; // 入栈/递归前标记，避免重复访问
    flood(r + 1, c); flood(r - 1, c); flood(r, c + 1); flood(r, c - 1);
}
for (int r = 0; r < rows; ++r) for (int c = 0; c < cols; ++c)
    if (grid[r][c] == '1') { ++islands; flood(r, c); }
// O(rows*cols) 时间、最坏 O(rows*cols) 栈/队列；输入不可改则另设 visited
```

13.6 排序、组合搜索与贪心

062 归并排序（Merge Sort）

答案 归并排序把区间拆成两半递归排序，再用双指针将两个有序子区间线性合并；相等时先取左侧元素可保持稳定性。每层总合并工作 O(n)，共 O(log n) 层，因此时间 O(n log n)，数组通常需 O(n) 临时空间。链表可通过拆链和合并做到较少额外存储，也适合外部排序。 每次分割都应尽量平衡；链表排序可用快慢指针拆分，避免随机访问。

```
// 归并时相等元素优先取左半，保留稳定次序。
void mergeSort(int left, int right) { if (right - left <= 1) return;
    int mid = left + (right - left) / 2; mergeSort(left, mid); mergeSort(mid, right);
    int i = left, j = mid; tmp.clear();
    while (i < mid || j < right) tmp.push_back(j == right || (i < mid && a[i] <= a[j]) ? a[i++] : a[j++]);
    std::copy(tmp.begin(), tmp.end(), a.begin() + left);
}
// 相等时先取左项保证稳定；O(n log n) 时间、数组临时空间 O(n)
```

063 快速排序（Quick Sort）

答案 选 pivot 后用双指针或 Lomuto/Hoare 分区把元素按大小放到两侧，再递归排序子区间。平均时间 O(n log n)，极端划分会退化 O(n²)；随机 pivot、三数取中或三路分区可降低有序输入和大量重复值的退化。原地版本空间通常为递归栈，且排序一般不稳定。 三路分区把小于、等于和大于 pivot 分开，重复键很多时效果更好。

```
// 三路分区把等于 pivot 的中间段一次排定。
void quickSort(int left, int right) { if (left >= right) return;
    int pivot = a[left + (right - left) / 2], low = left, i = left, high = right;
    while (i <= high) { if (a[i] < pivot) std::swap(a[low++], a[i++]);
        else if (a[i] > pivot) std::swap(a[i], a[high--]); else ++i; }
    quickSort(left, low - 1); quickSort(high + 1, right); // 中间等值段已完成
}
// 平均 O(n log n)，坏划分 O(n²)；三路划分利于大量重复值，排序不稳定
```

064 打家劫舍问题的状态转移是什么?

答案 按 0 下标数组定义，可令 dp[i] 为考虑前 i 间房的最大收益，转移为 max(dp[i-1], dp[i-2]+nums[i-1])，并正确设置 i=0、1 的边界。只需滚动保存前两项，时间 O(n)、空间 O(1)。环形街区不能直接套此式，应分别排除首间或末间求最大值。 该递推默认相邻房屋不能同时选择，若可选房屋为空则收益为零。

```
// prev1/prev2 分别表示前一间与前两间范围的最优收益。
long long prev2 = 0, prev1 = 0; // 前 0/1 间房的最优收益
for (int money : houses) { long long now = std::max(prev1, prev2 + money);
    prev2 = prev1; prev1 = now; // 不抢当前 vs 抢当前
}
return prev1; // O(n) 时间、O(1) 空间；允许一间不抢故初值为 0
// 环形街区需分别计算不含首间与不含末间两段再取最大
```

065 找出一个排序数组中的重复元素

答案 若数组已排序，扫描相邻元素相等即可找重复值；要列出所有重复值时每组只输出一次并跳过后续重复项。若只判断是否存在且值域/单调谓词满足条件，可二分计数，但不能对任意排序数组凭空二分重复值。完整扫描 O(n)，先排序则通常 O(n log n)，题意需说明返回一个还是全部。

```
// 相同元素在已排序数组中形成连续的一组。
if (a.empty()) return;
for (int i = 0; i < a.size();) {
    int j = i + 1; while (j < a.size() && a[j] == a[i]) ++j;
    if (j - i > 1) duplicates.push_back(a[i]); // 每组只报告一次
    i = j;
}
// 输入已排序，单次扫描 O(n)、O(1) 额外空间；返回一个重复值可提前停
```

066 拓扑排序（Topological Sorting）

答案 Kahn 算法统计入度为零节点并逐个删除其出边；若输出节点数少于 V，则图中有环。DFS 算法沿边访问，完成节点后压栈，最后逆完成顺序输出；遇到当前递归栈中的节点表示有向环。时间 O(V+E)，拓扑序可能不唯一，可用优先队列按指定规则取零入度节点。 有多个合法序时，使用最小堆维护零入度节点可得到字典序最小结果。

```
// 零入度队列逐步删除依赖；未输出顶点意味着图内有环。
std::queue<int> ready; for (int u = 0; u < n; ++u) if (indegree[u] == 0) ready.push(u);
while (!ready.empty()) { int u = ready.front(); ready.pop(); order.push_back(u);
    for (int v : graph[u]) if (--indegree[v] == 0) ready.push(v);
}
if (order.size() != n) return {}; // 仍有入度节点说明存在有向环
// O(V+E) 时间；有多个零入度点时拓扑序不唯一
```

067 括号生成问题如何剪枝?

答案 构造过程中左括号最多放 n 个，右括号只有在 left > right 时才能放，从而每个前缀都保持合法，不必生成无效串再过滤。达到长度 2n 即输出；结果数量为第 n 个 Catalan 数，增长很快，时间至少与输出量相关。n=0 时通常产生一个空字符串，需依接口约定处理。 左括号数量达到 n 后只能闭合，闭合数追平时必须继续开新括号。

```
// close 不得超过 open，所有递归前缀始终合法。
void generate(int open, int close) {
    if (path.size() == 2 * n) { out.push_back(path); return; }
    if (open < n) { path.push_back('('); generate(open + 1, close); path.pop_back(); }
    if (close < open) { path.push_back(')'); generate(open, close + 1); path.pop_back(); }
}
// close<=open 保证每个前缀合法；n=0 通常输出空串
// 时间至少与 Catalan 数量级的输出量相同
```

068 插入排序（Insertion Sort）

答案 把左侧维持为有序区间，每轮取下一个元素，向右移动所有更大的元素，再将它放入空位；移动而非交换可保持相同元素的相对顺序。算法稳定、原地，最好近乎有序时 O(n)，平均和最坏 $O(n^2)$。适合小规模或近乎有序数据，也常用于混合排序的短区间。 对含大量相等键的记录排序时，稳定性可以保留先前排序建立的次序。

```
// 每轮开始时左侧区间有序，将当前值插入其正确位置。
for (int i = 1; i < n; ++i) { int value = a[i], j = i;
    while (j > 0 && a[j - 1] > value) { a[j] = a[j - 1]; --j; } // 有序前缀右移
    a[j] = value;
}
// 相等元素不移动，故稳定；原地，近乎有序最好 O(n)，平均/最坏 O(n²)
```

069 数组中的第 K 大元素如何求解?

答案 快速选择按 pivot 分区，只递归包含目标排名的一侧，平均 O(n)、最坏 $O(n^2)$，适合一次静态查询；随机化 pivot 可降低常见退化。维护大小 K 的最小堆需 O(n log K) 时间、O(K) 空间，适合流式数据和 K 远小于 n。第 K 大对应升序下标 n-K，需处理 K 越界和重复值排名定义。

```
// partition 将 pivot 放到最终排名，只继续目标所在一侧。
// 三路 partition 避免重复值堆在某一侧；目标下标为 n-k
if (k < 1 || k > a.size()) throw InvalidInput{};
int target = static_cast<int>(a.size()) - k, left = 0, right = a.size() - 1;
while (left <= right) { int pivot = a[left + (right - left) / 2], low = left, i = left, high = right;
    while (i <= high) { if (a[i] < pivot) std::swap(a[low++], a[i++]);
        else if (a[i] > pivot) std::swap(a[i], a[high--]); else ++i; }
    if (target < low) right = low - 1; else if (target > high) left = high + 1; else return pivot;
}
// 快速选择平均 O(n)、最坏 O(n²)；随机 pivot 可降低常见退化
// 流式数据可改用大小 K 的最小堆，时间 O(n log K)、空间 O(K)
```

070 斐波那契数列（Fibonacci Sequence）

答案 迭代从 F0、F1 开始滚动更新两项，避免朴素递归反复计算，时间 O(n)、额外空间 O(1)。整数位数随 n 增长，定宽类型会溢出；若要求模 M，可在每步取模。快速倍增用 F(2k)、F(2k+1) 关系可把运算次数降到 O(log n)。Fibonacci 的增长近似指数级，测试时小心中间项溢出早于最终返回值。

```
// prev/current 始终保存相邻两项，迭代避免重复递归。
long long fib(int n) { if (n < 0) throw InvalidInput{}; if (n <= 1) return n; // F0=0、F1=1
    long long prev = 0, current = 1;
    for (int i = 2; i <= n; ++i) { long long next = prev + current; prev = current; current = next; }
    return current;
}
// O(n) 时间、O(1) 空间；定宽整数可能溢出，模运算按需逐步执行
```

071 旋转排序数组中的最小值

答案 无重复元素时取中点与右端比较：nums[mid] 大于右端说明最小值在右侧，否则最小值在左侧含 mid，逐步缩小区间到单点。重复值且 nums[mid]==nums[right] 时无法判断哪边有旋转点，只能 right--，最坏退化 O(n)。边界为空数组时要先定义返回方式。 若数组未发生旋转，算法仍会返回首元素；旋转次数为零也覆盖在内。

```
// 与右端比较后保留包含最小值的闭区间。
if (a.empty()) throw EmptyInput{};
int left = 0, right = static_cast<int>(a.size()) - 1;
while (left < right) { int mid = left + (right - left) / 2;
    if (a[mid] > a[right]) left = mid + 1;
    else if (a[mid] < a[right]) right = mid;
    else --right; // 重复值无法判定旋转点在哪边
}
return a[left]; // 无重复 O(log n)，含重复最坏 O(n)；未旋转也返回首元素
```

072 最大矩形问题（Maximal Rectangle）

答案 逐行扫描矩阵，当前为 1 的列高度加一，遇 0 清零；把每一行高度视为直方图，调用单调栈求最大矩形。若有矩形底边在该行，它的高恰好是各列连续 1 的高度，因此不会漏解。总时间 O(mn)、额外空间 O(n)，空行/空列应返回 0。 每列高度可原地保存在一维数组中，扫描下一行时只更新对应列。 每行均为零时该行最大面积为零，更新直方图后即可继续下一行。

```
// height[c] 是当前行为底时该列连续 1 的高度。
std::vector<int> height(cols, 0);
for (const auto& row : matrix) {
    for (int c = 0; c < cols; ++c) height[c] = row[c] == '1' ? height[c] + 1 : 0;
    best = std::max(best, largestHistogram(height)); // 当前行作矩形底边
}
return best; // 每行 O(cols)，总 O(rows*cols)，额外 O(cols)
// 空矩阵/空列返回 0；直方图子程序用单调栈求最大面积
```

073 最小生成树：Kruskal 算法

答案 将边按权重升序排列，依次检查两端是否属于不同并查集；不同则选入 MST 并合并，形成环的边跳过。边排序 O(E log E)，并查集合并近似线性；若最终选边少于 V-1，原图不连通，只得到最小生成森林。负权边可正常处理，前提是无向图。并查集按代表元判断两端是否已连通，而不是比较顶点编号本身。若有相同权重，MST 可能不唯一，但 Kruskal 仍会得到一个最优生成树。

```
// 仅合并不同并查集的边，已连通边会形成环而跳过。
std::sort(edges.begin(), edges.end(), [](const Edge& a, const Edge& b) { return a.weight < b.weight; });
for (const Edge& e : edges) if (dsu.unite(e.u, e.v)) {
    mst.push_back(e); total += e.weight;
    if (mst.size() == n - 1) break;
}
if (mst.size() != n - 1) reportForest(); // 原图不连通
// O(E log E) 排序加近线性并查集；负权无向边可正常处理
```

074 最小生成树：Prim 算法

答案 从任意顶点开始，把跨越已访问集合与未访问集合的候选边放入最小堆，每次取权重最小且终点尚未访问的边加入树。使用邻接表时堆优化复杂度通常 O(E log V)，稠密图也可用 $O(V^2)$ 数组实现。图不连通时需从多个连通分量启动，得到最小生成森林。 懒删除堆中已访问终点的旧候选边，可避免显式删除任意堆元素。

```
// 候选堆维护已选集合的外边界，每个顶点只纳入一次。
using Candidate = std::pair<int, int>; // (权重, 顶点)
std::priority_queue<Candidate, std::vector<Candidate>, std::greater<Candidate>> q;
for (int start = 0; start < n; ++start) if (!used[start]) { q.push({0, start});
    while (!q.empty()) { auto [w, u] = q.top(); q.pop(); if (used[u]) continue;
        used[u] = true; total += w;
        for (auto [v, cost] : graph[u]) if (!used[v]) q.push({cost, v});
    }
}
// 外层覆盖不连通分量得最小生成森林；堆优化常见 O(E log V)
```

13.7 路径规划与动态规划

075 最小路径和（Minimum Path Sum）

答案 dp[i][j] 表示从左上到该格的最小代价，只能由上方或左方到达，故取二者较小值并加当前格权重。首行只能从左来、首列只能从上来，不能把未初始化状态当作 0。滚动一维数组可降至 O(n) 空间，时间 O(mn)；负权格不影响该网格无环方向转移。 若题目允许向右和向下以外的移动，网格 DP 的无环依赖假设便不成立。

```
// dp[c] 在本行更新前是上方，更新后是左方最优代价。
if (rows == 0 || cols == 0) return 0;
const int inf = std::numeric_limits<int>::max() / 4;
std::vector<int> dp(cols, inf); dp[0] = grid[0][0];
for (int r = 0; r < rows; ++r) for (int c = 0; c < cols; ++c) {
    if (r == 0 && c == 0) continue;
    int fromAbove = dp[c], fromLeft = c ? dp[c - 1] : inf;
    dp[c] = grid[r][c] + std::min(fromAbove, fromLeft);
}
return dp[cols - 1]; // O(rows*cols) 时间、O(cols) 空间；避免 INF 加法溢出
```

076 最短路径问题：Bellman-Ford 算法

答案 初始化源点距离为 0、其余为无穷，对所有边重复松弛 V-1 轮；若一轮没有更新可提前结束。再扫描可达边，若仍能降低距离，则存在从源点可达的负权环；不可达的负权环不会影响源点答案。时间 O(VE)，无负权环时适用于有负边图。 检测负环时只从距离有限的顶点松弛，防止无穷值加边权产生伪更新。 第 V 轮仅需扫描边检查是否还能松弛，不必再完整执行一轮更新。

```
// 每轮松弛允许最短路再多经过一条边，V-1 轮覆盖简单路径。
std::vector<long long> dist(n, INF); dist[source] = 0;
for (int round = 1; round < n; ++round) { bool changed = false;
    for (auto [u, v, w] : edges) if (dist[u] != INF && dist[u] + w < dist[v]) {
        dist[v] = dist[u] + w; changed = true;
    }
    if (!changed) break;
}
for (auto [u, v, w] : edges) if (dist[u] != INF && dist[u] + w < dist[v]) reportReachableNegativeCycle();
// O(VE) 时间，可处理负边；只从可达点松弛以免 INF 溢出
```

077 最短路径问题：Dijkstra 算法

答案 Dijkstra 每次从最小堆取当前暂定距离最小的有效节点，再用它的距离松弛邻边；堆中旧条目若已过期应跳过。算法要求所有可达边权非负，否则已确定的距离还可能被后续负边改小。邻接表加二叉堆通常 O((V+E)logV)，单源不可达点保持无穷。 常见实现会把更短距离重新压入堆，并跳过过期堆项以简化 decrease-key。

```
// 堆顶暂定距离若已过期就跳过，避免重复处理旧状态。
std::vector<long long> dist(n, INF); dist[source] = 0;
using State = std::pair<long long, int>; std::priority_queue<State, std::vector<State>, std::greater<State>> q;
q.push({0, source});
while (!q.empty()) { auto [d, u] = q.top(); q.pop(); if (d != dist[u]) continue; // 跳过过期条目
    for (auto [v, w] : graph[u]) if (d + w < dist[v]) { dist[v] = d + w; q.push({dist[v], v}); }
}
// 仅适用于非负边；邻接表+堆 O((V+E)log V)，不可达点保持 INF
```

078 最长公共子串（Longest Common Substring）

答案 dp[i][j] 表示两个字符串分别以 i、j 位置结尾时的最长公共子串长度；字符相等时取左上状态加一，不同则清零，因为子串必须连续。遍历中记录最大长度及结束位置即可恢复答案。时间 O(mn)，滚动数组 O(min(m,n)) 空间，更新方向要避免覆盖尚未读取的左上状态。

```
// 子串必须连续，字符不等时以该字符结尾长度归零。
std::vector<int> dp(b.size() + 1, 0); int best = 0;
for (char x : a) for (int j = static_cast<int>(b.size()); j > 0; --j) {
    dp[j] = x == b[j - 1] ? dp[j - 1] + 1 : 0; // 不同字符必须清零
    best = std::max(best, dp[j]);
}
return best; // 子串连续；逆序更新保护上一行左上状态
// O(mn) 时间、O(n) 空间；可把较短串放在列维
```

079 最长公共子序列（Longest Common Subsequence, LCS）

答案 dp[i][j] 表示两个前缀的最长公共子序列长度；末尾字符相同则由左上加一，不同则取上方或左方较大值。子序列允许跳过字符，因此与公共子串的“不匹配归零”不同。时间 O(mn)，长度计算可用 O(min(m,n)) 滚动空间；恢复具体序列通常要保留方向信息或重新计算。若只求长度可滚动压缩，若要输出序列则应保存矩阵或使用 Hirschberg 重建。

```
// 子序列允许跳过字符，不匹配时取删去一侧末字符的较优状态。
std::vector<std::vector<int>> dp(a.size() + 1, std::vector<int>(b.size() + 1, 0));
for (int i = 1; i <= a.size(); ++i) for (int j = 1; j <= b.size(); ++j) {
    if (a[i - 1] == b[j - 1]) dp[i][j] = dp[i - 1][j - 1] + 1;
    else dp[i][j] = std::max(dp[i - 1][j], dp[i][j - 1]);
}
return dp.back().back(); // 子序列允许跳过字符，不要求连续
// O(mn) 时间；只求长度可滚动为 O(min(m,n)) 空间，重建需留决策
```

080 最长无重复字符子串如何求解？

答案 用滑动窗口保证其中字符互不重复，并用哈希表记录字符最近位置；遇到位置在当前窗口内的重复字符，左边界更新为旧位置加一。更新边界要取 max(left, last[c]+1)，否则可能因窗口外旧字符错误地左退。每个字符处理一次，时间 O(n)，空间取决于字符集。 字节数组适用于 ASCII；UTF-8 文本应按码点或题目指定的字符单位处理。

```
// 窗口左边界只能前进，哈希表记录字符最近出现下标。
std::unordered_map<char, int> last; int left = 0, best = 0;
for (int right = 0; right < s.size(); ++right) {
    auto it = last.find(s[right]);
    if (it != last.end()) left = std::max(left, it->second + 1); // 窗口左边界不后退
    last[s[right]] = right; best = std::max(best, right - left + 1);
}
return best; // 每字符处理一次，平均 O(n) 时间，空间与字符集有关
// UTF-8 输入若按码点判断，需先解码而不能直接以字节为字符
```

081 最长递增子序列（Longest Increasing Subsequence, LIS）

答案 $O(n^2)$ DP 对每个位置检查此前更小元素；优化做法维护长度为 L 的递增子序列能达到的最小尾值 tails[L-1]，二分找到首个不小于当前数的位置替换。tails 长度就是 LIS 长度，时间 O(n log n)、空间 O(n)。若要恢复具体序列，还需记录前驱和每个元素对应的 tails 位置。

```
// tails 保存各长度的最小结尾，用 std::lower_bound 替换但不直接存真实子序列。
std::vector<int> tails;
for (int x : a) { auto it = std::lower_bound(tails.begin(), tails.end(), x);
    if (it == tails.end()) tails.push_back(x); else *it = x;
}
return tails.size(); // tails[i] 是长度 i+1 的最小可行结尾
// std::lower_bound 保证严格递增；O(n log n) 时间、O(n) 空间
// 需要恢复具体序列时另存每项前驱和所在长度
```

13.8 栈、滑动窗口与数组优化

082 有效括号如何用栈判断?

答案 遇到左括号将其压栈；遇到右括号时栈必须非空且栈顶类型匹配，然后弹出。忽略其他字符还是将其判非法需服从题目定义，遍历结束栈也必须为空。每个字符至多入栈和出栈一次，时间 O(n)、空间 O(n)，只含一种括号时也可用计数器。对混合类型括号，建议用映射表匹配而非只统计左、右括号总数。遇到右括号后立即弹栈，能保证每个闭合符号都与最近未匹配左括号配对。

```
// 每个右括号只匹配最近未配对的左括号。
std::stack<char> st;
for (char c : s) {
    if (isOpening(c)) st.push(c);
    else if (isClosing(c)) { if (st.empty() || !matches(st.top(), c)) return false; st.pop(); }
    else if (otherCharactersAreInvalid) return false;
}
return st.empty(); // 所有左括号都必须匹配
// O(n) 时间、O(n) 空间；单一种类括号可仅用计数器
```

083 查找数组中的最大值和最小值

答案 非空数组先以首元素初始化 min 和 max，再遍历其余元素更新两者；这样全负数和重复值都能正确处理。时间 O(n)、额外空间 O(1)，空数组应返回错误状态而非读取首元素。若比较次数特别重要，可成对比较每组元素，再分别与全局最小和最大比较。

```
// 从首元素初始化可兼容全负数组，不依赖 0 作为初值。
if (a.empty()) return std::nullopt;
int low = a[0], high = a[0];
for (int x : a) { low = std::min(low, x); high = std::max(high, x); }
return std::pair<int, int>{low, high};
// 全负数也正确；O(n) 时间、O(1) 额外空间
// 需最少比较时可成对比较，再分别更新全局极值
```

084 查找数组中的重复元素

答案 用哈希集合记录已见值，第二次及之后出现时可加入结果；若结果只需列每个重复值一次，还要用第二个集合或标记输出过的键。平均时间 O(n)、空间 O(n)，值可哈希且内存允许时简单可靠。已知值域并可修改输入时可考虑原地标记，但必须处理负数和重复计数。 若要统计每个重复值出现次数，可用哈希映射计数而不是仅返回集合。

```
// 频次大于 1 的键才是重复值；若只求存在可提前返回。
std::unordered_map<int, int> frequency;
for (int x : a) ++frequency[x];
for (const auto& [value, count] : frequency) if (count > 1) duplicates.push_back(value);
// 计数表可同时得到重复次数；平均 O(n) 时间、O(n) 空间
// 只需判断是否有重复时，集合发现第二次即可提前返回
```

085 查找数组中缺失的最小正整数

答案 对每个值 x，只要 1≤x≤n 且 nums[x-1] 不等于 x，就交换到下标 x-1；交换直到越界、重复或归位。随后首个 nums[i] 不等于 i+1 的位置给出缺失最小正整数，若全匹配则答案为 n+1。时间 O(n)、额外空间 O(1)，原地修改输入是该复杂度成立的条件。 交换时若目标位置已有同值要停止，否则重复值会在两个下标间无限互换。

```
// 把值 x 放到下标 x-1，越界和重复值停止交换。
for (int i = 0; i < n; ++i) {
    while (a[i] >= 1 && a[i] <= n && a[a[i] - 1] != a[i]) std::swap(a[i], a[a[i] - 1]);
}
for (int i = 0; i < n; ++i) if (a[i] != i + 1) return i + 1;
return n + 1; // 1..n 都归位后答案为 n+1
// 越界和重复值直接停止交换；O(n) 时间、O(1) 空间但会修改输入
```

086 柱状图中的最大矩形（Largest Rectangle in Histogram）

答案 维护高度单调递增的下标栈；当前高度低于栈顶时弹出柱子，以其高度乘以当前右边界和新栈顶之间的宽度计算面积。末尾加 0 哨兵清栈，空数组返回 0。每根柱子至多入栈和弹出一次，时间与空间 O(n)；相同高度可保留一个代表或按一致规则弹出。 面积类型需容纳高度乘宽度，较大输入下 32 位整数可能溢出。 对零高柱子可正确清空较高柱子的最大延伸范围，不需额外分支。

087 桶排序（Bucket Sort）

答案 把数据按值域映射到多个桶，在桶内排序，再按桶序连接结果；桶区间和数量应根据输入分布选择。均匀分布时每桶较小，平均可接近 O(n)，但极端集中在单桶时复杂度取决于桶内排序，可能退化。桶排序需要额外空间，浮点数需先定义有效范围和映射边界。桶内元素分布若未知，可选插入排序等简单方法，也可再递归细分桶。桶索引计算要保证最大值落入有效桶，避免上界恰好越界。

```
// 桶索引按值域线性映射，最大边界需限制在最后一桶。
// 空输入直接返回；非空时假设 high>low 且桶数 n>0
if (a.empty()) return out;
std::vector<std::vector<double>> buckets(n);
for (double x : a) {
    int id = static_cast<int>((x - low) / (high - low) * n);
    id = std::min(n - 1, id); buckets[id].push_back(x);
}
for (auto& bucket : buckets) std::sort(bucket.begin(), bucket.end());
for (const auto& bucket : buckets) out.insert(out.end(), bucket.begin(), bucket.end());
// 均匀分布时平均接近 O(n)；集中一桶时退化为桶内排序复杂度
// 额外空间 O(n)，上界映射需 clamp 防止下标等于 n
```

088 正则表达式匹配（Regular Expression Matching）

答案 针对仅含 `.` 和 `*` 的匹配题，dp[i][j] 表示文本前 i 个字符与模式前 j 个字符是否匹配；普通字符或点按对角线转移。模式中的星号作用于前一字符，可选择出现零次（跳过两位）或在当前字符匹配时延长一次。初始化空文本与连续星号很关键，算法 O(mn) 时间和空间，不等于支持完整正则语法。

```
// 星号转移分为跳过前项与匹配后重复两种情况。
// dp[i][j]: 文本前 i 字符与模式前 j 字符匹配；模式不以 * 开头
std::vector<std::vector<bool>> dp(s.size() + 1, std::vector<bool>(p.size() + 1)); dp[0][0] = true;
for (int j = 2; j <= p.size(); ++j) if (p[j - 1] == '*') dp[0][j] = dp[0][j - 2];
for (int i = 1; i <= s.size(); ++i) for (int j = 1; j <= p.size(); ++j) {
    if (p[j - 1] == '*') dp[i][j] = dp[i][j - 2] || ((p[j - 2] == '.' || p[j - 2] == s[i - 1]) && dp[i - 1][j]);
    else dp[i][j] = (p[j - 1] == '.' || p[j - 1] == s[i - 1]) && dp[i - 1][j - 1];
}
// * 可取零次或延长一次；O(mn) 时间、空间，不代表完整正则语法
```

089 求解最大子数组和（Maximum Subarray Sum / Kadane's Algorithm）

答案 遍历每个数 x，维护以当前位置结尾的最大子数组和 cur=max(x,cur+x)，并用它更新全局最大值。若 cur 已为负，重新从 x 开始不会更差；全局答案应以首元素初始化，从而正确处理全负数组。时间 O(n)、空间 O(1)，空数组需按接口约定单独处理。 同步更新最大区间的起止下标，还能在线性扫描时恢复具体子数组。

```
// ending 是必须以当前位置结尾的最优和，best 是全局最优。
if (a.empty()) return 0; // 空数组按接口约定处理
int ending = a[0], best = a[0];
for (int i = 1; i < a.size(); ++i) {
    ending = std::max(a[i], ending + a[i]); // 从当前重开或延长前段
    best = std::max(best, ending);
}
return best; // 全负数也正确；O(n) 时间、O(1) 空间
```

090 滑动窗口最大值如何在线性时间求解？

答案 双端队列保存下标且对应值单调递减；每步先移除已离开窗口的队首，再从队尾删除值不大于当前值的下标，最后读取队首最大值。下标用于判断过期，重复最大值可保留较新的一个。每个元素最多入队出队一次，整体 O(n) 时间、O(k) 空间，k 为窗口宽度。 先等至少 k 个元素进入窗口再输出，避免把尚未完整的窗口当结果。

```
// 队列下标递增且对应值递减，过期从头、较小候选从尾删除。
if (k <= 0 || k > a.size()) return maxima;
std::deque<int> q; // 下标对应值单调递减，队首为窗口最大值
for (int i = 0; i < a.size(); ++i) {
    while (!q.empty() && q.front() <= i - k) q.pop_front();
    while (!q.empty() && a[q.back()] <= a[i]) q.pop_back();
    q.push_back(i); if (i >= k - 1) maxima.push_back(a[q.front()]);
}
// 每下标最多进出队一次，O(n) 时间、O(k) 空间；仅完整窗口输出
```

091 爬楼梯问题（Climbing Stairs）

答案 若每次可以爬 1 或 2 阶，到第 n 阶的最后一步来自 n-1 或 n-2，因此 dp[n]=dp[n-1]+dp[n-2]。按题目定义设置 n=0、n=1 的边界，再用两个变量滚动即可 O(n) 时间、O(1) 空间。n 大时计数可能快速超过整数范围，应使用大整数或按要求取模。

```
// 每阶最后一步只能来自前一阶或前两阶。
if (n == 0) return 1; // 空台阶通常有一种空走法
long long twoBack = 1, oneBack = 1; // F(0)=F(1)=1
for (int step = 2; step <= n; ++step) { long long ways = twoBack + oneBack;
    twoBack = oneBack; oneBack = ways;
}
return oneBack; // O(n) 时间、O(1) 空间；大 n 注意计数溢出
```

092 线性查找和二分查找有什么区别？二分查找如何实现？

答案 线性查找逐项比较，不要求有序，时间 O(n)；二分查找需要有序且能快速访问中点，反复丢弃不可能区间，时间 O(log n)。实现前固定闭区间 [l,r] 或半开区间 [l,r)，并对应调整循环条件和边界更新。计算中点用 l+(r-l)/2 可避免 l+r 溢出，重复值要明确找任意、首个还是末个。 查找下界时要保留满足边界条件的那一半，循环终止点往往就是插入位置。

```
// 线性扫描无需有序；二分半开区间返回 std::lower_bound 所在位置。
// 线性查找不要求有序，逐项比较 O(n)
for (int i = 0; i < a.size(); ++i) if (a[i] == target) return i;
// 二分要求有序且可 O(1) 访问中点；半开区间找首个 >= target
int left = 0, right = a.size();
while (left < right) { int mid = left + (right - left) / 2;
    if (a[mid] < target) left = mid + 1; else right = mid;
}
return left; // std::lower_bound 插入位置，O(log n)；mid 公式避免加法溢出
```

093 编辑距离（Levenshtein Distance）

答案 令 dp[i][j] 为两个前缀互相转换的最小编辑次数，首行/首列分别初始化为插入或删除全部字符的代价。字符相同取左上值，不同则取插入、删除、替换三者最小值加一。时间 O(mn)，滚动数组可将长度计算降为 O(min(m,n)) 空间；恢复编辑操作需额外保存路径。 允许插入、删除和替换的代价不同时，可分别赋权修改三种转移。

```
// diagonal 保存上一行左上，dp[j] 与 dp[j-1] 表示删除/插入候选。
std::vector<int> dp(b.size() + 1); std::iota(dp.begin(), dp.end(), 0); // 空前缀需插入 j 次
for (int i = 1; i <= a.size(); ++i) { int diagonal = dp[0]; dp[0] = i; // 删除 i 次
    for (int j = 1; j <= b.size(); ++j) { int old = dp[j];
        dp[j] = a[i - 1] == b[j - 1] ? diagonal : 1 + std::min({diagonal, dp[j], dp[j - 1]});
        diagonal = old; // 保存上一行左上值
    }
}
// O(mn) 时间、O(n) 空间；恢复操作序列需保存路径/决策
```

094 计数排序（Counting Sort）

答案 先求整数值域最小值和最大值，用偏移后的计数数组统计每个值，再按频次输出；值域跨度 k 小时效率高。时间 O(n+k)、空间 O(k)，若值域很大或稀疏则不合适。需要稳定排序记录对象时，计算累积位置并从输入尾部向前放置，重复元素相对顺序才保持。 k 表示 max-min+1 的值域跨度，不能只看输入元素数量 n。

```
// count[x-low] 统计偏移值出现次数，按计数回填整数。
if (a.empty()) return;
int low = *std::min_element(a.begin(), a.end()), high = *std::max_element(a.begin(), a.end());
std::vector<int> count(high - low + 1, 0);
for (int x : a) ++count[x - low];
int write = 0; for (int i = 0; i < count.size(); ++i) while (count[i]-- > 0) a[write++] = i + low;
// O(n+k) 时间、O(k) 空间，k=high-low+1；值域很大/稀疏时不适合
// 稳定排序记录需改用累积位置并从输入尾部放置
```

095 选择排序（Selection Sort）

答案 每轮在未排序区间扫描最小元素，与区间首元素交换，再缩小未排序边界；比较次数总是约 n(n-1)/2。空间 O(1)，交换次数至多 n-1，适合写入成本高而比较便宜的小规模场景。标准交换实现可能不稳定，存在相同键时不要假设原顺序保留。 与插入排序相比，选择排序的移动次数少，但几乎有序时仍做平方级比较。

```
// 每轮定位未排序区最小项，只做一次最终交换。
for (int i = 0; i < n; ++i) { int minIndex = i;
    for (int j = i + 1; j < n; ++j) if (a[j] < a[minIndex]) minIndex = j;
    if (minIndex != i) std::swap(a[i], a[minIndex]); // 每轮最多一次交换
}
// 比较次数约 n(n-1)/2、空间 O(1)，交换少但时间仍 O(n²)
// 标准交换版不稳定，近乎有序也不会明显减少比较
```

096 链表表示的两个数如何相加?

答案 从两个表示低位在前的链表头开始逐位相加，缺少节点时按 0 处理，并把和的个位写入新节点、十位作为进位。遍历条件必须包含任一链表未结束或进位非零，才能输出末尾进位。时间 O(max(m,n))，若数字按高位在前存储则需先反转、栈辅助或递归处理。 新建结果节点可避免修改输入链表；复用节点则要小心所有权与别名。

```
// carry 保存十进位；循环条件必须包含尚未输出的末尾进位。
Node dummy{}; Node* tail = &dummy; int carry = 0;
while (a || b || carry) { int sum = carry + (a ? a->value : 0) + (b ? b->value : 0);
    carry = sum / 10; tail->next = new Node{sum % 10}; tail = tail->next;
    if (a) a = a->next; if (b) b = b->next;
}
return dummy.next; // 循环包含末尾进位，低位在前
// O(max(m,n)) 时间；新建结果节点，不修改输入链表
```

097 除自身以外数组的乘积如何在 O(n) 且不除法求解?

答案 结果数组先写每个位置左侧所有元素的乘积，缺少左侧元素时为 1；再从右向左维护右侧乘积并乘入结果。整个过程不做除法，零值也自然处理，时间 O(n)、除输出外额外空间 O(1)。乘积可能远超元素类型范围，需根据约束使用更宽整数或大数。若输入允许任意精度要求，应先确认数值范围，不能因无除法忽略溢出。前缀和后缀乘积可分别独立检查，便于定位零值和边界初始化错误。

```
// 前缀和后缀乘积均跳过当前位置，不使用除法。
std::vector<long long> out(a.size(), 1); long long prefix = 1;
for (int i = 0; i < a.size(); ++i) { out[i] = prefix; prefix *= a[i]; }
long long suffix = 1;
for (int i = static_cast<int>(a.size()) - 1; i >= 0; --i) {
    out[i] *= suffix; suffix *= a[i];
}
// 前缀与后缀都不包含当前位置；O(n) 时间、输出外 O(1) 空间
// 无除法可自然处理零值，乘积类型仍需符合数值范围
```

13.9 STL 容器使用与选择

098 std::vector 中 emplace_back 和 push_back 有什么区别?

答案 push_back 接收已构造的 T，复制或移动到容器；emplace_back 转发构造参数，在尾部直接构造元素，例如 `v.emplace_back(a, b)`。已有 T 对象时两者通常都要移动/复制一次，emplace 并不天然更快；扩容时已有元素仍可能搬迁。还要留意完美转发可能选择不同构造函数重载。〔S2〕

099 std::vector 和 std::deque 如何选择?

答案 vector 元素连续，适合高效随机访问、缓存局部性和需要 data() 连续缓冲区的接口；尾插均摊 O(1)，但扩容可能搬迁元素并使指针、引用和迭代器失效。deque 随机访问也是 O(1)，两端插入删除通常 O(1)，但元素不保证整体连续且迭代器失效规则不同。若需频繁中间插入或稳定引用，还应比较 list 等容器的代价。〔S2〕

13.10 标准算法与容器适配器

100 std::priority_queue 怎样维护堆序？常见操作是什么复杂度?

答案 std::priority_queue 是容器适配器，默认在 std::vector 上维护堆序，要求底层支持随机访问和尾端插删。top 只访问堆顶，通常 O(1)；push 通过上浮恢复堆序，pop 把堆顶与尾部交换再下沉，比较次数为 O(log n)，底层扩容仍可能增加单次成本。默认 std::less 让最大的元素位于顶部，std::greater 则常用于最小堆。它不提供任意位置高效删改；需要 decrease-key 的任务可换带索引堆或采用懒删除，并考虑过期条目的空间代价。〔N150〕

```
std::priority_queue<int, std::vector<int>, std::greater<int>> pending;
pending.push(7); pending.push(2);
int smallest = pending.top(); // 2，查看不删除
pending.pop(); // 删除堆顶，不返回原值
```

101 std::sort 的复杂度保证与常见内省排序实现是什么关系？

答案 标准要求 std::sort 在通常非并行重载下进行 O(N log N) 次比较，并要求随机访问迭代器和满足严格弱序的比较关系，不强制某一种算法。常见标准库采用内省排序：以快速排序分区获得平均效率，递归深度过大时切换堆排序防止最坏退化，小区间使用插入排序降低常数。它不保证相等元素的原有顺序，需稳定性时选 std::stable_sort。链表迭代器不满足随机访问要求，应使用 list::sort；比较器不一致或含不恰当的浮点 NaN 关系也会破坏排序前提。〔N151、N152〕

13.11 索引结构与算法选择

102 AVL 与红黑树如何限制高度，读写性能能否只凭树名判断？

答案 AVL 要求每个节点左右子树高度差至多为 1，通过旋转和高度更新保持较严格平衡。红黑树用颜色、红节点不相邻、从节点到空叶路径黑节点数一致等约束间接限制高度，查找、插入、删除均可保持 O(log n)。AVL 通常更紧凑地限制树高，红黑树在更新场景常有较宽松的平衡维护，但实际速度还取决于节点布局、比较成本、缓存和读写比例，不能断言某一种始终更快。有序映射常采用平衡搜索树，具体实现要看库；C++ 标准并不强制 std::map 必须采用红黑树。选型应从范围查询、更新比例及稳定复杂度需求出发。〔N153、N154〕

103 跳表怎样利用随机层级加速有序访问，与平衡树有哪些取舍？

答案 跳表底层是按键有序的链表，再给一部分节点随机建立更高层的前进指针。查找从高层向前跳，到即将越过目标时下降一层，逐步逼近目标；插入先找各层前驱，再按新节点随机高度改链，删除亦需更新相关层。在通常的随机化假设下，查找和更新期望 O(log n)，最坏仍可能 O(n)，并非确定性的平衡树高度保证。它适合有序遍历和范围访问，维护逻辑直观，但多层指针增加空间和指针访问成本。Redis 较大有序集合可组合字典与跳表，跳表节点的跨度还能支持名次查询；小集合可能采用紧凑编码，不是所有 ZSet 始终使用跳表。〔N155〕

104 Trie 为什么适合前缀检索，怎样区分完整单词与已有路径？

答案 Trie 把键按字符或其他符号拆成路径，共同前缀共享节点；插入沿路径创建缺失节点，并在终点记录结束标记或关联值。查询完整词时不仅要走通路径，还要检查终点标记，否则会把某个长词的前缀误判为独立键。设键长 L，在子节点查找成本有界时，精确查找和插入可为 O(L)，前缀枚举还需加上访问和输出结果的成本。固定字母表数组快但可能浪费空间，稀疏映射、压缩 Trie 可减少节点开销。处理中文等文本要先约定按 UTF-8 字节、Unicode 码点还是其他单位建边，复杂度中的长度必须与该单位一致。〔N156〕

```
node = root
for symbol in key:
    node = node.children.get_or_create(symbol)
node.terminal = True  # 路径存在并不意味着完整键存在
# 查询完整词：走到终点后还要检查 terminal
# 前缀查询：走通前缀后，再枚举子树中的 terminal 节点
```

105 待排序数据装不进内存时，怎样实施外部归并排序？

答案 先按内存预算分块读入，每块在内存中排序后写成有序段；随后为若干有序段分别保留输入缓冲，用最小堆或败者树反复选择最小段首，写入输出缓冲，完成多路归并。若有序段太多，受内存和文件描述符限制，可分多轮合并。主要成本通常是存储 I/O，因此合并路数、块大小和顺序读写比单纯减少比较更重要。若每轮能合并 k 段，段数 R 通常约需 ceil(log_k R) 轮；重复键要定义稳定规则。还须预留临时空间，处理失败清理和原子发布，避免排序过程中损坏唯一原文件。〔N157〕

```
runs = []
while 仍有输入:
    chunk = 读取不超过内存预算的一块数据()
    sort(chunk)
    runs.append(写入临时有序段(chunk))
while len(runs) > 1:
    runs = 对 runs 分组并执行有缓冲的多路归并(runs)
# 只在完整结果验证后发布，归并途中保留可恢复状态。
```

106 邻接矩阵、邻接表与边列表各适合什么图操作？

答案 邻接矩阵用 V×V 单元记录边，空间 $O(V^2)$，测试某条边是否存在通常 O(1)，但枚举一个顶点的邻居要扫描一整行，适合稠密图或频繁邻接测试。邻接表为每个顶点保存相邻边，空间 O(V+E)，遍历其邻居与度数相关，适合稀疏图中的 DFS/BFS；若邻接容器无额外索引，判断任意边存在通常要扫描该顶点邻接项。边列表直接保存端点和权重，便于按边排序或逐边松弛，却不直接支持快速枚举邻居。无向图的邻接表往往把边存两次，自环、重边、方向和权重也须在建图前明确约定。〔N158〕

107 贪心算法何时能替代动态规划，怎样证明局部选择不会破坏最优解？

答案 动态规划把问题拆成状态与转移，复用重叠子问题，并在候选决策间比较；贪心每一步直接做一个局部选择，通常不再回看。二者都可能利用最优子结构，但有最优子结构不代表某个贪心规则就正确，还要证明选择性质，例如用交换论证把某个最优方案变成包含当前选择且不变差的方案。最早结束时间选活动可成立；任意币值下总选最大面额则会失败，如币值 1、3、4 凑 6 时，4+1+1 多于 3+3。无法证明时不要仅凭少量样例采纳贪心，可构造反例或使用合适 DP。性能比较还要依据状态规模和约束。〔N159、N160〕

第 14 章 Qt

14.1 核心机制与容器

001 QByteArray 的使用场景是什么？

答案 QByteArray 保存的是原始字节，不解释字符编码，适合 socket 收发、压缩数据、文件内容和二进制协议；例如 UTF-8 文本先编码后可放入 QByteArray。转回文本时用 QString::fromUtf8 等明确解码，不能按字节数当字符数；需要 Unicode 文本运算时用 QString。

```
QByteArray wire = QStringLiteral("猫").toUtf8(); // 显式编码成字节
wire.append('\n'); // 协议分隔符也是字节
send(socket, wire); // 二进制数据不附带字符集信息
const QString decoded = QString::fromUtf8(wire); // 读回时按 UTF-8 解码
const int characters = decoded.size(); // 字符计数不等于 wire.size()
QByteArray compressed = qCompress(wire); // 同一类型也可装任意二进制内容
```

002 QList、QVector、QLinkedList 有什么区别？

答案 Qt 6 中 QList 与 QVector 使用相同的连续存储实现，QVector 作为兼容别名保留；QLinkedList 已在 Qt 6 移除，旧代码应考虑 std::list 或连续容器。顺序访问、遍历和尾部追加通常优先 QList；中间插入会搬移元素，链表虽便于已知位置插入却有额外节点和缓存未命中成本。

```
QList<int> values{4, 1, 3}; // Qt 6 中 QList 连续存储
values.append(2); // 尾部追加与顺序遍历常见
std::sort(values.begin(), values.end()); // Qt 容器可接标准算法
// Qt 6 的 QVector 是 QList 的兼容别名，QLinkedList 已移除
std::list<int> nodes{4, 1, 3}; // 旧链表代码可评估 std::list
// 已知位置插入虽方便，链表节点和缓存未命中也有成本
```

003 QMap 和 QHash 有什么区别？

答案 QMap 按键排序，适合范围查询或需要稳定有序遍历的场合，查找、插入通常为 O(log n)；QHash 不保证顺序，按哈希平均查找更快，适合仅按键定位。哈希冲突会影响性能，自定义键要提供相等比较和 qHash；若结果要排序，可取出键后另行排序。若只需遍历顺序无关，应优先选更符合访问模式的容器，而非为排序额外付费。

```
QMap<QString, int> ordered{{QStringLiteral("b"), 2}, {QStringLiteral("a"), 1}};
for (auto it = ordered.cbegin(); it != ordered.cend(); ++it)
    consume(it.key(), it.value()); // QMap 按键排序
QHash<QString, int> fast; fast.insert(QStringLiteral("a"), 1);
const auto found = fast.constFind(QStringLiteral("a"));
if (found != fast.cend()) consume(found.value()); // 不依赖遍历顺序
// 自定义 QHash 键还需相等比较与 qHash 实现
```

004 QString 和 std::string 有什么区别？

答案 QString 是 Qt 的 Unicode 文本类型，便于与控件、翻译和字符串算法协作；std::string 只是字节容器，本身不携带 UTF-8、GBK 等编码信息。跨界时显式使用 toStdString/fromStdString 之外的编码转换，例如 toUtf8/fromUtf8；含非 ASCII 字符时不可直接按字节索引当作字符。

```
const QString text = QStringLiteral("雪");
const QByteArray utf8 = text.toUtf8(); // 明确写出编码
std::string bytes(utf8.constData(), static_cast<std::size_t>(utf8.size()));
const QString restored = QString::fromUtf8(bytes.data(), static_cast<qsizetype>(bytes.size()));
// std::string 只保存字节，不携带编码；不要按字节下标当 Unicode 字符
showText(restored); // Qt 控件继续使用 QString
```

005 QVariant 是什么？有什么应用场景？

答案 QVariant 是带运行时类型信息的值容器，便于统一承载属性值、模型角色数据或查询结果，例如一个 DisplayRole 返回 QString、另一个 EditRole 返回整数。读取时用 value<T>() 或 toInt(&ok) 等并检查转换结果；自定义类型需注册元类型，接口设计时仍应明确预期类型，避免滥用动态类型。

```
QVariant display = QStringLiteral("ready"); // DisplayRole 可返回文本
QVariant edit = 42; // EditRole 也可返回数值
bool ok = false;
const int count = edit.toInt(&ok); // 动态转换要检查结果
if (ok) updateCount(count);
QVariant custom = QVariant::fromValue(MyValue{});
qRegisterMetaType<MyValue>(); // 自定义类型跨元对象边界前注册
```

006 Qt QObject 父子对象树如何管理生命周期？

答案 QObject 父对象析构时会递归删除仍归它所有的子对象，因此 QWidget 常把子控件设父对象来自动回收；父子对象必须处于同一线程；moveToThread() 会把无父对象的目标及其子对象一起迁移；不能有多个所有者重复 delete。无父对象者由调用方用栈对象、智能指针或明确所有权管理；deleteLater() 将删除延后到对象所属线程处理：若主事件循环已经停止，事件不会再处理；无事件循环的工作线程中，对象会在线程结束时销毁。

```
QObject owner; // 栈对象先构造，也会先于局部 child 析构
auto *child = new QObject(&owner); // parent 是唯一所有者且同线程
auto *independent = new Worker; // 无 parent，由调用方负责
independent->moveToThread(&workerThread); // 只能移动无 parent 对象及其子树
connect(&workerThread, &QThread::finished, independent, &QObject::deleteLater);
workerThread.start(); // 退出时须 quit()、wait()，并让事件循环处理延迟删除
child->deleteLater(); // owner 仍会回收；延迟删除依赖所属线程事件循环
```

007 Qt 信号槽依赖什么机制？连接类型如何影响调用？

答案 带 Q_OBJECT 的类由 moc 生成元对象代码，Qt 据此支持自定义信号、槽、属性和运行时调用；新式函数指针 connect 可在编译期检查签名。AutoConnection 在发射线程就是接收对象线程时直调，否则排队到接收线程事件循环；Direct 在发射线程同步调用，Queued 异步执行，BlockingQueued 会等待且同线程可能死锁。

```
connect(worker, &Worker::resultReady, window, &Window::showResult);
emit worker->resultReady(result); // AutoConnection 跨线程时排队
connect(worker, &Worker::progress, window, &Window::showProgress, Qt::QueuedConnection);
connect(this, &Controller::localUpdate, this, &Controller::updateNow, Qt::DirectConnection);
// DirectConnection 在发射线程同步执行；不要用它跨线程碰 GUI
// 同线程 BlockingQueuedConnection 会死锁，接收线程还必须运行事件循环
```

008 Qt 的隐式共享与写时复制是什么？

答案 Qt 隐式共享值类型以引用计数让多个副本暂时共用数据，复制通常只增加引用计数；其中一个副本写入时若仍共享，才分离并复制数据，这就是写时复制。它适合按值传递 QByteArray、QString 等，但不能把它当同步机制：不同线程共享对象时仍须遵循读写安全规则，迭代器失效也要按具体容器处理。

```
QByteArray original("abc");
QByteArray copy = original; // 隐式共享：通常只复制引用计数头
copy[0] = 'A'; // 共享时写入会先分离再修改
Q_ASSERT(original == QByteArray("abc")); // 值对象彼此独立
Q_ASSERT(copy == QByteArray("Abc"));
// 写时复制减少值传递开销，但不替代跨线程读写同步
```

009 Qt 容器类和 STL 容器类有什么区别？

答案 Qt 容器与 STL 容器在接口、迭代器规则、算法生态和 Qt API 适配上不同；Qt 容器常有隐式共享，STL 则可配合标准算法、allocator 和范围库。Qt 6 中 QList 与 QVector 底层均连续，旧版表现不同；按项目版本、库边界和性能测量选用，跨库接口优先避免不必要的容器转换。

010 一个信号可以连接多个槽吗？一个槽可以连接多个信号吗？

答案 可以，一个信号可连接多个槽，发射时会逐个调用已连接的接收端；一个槽也可接收多个信号，适合把多个来源汇入同一处理逻辑。参数类型须兼容，槽可以少接收信号末尾参数；动态初始化时注意重复 connect 会造成重复响应，可保存 QMetaObject::Connection 并按需断开。

```
const auto first = connect(button, &QPushButton::clicked, this, &Window::save);
const auto second = connect(button, &QPushButton::clicked, this, &Window::submit);
// 一次发射会调用两个接收端；同一槽也可连接多个信号
disconnect(first); // 按需移除某条连接
Q_ASSERT(second); // 检查连接句柄仍代表有效连接
// 重复初始化会重复响应；UniqueConnection 适用于成员函数连接
// Lambda 需保存 QMetaObject::Connection 管理其连接生命周期
```

011 信号与槽的优缺点是什么?

答案 信号槽让发送者只声明事件而不依赖接收者，函数指针形式通常能在编译期检查参数，也能通过队列连接跨线程调度。代价是连接与元对象分发有开销，调试异步调用链也需追踪发射点和连接点；跨线程排队不等于自动保护共享数据，GUI 控件仍须在主线程操作。要排查时可用 QObject::dumpObjectInfo() 查看对象连接关系，并记录关键发射点。

```
connect(source, &Source::changed, view, &View::refresh, Qt::QueuedConnection);
emit source->changed(value); // 发送方只发布事件，不依赖具体 UI 实现
// 跨线程排队仅调度调用，不会锁住外部共享变量
// 用 QObject::dumpObjectInfo() 查看连接与对象信息
qInfo() << "Source::changed emitted" << value; // 记录异步链路关键点
// GUI 接收端在主线程处理；锁和对象亲和性仍需独立遵守
```

012 如何自定义信号和槽?

答案 在 QObject 子类中声明 signals: void dataReady(const QByteArray &data);，发射处写 emit dataReady(bytes)，Qt 信号通常不自行实现函数体。槽可用普通成员函数或 Lambda，通过 QObject::connect(sender, &Sender::dataReady, receiver, &Receiver::handleData) 连接；自定义信号类需 Q_OBJECT，并确保构建系统运行 moc。

```
class Worker : public QObject {
    Q_OBJECT
signals:
    void dataReady(const QByteArray &bytes); // 信号通常无需函数体
public slots:
    void run() { emit dataReady(makePayload()); }
};
connect(worker, &Worker::dataReady, receiver, &Receiver::handleData);
// 含 Q_OBJECT 的类需要构建系统运行 moc
```

14.2 界面与绘图

001 Graphics View 框架是什么？适用于什么场景?

答案 Graphics View 由场景、视图和图元组成，适合需要大量可选取、拖动、缩放或碰撞检测的二维对象，例如流程图编辑器、节点画布和地图标注。它管理场景坐标与视口变换，可让多个 view 显示同一 scene；若只是普通表单控件，直接用 Widgets 布局更简单。大量图元仍需通过索引、局部更新和绘制缓存实测优化。

```
auto *scene = new QGraphicsScene(this); // 管理图元和场景坐标
auto *item = scene->addEllipse(QRectF(0, 0, 24, 24), QPen(Qt::blue), QBrush(Qt::cyan));
item->setFlag(QGraphicsItem::ItemIsSelectable);
auto *view = new QGraphicsView(scene, this); // 视图处理滚动与变换
view->scale(1.5, 1.5); // 编辑器类场景可缩放和平移
// 普通表单无需 Graphics View；大场景再测索引和局部更新
```

002 QGraphicsScene、QGraphicsView、QGraphicsItem 的关系是什么?

答案 QGraphicsItem 是可绘制和交互的单个图元，可有父子变换关系；QGraphicsScene 管理图元集合、场景坐标、选择与碰撞查询；QGraphicsView 把场景映射到滚动视口并处理缩放、平移。一个 scene 能被多个 view 同时观察，例如主编辑区和缩略预览；item 不一定继承 QObject，需信号槽时可用 QGraphicsObject。

```
auto *scene = new QGraphicsScene(this);
auto *shape = scene->addRect(QRectF(0, 0, 80, 40)); // QGraphicsItem
shape->setFlag(QGraphicsItem::ItemIsMovable);
mainView->setScene(scene); // view 映射同一个 scene
previewView->setScene(scene); // 多个 view 可同时观察
previewView->scale(0.25, 0.25);
// 需 QObject 信号槽的图元可改用 QGraphicsObject
```

003 QPaintDevice、QPaintEngine、QPainter 的关系是什么?

答案 QPaintDevice 是绘制目标抽象，QWidget、QImage、QPixmap、打印机等都能作为设备；QPainter 提供画线、文本和变换等高层指令。实际输出由设备对应的 QPaintEngine 执行，例如窗口后端或图像光栅引擎，因此同一段 painter 代码可画到屏幕、图片或 PDF，但格式与性能受设备影响。

```
QImage image(QSize(160, 80), QImage::Format_ARGB32_Premultiplied);
image.fill(Qt::transparent); // QImage 也是 paint device
{
    QPainter painter(&image); // QPainter 发出绘图指令
    painter.drawText(QPoint(8, 32), QStringLiteral("preview"));
} // painter 结束后再读取或显示 image
widget->update(); // QWidget 由窗口绘制后端输出，不要长期持有 painter
```

004 QPainter 的基本使用方法是什么?

答案 自定义 QWidget 通常重写 paintEvent()，在函数内创建 QPainter painter(this)，按需设置 QPen、QBrush、字体和变换，再调用 drawLine、drawText 或 drawImage；函数返回后绘制结束。不要在任意时刻直接绘制 QWidget，数据变化后调用 update() 请求合并重绘，而非手动调用 paintEvent()；绘制设备若是 QImage，则在该图像上创建 painter。

```
void PlotWidget::setValue(double value) {
    value_ = value;
    update(); // 请求事件循环合并重绘
}
void PlotWidget::paintEvent(QPaintEvent *event) {
    QPainter painter(this); // 仅在 paintEvent 中画 QWidget
    painter.setClipRegion(event->region()); // 只画失效区域
    painter.drawLine(QPoint(8, 8), mapValue(value_));
} // 不直接调用 paintEvent，也不在其中做 I/O
```

005 QPixmap、QImage、QBitmap 有什么区别? 使用场景是什么?

答案 QImage 是可直接访问像素的硬件无关图像，适合解码、逐像素处理和后台线程绘制；QPixmap 更适合界面显示，通常与窗口系统图像资源集成，跨平台实现不应假设它必然由 GPU 加速。QBitmap 是单色位图，常用于遮罩；实际图像处理可在线程中生成 QImage，再把结果交给 GUI 线程显示。

```
// 后台线程解码和逐像素处理使用 QImage
QImage decoded = QImageReader(path).read();
if (decoded.isNull()) report(QStringLiteral("image decode failed"));
QImage processed = transform(decoded); // 线程间传值时按数据所有权处理
QMetaObject::invokeMethod(this, [this, processed] {
    label->setPixmap(QPixmap::fromImage(processed)); // GUI 线程显示
}, Qt::QueuedConnection);
// QBitmap 仅适合单色蒙版；不要假设 QPixmap 一定由 GPU 加速
```

006 QWidget、QDialog、QMainWindow 有什么区别?

答案 QWidget 是 Widgets 控件和窗口的基础，可作为普通控件或顶层窗口；QDialog 增加对话框语义与 accept/reject 结果，适合短交互；QMainWindow 提供菜单栏、工具栏、状态栏和中央区域等主窗体结构。三者都需在 GUI 线程创建和操作，子控件通常交给布局管理器与 QObject 父子树管理。

007 Qt Designer 和手写代码创建界面有什么区别?

答案 Qt Designer 用拖放编辑 .ui 表单，构建时由 uic 生成界面初始化代码，适合稳定、常规的控件布局，调整视觉结构也直观。手写代码便于按数据动态生成组件、复用构造逻辑和做复杂定制；两者可以混用，例如 .ui 搭静态页面、代码绑定模型和运行时控件，避免重复手工维护生成文件。Designer 生成文件不建议直接手改；需复用自定义控件时可在表单中提升为子类。

008 Qt 中如何实现反走样（抗锯齿)?

答案 在绘制代码中调用 painter.setRenderHint(QPainter::Antialiasing)，可减少斜线和曲线边缘锯齿；文字可另外启用 TextAntialiasing，缩放图像时还可设 SmoothPixmapTransform。提示应在每次绘制时设置，效果依赖后端、设备像素比和变换；高频绘制需测量开销，不能只靠开抗锯齿解决低分辨率素材问题。

```
void Canvas::paintEvent(QPaintEvent *) {
    QPainter painter(this);
    painter.setRenderHint(QPainter::Antialiasing); // 曲线与斜线
    painter.setRenderHint(QPainter::TextAntialiasing); // 文字单独控制
    painter.setRenderHint(QPainter::SmoothPixmapTransform); // 缩放图像
    painter.drawEllipse(QPointF(40, 40), 25, 25);
} // 效果受设备像素比、变换和绘图后端影响
// 高频绘制要在目标设备测量开销
```

009 Qt 中如何实现拖拽功能?

答案 拖放源端在 mouseMoveEvent 等事件中创建 QDrag，设置 QMimeData 携带文本、URL 或自定义格式，再调用 exec()；目标端启用 setAcceptDrops(true)，按需重写 dragEnterEvent、dragMoveEvent 和 dropEvent 并检查 MIME 格式。拖动对象使用 MIME 数据传递而非裸指针；若跨应用拖放，还需考虑动作类型、文件权限和平台行为。

```
void SourceWidget::mouseMoveEvent(QMouseEvent *event) {
    if (!dragging_) return;
    auto *mime = new QMimeData;
    mime->setText(selectedText()); // MIME 可跨 widget/应用传值
    QDrag drag(this); drag.setMimeData(mime);
    drag.exec(Qt::CopyAction | Qt::MoveAction);
}
void TargetWidget::dragEnterEvent(QDragEnterEvent *event) {
    if (event->mimeData()->hasText()) event->acceptProposedAction();
} // dropEvent 中再次校验 MIME，再读取文本
```

010 Qt 中有哪些常用的布局管理器?

答案 QHBoxLayout、QVBoxLayout 管理横向和纵向排列，QGridLayout 处理网格，QFormLayout 适合标签加输入框的表单，QStackedLayout 切换一组页面中的当前页。布局根据 sizeHint、伸缩因子和边距分配尺寸，可嵌套组合并随窗口缩放；一般把布局设到父 widget 上，不要同时手动设置子控件几何位置。

```
auto *form = new QFormLayout;
form->addRow(tr("Name"), nameEdit); // 标签加输入框
auto *grid = new QGridLayout;
grid->addWidget(formWidget, 0, 0); // 网格可放子布局
auto *pages = new QStackedLayout;
pages->addWidget(pageA); pages->addWidget(pageB);
pages->setCurrentWidget(pageA); // 页面切换不重设子控件坐标
parentWidget->setLayout(grid); // 布局接管几何分配和缩放
```

011 Qt 样式表（QSS）是什么？如何使用?

答案 Qt Style Sheets 是类似 CSS 的控件样式规则，可用 QWidget::setStyleSheet() 或应用级设置统一背景、边框、字体及伪状态，例如 QPushButton:hover。选择器按对象名、类和属性匹配，范围过大的应用级样式可能覆盖平台原生观感；复杂绘制、布局和交互仍由控件或 QStyle 管理，QSS 不是完整 CSS。

```
button->setObjectName(QStringLiteral("saveButton"));
button->setProperty("accent", true); // 属性可供选择器匹配
qApp->setStyleSheet(QStringLiteral(
    "QPushButton[accent=\"true\"] { background: #2878c8; }"
    "QPushButton:hover { border: 1px solid white; }"));
// 应用级规则会覆盖较大范围；复杂绘制仍由控件/QStyle 实现
button->setEnabled(false); // 样式不会替代交互状态逻辑
```

012 Qt 的事件处理机制是什么？事件传递顺序是怎样的?

答案 事件循环从系统或队列取出事件并交给目标 QObject 的 event()，默认再分派到 mousePressEvent、paintEvent 等处理函数；也可用 sendEvent 同步发送或 postEvent 排队。某些被忽略的输入事件会继续传播到父控件，但并非所有事件都按固定链路冒泡；处理后 accept()，不处理则按事件类型决定是否 ignore()。

```
class Target : public QWidget {
    void mousePressEvent(QMouseEvent *event) override {
        if (event->button() == Qt::LeftButton) event->accept();
        else event->ignore(); // 某些输入事件可继续向父控件传播
    }
};
QCoreApplication::postEvent(target, new MyEvent(payload)); // 排队稍后分发
// sendEvent() 同步重入调用；事件循环调用 event() 再分派专用处理函数
```

013 event() 与事件过滤器有什么区别?

答案 event() 是对象统一的事件入口，适合拦截多类事件或处理自定义 QEvent，并可调用基类维持默认分派。事件过滤器由另一个 QObject 对目标 installEventFilter()，在目标处理前观察或截断事件；过滤函数返回 true 表示已处理，返回 false 才继续交给目标，过滤器与目标必须处于同一线程。

```
target->installEventFilter(this); // filter 与 target 必须同线程
bool Filter::eventFilter(QObject *watched, QEvent *event) {
    if (watched == target && event->type() == QEvent::MouseButtonPress) {
        handlePress(static_cast<QMouseEvent *>(event));
        return true; // 已处理，目标不会再收到该事件
    }
    return QObject::eventFilter(watched, event); // 保持默认过滤行为
}
```

014 paintEvent()、resizeEvent()、mousePressEvent()等事件函数的作用是什么?

答案 paintEvent() 负责绘制失效区域，通常由 update() 触发；resizeEvent() 在尺寸变化后通知控件，可重算缓存或布局相关状态；mousePressEvent() 等处理输入，并可读取按钮、位置和修饰键。它们是事件分派的专用虚函数，重写后应按需调用基类实现，且耗时工作不应放进 GUI 事件回调。

```
void Canvas::resizeEvent(QResizeEvent *event) {
    cachedSize_ = event->size(); // 重算与尺寸相关的缓存
    QWidget::resizeEvent(event); // 按需保留基类行为
}
void Canvas::mousePressEvent(QMouseEvent *event) {
    if (event->button() == Qt::LeftButton) { beginDrag(event->position()); return; }
    QWidget::mousePressEvent(event);
}
void Canvas::paintEvent(QPaintEvent *) { QPainter(this).drawImage(rect(), cachedImage_); }
// 事件回调只更新轻状态；耗时工作放到 worker
```

015 如何实现双缓冲绘图?

答案 双缓冲先把完整帧画入离屏图像，再一次性呈现，可避免逐笔刷新造成闪烁；但 QWidget 默认已采用双缓冲，一般只需在 paintEvent 中完整绘制更新区域。自定义动画可复用 QImage/QPixmap 缓存静态背景，再绘制动态层；缓冲区尺寸、设备像素比与失效更新范围要匹配窗口变化。只有在非 QWidget 设备上自行缓存时，才需要针对脏区域复制或重绘缓冲内容。

```
// QWidget 已由 backing store 双缓冲，通常不需手写整窗缓冲
void Canvas::setStaticLayer(const QImage &layer) {
    cachedLayer_ = layer; update();
}
void Canvas::paintEvent(QPaintEvent *event) {
    QPainter painter(this); painter.setClipRegion(event->region());
    painter.drawImage(QPoint(0, 0), cachedLayer_); // 重用静态层
    drawMovingObjects(painter); // 每帧只重绘变化部分
}
// 缓存需随尺寸/DPR 更新；避免保存多份全尺寸图像
```

016 如何实现自定义控件?

答案 先选择合适基类，普通自绘控件继承 QWidget，需保留按钮语义则继承 QPushButton；再重写 paintEvent 和必要的鼠标、键盘或尺寸事件，并定义信号向外报告交互。提供 sizeHint()/minimumSizeHint() 让布局合理分配空间，设置焦点策略、可访问性和属性；资源应由父子树或 RAII 管理。

```
class Meter : public QWidget {
    Q_OBJECT
public:
    QSize sizeHint() const override { return QSize(120, 32); }
signals:
    void valueActivated(int value); // 交互用信号报告给外部
protected:
    void paintEvent(QPaintEvent *) override { QPainter(this).drawText(rect(), text_); }
    void mousePressEvent(QMouseEvent *) override { emit valueActivated(value_); }
};
// QWidget 子类留在 GUI 线程；父子树或 RAII 管理其资源
```

017 模态对话框和非模态对话框有什么区别?

答案 模态对话框暂时阻止指定范围内的其他窗口交互，常用于必须先确认的短操作；非模态对话框通过 show() 打开，用户可继续使用主窗体，适合查找、监控和设置面板。QDialog::exec() 会启动嵌套事件循环并同步返回结果，open() 或 show() 配合 finished 信号通常更利于避免重入和保持异步流程。

```
auto *dialog = new SettingsDialog(this); // parent 保证窗口对象活到关闭
connect(dialog, &QDialog::finished, dialog, &QObject::deleteLater);
connect(dialog, &QDialog::finished, this, &Window::applySettings);
dialog->open(); // 非模态流程，不启动同步嵌套循环
// 短操作可用 exec() 并检查 Accepted/Rejected
// exec() 返回前 GUI 线程仍可重入处理其他事件
```

14.3 并发与网络

001 QMutex、QReadWriteLock、QSemaphore 有什么区别?

答案 QMutex 保证临界区同一时刻只有一个线程进入，适合保护普通共享状态；QReadWriteLock 允许并发读、独占写，适合读多写少但写者等待规则需考虑饥饿。QSemaphore 管理可用许可数，例如连接池最多开放 8 个并发槽位；用完要 release，避免资源计数泄漏。临界区可用 QMutexLocker 等 RAII 守卫，确保提前 return 或异常路径也会解锁。

```
QMutex mutex;
{
    QMutexLocker lock(&mutex); // 所有访问共享状态的路径用同一把锁
    ++sharedCount_;
} // 提前 return 也会释放
QReadWriteLock rw;
{ QReadLocker read(&rw); inspect(sharedData_); } // 多读者
{ QWriteLocker write(&rw); sharedData_ = next; } // 写者独占
QSemaphore slots(8); // acquire 后使用许可，完成必须 release
```

002 Qt 中有哪些多线程方式？QThread 应如何使用？

答案 简单一次性计算可用 QtConcurrent 或 QThreadPool；需要独立事件循环的长驻任务常用 QThread 加 worker-object：创建 QObject worker，moveToThread() 后以队列信号槽交互，并让 worker 在线程结束时 deleteLater。QThread 对象本身属于创建它的线程；GUI 控件留在主线程，退出时让 worker 协作检查取消请求，再 quit() 并 wait() 回收；quit() 不会打断正在执行的长函数。

```
auto *thread = new QThread(this); // QThread 对象属于创建线程
auto *worker = new Worker; worker->moveToThread(thread); // worker 无 parent
connect(thread, &QThread::started, worker, &Worker::process);
connect(worker, &Worker::finished, thread, &QThread::quit);
connect(thread, &QThread::finished, worker, &QObject::deleteLater);
thread->start(); // 工作槽需要 worker 线程事件循环
// 关闭时请求协作取消，再 quit() 并 wait(); quit 不打断长函数
```

003 QtConcurrent 是什么？有什么应用场景？

答案 QtConcurrent 提供高层并行算法和 QtConcurrent::run()，通常把工作提交到 QThreadPool，避免手工管理每个线程，适合独立的计算、压缩或批量处理任务。用 QFuture 表示结果，需要进度或完成通知时配 QFutureWatcher；它不适合操作 GUI 控件，结果应通过 watcher 信号或队列槽回到主线程。

```
QFuture<Result> future = QtConcurrent::run([input] { return calculate(input); });
auto *watcher = new QFutureWatcher<Result>(this);
connect(watcher, &QFutureWatcher<Result>::finished, this, [watcher] {
    display(watcher->result()); // 回主线程显示结果
    watcher->deleteLater();
});
watcher->setFuture(future); // pool 管理工作线程
// 任务不可直接访问 QWidget；长任务需设计取消与错误传播
```

004 Qt 中如何实现线程安全？

答案 先确定共享状态由哪个线程拥有；能避免共享时优先复制数据或消息传递，需要共享才用 QMutex/QReadWriteLock 保护所有相关读写。QObject 的线程亲和性不代表其方法可任意跨线程并发调用，事件驱动对象应在所属线程使用；跨线程用队列信号槽，并处理取消、退出和对象销毁顺序。互斥量只能保护遵守同一锁约定的访问者，锁外读写仍会产生数据竞争。

```
// 优先让一个线程独占可变 QObject 状态
connect(worker, &Worker::resultReady, controller, &Controller::applyResult,
        Qt::QueuedConnection);
void Controller::applyResult(const Result &result) {
    model_->replace(result); // 模型只在所属线程被修改
    emit viewNeedsRefresh();
}
// 若必须共享裸数据，对所有读写统一加同一把锁
// 线程亲和性本身不等于对象方法线程安全
```

005 Qt 中如何实现线程间通信？

答案 最常见做法是发送者发信号、接收者位于工作线程或主线程，并使用 QueuedConnection，让槽在接收对象所属线程的事件循环处理。大数据也可用线程安全队列、QWaitCondition 或 QMutex 传递；参数应可复制或注册元类型，且线程事件循环必须运行，不能从 worker 直接更新 QWidget。

```
connect(worker, &Worker::resultReady, window, &Window::showResult,
        Qt::QueuedConnection);
worker->startWork(); // 发射端可在工作线程
// Qt 把槽排到 window 所在线程的事件循环
// 参数类型需可复制；自定义类型跨队列要注册元类型
// 事件循环未运行时队列槽不会处理
// worker 只传结果，不直接访问 QWidget
```

006 如何避免多线程中的死锁?

答案 给多把锁规定全局固定获取顺序，并尽量缩小持锁区间，避免持锁等待另一个线程的信号、事件循环或 I/O。QMutex::tryLock(timeout) 可避免无限等待，但超时后仍需明确回滚策略；BlockingQueuedConnection、互相 join/wait 也会形成等待环，应做超时日志和可取消任务。

```
// 多锁统一规定顺序：先 accountMutex，再 ledgerMutex
QMutexLocker first(&accountMutex);
if (!ledgerMutex.tryLock(100)) {
    rollbackLocalChanges(); // 超时也要恢复一致状态
    return;
}
QMutexLocker second(&ledgerMutex, QMutexLocker::AdoptLock);
commitTransfer(); // 尽量缩短临界区
// 持锁时不要同步等待信号、I/O、另一线程 wait() 或阻塞队列槽
```

14.4 数据、模型与事件

001 Delegate 的作用是什么？如何自定义 Delegate?

答案 Delegate 把模型项渲染到视图并提供可选编辑器，避免为每个单元格创建真实控件。自定义时继承 QStyledItemDelegate，重写 paint() 和 sizeHint() 控制显示，需要编辑再实现 createEditor、setEditorData、setModelData；例如状态列可画彩色圆点，编辑列用 QComboBox。

```
void StatusDelegate::paint(QPainter *p, const QStyleOptionViewItem &o,
                           const QModelIndex &i) const {
    const bool active = i.data(Qt::UserRole).toBool();
    p->setBrush(active ? Qt::green : Qt::gray);
    p->drawEllipse(o.rect.center(), 5, 5); // 绘制可见单元格，无常驻 QWidget
}
QWidget *StatusDelegate::createEditor(QWidget *parent, const QStyleOptionViewItem &,
                                      const QModelIndex &) const {
    return new QComboBox(parent); // 编辑时才创建 editor
} // setEditorData/setModelData 在编辑器和 model 间传值
```

002 Model / View 中的数据角色（Role）是什么?

答案 Role 是 QModelIndex 上按用途索取数据的整数标识：DisplayRole 给视图展示，EditRole 给编辑器，DecorationRole 常放图标，ToolTipRole 给提示。模型的 data(index, role) 按角色返回 QVariant；自定义角色从 Qt::UserRole 起定义，并在 QML 等场景通过 roleNames() 暴露稳定名称。

```
QVariant TaskModel::data(const QModelIndex &i, int role) const {
    if (!i.isValid() || i.row() >= tasks_.size()) return {};
    const Task &task = tasks_[i.row()];
    if (role == Qt::DisplayRole) return task.title;
    if (role == Qt::DecorationRole) return task.icon;
    if (role == Qt::UserRole) return task.done;
    return {};
}
// 自定义角色集中声明；QML 列表模型还实现 roleNames()
// 视图按角色取值，不要把所有数据塞成一个显示字符串
```

003 QAbstractItemModel 的作用是什么?

答案 QAbstractItemModel 定义视图读取行列、父子层级、数据和编辑状态的协议，使同一数据源可连接不同 View。简单列表可继承 QAbstractListModel，只需正确实现 rowCount()、data()，表格还实现 columnCount()；插删数据必须用 beginInsertRows/endInsertRows 等通知视图维护索引和选择。

```
int TaskModel::rowCount(const QModelIndex &parent) const {
    return parent.isValid() ? 0 : static_cast<int>(tasks_.size());
}
QVariant TaskModel::data(const QModelIndex &i, int role) const {
    if (!i.isValid() || i.row() < 0 || i.row() >= rowCount()) return {};
    if (role == Qt::DisplayRole) return tasks_[i.row()].title;
    return {};
}
// QAbstractListModel 把数据接口提供给 QListView/QML；修改时必须发模型通知
```

004 QCoreApplication::processEvents() 的作用是什么?

答案 processEvents() 在当前线程同步分发待处理事件，常用于少量必须保持响应的初始化循环，但可能让尚未完成的对象被重入调用，形成状态竞争。它也不能让耗时计算变快，频繁调用会掩盖主线程阻塞；长任务应移到 worker 或拆成定时小步，并由事件循环自然调度。尤其不能在模型通知、半初始化控件或持锁期间嵌套处理任意事件。

```
// 不用 processEvents() 在长循环里反复嵌套分发任意回调
auto *watcher = new QFutureWatcher<Result>(this);
connect(watcher, &QFutureWatcher<Result>::finished, this, [watcher] {
    showResult(watcher->result()); watcher->deleteLater();
});
watcher->setFuture(QtConcurrent::run([input] { return compute(input); }));
// 少量可切成 QTimer::singleShot() 小步任务
// 持锁、改模型通知或对象半初始化时绝不嵌套处理事件
```

005 QDataStream 和 QTextStream 有什么区别?

答案 QDataStream 将 Qt/C++ 值按二进制格式序列化，适合结构化存档或协议，但需固定 stream version、字节序及字段顺序以兼容不同版本。QTextStream 面向可读文本，处理字符编码、行文本和格式化；跨机器传文件时显式约定编码，Qt 6 的默认文本编码是 UTF-8。

```
QDataStream binary(&file);
binary.setVersion(QDataStream::Qt_6_0); // 固定序列化版本
binary.setByteOrder(QDataStream::BigEndian); // 文件格式显式约定字节序
binary << quint32(1) << record.id << record.name; // 结构化二进制
if (binary.status() != QDataStream::Ok) report(binary.status());
QTextStream text(&textFile); text.setEncoding(QStringConverter::Utf8);
text << record.name << '\n'; // 面向可读文本
// 读不可信长度时先限幅，再分配内存
```

006 QFile、QFileInfo、QDir 的作用是什么?

答案 QFile 提供文件打开、读取、写入和复制等 I/O；QFileInfo 查询路径对应的大小、时间、权限等元数据；QDir 用于遍历、创建目录和路径组合。比如用 QDir::entryInfoList() 列出文件，再由 QFile 打开目标；检查 open() 返回值与 errorString()，路径应按输入来源做校验。

```
QDir dir(rootPath);
for (const QFileInfo &info : dir.entryInfoList(QDir::Files)) {
    if (!isAllowed(info.fileName())) continue; // 校验输入路径/名称
    QFile file(info.filePath());
    if (!file.open(QIODevice::ReadOnly)) { qWarning() << file.errorString(); continue; }
    consume(file.readAll()); // QFile 析构关闭文件
    qInfo() << info.size() << info.lastModified();
}
// 组合路径用 QDir，不手工拼平台分隔符
```

007 QListView、QTableView、QTreeView 有什么区别?

答案 QListView 以一维方式显示项，QTableView 展示行列数据，QTreeView 展示 QModelIndex 父子层级；三者都可连接同一个自定义 model。大量数据应选 model/view 而非为每条记录创建 widget，例如文件列表用 QFileSystemModel 接 QListView，表格用 QAbstractTableModel 接 QTableView。

```
auto *model = new QFileSystemModel(this);
model->setRootPath(rootPath); // 层级数据由模型管理
listView->setModel(model); // 一维项
tableView->setModel(model); // 行列数据
treeView->setModel(model); // 父子层级
treeView->setRootIndex(model->index(rootPath));
// 大量记录用 model/view 和 delegate，避免每行创建 QWidget
```

008 QNetworkAccessManager 的作用是什么?

答案 QNetworkAccessManager 统一发起 HTTP 等请求，异步返回 QNetworkReply，可监听 finished、downloadProgress 和错误信号，不会像同步等待那样阻塞界面。通常每个线程复用一个 manager，manager 与 reply 在其所属线程事件循环工作；完成后读 reply 数据、处理错误并 deleteLater()，不能立刻跨线程读取。

```
auto *manager = new QNetworkAccessManager(this); // 在线程内复用
QNetworkReply *reply = manager->get(QNetworkRequest(QUrl(url)));
connect(reply, &QNetworkReply::finished, this, [reply] {
    if (reply->error() == QNetworkReply::NoError) consume(reply->readAll());
    else qWarning() << reply->errorString();
    reply->deleteLater(); // 读完再延迟销毁
});
// manager/reply 由所属线程事件循环驱动，不在别的线程直接读取
```

009 QSettings 的作用是什么？如何使用？

答案 QSettings 按组织名、应用名或显式文件名保存键值配置，平台默认可落到注册表、INI 等位置。示例：settings.setValue("window/size", size)，重启后用 value(key, default).toSize() 读取；用 beginGroup()/endGroup() 管理层级，敏感凭据不应因为方便就明文存入普通配置。

```
QSettings settings;
settings.beginGroup(QStringLiteral("window"));
settings.setValue(QStringLiteral("size"), size());
settings.endGroup();
settings.sync(); // 写回后检查持久化状态
if (settings.status() != QSettings::NoError) reportSettingsError();
const QSize saved = settings.value(QStringLiteral("window/size"), QSize(800, 600)).toSize();
// 普通配置不存明文口令，升级时按版本迁移键值
```

010 QSqlDatabase、QSqlQuery、QSqlTableModel 的作用是什么？

答案 QSqlDatabase 表示驱动和连接，负责设置数据库路径、主机与登录参数；QSqlQuery 执行 SQL、绑定参数并遍历结果；QSqlTableModel 把单表接到 Qt 视图并可编辑。数据库连接只能在创建它的线程访问，后台线程需建立自己的命名连接；跨线程共享同一连接是不安全的。连接名应避免冲突，多个数据库或后台任务宜显式命名而不依赖默认连接。

```
const QString name = QUuid::createUuid().toString();
QSqlDatabase db = QSqlDatabase::addDatabase(QStringLiteral("QSQLITE"), name);
db.setDatabaseName(filePath);
if (!db.open()) return report(db.lastError());
{ QSqlQuery query(db); query.prepare(QStringLiteral("SELECT name FROM user WHERE id=?"));
  query.addBindValue(userId); if (!query.exec()) report(query.lastError()); }
// 连接在哪个线程创建就在哪个线程使用
db.close(); db = {}; QSqlDatabase::removeDatabase(name); // 先销毁 query/model/副本
```

011 QSqlQueryModel、QSqlTableModel、QSqlRelationalTableModel 有什么区别？

答案 QSqlQueryModel 用自定义查询结果构建只读模型，适合 join 或聚合报表；QSqlTableModel 围绕一张表并提供修改、插删和提交策略；QSqlRelationalTableModel 在单表基础上把外键值映射为关联表字段。复杂多表编辑通常需自定义模型或拆分事务，不能把只读查询模型当成可写表。

```
QSqlQueryModel reportModel(this);
reportModel.setQuery(QStringLiteral("SELECT department, count(*) FROM staff GROUP BY department"), db);
// 自定义 JOIN/聚合通常用查询模型，只读展示
QSqlTableModel editable(this, db); editable.setTable(QStringLiteral("employee"));
editable.select(); // 单表编辑使用 QSqlTableModel
QSqlRelationalTableModel related(this, db);
related.setTable(QStringLiteral("order")); // 外键可映射关联表显示
// 多表编辑需自定义提交/事务边界
```

012 QTcpSocket 和 QUdpSocket 的使用场景是什么？

答案 QTcpSocket 建立面向连接的可靠有序字节流，适合登录、文件传输等需要完整顺序的业务，但 TCP 不保留应用消息边界。QUdpSocket 发送独立数据报，适合发现、组播、实时状态；它可能丢包、乱序或重复，需由协议自行处理。两者异步读写都依赖对象线程的事件循环。UDP 的最大载荷受路径 MTU 影响，过大数据报会分片并增大丢失概率。

```
QTcpSocket tcp; tcp.connectToHost(host, port); // 可靠有序字节流
connect(&tcp, &QTcpSocket::readyRead, this, &Client::appendTcpBytes);
QUdpSocket udp; udp.bind(QHostAddress::AnyIPv4, 0);
udp.writeDatagram(payload, serverAddress, serverPort); // 独立数据报
// TCP 需自行定义消息边界；UDP 要处理丢失、乱序与重复
// 两类 socket 都需在有事件循环的所属线程中使用
```

013 QTimer 的使用方法是什么？

答案 创建 QTimer 并连接 timeout()，调用 setInterval(ms) 和 start() 即可周期触发；setSingleShot(true) 或 singleShot() 用于延时一次执行。定时器事件由对象所属线程的事件循环投递，回调繁忙时会延迟，不能用于硬实时测量；精确经过时间应用 QElapsedTimer 计时。

```
QElapsedTimer elapsed; elapsed.start(); // 测实际经过时间
QTimer timer(this); timer.setInterval(500); // 事件循环驱动的周期回调
connect(&timer, &QTimer::timeout, this, [this] { refreshView(); });
timer.start();
// 所属线程必须有事件循环；回调繁忙会延迟触发
// QTimer 不是硬实时计时器，不用 timeout 间隔测精确耗时
```

014 Qt 中有哪些实现定时器的方式?

答案 QTimer 提供信号槽、单次/周期启动和调整间隔，适合绝大多数界面定时任务；QObject::startTimer() 返回 ID，需在 timerEvent() 中分辨并处理；QBasicTimer 是轻量封装，可管理该 ID。三种都由所属线程事件循环驱动，精度受系统调度和繁忙事件队列影响。

```
QTimer highLevel(this); // timeout 信号和单次/周期 API
connect(&highLevel, &QTimer::timeout, this, &Worker::tick);
highLevel.setSingleShot(true); highLevel.start(200);
const int timerId = startTimer(1000); // 低层 ID 由对象管理
void Worker::timerEvent(QTimerEvent *e) {
    if (e->timerId() == timerId) doSmallStep();
    else QObject::timerEvent(e);
}
// 两种方式都依赖对象线程事件循环
```

015 Qt 的事件循环机制是什么?

答案 事件循环从系统窗口事件和 Qt 队列取出事件，分发到 QObject 的 event()，并运行排队槽、定时器及网络通知；GUI 主循环由 QApplication/QCoreApplication::exec() 启动，工作线程可用 QThread::exec()。某线程没有事件循环时，定时器和队列槽等事件驱动功能不能正常投递，长时间阻塞也会暂停处理。

```
int main(int argc, char **argv) {
    QApplication app(argc, argv); // 创建 GUI 应用上下文
    MainWindow window; window.show();
    QTimer::singleShot(0, &window, &MainWindow::startAsyncWork);
    return app.exec(); // 网络/定时器/排队槽由主循环分发
}
// 工作线程需自己的 exec() 才能处理该线程的 queued slot 和 QTimer
// 长时间阻塞某线程会暂停该线程的事件处理
```

016 Qt 的模型 / 视图（Model / View）架构是什么?

答案 Model 保存并提供数据，View 负责浏览与选择，Delegate 负责单元格绘制和编辑，三者经 QModelIndex、数据角色和模型通知协作。模型发出 dataChanged 或 begin/end 插入删除通知后，视图局部更新；同一模型可接多个视图，例如表格与汇总列表同步观察一份业务数据。

```
class TaskModel : public QAbstractListModel { /* 数据与 role API */ };
TaskModel model;
QListView list; QTableView table;
list.setModel(&model); table.setModel(&model); // 两个视图观察同一份数据
model.setData(model.index(0, 0), QStringLiteral("done"), Qt::EditRole);
// 模型发 dataChanged/插删通知后视图更新
// Delegate 管绘制与编辑，View 管选择和呈现
```

017 Qt 中如何实现 JSON 数据的解析和生成?

答案 用 QJsonDocument::fromJson(bytes, &error) 解析，检查 QJsonParseError 后按根类型读取 QJsonObject 或 QJsonArray；字段还应验证类型和默认值。生成时组装对象/数组，再用 toJson(QJsonDocument::Compact) 得到 QByteArray；适合 HTTP API，若需大文档流式处理则考虑增量解析而非一次载入内存。

```
QJsonParseError error;
const QJsonDocument doc = QJsonDocument::fromJson(bytes, &error);
if (error.error != QJsonParseError::NoError || !doc.isObject()) return report(error);
const QJsonObject object = doc.object();
if (!object.value("id").isDouble()) return reportSchemaError();
QJsonObject response{{"ok", true}, {"id", object.value("id")}};
const QByteArray output = QJsonDocument(response).toJson(QJsonDocument::Compact);
// 大文档流式读取可用 QJsonStreamReader 等分段方案
```

018 Qt 中如何实现 WebSocket 通信?

答案 客户端用 QWebSocket::open() 发起 ws/wss 握手，连接 connected、textMessageReceived、binaryMessageReceived、errorOccurred 和 disconnected 后收发完整 WebSocket 消息；服务端用 QWebSocketServer 监听并管理接入 socket。生产环境要验证 TLS 证书、鉴权、消息大小和断线重连，事件驱动 socket 必须在有事件循环的所属线程操作。

```
QWebSocket socket; // 对象需存活在事件循环线程
connect(&socket, &QWebSocket::connected, this, [&socket] {
    socket.sendTextMessage(QStringLiteral("hello"));
});
connect(&socket, &QWebSocket::textMessageReceived, this, &Client::onMessage);
connect(&socket, &QWebSocket::errorOccurred, this, &Client::onError);
connect(&socket, &QWebSocket::disconnected, this, &Client::scheduleReconnect);
socket.open(QUrl(QStringLiteral("wss://example.invalid/socket")));
// 校验证书、限制消息大小，并为断线重连设置退避
```

019 Qt 中如何实现 XML 数据的解析?

答案 QXmlStreamReader 是前向流式解析器，逐个读取 startElement、属性和文本，内存占用适合大文件；解析中要检查 hasError() 并报告行列位置。需要生成时用 QXmlStreamWriter 保证转义正确；若业务必须随机查询完整树，可考虑 DOM，但 Qt 6 已移除旧 SAX 类，应按目标版本选 API。

```
QXmlStreamReader xml(&file); // 前向解析，适合大文件
while (!xml.atEnd()) {
    xml.readNext();
    if (xml.isStartElement() && xml.name() == u"item")
        consume(xml.attributes().value(u"id").toString());
}
if (xml.hasError()) report(xml.errorString(), xml.lineNumber(), xml.columnNumber());
QXmlStreamWriter writer(&out); writer.writeStartDocument();
writer.writeTextElement(QStringLiteral("name"), userText); // 自动转义
writer.writeEndDocument(); // DOM 仅在需要随机访问整棵树时载入
```

020 Qt 中如何读写文本文件和二进制文件?

答案 文本可用 QFile 加 QTextStream，读写时明确 UTF-8 等编码、换行和错误状态；二进制原始字节直接通过 QFile::read/write，结构化二进制可用 QDataStream。持久化格式要固定 QDataStream::Version 并在格式中写版本号，读取不可信输入时检查长度和流状态，避免错误数据导致超大分配。

```
QFile binary(path);
if (!binary.open(QIODevice::WriteOnly)) return report(binary.errorString());
QDataStream out(&binary); out.setVersion(QDataStream::Qt_6_0);
out << quint32(formatVersion) << payload; // 结构化二进制
QFile textFile(textPath); if (!textFile.open(QIODevice::WriteOnly)) return;
QTextStream text(&textFile); text.setEncoding(QStringConverter::Utf8);
text << QString::fromUtf8(message); // 编码与字节序显式约定
// 读取外部文件要验证长度和流状态后再使用数据
```

021 Qt 中如何连接数据库?

答案 先用 QSqlDatabase::addDatabase("QSQLITE", connectionName) 创建连接，设置文件名并 open()，随后检查 lastError()；其他驱动还需主机、用户名、密码等参数。用完先销毁所有 QSqlQuery/QSqlTableModel 和数据库副本，再 removeDatabase(connectionName)，避免仍有引用时移除产生警告或悬空使用。

```
const QString connection = QUuid::createUuid().toString();
{
    QSqlDatabase db = QSqlDatabase::addDatabase(QStringLiteral("QSQLITE"), connection);
    db.setDatabaseName(filePath);
    if (!db.open()) return report(db.lastError());
    { QSqlQuery query(db); runQueries(query); } // query 先离开作用域
    db.close();
} // 清除所有 QSqlDatabase 副本、model 后再 remove
QSqlDatabase::removeDatabase(connection); // 否则仍引用连接会告警
```

022 Qt 中有哪些网络编程相关的类?

答案 TCP 服务端/客户端用 QTcpServer、QTcpSocket，UDP 用 QUdpSocket，HTTP 请求用 QNetworkAccessManager，WebSocket 用 QWebSocket/QWebSocketServer。文件与配置可由 QFile/QSettings 处理，SQL 用 Qt SQL；网络对象通常依赖其线程事件循环，选择模块前确认目标 Qt 版本、构建组件和平台插件已部署。

023 exec() 函数的作用是什么?

答案 QCoreApplication::exec() 启动应用主事件循环并在退出时返回状态码；QDialog::exec() 启动嵌套局部循环并返回接受/拒绝结果，常用于模态对话框。嵌套 exec 会带来重入风险，异步界面更适合 show()/open() 后监听 finished；QThread::exec() 则为工作线程处理其队列事件。

```
QCoreApplication app(argc, argv);
QTimer::singleShot(0, &worker, &Worker::processChunk); // 让循环自然回到事件队列
return app.exec();
// processEvents() 会同步重入当前线程所有已排队事件
// 不能在半初始化对象、发模型通知或持锁时调用
// 长任务应异步拆分或交给 worker，而非插入嵌套事件泵
```

024 startTimer() 和 QTimer 有什么区别?

答案 QTimer 把超时封装为 timeout() 信号，支持 start/stop、周期及单次配置，调用代码更清晰；startTimer() 直接返回定时器 ID，需要在 timerEvent() 判断 ID 并自行管理生命周期。它们都不创建独立计时线程，定时回调在对象亲和线程运行并受该线程事件循环负载影响。

```
class TimerOwner : public QObject {
    void startLowLevel() { timerId_ = startTimer(250); }
    void timerEvent(QTimerEvent *event) override {
        if (event->timerId() == timerId_) poll();
        else QObject::timerEvent(event);
    }
    int timerId_ = 0;
};
QTimer high(this); connect(&high, &QTimer::timeout, this, &TimerOwner::poll);
// 高层信号 API 更清楚；两者精度都受事件循环调度
```

025 如何使用 QTcpServer 实现服务器?

答案 创建 QTcpServer 后调用 listen(address, port)，收到 newConnection() 时用 nextPendingConnection() 取 socket，并连接 readyRead、disconnected 和 error 信号；每个连接要有独立缓冲并及时释放。TCP 是字节流，接收端需按长度前缀或分隔符拼出完整消息，同时限制最大包长并处理客户端断开。

```
auto *server = new QTcpServer(this);
connect(server, &QTcpServer::newConnection, this, [server] {
    while (server->hasPendingConnections()) {
        auto *client = server->nextPendingConnection(); // server 暂时拥有
        connect(client, &QTcpSocket::readyRead, this, [client] {
            appendAndParse(client, client->readAll()); // TCP 字节流
        });
        connect(client, &QTcpSocket::disconnected, client, &QObject::deleteLater);
    }
});
if (!server->listen(QHostAddress::Any, port)) report(server->errorString());
// 限制每连接缓冲与消息长度
```

026 如何使用 Qt 实现 HTTP 请求?

答案 创建并复用 QNetworkAccessManager，以 get(QNetworkRequest(url)) 或 post(request, body) 异步发送请求；给 QNetworkReply 接 finished/errorOccurred 槽，读取状态码、头部和响应体后 deleteLater()。请求头可设 Content-Type、认证信息，HTTPS 需验证证书；不要在 GUI 线程调用 waitFor... 阻塞等待。

```
auto *manager = new QNetworkAccessManager(this);
auto *reply = manager->post(QNetworkRequest(QUrl(url)), body);
connect(reply, &QNetworkReply::finished, this, [reply] {
    if (reply->error() != QNetworkReply::NoError) report(reply->errorString());
    else if (reply->attribute(QNetworkRequest::HttpStatusCodeAttribute).toInt() >= 400)
        reportHttpStatus(reply->attribute(QNetworkRequest::HttpStatusCodeAttribute));
    else consume(reply->readAll());
    reply->deleteLater();
});
// GUI 线程不要 waitFor...; HTTPS 必须验证证书
```

027 如何使用 Qt 操作 SQLite 数据库?

答案 确保部署了 Qt SQL 的 SQLite 驱动插件，建立命名 QSQLITE 连接并设置数据库文件路径；open() 成功后使用 QSqlQuery 的 prepare()/bindValue() 执行参数化语句，避免拼接用户输入。读多行要检查 exec()/next() 与 lastError()；关闭连接前销毁 query 和 model，数据库操作保持在连接所属线程。

```
const QString connection = QUuid::createUuid().toString();
{
    QSqlDatabase db = QSqlDatabase::addDatabase(QStringLiteral("QSQLITE"), connection);
    db.setDatabaseName(filePath);
    if (!db.open()) return report(db.lastError());
    QSqlQuery query(db); query.prepare(QStringLiteral("INSERT INTO user(name) VALUES(?)"));
    query.addBindValue(userName); if (!query.exec()) report(query.lastError());
} // 先析构 query/model/db 副本
QSqlDatabase::removeDatabase(connection); // 部署也要包含 QSQLITE 驱动
```

028 如何处理网络数据的粘包和拆包?

答案 TCP 只提供连续字节流，readyRead() 一次可能收到半包或多包，因此把 readAll() 追加到缓冲区后按协议循环解析。常用固定头部长度前缀方案：先等够头部，解出长度并检查上限，再等完整 payload；分隔符协议需处理转义或内容编码，不能假设一次 read 就是一条消息。长度字段要固定字节序并防整数溢出，处理完成后从缓冲区移除对应完整帧。

```
buffer.append(socket->readAll()); // readyRead 一次可能含半帧或多帧
while (buffer.size() >= 4) {
    const auto *head = reinterpret_cast<const uchar *>(buffer.constData());
    const quint32 length = qFromBigEndian<quint32>(head); // 固定大端长度
    if (length > maxPayload || length > std::numeric_limits<quint32>::max() - 4u) { socket->abort(); return; }
    if (buffer.size() < 4 + length) break; // 半帧留待下次数据
    handle(buffer.mid(4, length)); buffer.remove(0, 4 + length);
}
// 解析后移除完整包；长度上限避免溢出和内存耗尽
```

029 如何实现数据库的事务处理?

答案 确认当前 QSqlDriver 支持事务后调用 db.transaction()，用多个 QSqlQuery 完成一组变更并逐条检查结果；全成功才 commit()，任一步失败就 rollback()。事务应尽量短，并检查提交本身的返回值；事务只保证数据库操作原子性，不自动解决业务层重试、并发冲突或外部服务副作用。

```
if (!db.transaction()) return report(db.lastError());
bool ok = false; QSqlError queryError;
{ QSqlQuery q(db); q.prepare(sql); q.addBindValue(value);
  ok = q.exec(); if (!ok) queryError = q.lastError(); }
if (!ok) { db.rollback(); return report(queryError); } // query 先销毁
if (!db.commit()) { const auto error = db.lastError(); db.rollback(); report(error); }
// 事务尽量短；外部服务副作用不由数据库回滚
```

030 如何自定义 Model?

答案 列表模型优先继承 QAbstractListModel，实现 rowCount() 和 data()；表格模型再实现 columnCount()，编辑功能加 flags()、setData() 并在修改后发 dataChanged。插入、删除、重置前后必须配对调用 beginInsertRows/endInsertRows 等 API，索引由 model 创建和解释，树模型还须正确实现 index()、parent()。

```
bool Model::insertRows(int row, int count, const QModelIndex &parent) {
    if (parent.isValid() || count <= 0 || row < 0 || row > rows_.size()) return false;
    beginInsertRows(parent, row, row + count - 1); // 先通知视图
    rows_.insert(row, count, Item{});
    endInsertRows(); // 修改后闭合通知
    return true;
}
QVariant Model::data(const QModelIndex &i, int role) const { /* bounds + role */ }
// 修改已有值后发 dataChanged；移除也用 begin/endRemoveRows
```

14.5 高级特性与 QML

001 Loader、Repeater、ListView 的作用是什么?

答案 Loader 根据 source 或 sourceComponent 动态创建一段 QML 子树，适合延迟打开低频页面；切换 source 会销毁旧对象，使用前检查 status/item。Repeater 会一次为模型每项创建 delegate，适合数量较小的固定集合；ListView 按可视区域创建并可复用 delegate，更适合长列表，模型数据应独立于 delegate 生命周期。

```
// QML: Loader 仅在需要时创建低频页面
Loader { id: settings; active: settingsVisible; source: "Settings.qml" }
onLoaded: settings.item.resetView()
onStatusChanged: if (status === Loader.Error) console.warn(errorString())
// Repeater 会为所有项创建 delegate，适合小集合
Repeater { model: 5; delegate: Text { text: index } }
// 长列表用 ListView 的可视 delegate；model 状态不能放在会销毁的 delegate 内
```

002 QLibrary 的作用是什么?

答案 QLibrary 在运行时打开普通动态库，用 load() 加载、resolve("functionName") 取得导出函数地址，结束后 unload()；返回指针要匹配准确 ABI、调用约定和函数签名。库文件名及依赖按平台部署，加载失败应记录 errorString()；需要 Qt 插件元数据、接口检查和 QObject 实例时应使用 QPluginLoader。

```
QLibrary library(fileName);
if (!library.load()) return report(library.errorString());
using CreateFn = QObject *(*)(); // 导出函数 ABI 必须一致
auto create = reinterpret_cast<CreateFn>(library.resolve("createPlugin"));
if (!create) return report(library.errorString());
QObject *instance = create(); // 调用期间库必须仍加载
use(instance);
destroyWithLibraryContract(instance);
library.unload(); // 所有函数指针/对象失效后才卸载
```

003 QML 中如何实现动画效果?

答案 QML 可用 NumberAnimation、ColorAnimation 或 PropertyAnimation 改变属性，写在 Behavior 中可响应值变化，也可由 states/Transition 驱动状态切换。例：给 opacity 配 200ms 动画并用 easing 控制曲线；动画期间避免另一段逻辑持续覆盖同一属性，循环动画需设置明确的停止条件。

```
// QML
Rectangle {
    id: panel
    opacity: expanded ? 1.0 : 0.0
    Behavior on opacity {
        NumberAnimation { duration: 200; easing.type: Easing.OutCubic }
    }
}
// 状态切换也可由 states/Transition 驱动
// 多个动画不要同时命令式写同一属性；隐藏组件时停止循环动画
```

004 QML 中的信号和槽如何使用?

答案 QML 组件用 signal valueChosen(int value) 声明信号，用 onValueChosen: ... 处理本组件信号，也可通过 Connections 监听动态目标。C++ 端 QObject 信号可连接 QML handler，QML 信号也能由 C++ connect；参数类型须可由 QML 元类型系统识别，处理长任务时不要阻塞界面线程。

```
// QML
Item {
    signal valueChosen(int value)
    Button { onClicked: valueChosen(7) }
    onValueChosen: console.log("chosen", value)
}
// C++ 接 QObject 信号时用 typed connect
connect(&backend, &Backend::finished, this, &Window::showResult);
// QML 参数需可被元类型系统识别；重活不能同步阻塞 UI
```

005 QML 中的属性绑定是什么?

答案 属性绑定是表达式依赖：如 width: parent.width * 0.5，依赖值变化时引擎重新计算并更新属性，适合把布局和状态关系写清楚。若随后用命令式赋值直接写 width，原绑定通常会被移除；需要恢复时使用 Qt.binding()，表达式应保持纯计算，避免副作用造成重复执行。必要时可用 qmllint 检查绑定断裂和未限定属性引用，减少运行时才出现的问题。

```
// QML 属性绑定会跟踪 parent.width 变化
Rectangle { width: parent.width * 0.5 }
// 直接赋值通常会移除原绑定
target.width = 240
// 需要重新绑定时显式恢复表达式
target.width = Qt.binding(function() { return parent.width * 0.5 })
// 绑定应是无副作用计算；工具可检查绑定被意外打断
```

006 QML 和 C++ 如何交互?

答案 用 QObject 子类向 QML 暴露 Q_PROPERTY、信号、槽和 Q_INVOKABLE，再按需通过 qmlRegisterType/QML_ELEMENT 注册类型或提供实例。上下文属性虽方便但依赖不显式、工具难分析；优先考虑注册类型或根对象属性。明确 C++ 对象仍由谁持有，QML 引擎对方法返回 QObject 的所有权规则需特别检查。

```
class Backend : public QObject {
    Q_OBJECT
    Q_PROPERTY(QString title READ title NOTIFY titleChanged)
public:
    Q_INVOKABLE void refresh() { /* 异步触发工作 */ }
signals: void titleChanged();
};
qmlRegisterType<Backend>("App", 1, 0, "Backend"); // 类型可见
// QML 引擎使用对象时确认 QObject owner 与销毁顺序
```

007 Q_INVOKABLE 宏的作用是什么?

答案 Q_INVOKABLE 把成员函数写入元对象信息，可经 QMetaObject::invokeMethod() 按名称调用，也可让 QML 调用该方法；所在类须继承 QObject 并带 Q_OBJECT。它适合简单命令或查询，不应把耗时工作直接同步暴露给 QML，否则会卡住对应线程；参数和返回类型须能被元对象系统处理。

```
class Backend : public QObject {
    Q_OBJECT
public:
    Q_INVOKABLE QString greeting(const QString &name) const {
        return tr("Hello, %1").arg(name);
    }
};
QMetaObject::invokeMethod(&backend, "refresh", Qt::QueuedConnection);
// Q_INVOKABLE 写入元对象；同步耗时函数会卡住调用线程
```

008 Q_PROPERTY 宏的作用是什么?

答案 Q_PROPERTY 在 QObject 子类中声明可反射访问的静态属性，例如 Q_PROPERTY(QString title READ title WRITE setTitle NOTIFY titleChanged)，供 QML、Designer 和元对象 API 使用。写入值变化时发出 NOTIFY 信号，QML 绑定才会重新计算；属性读写函数和通知应对应同一状态，勿只把成员变量列出来却不维护一致性。

```
class Backend : public QObject {
    Q_OBJECT
    Q_PROPERTY(QString title READ title WRITE setTitle NOTIFY titleChanged)
public:
    QString title() const { return title_; }
    void setTitle(const QString &v) { if (title_ == v) return; title_ = v; emit titleChanged(); }
signals: void titleChanged();
private: QString title_;
};
// NOTIFY 触发 QML 绑定更新；属性读写必须共享同一状态
```

009 Qt Widgets 与 Qt Quick/QML 如何选择?

答案 Qt Widgets 提供成熟桌面控件和原生风格集成，适合表单、传统工具和大量既有 QWidget 代码；Qt Quick/QML 用声明式层次、属性绑定和动画，适合触控、动态视觉界面。也可混合使用：QML 构建外观，C++ 提供模型和业务逻辑；选型还要考虑平台、团队经验与模块部署。Widgets 和 Quick 可以通过容器桥接混用，但跨界会增加事件、样式和构建复杂度。

010 Qt 中如何实现动态库的加载?

答案 普通 C ABI 动态库用 QLibrary 加载并 resolve 导出函数；Qt 插件则由共享库实现稳定接口，接口用 Q_DECLARE_INTERFACE 声明、插件类用 Q_PLUGIN_METADATA 导出，再由 QPluginLoader::instance() 取对象并 qobject_cast 校验。插件必须与宿主的 Qt/编译器 ABI、接口版本和部署目录兼容，释放实例前不能卸载库。

```
// 普通 C ABI 用 QLibrary; Qt 插件更适合 QPluginLoader
QPluginLoader loader(pluginPath);
QObject *object = loader.instance();
if (!object) return report(loader.errorString());
auto *plugin = qobject_cast<MyInterface *>(object);
if (!plugin) return report(QStringLiteral("interface mismatch"));
plugin->start();
// 销毁插件对象及其调用者后才能卸载库; 宿主/插件 ABI 必须匹配
```

011 Qt 中如何实现属性系统?

答案 静态属性在类中以 Q_PROPERTY 声明并由 moc 记录；运行时通过 QObject::property()/setProperty() 按名称访问。名称未声明为静态属性时，setProperty() 可附加动态属性，适合临时标记或 QSS 属性选择器；静态属性写入仍遵循 WRITE/read-only 规则，不能因为使用反射就绕过类型和线程约束。

```
// 静态属性在 QObject 子类中用 Q_PROPERTY + moc 声明
Q_PROPERTY(int count READ count WRITE setCount NOTIFY countChanged)
object->setProperty("count", 3); // 走静态属性的 WRITE
object->setProperty("busy", true); // 未声明时添加动态属性
const QVariant busy = object->property("busy");
// 动态属性无强类型访问器和专属 NOTIFY
// 常驻业务状态优先声明静态属性
```

012 Qt 的反射机制是如何实现的?

答案 Qt 元对象系统由 moc 在构建阶段为含 Q_OBJECT 的类生成表格元信息，支持信号槽、属性、类名查询和 invokeMethod 等运行时能力。它解决的是 Qt 对象层需要的有限反射，不会自动枚举任意 C++ 成员或模板；构建系统需启用 AUTOMOC，普通 C++ 反射需求仍需其他机制或显式注册。

```
class Device : public QObject {
    Q_OBJECT
    Q_PROPERTY(QString name READ name)
public slots: void start() { /* ... */ }
};
const QMetaObject *meta = device->metaObject(); // moc 生成元数据
int propertyIndex = meta->indexOfProperty("name");
int methodIndex = meta->indexOfMethod("start()");
QMetaObject::invokeMethod(device, "start", Qt::QueuedConnection);
// 这套反射不会自动枚举任意 C++ 成员; 构建启用 AUTOMOC
```

013 Qt 的对象序列化如何实现?

答案 Qt 没有自动把任意 QObject 实例及其属性完整持久化的通用序列化器；通常显式选取字段，用 QDataStream 自定义 operator<< / >>，或转成 QVariantMap/QJsonObject 再保存。读写双方要约定格式版本、默认值和兼容策略；QObject 的指针、父子树和信号连接不会因写出属性而自动恢复。

```
QJsonObject record{{"version", 2}, {"name", object.name()}};
const QByteArray bytes = QJsonDocument(record).toJson(QJsonDocument::Compact);
QFile file(path); if (!file.open(QIODevice::WriteOnly)) return report(file.errorString());
if (file.write(bytes) != bytes.size()) return report(file.errorString());
QJsonParseError error; const auto doc = QJsonDocument::fromJson(file.readAll(), &error);
if (error.error != QJsonParseError::NoError) return report(error.errorString());
restoreFields(doc.object()); // QObject 指针树/连接不会由属性自动恢复
// 长文档和兼容格式需限制大小并显式迁移 version
```

014 Qt 的插件机制是什么? 如何实现插件?

答案 插件提供者先定义版本化纯虚接口，宿主和插件共享接口头文件；插件实现 QObject 类、使用 Q_OBJECT 与 Q_PLUGIN_METADATA，接口用 Q_DECLARE_INTERFACE 标记。宿主遍历插件目录，用 QPluginLoader 加载、检查 errorString() 并 qobject_cast；插件实例销毁前不得 unload，且部署要匹配 Qt 次版本、编译器 ABI 和依赖。

```
QPluginLoader loader(pluginPath);
QObject *root = loader.instance(); // loader 持有库和根对象
if (!root) return report(loader.errorString());
auto *iface = qobject_cast<MyInterface *>(root); // 检查接口
if (!iface) return report(QStringLiteral("unsupported plugin"));
iface->initialize();
connect(this, &Host::closing, root, &Plugin::stop);
// 停止调用者并销毁对象后才 unload; 检查版本和 ABI
```

015 动态属性（Dynamic Property）是什么?

答案 动态属性是在运行时通过 setProperty("busy", true) 附加到 QObject 的名称和值，不需在类源码中预先定义，可用 property() 查询或传给基于属性选择器的 QSS。它不自动提供强类型访问器或属性专属 NOTIFY 信号，属性变化会产生 QDynamicPropertyChangeEvent；常驻业务状态更适合声明 Q_PROPERTY。

```
widget->setProperty("busy", true); // 运行时附加属性
const QVariant value = widget->property("busy");
if (!value.isValid()) return; // 检查属性是否存在
applyBusyStyle(value.toBool());
connect(widget, &QObject::destroyed, this, &Controller::clearState);
// 动态属性变化还会产生 QDynamicPropertyChangeEvent
// 常驻业务状态改用有类型访问器的 Q_PROPERTY
```

14.6 工程实践

001 .pro 文件的作用是什么？常用配置有哪些?

答案 .pro 是 qmake 项目描述文件，告诉构建系统生成应用、库还是子项目，并枚举源文件、模块、宏和链接依赖。常见项有 TEMPLATE、QT、SOURCES、HEADERS、CONFIG、INCLUDEPATH、LIBS；例如加入 network 模块后还要链接对应 Qt 库。Qt 6 仍可构建 qmake 项目，新项目也可按团队工具链采用 CMake。

```
TEMPLATE = app
QT += widgets network sql
CONFIG += c++20
SOURCES += main.cpp worker.cpp
HEADERS += worker.h
FORMS += mainwindow.ui
# qmake 从 .pro 生成构建文件；CMake 项目改用 Qt6::Widgets 等 targets
```

002 C++11 / 14 / 17 / 20 的新特性在 Qt 中如何应用?

答案 现代 C++ 能简化 Qt 代码：C++11 的 lambda 常用于 connect，智能指针和移动语义改善所有权；C++17 的结构化绑定、std::optional 可用于数据接口，C++20 协程可与合适异步库结合。Qt 版本与编译器决定可用标准和 Qt API，标准语言特性不等于 Qt 自动支持协程，构建时应显式设置 CXX_STANDARD。

```
connect(button, &QPushButton::clicked, this, [this, id = record.id] {
    saveRecord(id); // lambda 按值捕获异步所需数据
});
std::optional<Result> result = findResult(id); // C++17 接口可表达缺失值
if (result) show(*result);
// CXX_STANDARD 由项目工具链显式设置；语言标准不等于 Qt API 版本
```

003 Lambda 表达式在 Qt 信号槽中如何使用?

答案 新式 connect 可直接把信号连接到 lambda，例如 connect(button, &QPushButton::clicked, this, [this]{ save(); });，上下文对象销毁时连接会断开。捕获局部变量按值保存或确保引用在连接期内有效；长生命周期异步回调需防止访问已释放对象，简单逻辑适用 lambda，复杂业务仍宜拆到具名槽。

```
connect(button, &QPushButton::clicked, this, [this, recordId] {
    save(recordId); // context 销毁时连接自动断开
});
const auto connection = connect(source, &Source::ready, this,
    [this](const Result &r) { show(r); });
disconnect(connection); // 动态连接可保存句柄
// 引用捕获的局部变量必须活到回调结束；复杂逻辑放具名槽
```

004 MVC、MVP、MVVM 在 Qt 中如何应用?

答案 MVC 把数据、显示和输入协调分开，Qt 的 Model/View/Delegate 是数据视图领域的成熟实现；MVP 由 Presenter 作为中介调用 View 接口，便于测试。MVVM 让 ViewModel 暴露可绑定状态，适合 QML 属性绑定，但它不是 Qt 强制架构；根据界面复杂度和测试边界选取，避免为简单页面套多层抽象。

005 Qt 5 和 Qt 6 有什么主要区别?

答案 Qt 6 延续大量 Qt 5 API，但提高了工具链要求、重整图形栈与部分模块，QList/QVector 的实现关系也变了；Qt 6 工程通常以 CMake 作为主流构建选择。迁移应对照目标小版本的 Porting Guide，检查已移除模块、重命名或行为变化、信号签名和部署插件，并在实际目标平台编译运行。

006 Qt 6 中移除了哪些模块?

答案 Qt 6 移除了 Qt Script、Qt XmlPatterns、Qt Quick Controls 1 等旧模块，Qt WebKit 不再随 Qt 提供且 WebEngine 不是源码级等价替换；Qt State Machine 等内容也调整了模块归属。Qt Multimedia 接口经过重做，不能简单归为整个模块移除；迁移时按目标小版本官方清单确认替代 API 和兼容层。

007 Qt 6 的新特性有哪些?

答案 Qt 6 用 RHI 抽象 Qt Quick 图形后端，使场景可使用 Vulkan、Metal、Direct3D 或 OpenGL 等后端，具体可用性由平台和构建决定；还更新了 QML 工具链、模块化和高 DPI 行为。它不是保证所有设备都自动加速，应用需验证渲染后端、驱动、着色器与目标硬件兼容性。

008 Qt Creator 的使用技巧有哪些?

答案 Qt Creator 中可用 Kits 管理 Qt 版本、编译器、调试器和目标设备，先确认 Kit 匹配再构建；代码导航、符号搜索和安全重构能减少手动查找。调试时查看调用栈、线程和局部变量，必要时给 Qt/应用配置符号；Designer 用于编辑 .ui，性能分析则应使用 profiler 找到热点再优化。

009 Qt Test 框架是什么？如何使用?

答案 Qt Test 是随 Qt 提供的单元测试框架，测试类继承 QObject，在 private slots 中写 initTestCase、测试函数和清理函数，用 QVERIFY/QCOMPARE 断言。由 QTEST_MAIN 或 QTEST_GUILESS_MAIN 提供入口，CMake 链接 Qt::Test 后可在 CI 运行；测试优先覆盖业务逻辑，GUI 行为可用 QTest 鼠标键盘事件验证。

```
class ModelTest : public QObject {
    Q_OBJECT
private slots:
    void rowCountIsUpdated() {
        TaskModel model; QCOMPARE(model.rowCount(), 0);
        model.append(Task{QStringLiteral("one")});
        QCOMPARE(model.rowCount(), 1);
    }
};
QTEST_GUILESS_MAIN(ModelTest) // CMake target_link_libraries(... Qt6::Test)
// GUI 交互可用 QTest::mouseClick; 外部依赖要用 fake 隔离
```

010 Qt 中如何调用平台特定的 API?

答案 跨平台功能优先调用 Qt API；确需原生能力时按 Q_OS_WIN、Q_OS_LINUX 等宏隔离代码，并将系统调用封装在平台适配层。窗口句柄或原生接口可能随平台后端、窗口生命周期和高 DPI 改变，使用前检查文档和返回值；让通用业务层不依赖平台头文件，便于其他平台构建。若要跨线程访问原生句柄，还需遵守操作系统线程与窗口有效期要求。

```
QWindow *window = /* 已创建的平台窗口 */;
#ifdef Q_OS_WIN
    const WId handle = window->winId(); // 仅在平台适配层调用原生 API
    useWin32Handle(handle);
#elif defined(Q_OS_MACOS)
    useCocoaWindow(window->winId());
#else
    usePortableQtPath(window);
#endif
// 检查句柄有效期/DPI; 平台头文件不要泄漏到业务层
```

011 Qt 中有哪些调试工具?

答案 定位问题时先用 Qt Creator 或 GDB/LLDB 查看断点、调用栈与线程；qDebug/qWarning 可补充运行日志，Qt Test 回归具体行为。内存问题用 ASan/LSan 或 Valgrind，性能用采样 profiler/QElapsedTimer；发布环境应保留符号化崩溃栈，日志不得泄露口令和令牌。

```
QElapsedTimer timer; timer.start();
runRepresentativeOperation();
qInfo() << "operation ms" << timer.elapsed(); // 局部测量而非结论
// Qt Creator Profiler/GDB 看调用栈、线程、CPU 热点
// ASan/LSan 或 Valgrind 查内存; 发布包保留匹配符号
// 用同一 Release 配置、同一输入复测改动前后
```

012 Qt 如何实现跨平台?

答案 Qt 通过平台抽象层和插件封装窗口系统、输入、文件与网络等差异，让同一套 API 在多个系统构建；Widgets 的平台插件会接入 Windows、Cocoa、X11 或 Wayland 等后端。跨平台不等于行为完全一致，字体、路径权限、屏幕坐标、编码和打包插件仍需目标平台测试，平台专属功能要隔离实现。

013 Qt 应用出现内存泄漏如何定位和处理?

答案 用 ASan/LSan、Valgrind 或系统堆分析器重现并记录分配栈，区分真正泄漏、仍可达缓存和延迟释放。检查未纳入 QObject 父子树的裸指针、shared_ptr 环、定时器与 lambda 捕获、重复创建资源；修复时明确单一所有者或 RAII，反复打开关闭页面并检查退出路径。

```
QPointer<Worker> worker = page->worker(); // 观察 QObject 是否仍存活
connect(worker, &Worker::done, this, [worker] {
    if (!worker) return; // 异步时对象可能已销毁
    consume(worker->result());
});
page->close(); // 父子树回收其子对象
// 用 sanitizer/堆快照定位孤儿对象、环和重复创建资源
// 修复后反复打开关闭页面并检查退出路径
```

014 Qt 界面如何实现国际化和运行时切换?

答案 用户可见文本用 tr() 标记，lupdate 提取到 .ts，译者在 Qt Linguist 编辑后用 lrelease 生成 .qm；运行时加载 QTranslator 并 installTranslator()。切换语言要重新执行控件 retranslateUi()/刷新模型展示文本，还需考虑复数、动态拼接文本和长词造成的布局变化；安装顺序影响翻译查找。

```
QTranslator translator; // 生命周期覆盖应用翻译期间
if (!translator.load(locale, QStringLiteral(":/i18n/app"))) reportLoadError();
qApp->installTranslator(&translator);
ui->retranslateUi(window); // Designer 控件刷新
model->refreshTranslatedLabels(); // 自定义模型文本也需重算
// 用户文案统一 tr()；复数用 n 参数，不拼接词片段
// 长词会改变布局，应检查窄窗口和切换时机
```

015 Qt 程序崩溃的常见原因有哪些?

答案 常见崩溃源包括空/悬空指针、越界、重复释放、对象树错误、跨线程访问 QWidget，以及异步槽或 lambda 使用已销毁对象。第三方库 ABI 不匹配、资源插件缺失也会导致启动或调用失败；用符号栈和 sanitizer 将故障定位到实际访问点，并检查关闭窗口、取消任务等生命周期边界。保留最小可复现输入和精确二进制版本，便于把偶发故障关联到符号栈。

016 qDebug()、qWarning()、qCritical() 有什么区别?

答案 qDebug 用于诊断信息，qInfo 表示一般运行信息，qWarning 提醒可恢复异常，qCritical 表示严重错误；qFatal 输出后会终止进程。日志级别本身不自动保证问题处理，程序仍要检查错误返回值；可通过 QT_NO_DEBUG_OUTPUT 等构建配置关闭调试输出，生产日志需设置滚动和隐私策略。

```
qDebug() << "cache miss" << key; // 诊断细节
qInfo() << "service started"; // 运行信息
qWarning() << "retrying" << error; // 可恢复问题
qCritical() << "database unavailable"; // 严重错误仍需明确处理
if (!query.exec()) handle(query.lastError()); // 日志等级不代替错误检查
// qFatal 会终止进程；生产日志需轮转并脱敏
```

017 qmake 和 CMake 有什么区别?

答案 qmake 用 .pro 描述 Qt 工程，Qt 集成直接、旧项目常见；CMake 是通用构建系统，Qt 6 官方文档主推其 imported targets、AUTOMOC/AUTOUIC/AUTORCC 与跨项目生态。两者都能构建 Qt 项目，选择应看依赖和团队工具；迁移时将 Qt 模块、资源和代码生成规则逐项映射，不只是改文件名。

```
cmake_minimum_required(VERSION 3.21)
project(App LANGUAGES CXX)
set(CMAKE_AUTOMOC ON)
set(CMAKE_AUTOUIC ON)
find_package(Qt6 REQUIRED COMPONENTS Widgets Network)
add_executable(app main.cpp window.cpp window.ui)
target_link_libraries(app PRIVATE Qt6::Widgets Qt6::Network)
# 旧 qmake 工程把模块/资源/uic 规则逐项映射再迁移
```

018 windeployqt 和 macdeployqt 的作用是什么?

答案 windeployqt 收集 Windows 运行所需的 Qt DLL、插件和 QML 依赖；macdeployqt 把相关库与插件放入 .app 并修整部分库路径。它们不能自动发现全部第三方依赖或替代签名、公证；部署后在未安装 Qt 的干净环境启动，检查平台插件、翻译、图片格式和多媒体后端。

019 不同平台下 Qt 程序的打包和部署有什么区别?

答案 Windows 常随程序部署 DLL 和 plugins/platforms；macOS 要维护 .app 内框架路径、代码签名和公证；Linux 则要面对发行版 ABI、系统库与平台插件版本。对应部署工具能复制一部分依赖，不能覆盖所有运行环境差异；在目标 OS/架构的干净机器测试安装、升级和卸载流程。

020 信号槽连接失败的常见原因有哪些?

答案 先检查 connect() 是否成功、信号槽签名和参数类型是否兼容；旧式字符串连接容易因拼写或 const/reference 差异失败，新式函数指针连接更能编译期报错。自定义信号需 QObject 派生并有 Q_OBJECT、构建时 moc 正常运行；跨线程槽要有接收线程事件循环，连接对象也须仍存活。

```
const auto connection = QObject::connect(
    sender, &Sender::ready, receiver, &Receiver::consume);
if (!connection) qWarning() << "connect failed"; // 新式签名编译期检查
// 跨线程队列槽依赖接收者事件循环
// 自定义信号类需要 QObject + Q_OBJECT + 正常 moc
// 重复初始化要么只连接一次，要么保存句柄按需断开
```

021 大量数据显示时如何优化性能?

答案 大数据视图用 QAbstractItemModel 配 QListView/QTableView，配合 Delegate 绘制可见项，避免为每行创建数千个 QWidget；耗时数据可用 canFetchMore()/fetchMore() 分批载入。批量修改时成组发模型通知、合并 UI 刷新，并测量 data()/paint() 热点；缓存需有上限且数据变化时失效。

```
bool Model::canFetchMore(const QModelIndex &parent) const {
    return !parent.isValid() && loaded_ < total_;
}
void Model::fetchMore(const QModelIndex &parent) {
    const int count = qMin(batchSize, total_ - loaded_);
    beginInsertRows(parent, loaded_, loaded_ + count - 1);
    loadNextBatch(count); loaded_ += count; endInsertRows();
}
// delegate 绘制可见行；缓存必须设上限并在数据改变时失效
```

022 如何优化 Qt 应用程序的启动速度?

答案 先用启动剖析记录耗时，再推迟非首屏模块、缩略图和网络请求，减少 main() 里的同步 I/O；可用 Loader 延后实例化 QML 页面，并把独立初始化放后台。资源系统主要改善资源组织与读取部署，不会自动减少加载时间；优先加载用户立即需要的内容并报告异步进度。通过预热、延迟和并行初始化需分别测量首屏时间与总启动时间，不能只看其中一个。

```
// QML：首页先显示，低频页面延后构造
ApplicationWindow {
    Component.onCompleted: Qt.callLater(() => settingsLoader.active = true)
    Loader { id: settingsLoader; active: false; source: "Settings.qml" }
}
// 网络与磁盘初始化异步执行，首屏只载入必要资源
// 分别测首屏时间与全部启动时间，延迟加载不一定缩短总耗时
```

023 如何优化 Qt 界面的绘制性能?

答案 先用 profiler 确认是 CPU 绘制、布局还是数据更新造成重绘；随后缩小 update() 区域、缓存静态层、避免 paintEvent 做 I/O 或复杂计算，并减少每帧创建临时对象。列表采用 Model/View 和 delegate 虚拟化；Qt Quick 图形后端是否加速依平台而定，要以目标设备测帧率和内存。

```
void Chart::setPoint(const QPointF &p) {
    const QRect dirty = markerRect_ | nextMarkerRect(p);
    marker_ = p; markerRect_ = nextMarkerRect(p);
    update(dirty); // 仅请求局部重绘
}
void Chart::paintEvent(QPaintEvent *e) {
    QPainter painter(this); painter.setClipRegion(e->region());
    drawCachedGrid(painter); drawMarker(painter, marker_);
}
// 不在 paintEvent 做 I/O/复杂计算；Model/View 可虚拟化大列表
```

024 如何减小 Qt 程序的体积?

答案 使用 Release 优化构建并去除符号文件，移除未使用 Qt 模块、插件、翻译和大资源，再检查依赖闭包；平台部署工具可按实际使用裁剪插件。静态链接常让单文件变大且不自动消除依赖，还需处理静态插件注册和许可证义务；比较安装包而非只看主可执行文件。不同平台可用 strip 或符号分离工具，但需留存对应符号以支持线上诊断。

025 如何减少 Qt 应用程序的内存占用?

答案 通过 profiler 和堆快照找出高水位来自图像、缓存、模型还是泄漏；限制队列与缓存容量，按目标尺寸解码图片，及时释放不再显示的临时数据。QObject 用父子关系明确生命周期，普通资源用 RAII；隐式共享有助减少副本但修改可能触发分离，避免无依据地反复 squeeze 容器。高分辨率图片可先生成缩略图，避免同时保留多个全尺寸 QPixmap。

```
QImageReader reader(path);
reader.setScaledSize(targetPixels); // 按目标显示尺寸解码
QImage thumbnail = reader.read();
if (thumbnail.isNull()) return report(reader.errorString());
cache.insert(key, thumbnail); // 缓存设置最大容量和淘汰策略
if (cache.size() > maxEntries) evictOldest();
thumbnail = {}; // 不再显示时释放引用
// 隐式共享能减副本，写操作仍可能触发 detach
```

026 如何分析 Qt 应用程序的性能瓶颈?

答案 在代表性场景下用 Qt Creator Profiler 或系统采样器定位 CPU、内存、QML 和渲染热点，先记录基线；QElapsedTimer 可量测局部关键路径，但不要把单次测量当结论。检查主线程阻塞、频繁布局/重绘和网络等待，针对最热环节改动后用相同输入复测。避免用 Debug 构建测量发布性能，编译优化和符号设置会显著改变结果。

```
QElapsedTimer timer; timer.start();
const auto result = runHotPath(input);
const qint64 elapsedMs = timer.elapsed();
qInfo() << "hot path ms" << elapsedMs; // 测真实路径
// profiler 先找 CPU/内存/渲染热点，不凭猜测优化
// 改动后用相同 Release 构建、设备和输入复测
// 单次耗时会受调度影响，应重复采样
```

027 如何处理不同平台的路径分隔符问题?

答案 路径拼接和遍历用 QDir、QFileInfo、QFile 等 API，避免手工拼接 Windows 的反斜杠；Qt 通常接受正斜杠作为跨平台路径分隔符。用户输入路径要规范化并验证根目录边界，不能仅靠 cleanPath() 防止路径穿越；区分文件系统路径、URL 和资源路径 :/。处理外部不可信路径时还应防止软链接和符号链接绕出允许目录。

```
QDir root(allowedRoot);
const QString candidate = root.filePath(userRelativePath); // 不手拼斜杠
const QString clean = QDir::cleanPath(candidate);
if (!isWithinAllowedRoot(clean, root.absolutePath())) return reject();
QFile file(clean);
if (!file.open(QIODevice::ReadOnly)) return report(file.errorString());
consume(file.readAll());
// cleanPath 不阻止软链接逃逸；边界校验按真实文件系统路径做
```

028 如何实现 Qt 应用程序的单例模式?

答案 若确实需要进程内唯一服务，可将构造函数私有、禁用拷贝并提供 instance()，由 C++11 静态局部初始化保证线程安全。QObject 单例应在 QApplication 创建后再实例化，并避免再交给另一父对象或智能指针重复释放；多数可替代服务可由应用对象持有并依赖注入，降低全局耦合。

```
class Service : public QObject {
    Q_OBJECT
public:
    static Service &instance() { static Service value; return value; }
    Service(const Service &) = delete;
    Service &operator=(const Service &) = delete;
private:
    Service() = default; // 不交给另一个 parent/shared_ptr 重复拥有
};
// 首次调用应在 QApplication 构造之后；优先考虑依赖注入
```

029 如何实现 Qt 应用程序的崩溃捕获和日志记录?

答案 普通错误应通过异常/错误码和 Qt 日志处理；崩溃时优先配置系统 core dump、Windows minidump 或崩溃收集器保存符号化栈和版本信息。SIGSEGV 等致命信号处理器中不能安全调用 QFile、qDebug 或分配内存，通常不可恢复继续运行；可记录最少信号信息后交给平台崩溃机制。

030 如何实现 Qt 应用程序的日志系统?

答案 用 qDebug/qInfo/qWarning 等按级别记录，通过 qInstallMessageHandler() 集中添加时间、线程 ID、分类并写入滚动文件；处理器可能由多个线程并发调用，写文件需同步。handler 内避免再次调用 Qt 日志以免递归，也不应阻塞主线程做网络上传；敏感字段应脱敏。

```
static QMutex logMutex;
void messageHandler(QtMsgType type, const QMessageLogContext &, const QString &msg) {
    QMutexLocker lock(&logMutex); // handler 可从多线程并发进入
    writeRotatingLog(type, msg.toUtf8()); // 自有写入函数
    if (type == QtFatalMsg) std::abort();
}
qInstallMessageHandler(messageHandler);
// handler 内不能再调 qDebug/qWarning，否则递归；不要同步上传网络
```

031 如何实现 Qt 应用程序的权限管理?

答案 Qt 只提供界面和系统 API，不会替业务定义权限模型；由可信后端或本地认证层验证身份和角色，再对每项敏感操作执行授权检查。隐藏按钮或禁用菜单仅改善体验，不能替代业务校验；通过集中权限服务管理策略，并记录拒绝和审计事件，避免把权限仅存于可篡改配置。服务器端权限检查应覆盖直接 API 调用，不能只依靠客户端界面限制。

032 如何实现 Qt 应用程序的皮肤切换?

答案 Widgets 可准备多套 QSS，在应用启动或用户切换时读取样式表并调用 qApp->setStyleSheet()；集中定义颜色和控件规则，减少组件覆盖。QML 可用统一 Theme 对象暴露调色板、间距和字体，让绑定自动更新；需重建或刷新缓存颜色，并检查原生平台样式、状态和可访问性。

```
QFile qssFile(stylePath);
if (!qssFile.open(QIODevice::ReadOnly)) return report(qssFile.errorString());
qApp->setStyleSheet(QString::fromUtf8(qssFile.readAll())); // Widgets 换肤
// QML: Theme 属性可绑定并驱动组件重绘\ntheme.accent = "#2878c8"; theme.spacing = 12;
// 组件按 theme 属性刷新自绘缓存；选择器尽量限制作用范围
// 验证焦点、禁用态、平台原生外观和可访问性
```

033 如何实现 Qt 应用程序的自动更新功能?

答案 客户端用 QNetworkAccessManager 检查版本并下载更新包，先校验数字签名/哈希和目标平台，再由独立 updater 在主程序退出后替换文件。更新目录应避免半写入，保留回滚版本并处理中断、权限和磁盘空间；TLS 传输本身不能替代包签名，更新清单也要防降级攻击。把更新状态机设计为可恢复步骤，可在重启后判断下载、校验或替换是否完成。

034 如何实现 Qt 应用程序的配置管理?

答案 用 QSettings 统一封装配置读写，设置组织名、应用名和默认值，通过 beginGroup() 分模块管理；退出前必要时 sync() 并检查 status()。适合窗口位置、偏好和非敏感参数，凭据应交给系统密钥链或安全凭据存储；为配置结构加版本迁移，避免升级后旧值被错误解释。同一应用的多个进程并发改写配置时，需关注 sync() 和冲突策略。

```
QSettings settings;
settings.beginGroup(QStringLiteral("config"));
const int version = settings.value("version", 0).toInt();
if (version < 2) migrateV1ToV2(settings); // 配置结构版本迁移
settings.setValue("version", 2);
settings.endGroup(); settings.sync();
if (settings.status() != QSettings::NoError) reportSettingsError();
// 凭据放系统密钥链，不写普通 INI/注册表键
```

035 如何实现一个自定义的标题栏?

答案 用 Qt::FramelessWindowHint 移除系统装饰，再通过布局创建标题文本和最小化/最大化/关闭按钮；鼠标拖动可记录按下偏移并移动窗口，也可调用平台支持的系统移动接口保留贴边行为。还要实现双击最大化、边缘缩放、屏幕工作区、高 DPI 与键盘可访问性，交互差异需在目标平台验证。

```
setWindowFlags(Qt::Window | Qt::FramelessWindowHint);
auto *bar = new QWidget(this); auto *layout = new QHBoxLayout(bar);
layout->addWidget(titleLabel); layout->addWidget(closeButton);
connect(closeButton, &QPushButton::clicked, this, &QWidget::close);
void TitleBar::mousePressEvent(QMouseEvent *e) {
    dragOffset_ = e->globalPosition().toPoint() - window()->pos();
}
void TitleBar::mouseMoveEvent(QMouseEvent *e) {
    if (e->buttons() & Qt::LeftButton)
        window()->move(e->globalPosition().toPoint() - dragOffset_);
}
// 还需双击最大化、边缘缩放、高 DPI 与键盘操作
```

036 如何实现串口通信工具?

答案 链接 Qt SerialPort，先用 QSerialPortInfo::availablePorts() 枚举设备，再配置端口名、波特率、数据位、校验和停止位并 open(QIODevice::ReadWrite)。readyRead() 中读取并追加缓冲，按设备协议解析完整帧；检查错误和热拔插，并在 GUI 中避免 waitForReadyRead() 长时间阻塞。

```
QSerialPort port(this); port.setPortName(portName);
port.setBaudRate(QSerialPort::Baud115200);
port.setDataBits(QSerialPort::Data8); port.setParity(QSerialPort::NoParity);
port.setStopBits(QSerialPort::OneStop);
if (!port.open(QIODevice::ReadWrite)) return report(port.errorString());
connect(&port, &QSerialPort::readyRead, this, [this] { buffer_ += port.readAll(); parseFrames(buffer_); });
// 检查错误/拔出；GUI 线程不 waitForReadyRead() 阻塞
```

037 如何实现图片查看器?

答案 QFileDialog 选图后用 QImageReader 检查格式、尺寸并解码，再以 QPixmap 显示在 QLabel 或由自绘控件按窗口缩放绘制；支持滚轮缩放、拖动和旋转时维护变换矩阵。超大图片可请求按目标大小解码，别在主线程反复全尺寸转换；保存时检查格式和写入错误。解码前设定像素尺寸上限，也可避免恶意图片触发内存耗尽。

```
QImageReader reader(path);
reader.setAutoTransform(true); // 应用 EXIF 方向
reader.setScaledSize(targetPixels); // 避免解完整超大图
QImage image = reader.read();
if (image.isNull()) return report(reader.errorString());
preview->setPixmap(QPixmap::fromImage(image)); // 在 GUI 线程显示
// 保存用 QImageWriter 并检查错误；外部输入限制像素数
// 滚轮缩放由变换矩阵控制，别反复全尺寸重采样
```

038 如何实现截图功能?

答案 桌面截图可取 QGuiApplication::screens() 中目标屏幕并用 QScreen::grabWindow(0) 获取 QPixmap，再预览或通过 save() 保存；多屏区域坐标和 devicePixelRatio 需正确换算。截取其他应用窗口受系统权限和平台限制，Wayland 等环境可能不允许任意全屏抓取，应提供授权或系统门户路径。

```
QScreen *screen = QGuiApplication::primaryScreen();
if (!screen) return report(QStringLiteral("no screen"));
QPixmap shot = screen->grabWindow(0); // 平台权限可能限制
if (shot.isNull()) return report(QStringLiteral("capture unavailable"));
if (!shot.save(outputPath)) return report(QStringLiteral("save failed"));
// 多屏先按窗口所在 QScreen 选择并换算 DPR/逻辑坐标
// Wayland 等环境应走平台授权门户
```

039 如何实现数据可视化图表?

答案 图表可选 Qt Graphs、旧有 Qt Charts、第三方库或自绘；折线适合趋势、柱状适合分类比较，数据和坐标轴最好由模型层管理。注意版本变化：Qt Charts 自 Qt 6.10 起已弃用，新项目应评估 Qt Graphs；动态图表要限制刷新率和点数，并测试目标后端性能。图表库通常面向数据可视化而非高频海量散点，超大数据应抽稀或分层聚合。

040 如何实现窗口的无边框和拖动?

答案 通过 setWindowFlags(Qt::FramelessWindowHint) 创建无系统边框窗口，自定义标题区域处理鼠标拖动：记录按下时全局坐标与窗口位置，仅左键移动时更新位置。手动方案还要实现边缘缩放、双击最大化和多屏边界；优先使用系统窗口移动/调整大小接口时需按目标 Qt 和平台确认支持。

```
setWindowFlags(Qt::Window | Qt::FramelessWindowHint);
void Window::mousePressEvent(QMouseEvent *e) {
    if (e->button() == Qt::LeftButton) dragOffset_ = e->globalPosition().toPoint() - pos();
}
void Window::mouseMoveEvent(QMouseEvent *e) {
    if (e->buttons() & Qt::LeftButton) move(e->globalPosition().toPoint() - dragOffset_);
}
// 优先用平台原生 startSystemMove；手动窗口还需边缘缩放
// 多屏边界、最大化恢复和无障碍键盘操作都要处理
```

041 如何实现系统托盘功能?

答案 创建 QSystemTrayIcon，设置有效图标和提示文本，调用 setContextMenu() 绑定 QMenu，并连接 activated() 响应单击、双击或菜单触发。先检查 isSystemTrayAvailable()，关闭主窗时可隐藏而不退出，并提供从托盘恢复和真正退出的路径；不同桌面环境对托盘激活行为可能不同。

```
QSystemTrayIcon tray(QIcon(":/icons/app.png"), this);
if (!QSystemTrayIcon::isSystemTrayAvailable()) reportNoTray();
auto *menu = new QMenu(this); menu->addAction(tr("Show"), this, &Window::showNormal);
menu->addAction(tr("Quit"), qApp, &QCoreApplication::quit);
tray.setContextMenu(menu);
connect(&tray, &QSystemTrayIcon::activated, this, [this](auto reason) {
    if (reason == QSystemTrayIcon::Trigger) showNormal();
});
tray.show(); // 关闭窗口可 hide 而不是退出，并保留真正退出入口
```

042 如何实现网络调试助手?

答案 界面可组合 QTcpSocket 客户端、QTcpServer 服务端和 QUdpSocket 数据报模式，用状态区显示连接、错误、发送和接收日志。以 QByteArray 缓冲收发并明确十六进制/ASCII 编码、帧边界和最大长度；自动发送用 QTimer，限制高频速率和日志容量，避免大量文本刷新拖慢界面。

043 如何实现聊天软件的界面?

答案 消息列表用 QAbstractListModel 保存发送者、时间、正文、状态和头像等字段，QListView 配合自定义 Delegate 绘制左右气泡；输入栏、附件和发送动作由独立控件负责。分页历史和新消息分批更新模型，长消息按宽度换行；网络收发放后台异步层，不能在 delegate 绘制时读取数据库或阻塞请求。

```
class MessageModel : public QAbstractListModel { /* roles: body/time/status */ };
MessageModel model; QListView list; list.setModel(&model);
list.setItemDelegate(new BubbleDelegate(&list)); // delegate 绘制气泡而非每行 QWidget
connect(sendButton, &QPushButton::clicked, this, [this] {
    emit sendRequested(input->text()); input->clear();
});
connect(networkWorker, &Worker::messageReceived, &model, &MessageModel::append,
        Qt::QueuedConnection);
// 分页载入历史；绘制时不查数据库/不做网络请求
```

044 如何实现视频播放器?

答案 Qt 6 可用 QMediaPlayer::setSource() 载入媒体，通过 setVideoOutput(QVideoWidget*) 显示画面，再提供 play()/pause()、positionChanged 和 durationChanged 进度控制。音量由 QAudioOutput 管理；格式与编解码能力依平台后端，需检查错误和缓冲状态，并对 Qt 5 旧接口及 QMediaPlaylist 移除情况做版本迁移。

```
auto *player = new QMediaPlayer(this);
auto *audio = new QAudioOutput(this); player->setAudioOutput(audio);
auto *video = new QVideoWidget(this); player->setVideoOutput(video);
connect(player, &QMediaPlayer::errorOccurred, this, &Window::showMediaError);
connect(player, &QMediaPlayer::positionChanged, slider, &QSlider::setValue);
player->setSource(QUrl::fromLocalFile(filePath)); player->play();
// 检查后端格式/缓冲状态；音量由 QAudioOutput 管理
```

045 如何对 Qt 应用程序进行单元测试?

答案 用 Qt Test 测纯业务对象、模型数据和信号行为，QSignalSpy 可验证信号发射；把网络、时钟和文件系统依赖抽象后用 fake/mock 隔离，减少不稳定外部条件。UI 可通过 QTest::mouseClick/keyClicks 模拟交互，但截图和时序测试需稳定环境；在 CI 为关键回归自动构建并运行测试。

```
QSignalSpy spy(&model, &Model::dataChanged);
model.setData(model.index(0, 0), QStringLiteral("new"));
QCOMPARE(spy.size(), 1); // 验证信号和状态
QTest::mouseClick(widget.submitButton(), Qt::LeftButton);
QCOMPARE(widget.statusText(), QStringLiteral("Done"));
// 把网络/时钟/文件系统依赖注入 fake，测试可重复
// CI 链接 Qt6::Test 并运行 QTest suite
```

046 如何自定义消息处理函数?

答案 通过 qInstallMessageHandler(handler) 替换默认日志出口，handler 接收 QtMsgType、上下文和消息，可统一写控制台或文件并附时间、线程标识。安装后由当前进程日志调用它，函数应线程安全且避免递归调用 qDebug；保存前做好轮转、编码和脱敏，退出时安全刷新文件。

```
static QMutex outputMutex;
void handler(QtMsgType type, const QMessageLogContext &ctx, const QString &msg) {
    QMutexLocker lock(&outputMutex); // 可能由多个线程调用
    appendWithTimestamp(type, ctx.category, msg.toUtf8());
    // 不在 handler 中再次调用 qDebug，避免递归
}
auto oldHandler = qInstallMessageHandler(handler);
// 退出时刷新并恢复 oldHandler；写入做轮转和隐私脱敏
```

047 如何解决 Qt 中文乱码问题?

答案 先确认源文件为 UTF-8、编译器和终端编码正确；QString 保存 Unicode 文本，读网络 QByteArray 时按协议用 fromUtf8 等明确解码，写文件也指定编码。Qt 6 默认 UTF-8，Qt 5 的本地 8 位编码 API 不应直接照搬；检查同一文本在日志、控件和导出文件各环节的字节转换。

```
QByteArray incoming = socket->readAll(); // 网络层先得到字节
QString text = QString::fromUtf8(incoming); // 按协议编码解码
QFile file(path); if (!file.open(QIODevice::WriteOnly | QIODevice::Text)) return;
QTextStream out(&file); out.setEncoding(QStringConverter::Utf8);
out << text; // 文件编码也明确为 UTF-8
qInfo().noquote() << text; // 日志终端另检查编码设置
// QString 是 Unicode 文本；不要把本地 8 位编码当 UTF-8
```

048 如何解决 Qt 多线程中的数据竞争问题?

答案 先确定数据竞争发生在哪些共享字段，再让单一线程拥有对象或用 QMutex/QReadWriteLock 把所有访问纳入临界区；尽量以不可变副本和排队消息传递结果。QueuedConnection 只把槽排到接收者线程，并不会保护外部共享变量；避免多个线程直接调用有事件的 QObject，也不要跨线程操作 QWidget。

```
connect(worker, &Worker::resultReady, controller, &Controller::apply,
        Qt::QueuedConnection); // 消息传递优先
QMutexLocker lock(&stateMutex); // 所有访问共享状态的路径都持同锁
sharedState = nextState;
// 队列连接只安排槽执行，不保护另一个共享变量
// GUI 控件只在 GUI 线程访问；可改传不可变副本
// ThreadSanitizer/sanitizer 用于复现并定位竞争
```

049 如何解决 Qt 界面卡顿问题?

答案 用 profiler 判断卡顿来自主线程计算、同步磁盘/网络、布局还是过量重绘，再把独立耗时任务移到 worker 并异步汇报进度。大数据展示改用 Model/View、批量提交模型通知并合并刷新；setUpdatesEnabled(false) 只适合短暂批量改控件，需恢复更新且不能解决阻塞原因。

```
auto *watcher = new QFutureWatcher<Result>(this);
connect(watcher, &QFutureWatcher<Result>::finished, this, [this, watcher] {
    model->apply(watcher->result()); watcher->deleteLater();
});
watcher->setFuture(QtConcurrent::run([input] { return crunch(input); }));
// 数据批量更新用 begin/end 模型通知；合并 update 区域
// setUpdatesEnabled(false) 只包围短批量界面修改并恢复
// profile 找到主线程热点后再优化
```

050 如何解决 Qt 程序假死的问题?

答案 假死通常是 GUI 线程长时间执行计算、同步 I/O、等待锁或嵌套等待，导致事件循环不能处理输入和重绘；用采样器和线程栈确认阻塞点。改成 QThread/QtConcurrent 或异步网络流程，通过信号回传结果并支持取消；processEvents() 会引入重入，只适合谨慎的短期过渡。

```
QThreadPool::globalInstance()->start([this] {
    const Result result = blockingIoOrCompute(); // 移出 GUI 事件回调
    QMetaObject::invokeMethod(this, [this, result] {
        apply(result); // 结果回 GUI 线程
    }, Qt::QueuedConnection);
});
// 任务要有取消/超时；分析线程栈定位锁、同步 I/O 或等待环
// processEvents 会重入，不应用来掩盖长期阻塞
```

051 如何解决 Qt 程序在高分屏下显示模糊的问题?

答案 Qt 6 默认启用高 DPI 缩放，布局尺寸通常按逻辑像素处理，图片资源需提供合适倍率或 SVG；读取原始像素时用 devicePixelRatio 区分物理尺寸。避免硬编码固定像素和手工重复缩放；Qt 5 的高 DPI 属性设置有版本差异，迁移时按目标版本核对而不要照抄旧开关。多屏窗口移动后应读取当前 QScreen，而不是假设主屏幕缩放比例恒定。

```
QScreen *screen = window->screen(); // 多屏移动后重新查询
const qreal dpr = screen ? screen->devicePixelRatio() : 1.0;
const QSize physical = (logicalSize * dpr).toSize();
QImage image(physical, QImage::Format_ARGB32_Premultiplied);
image.setDevicePixelRatio(dpr); // 布局仍使用逻辑像素
// 图片资源提供合适倍率或 SVG；避免手工重复缩放
// Qt 5 高 DPI 开关随版本变化，按目标版本确认
```

052 如何设计一个 Qt 项目的架构?

答案 按职责划分界面层、业务用例层和数据/设备适配层，让 Widget/QML 负责呈现，业务逻辑尽量不依赖具体控件；Model/View 管理大批数据，信号槽连接边界事件。公共日志、配置和网络模块通过明确接口复用；用依赖注入隔离外部服务，模块规模应随实际复杂度增长而非先建空壳。模块之间通过纯 C++ 接口或信号槽依赖，降低替换界面框架的成本。

053 如何避免不必要的信号槽连接?

答案 重复 connect 默认会建立多条连接，同一信号每次发射就可能多次调用槽；固定初始化只连接一次，动态连接可用 Qt::UniqueConnection 防止重复的成员槽连接。UniqueConnection 对 lambda/functor 形式不适用，需保存 QMetaObject::Connection 并按生命周期断开；大量条目应通过 model/view 批量表达状态。

```
QMetaObject::Connection connection;
void Controller::startObserving() {
    if (connection) return; // 固定初始化只连一次
    connection = connect(source, &Source::changed, this, &Controller::refresh);
}
void Controller::stopObserving() { QObject::disconnect(connection); connection = {}; }
// UniqueConnection 适用于成员槽，不支持 lambda/functor 去重
// 动态生命周期可保存句柄并显式断开
```

054 如何配置 Qt 的静态编译?

答案 先用官方配置流程构建一套 static Qt Kit（如源码配置 -static），应用再链接该 Kit 的静态模块；CMake/qmake 配置要匹配实际安装，平台插件和图片格式等静态插件可能需显式导入。静态部署会增加二进制体积、依赖和升级成本，检查每个第三方库及 Qt 许可证义务，不能只加 CONFIG += static 就认为 Qt 已静态化。

055 智能指针在 Qt 中如何使用?

答案 QScopedPointer 管理独占作用域资源，QSharedPointer 用引用计数共享普通对象，QWeakPointer 可打破共享指针环；C++ 的 unique_ptr/shared_ptr 也可用于非 QObject 资源。QObject 通常由父子树拥有，QPointer 只观察对象是否仍存活；不要让 parent 删除和 shared_ptr 同时拥有同一个 QObject，以免双重释放。

```
auto owned = std::make_unique<PlainResource>(); // 普通 C++ 资源独占
QPointer<QObject> observer = object; // 只观察 QObject，不拥有
if (observer) observer->deleteLater(); // 由对象线程事件循环处理
QSharedPointer<Cache> shared = QSharedPointer<Cache>::create(); // 共享非 QObject 数据
QWeakPointer<Cache> weak = shared.toWeakRef(); // 可观察而不形成强引用环
// QObject 用 parent 或单一智能指针所有权，不能两者都拥有
```

第 15 章 OpenCV 与计算机视觉

15.1 图像基础与常用处理

001 Canny 边缘检测

答案 先将图像转成单通道灰度图，必要时先用高斯滤波抑制噪声，再调用 Canny(gray, edges, 50, 150)。算法计算梯度、做非极大值抑制，并用双阈值滞后连接边缘：强边缘保留，介于两阈值的弱边缘仅在连到强边缘时保留；阈值应结合图像对比度调试。

```
cv::Mat gray = cv::imread(path, cv::IMREAD_GRAYSCALE);
if (gray.empty()) return reportReadFailure();
cv::Mat smooth, edges;
cv::GaussianBlur(gray, smooth, cv::Size(5, 5), 0); // 先抑制噪声
cv::Canny(smooth, edges, lowThreshold, highThreshold);
show(edges);
// 高阈值找强边缘，低阈值参与滞后连接；按图像对比度调参
```

002 Haar 级联分类器进行人脸检测

答案 先用 CascadeClassifier::load 加载与 OpenCV 版本匹配的 XML 模型，将输入转灰度，必要时 equalizeHist，再以 detectMultiScale 检测矩形。scaleFactor 控制逐级缩小图像的比例，minNeighbors 用于合并相邻候选框；返回框可用 rectangle 绘制。Haar 速度快、无需 GPU，但侧脸、遮挡和光照变化会显著降低召回率。

```
cv::CascadeClassifier detector;
if (!detector.load(cascadeXml) || detector.empty()) return reportModelError();
cv::Mat gray, equalized;
cv::cvtColor(frame, gray, cv::COLOR_BGR2GRAY);
cv::equalizeHist(gray, equalized); // 光照允许时增强局部对比
std::vector<cv::Rect> faces;
detector.detectMultiScale(equalized, faces, 1.1, 3, 0, cv::Size(30, 30));
for (const auto &face : faces) cv::rectangle(frame, face, cv::Scalar(0, 255, 0), 2);
// scaleFactor/minNeighbors 改变速度与候选框过滤强度
```

003 Mat 和 IplImage 的区别

答案 IplImage 是旧 C API 的结构体，常需 cvLoadImage/cvReleaseImage 手动管理，字段和步长也由调用者处理；cv::Mat 是 C++ 矩阵类，析构时通过引用计数释放存储，支持 ROI 和丰富运算。Mat 的普通复制通常共享像素数据，独立副本需 clone；新代码优先用 Mat，旧接口可用 cvarrToMat 过渡。

```
IplImage *legacy = cvLoadImage(path, CV_LOAD_IMAGE_COLOR);
if (!legacy) return reportReadFailure();
cv::Mat borrowed = cv::cvarrToMat(legacy); // 仍依赖 legacy 的数据生命周期
cv::Mat owned = borrowed.clone(); // 复制像素后才可独立使用
cvReleaseImage(&legacy); // 旧 C API 必须显式释放
process(owned); // cv::Mat 析构按引用计数回收自己的缓冲区
// 新代码优先用 Mat；ROI/普通复制仍可能共享像素
```

004 OpenCV 如何读写图像并进行 BGR 灰度转换?

答案 用 imread(path, IMREAD_COLOR) 读入并检查 empty()；默认彩色顺序是 BGR，可用 cvtColor(img, gray, COLOR_BGR2GRAY) 转灰度，只有明确需要 RGB 时才转 RGB。imshow 后用 waitKey 处理窗口事件，按 q 退出；imwrite 会按扩展名选择编码器，应检查返回值，且 GUI 窗口功能在无桌面环境可能不可用。

```
cv::Mat bgr = cv::imread(path, cv::IMREAD_COLOR); // 默认 BGR
if (bgr.empty()) return reportReadFailure();
cv::Mat gray; cv::cvtColor(bgr, gray, cv::COLOR_BGR2GRAY);
cv::imshow("gray", gray);
const int key = cv::waitKey(1); // 驱动 HighGUI 窗口事件
if (key == 'q') closeWindow();
if (!cv::imwrite(outputPath, gray)) reportWriteFailure();
// 无桌面环境不要依赖 imshow；编码器由扩展名选择
```

005 OpenCV 用什么语言编写？支持哪些平台?

答案 OpenCV 核心主要以 C++ 实现，C++ 是最完整的接口，常见官方语言绑定有 Python、Java，另有受构建配置影响的其他绑定。可在 Windows、Linux、macOS、Android、iOS 等平台编译或使用预构建包；具体模块、摄像头后端、GPU 和编解码能力取决于平台依赖及构建选项。

006 SIFT、SURF 和 ORB 的区别

答案 SIFT 在尺度空间检测关键点并生成通常为 128 维浮点描述子，匹配常用 L2；SURF 用积分图等方法加速尺度不变特征，但常由 opencv_contrib 的 xfeatures2d 提供。ORB 以 FAST 检点、方向化 BRIEF 描述，输出二进制串，宜用 Hamming 距离，通常速度和内存更省；实际检测效果还受尺度、模糊和版本构建影响。

```
auto sift = cv::SIFT::create(); // 通常产生浮点描述子
auto orb = cv::ORB::create(500); // 二进制描述子
std::vector<cv::KeyPoint> ks, ko; cv::Mat ds, dorb;
sift->detectAndCompute(gray, cv::noArray(), ks, ds);
orb->detectAndCompute(gray, cv::noArray(), ko, dorb);
cv::BFMatcher l2(cv::NORM_L2), hamming(cv::NORM_HAMMING);
l2.match(ds, siftMatches); hamming.match(dorb, orbMatches);
// SURF 需对应 contrib/build 配置；算法效果还受尺度和模糊影响
```

007 cv::findContours() 的作用 & 绘制轮廓

答案 findContours 从 8 位二值图中追踪边界，输出轮廓点集和可选的父子层级；RETR_EXTERNAL 只取外轮廓，RETR_TREE 保留嵌套关系，CHAIN_APPROX_SIMPLE 会压缩直线段上的冗余点。之后可用 drawContours 绘制，也可算面积、周长或包围框。OpenCV 3.2 起通常不修改输入，但跨版本或需保留掩码时传入副本最稳妥。

```
cv::Mat work = binary.clone(); // 保留原掩码，兼顾版本差异
std::vector<std::vector<cv::Point>> contours;
std::vector<cv::Vec4i> hierarchy;
cv::findContours(work, contours, hierarchy, cv::RETR_TREE, cv::CHAIN_APPROX_SIMPLE);
for (std::size_t i = 0; i < contours.size(); ++i) {
    const double area = cv::contourArea(contours[i]);
    if (area > minArea) cv::drawContours(canvas, contours, static_cast<int>(i), cv::Scalar(0, 255, 0), 2);
}
// RETR_TREE 保留层级；外轮廓场景可用 RETR_EXTERNAL
```

008 cv::split() 与 cv::merge()

答案 split 把多通道 Mat 拆成若干单通道 Mat，例如 BGR 拆为 B、G、R；merge 则要求各输入尺寸和深度兼容，再按给定顺序组合成多通道图像。它们适合分别处理色彩平面，但会产生额外矩阵和带宽开销；若只需访问像素通道，考虑 Vec 类型或直接按行遍历，并牢记 OpenCV 彩色图通常为 BGR。

```
std::vector<cv::Mat> channels;
cv::split(bgr, channels); // B、G、R 单通道矩阵
channels[2] = cv::Mat::zeros(bgr.size(), CV_8UC1); // 仅保留 B/G
cv::Mat merged;
cv::merge(channels, merged); // 尺寸、深度必须兼容
// 额外 split/merge 会占带宽和内存
cv::Vec3b pixel = bgr.at<cv::Vec3b>(y, x); // 单点访问可直接读通道
// OpenCV 彩色顺序通常是 BGR
```

15.2 特征、变换与视觉算法

009 什么是 OpenCV？它的主要特性有哪些？

答案 OpenCV 是开源的计算机视觉与图像处理库，核心模块提供 Mat、几何变换、滤波、阈值和形态学等基础能力；常用扩展覆盖特征检测、标定、视频 I/O、目标检测及机器学习。C++/Python 接口可组合传统算法与 DNN 推理。CPU 优化较普遍，CUDA 等加速需相应构建和硬件支持，并非每个函数都能自动使用 GPU。

010 光流 (Optical Flow) & Farneback 方法

答案 光流估计两帧间像素位移；Farneback 在局部邻域以多项式近似灰度变化，再从系数变化估计稠密运动。OpenCV 用 calcOpticalFlowFarneback(prev, next, flow, ...) 输出 CV_32FC2 位移场，flow(y,x) 的两个分量是水平、垂直位移。需输入尺寸一致，快速大运动、弱纹理、遮挡和亮度突变会破坏局部匹配假设。

```
cv::Mat flow;
cv::calcOpticalFlowFarneback(previousGray, currentGray, flow,
                             0.5, 3, 15, 3, 5, 1.2, 0);
// flow 为 CV_32FC2；两个分量分别是水平/垂直位移
const cv::Point2f motion = flow.at<cv::Point2f>(y, x);
drawArrow(frame, cv::Point(x, y), cv::Point2f(x, y) + motion);
// 两帧尺寸应一致；弱纹理、遮挡和快速运动会破坏局部假设
previousGray = currentGray.clone(); // 为下一次保留独立帧
```

011 单应性矩阵 (Homography) & 计算

答案 单应矩阵 H 是 3×3 投影变换，齐次坐标满足 x′ ~ Hx，可描述同一平面在两视角中的映射，也可近似纯旋转相机的成像变化。至少需要四组不退化的对应点；实际常用 findHomography(..., RANSAC) 在多组匹配中剔除外点，再由 warpPerspective 映射。场景存在明显深度视差时，一个 H 无法同时对齐所有物体。

```
cv::Mat inliers;
cv::Mat H = cv::findHomography(srcPoints, dstPoints, cv::RANSAC, 3.0, inliers);
if (H.empty()) return reportDegenerateMatches();
cv::Mat warped;
cv::warpPerspective(source, warped, H, outputSize, cv::INTER_LINEAR,
                    cv::BORDER_CONSTANT);
// 至少四对不退化对应点；mask 标出 RANSAC 内点
// 单一 H 不适用于明显深度视差场景
```

012 图像拼接 (Image Stitching)

答案 先检测并描述两幅图的特征，以匹配距离和比率检验筛选对应点，用 RANSAC 估计单应矩阵，再将图像变换到统一画布并融合重叠区域。需先计算变换后边界并平移坐标，避免负坐标被裁掉；曝光补偿和羽化/多频带融合可减轻接缝。单一单应性适合平面场景或相机绕光心旋转，近景视差会造成重影。

```
auto orb = cv::ORB::create(1500);
orb->detectAndCompute(leftGray, cv::noArray(), leftKeys, leftDesc);
orb->detectAndCompute(rightGray, cv::noArray(), rightKeys, rightDesc);
cv::BFMatcher(cv::NORM_HAMMING).knnMatch(rightDesc, leftDesc, pairs, 2);
filterRatioMatches(pairs, good); // 比率检验减少歧义匹配
cv::Mat H = cv::findHomography(rightPoints, leftPoints, cv::RANSAC, 3.0, inliers);
if (H.empty()) return reportNoAlignment();
cv::warpPerspective(rightImage, warped, H, canvasSize); // 先计算平移画布边界
blendOverlap(leftImage, warped); // 羽化/曝光补偿缓解接缝
// 平面/纯旋转较适用；近景视差会重影
```

013 图像旋转

答案 用 getRotationMatrix2D(center, angle, scale) 生成 2×3 仿射矩阵，再以 warpAffine 采样输出；角度按 OpenCV 定义传入，正负方向应按坐标系确认。若沿用原宽高，旋转后边角可能被裁剪；要完整保留内容，应根据 |cosθ|、|sinθ| 计算新画布宽高，并相应平移矩阵中的中心坐标。

```
const cv::Point2f center(image.cols / 2.0f, image.rows / 2.0f);
cv::Mat M = cv::getRotationMatrix2D(center, angleDegrees, 1.0);
const double radians = angleDegrees * CV_PI / 180.0;
const int outW = static_cast<int>(std::ceil(std::abs(image.cols * std::cos(radians)) +
                                            std::abs(image.rows * std::sin(radians))));
const int outH = static_cast<int>(std::ceil(std::abs(image.rows * std::cos(radians)) +
                                            std::abs(image.cols * std::sin(radians))));
M.at<double>(0, 2) += outW / 2.0 - center.x; M.at<double>(1, 2) += outH / 2.0 - center.y;
cv::warpAffine(image, rotated, M, cv::Size(outW, outH)); // 平移后保留完整画布
```

014 图像金字塔 (Image Pyramid)

答案 高斯金字塔每层先低通再下采样，可用 pyrDown 构造，常用于多尺度检测和配准；拉普拉斯金字塔记录相邻高斯层之间的高频残差，形式上是 G_i 减去上采样的 G_{i+1}。它可用于多尺度融合，重建时逐层上采样并加回残差；尺寸取整和边界处理须保持一致。

```
std::vector<cv::Mat> gaussian{image};
for (int level = 0; level < levels; ++level) {
    cv::Mat next; cv::pyrDown(gaussian.back(), next); gaussian.push_back(next);
}
std::vector<cv::Mat> laplacian;
for (std::size_t i = 0; i + 1 < gaussian.size(); ++i) {
    cv::Mat up, detail; cv::pyrUp(gaussian[i + 1], up, gaussian[i].size());
    cv::subtract(gaussian[i], up, detail); laplacian.push_back(detail);
}
// 重建需用相同尺寸取整与边界规则逐层上采样后加回
```

015 图像阈值化 / 二值化

答案 threshold(gray, binary, 127, 255, THRESH_BINARY) 对每个像素独立比较：大于阈值写入 maxval 255，其余写 0；等于阈值也归入 0。输入通常是灰度图，固定阈值适用于光照均匀的场景；照明不均可用 adaptiveThreshold，未知双峰分布可先试 Otsu 自动选阈值。

```
cv::Mat binary;
const double used = cv::threshold(gray, binary, 127, 255, cv::THRESH_BINARY);
// 大于 127 的像素变 255；等于阈值及更小值变 0
cv::threshold(gray, otsu, 0, 255, cv::THRESH_BINARY | cv::THRESH_OTSU);
// Otsu 根据直方图选阈值，仍要求单通道输入
// 光照不均可改 adaptiveThreshold；标签图不要线性插值
```

016 如何使用 ORB 进行特征检测

答案 创建 ORB 后调用 detectAndCompute(gray, mask, keypoints, descriptors)，它同时检测关键点并计算方向化 BRIEF 二进制描述子；可用 BFMatcher(NORM_HAMMING) 做匹配，kNN 后再用距离比筛掉歧义点。max features、scaleFactor 和 nlevels 控制点数及尺度层级。ORB 很快且无需浮点描述子，但大尺度变化和严重模糊下不一定稳定。

```
auto orb = cv::ORB::create(1000, 1.2f, 8);
std::vector<cv::KeyPoint> keypoints; cv::Mat descriptors;
orb->detectAndCompute(gray, mask, keypoints, descriptors);
cv::BFMatcher matcher(cv::NORM_HAMMING);
matcher.knnMatch(descriptors, referenceDescriptors, pairs, 2);
for (const auto &pair : pairs)
    if (pair.size() == 2 && pair[0].distance < 0.75f * pair[1].distance) good.push_back(pair[0]);
// ORB 描述子是二进制串，匹配用 Hamming 距离
```

017 如何调整图像大小

答案 resize(src, dst, Size(w,h)) 指定像素尺寸，也可用 fx、fy 指定缩放比例；缩小时 INTER_AREA 通常能减少混叠，放大常用 INTER_LINEAR 或较锐的 INTER_CUBIC，类别标签图应使用 INTER_NEAREST。保持宽高比时按同一比例计算两边，并另行补边或裁剪；检查输入非空，视频还要正确处理每帧色彩和 stride。

```
if (source.empty()) return reportReadFailure();
cv::Mat smaller;
cv::resize(source, smaller, cv::Size(), 0.5, 0.5, cv::INTER_AREA); // 缩小
cv::Mat larger;
cv::resize(source, larger, cv::Size(), 2.0, 2.0, cv::INTER_CUBIC); // 放大
// 相同比例保持宽高比；标签/掩码使用 INTER_NEAREST
// resize 输出是 Mat，逐行访问仍须尊重 step/stride
```

018 模板匹配 (Template Matching)

答案 matchTemplate 在源图上逐位置滑动模板，产生尺寸约为 (W−w+1)×(H−h+1) 的响应图，再用 minMaxLoc 找极值：平方差类取最小，相关/归一化相关类通常取最大。需保证模板不大于搜索图并选合适方法；该方法不天然处理尺度旋转变化，对亮度变化也敏感，可先限制 ROI 或做图像金字塔搜索。

```
if (templ.cols > image.cols || templ.rows > image.rows) return;
cv::Mat response;
cv::matchTemplate(image, templ, response, cv::TM_CCOEFF_NORMED);
double minScore = 0.0, maxScore = 0.0; cv::Point minLoc, maxLoc;
cv::minMaxLoc(response, &minScore, &maxScore, &minLoc, &maxLoc);
const cv::Point best = maxLoc; // 相关度方法取最大；平方差取最小
cv::rectangle(image, best, best + templ.size(), cv::Scalar(0, 255, 0), 2);
// 尺度/旋转不天然鲁棒；可先限制 ROI 或建金字塔
```

15.3 视频分析与相机标定

019 直方图均衡化

答案 equalizeHist 对单通道 8 位灰度图重映射累计直方图，拉开集中在窄范围的亮度分布；它会同时改变噪声和局部对比度。彩色图不要独立均衡 B、G、R，否则易偏色，可转 YCrCb 或 Lab 只处理亮度通道；光照变化剧烈时 CLAHE 以局部块均衡并限制对比度，通常更自然。

```
cv::Mat equalized;
cv::equalizeHist(gray, equalized); // 仅单通道 8-bit 灰度
cv::Mat ycrcb; cv::cvtColor(bgr, ycrcb, cv::COLOR_BGR2YCrCb);
std::vector<cv::Mat> planes; cv::split(ycrcb, planes);
cv::equalizeHist(planes[0], planes[0]); // 仅均衡亮度 Y
cv::merge(planes, ycrcb);
cv::Mat result; cv::cvtColor(ycrcb, result, cv::COLOR_YCrCb2BGR);
// 彩色直接均衡 B/G/R 容易偏色；强光差可用 CLAHE
```

020 视频中颜色跟踪

答案 逐帧把 BGR 转 HSV，用 inRange 生成颜色掩码，再用开/闭运算去小噪声或填孔，按面积筛轮廓并以矩计算质心；相邻帧可用最近邻或卡尔曼滤波保持目标身份。HSV 对亮度有一定解耦，但色相受照明影响；红色跨 Hue 首尾边界时要分两段阈值后 OR 合并。

```
cv::VideoCapture capture(device);
if (!capture.isOpened()) return reportCameraFailure();
while (capture.read(frame)) {
    cv::cvtColor(frame, hsv, cv::COLOR_BGR2HSV);
    cv::inRange(hsv, lowerHsv, upperHsv, mask);
    cv::morphologyEx(mask, clean, cv::MORPH_OPEN, kernel); // 去小噪点
    cv::findContours(clean, contours, cv::RETR_EXTERNAL, cv::CHAIN_APPROX_SIMPLE);
    selectLargestAndUpdateCentroid(contours); // 面积筛选/跨帧关联
}
// 红色 Hue 跨首尾范围时分两次 inRange 后 OR 合并
```

021 透视变换 (Perspective Transform)

答案 四对点按源点与目标点一一对应传给 getPerspectiveTransform，得到 3×3 矩阵后用 warpPerspective(src, dst, H, outputSize)。点的顺序必须一致、不能三点共线导致退化；四点适合直接求解，多于四对带噪声的点应使用 findHomography 和 RANSAC。输出画布、边界填充值和插值方式需按用途指定。

```
const std::vector<cv::Point2f> sourceCorners{a, b, c, d};
const std::vector<cv::Point2f> targetCorners{p, q, r, s}; // 顺序一一对应
cv::Mat H = cv::getPerspectiveTransform(sourceCorners, targetCorners);
if (H.empty()) return reportDegenerateCorners();
cv::warpPerspective(source, output, H, outputSize, cv::INTER_LINEAR,
                    cv::BORDER_CONSTANT, cv::Scalar());
 // 点不可退化/共线；多于四对含噪点改 findHomography + RANSAC
// 输出尺寸和边界填充值按目标画布设置
```

022 高斯模糊

答案 GaussianBlur(src, dst, Size(5,5), 0) 用二维高斯核加权邻域，抑制高频噪声并为边缘检测等步骤做预平滑；核宽高通常取正奇数，sigmaX 为 0 时由核尺寸推导，sigmaY 默认沿用 sigmaX。核越大平滑越强，也越会模糊细节；边缘附近的填充由 borderType 决定。

```
cv::Mat blurred;
cv::GaussianBlur(source, blurred, cv::Size(5, 5), 0, 0, cv::BORDER_DEFAULT);
// 奇数核宽高；sigmaX 为 0 时由核尺寸推导
// sigmaY 为 0 时沿用 sigmaX
cv::Canny(blurred, edges, low, high); // 平滑降低噪声误检
// 核越大平滑越强，也会损失边缘细节；检查输入为空
```

023 cv::Mat 的拷贝构造和 clone() 有什么区别?

答案 Mat 的拷贝构造和赋值通常只复制矩阵头并增加引用计数，像素仍共享，因此修改元素会影响另一视图；ROI 也属于共享视图。clone() 分配连续的新存储并复制元素，适合异步保存、跨线程传递或需要独立改写的场景。仅重新赋值某个 Mat 头不会自动复制；多线程共享时仍需避免无保护的并发写。〔S14〕

```
cv::Mat shared = image; // 浅拷贝矩阵头，像素共享
shared.at<cv::Vec3b>(0, 0)[0] = 255; // 修改会影响 image
cv::Mat independent = image.clone(); // 分配并复制像素
independent.at<cv::Vec3b>(0, 0)[0] = 0; // 原图不变
enqueueForWorker(independent); // 异步/跨线程独立修改更安全
// 只改 Mat 头的赋值不会深拷贝；并发写仍要同步
```

024 OpenCV 的 ROI 为什么可能不是连续内存？处理时要注意什么？

答案 ROI 是父 Mat 数据的矩形视图，通常只创建新矩阵头；每行仍按父图的 step 前进，末尾还有 ROI 右侧未包含的像素，所以不连续。用 ptr<T>(row) 或 step 逐行访问，只有 isContinuous() 为真时才可把全部元素当一段线性内存遍历。clone() 可获得独立连续副本，但会产生一次完整拷贝。〔S14〕

```
cv::Mat roi = image(rect); // 通常仅新建头，仍沿用父图 stride
for (int y = 0; y < roi.rows; ++y) {
    const auto *row = roi.ptr<std::uint8_t>(y);
    processRow(row, static_cast<std::size_t>(roi.cols) * roi.elemSize());
}
if (!roi.isContinuous()) handleRowsSeparately(); // 不可当成单段内存
cv::Mat packed = roi.clone(); // 需要连续独立存储时付出一次复制
// step 描述行间距，不能只用 cols 推出下行地址
```

025 相机标定要估计哪些参数？基本流程是什么？

答案 用棋盘格或 Charuco 等已知几何图案拍摄多个位置、倾角和距离的图像，检测角点后以 calibrateCamera 联合估计内参矩阵 fx、fy、cx、cy、畸变系数，以及每张图相对标定板的旋转和平移。观察重投影误差并剔除模糊/重复视角；之后可用 undistort 去畸变。图案实际尺寸影响尺度换算，但内参质量更依赖覆盖全画面和姿态多样性。〔S14〕

```
// 每个视角保存同一标定板的已知点与检测角点
std::vector<std::vector<cv::Point3f>> objectPoints;
std::vector<std::vector<cv::Point2f>> imagePoints;
cv::Mat cameraMatrix, distortion; std::vector<cv::Mat> rvecs, tvecs;
const double rms = cv::calibrateCamera(objectPoints, imagePoints, imageSize,
    cameraMatrix, distortion, rvecs, tvecs);
if (!std::isfinite(rms)) return reportCalibrationFailure();
cv::undistort(testImage, corrected, cameraMatrix, distortion);
// 多姿态覆盖全画面；模糊/重复视角剔除
```

第 16 章 音视频与 FFmpeg

16.1 容器、编码与流媒体协议

001 AAC 编码的 ADTS 和 ADIF 格式有什么区别?

答案 ADTS 通常在每个 AAC access unit 前附带同步头、采样率索引和帧长等信息，接收端可从流中逐帧找边界；ADIF 把头信息集中放在文件开头，适合已知完整文件而不利于从中间随机同步。两者是 AAC 的传输封装形式，不能据此判断编码质量，也不能简单互换扩展名。

002 AVFormatContext、AVCodecContext、AVStream 的作用分别是什么?

答案 AVFormatContext 代表输入或输出容器，维护文件 I/O、时长和流列表；AVStream 描述其中一条音视频流及其时间基和 AVCodecParameters。AVCodecContext 是具体解码器/编码器的运行配置与状态，打开 codec 后参与处理。解码时通常从 codecpar 复制参数到 context，不能把 Stream 本身当作解码器。

```
AVFormatContext *format = nullptr;
if (avformat_open_input(&format, path, nullptr, nullptr) < 0) return;
if (avformat_find_stream_info(format, nullptr) < 0) {
    avformat_close_input(&format); return reportProbeError();
}
int index = av_find_best_stream(format, AVMEDIA_TYPE_VIDEO, -1, -1, nullptr, 0);
if (index >= 0) {
    AVStream *stream = format->streams[index]; // 容器中的轨道/时间基
    AVCodecContext *codec = avcodec_alloc_context3(decoder);
    avcodec_parameters_to_context(codec, stream->codecpar); // 配置解码状态
    if (avcodec_open2(codec, decoder, nullptr) >= 0) decode(codec);
    avcodec_free_context(&codec);
}
avformat_close_input(&format); // 成功打开后的所有出口都关闭
```

003 AVPacket 和 AVFrame 有什么区别？分别存储什么数据?

答案 AVPacket 通常保存压缩码流引用、所属流索引和 PTS/DTS 等时间信息，供解码器接收或由编码器产生；AVFrame 保存解码后的图像平面或音频样本，编码前也把它作为输入。一个 packet 不保证恰好对应一个可显示 frame，解码还可能延迟。用 av_packet_unref/av_frame_unref 复用释放引用，避免泄漏。

```
AVPacket *packet = av_packet_alloc(); // 压缩码流及 stream_index/时间戳
AVFrame *frame = av_frame_alloc(); // 解码后的平面图像或音频样本
while (av_read_frame(format, packet) >= 0) {
    if (packet->stream_index == videoIndex) sendAndReceive(decoder, packet, frame);
    av_packet_unref(packet); // 释放 packet 持有的数据引用
}
av_frame_unref(frame); // 可复用 frame 时释放本轮缓冲引用
av_packet_free(&packet); av_frame_free(&frame); // 释放对象本身
// 一个 packet 不保证只产出一帧；decoder 可能缓存延迟帧
```

004 FFmpeg 的主要组件有哪些？各自的作用是什么?

答案 libavformat 负责读写容器和解复用/复用，libavcodec 提供编解码器，libavutil 提供时间、内存、日志等基础设施；libswscale 做图像缩放和像素格式转换，libswresample 做音频重采样。另有 libavfilter 处理滤镜图、libavdevice 接入采集设备。构建时可裁剪组件，程序应检查实际编译启用的格式和编码器。

005 FLV 文件格式的特点是什么？FLV Tag 的类型有哪些？

答案 FLV 文件由标记 FLV 的文件头和一系列 Tag 组成，每个 Tag 带类型、数据长度、时间戳及流 ID，后接 PreviousTagSize 便于定位。常见 Tag 是音频、视频和脚本数据，脚本数据常携带 onMetaData。Tag 内部的音视频编码格式仍由 codec 标识，不能仅凭 FLV 容器名判断实际编码。

006 H.264 和 H.265 的主要区别是什么？压缩效率和编码复杂度如何对比？

答案 H.265/HEVC 使用更灵活的编码树块、预测和变换结构，相比 H.264 常能以较低码率达到相近主观质量；收益随内容、分辨率、编码器和参数而变，不能保证固定节省比例。HEVC 编码通常更耗计算，解码硬件与终端兼容面也须确认；若播放端不支持，较高压缩率没有实际收益。

007 H.264 的 Profile 和 Level 分别代表什么含义？

答案 Profile 规定可使用的编码工具与能力子集，例如 Baseline、Main、High；Level 限定最大处理能力，如图像尺寸、宏块率、码率和解码缓冲要求。设备要同时满足 profile、level 和实际像素格式、分辨率、帧率组合。标记高 level 并不会提升画质，只表示码流可能需要更强解码能力。

```
AVCodecContext *encoder = avcodec_alloc_context3(codec);
encoder->width = width; encoder->height = height;
encoder->pix_fmt = AV_PIX_FMT_YUV420P; encoder->time_base = AVRational{1, 30};
encoder->profile = FF_PROFILE_H264_HIGH; // 工具集子集
encoder->level = 40; // 约束能力上限，数值按 codec 定义
if (avcodec_open2(encoder, codec, nullptr) < 0) reportUnsupportedConfig();
avcodec_free_context(&encoder);
// 还要验证目标设备能解该 profile/level/尺寸/帧率组合
```

008 HLS 协议的原理是什么？M3U8 文件的作用是什么？

答案 HLS 将媒体切为若干分片，由 M3U8 播放列表给出分片 URI、时长、序号及可选的多码率变体；客户端按列表缓冲播放，并可依带宽切换变体。分片常用 MPEG-TS 或 fMP4，直播列表会周期更新。分片时长、关键帧边界、缓存策略共同决定切换和端到端延迟。

009 IDR 帧和 I 帧有什么区别？

答案 I 帧的图像内容不依赖其他帧，可独立解码；H.264 的 IDR 是一种特殊的 I 图像，会标记参考图像状态刷新，使其后的图像不再引用 IDR 前的参考帧。普通 I 帧可能出现在开放 GOP 中，后续帧仍可能依赖之前图像，因此 seek、分段和错误恢复时应识别真正的随机访问点。

010 MP4 文件的基本结构是怎样的？什么是 moov、mdat、ftyp？

答案 MP4/ISO BMFF 由层级化 box 组成：ftyp 描述品牌及兼容标识，mdat 存放实际媒体样本，moov 保存轨道、时序、编解码配置和样本索引。moov 若在文件尾，播放端可能先需下载较多数据才能开始；faststart 重排可把 moov 移到前部，但大文件重排会涉及文件读写。

011 RTMP 协议的特点是什么？适用于什么场景？

答案 RTMP 常用于采集端向服务器推送直播流，基于 TCP，默认常用端口 1935，并以握手、chunk 和多路复用消息传输音视频；TCP 保证字节流有序，但重传和队头阻塞会增加时延，延迟取决于缓冲配置与网络。RTMPS 通常在 RTMP 上使用 TLS；浏览器播放通常会转成 HLS/DASH 或 WebRTC。

012 RTP 和 RTCP 分别是什么？它们在流媒体传输中的作用是什么？

答案 RTP 在实时媒体包中带序列号、采样时钟时间戳和负载类型，帮助接收端重排、检测丢包并按媒体时钟播放；它通常运行于 UDP，不保证送达。RTCP 配套交换发送/接收报告、丢包和抖动统计及 NTP/RTP 时钟映射，可用于音视频同步和拥塞控制反馈，但自身不负责重传保证。

013 RTSP 协议的工作流程是什么？主要包含哪些命令？

答案 客户端通常先 OPTIONS 查询能力，再 DESCRIBE 获取 SDP，之后 SETUP 协商传输方式和端口，PLAY 开始播放；可用 PAUSE 暂停，最终 TEARDOWN 结束会话。RTSP 是会话控制协议，媒体多数经 RTP/RTCP 传输，也可交织在 TCP 中；实际命令顺序会随服务器能力和认证流程变化。

014 SwrContext 的作用是什么？如何使用它进行音频重采样？

答案 SwrContext 是 libswresample 的转换状态，可在采样率、样本格式或声道布局不同的输入输出间重采样。先按输入输出参数配置并初始化，再逐帧传入平面数据及 linesize，按 swr_get_out_samples/延迟估算提供足够输出空间；转换量可能少于输入量。收尾时以空输入持续调用 swr_convert 取出滤波器尾部延迟样本。

```
SwrContext *swr = nullptr;
check(swr_alloc_set_opts2(&swr, &outLayout, AV_SAMPLE_FMT_S16, 44100,
                          &inLayout, AV_SAMPLE_FMT_FLTP, 48000, 0, nullptr));
check(swr_init(swr)); // 参数在初始化前确定
int capacity = swr_get_out_samples(swr, input->nb_samples);
allocateOutputPlanes(capacity, outLayout);
int samples = swr_convert(swr, output, capacity, input->extended_data, input->nb_samples);
writeSamples(output, samples); // 输出数与输入数可不同
int tail = swr_convert(swr, output, capacity, nullptr, 0); // 取滤波器尾部
freeOutputPlanes(); av_channel_layout_uninit(&inLayout); av_channel_layout_uninit(&outLayout); swr_free(&swr);
```

015 SwsContext 的作用是什么？如何使用它进行 YUV 到 RGB 的转换？

答案 SwsContext 保存像素格式转换和缩放配置，可由 sws_getContext 创建，再用 sws_scale 按输入平面指针、行跨度和行范围转换；例如 YUV420P 转 RGB24 时必须传对 Y/U/V plane 与 linesize。还要确认 BT.601/709 等矩阵以及 full/limited range，色彩元数据不一致会导致偏色或黑位发灰。

```
SwsContext *sws = sws_getContext(srcW, srcH, AV_PIX_FMT_YUV420P,
    dstW, dstH, AV_PIX_FMT_RGB24, SWS_BILINEAR, nullptr, nullptr, nullptr);
if (!sws) return reportAllocationFailure();
allocateRgbFrame(dstW, dstH, &dstData, &dstStride); // 含 padding
sws_scale(sws, src->data, src->linesize, 0, srcH, dstData, dstStride);
// 输入 plane 指针和每行 stride 必须来自 AVFrame
const int *matrix = sws_getCoefficients(SWS_CS_ITU709);
sws_setColorspaceDetails(sws, matrix, 0, matrix, 1, 0, 1 << 16, 1 << 16);
freeRgbFrame(&dstData); sws_freeContext(sws); // 同时释放源/目标 AVFrame
```

016 UDP 和 TCP 在流媒体传输中各有什么优缺点？

答案 TCP 提供可靠、有序字节流，丢包时重传且后续数据会被队头阻塞，适合容忍缓冲的传输；UDP 开销低、可独立处理丢失和过期包，但不保证送达、顺序，也不会自动控制拥塞。实时 RTP/UDP 可结合 NACK、FEC 和抖动缓冲恢复；选择取决于时延预算、网络策略与应用层机制。

017 WebRTC 的主要特点是什么？适用于什么场景？

答案 WebRTC 面向端到端实时通信，通常支持媒体采集、回声消除、降噪、拥塞控制、加密及音视频同步。ICE 利用候选地址建立连通，STUN 辅助发现公网映射，TURN 在直连失败时中继；应用仍需自行提供信令交换 SDP/候选信息。典型场景是通话和互动直播，网络穿透与服务部署会影响成本。

018 YUV420P、YUV420SP（NV12、NV21）的数据排列方式有何不同？

答案 YUV420P/I420 有独立的 Y、U、V 三个平面，U/V 分辨率各约为宽高一半；NV12 是 Y 平面加交错 UV 色度平面，NV21 则交错 VU。它们在相同采样下色度信息量接近，但内存布局不同，不能当作相同格式直接传给解码器或纹理；各平面的 stride 也可能有对齐填充。

```
// planar YUV420P: Y、U、V 独立 plane
const std::uint8_t *y = frame->data[0], *u = frame->data[1], *v = frame->data[2];
const int yStride = frame->linesize[0], uStride = frame->linesize[1];
samplePlanar(y, u, v, yStride, uStride);
// semiplanar NV12: data[1] 内 UV 交错; NV21 为 VU
const std::uint8_t *uv = nv12->data[1];
sampleInterleavedUv(uv, nv12->linesize[1]);
// 每行用 stride 前进; plane 次序不符会出现明显偏色
```

019 YUV420、YUV422、YUV444 的区别是什么？

答案 4:4:4 中每个亮度样本对应一个 U、V 样本；4:2:2 每两个水平方向亮度样本共享一组色度；4:2:0 再在垂直方向也约减半。抽样减少色度带宽和存储，通常对人眼影响较小，但会损失彩色边缘细节。采样比例说明分辨率关系，平面或交错排列则由具体 pixel format 决定。

```
const std::size_t y = static_cast<std::size_t>(width) * height;
const std::size_t cw = (width + 1) / 2, ch = (height + 1) / 2;
const std::size_t bytes420 = y + 2 * cw * ch; // U/V 向上取整
const std::size_t bytes422 = y + 2 * cw * height;
const std::size_t bytes444 = 3 * y;
// 采样越稀，色度边缘细节越少; 实际 frame 还有 stride padding
report(bytes420, bytes422, bytes444);
```

16.2 FFmpeg 解复用、解码与时间戳

020 av_read_frame()读取的是什么数据?

答案 av_read_frame(fmt, pkt) 从解复用器读取下一个压缩 packet，成功返回 0，EOF 返回 AVERROR_EOF，其他负值表示错误；packet 的 stream_index 区分音频、视频或字幕，时间戳以对应 AVStream 的 time_base 表示。它不负责解码，数据也未必与单个画面一一对应。处理后调用 av_packet_unref 释放其引用。

```
AVPacket *packet = av_packet_alloc();
if (!packet) return reportAllocationFailure();
while ((ret = av_read_frame(format, packet)) >= 0) {
    const AVStream *stream = format->streams[packet->stream_index];
    if (packet->stream_index == videoIndex) decode(packet); // 仍是压缩 packet
    inspect(packet->pts, packet->dts, stream->time_base);
    av_packet_unref(packet); // 每次读完都归还 payload 引用
}
if (ret != AVERROR_EOF) report(ret); // 区分读错误与正常结束
av_packet_free(&packet); // 再释放 packet 对象
```

021 av_rescale_q()函数的作用是什么？如何进行时间戳转换?

答案 av_rescale_q(a, bq, cq) 计算与 a×bq/cq 等价的有理数缩放，并做舍入与溢出保护，把以 bq 为单位的时间戳转成 cq 单位。例如流时间基 1/90000 转到编码器 1/25，应以 rescale_q 将输入 PTS 换成编码器刻度。传参时先确认源、目标 time_base 次序，不能把分子分母倒置。

```
if (pts != AV_NOPTS_VALUE) {
    const AVRational streamTb = stream->time_base; // 源时间刻度
    const AVRational encoderTb{1, 25}; // 目标每帧 1/25 秒
    const std::int64_t encoderPts = av_rescale_q(pts, streamTb, encoderTb);
    submitTimestamp(encoderPts); // 先确认 bq/cq 次序
}
// 不要直接把 PTS 当毫秒；有理数缩放还可避免乘法溢出
```

022 avcodec_send_packet()和 avcodec_receive_frame()的作用是什么?

答案 send_packet 把压缩包送入解码器，receive_frame 应循环取出当前所有可用帧；send 返回 EAGAIN 时先 receive 至 EAGAIN，再重送同一 packet；receive 返回 EAGAIN 则继续供包。输入 EOF 后发送一次 NULL flush，再持续 receive 到 AVERROR_EOF，取完延迟帧；EAGAIN 是状态切换信号，不是解码错误。

```
// receiveAvailable 循环 receive_frame：处理每帧并 unref，直到 EAGAIN/EOF
int ret = avcodec_send_packet(decoder, packet);
if (ret == AVERROR(EAGAIN)) { receiveAvailable(decoder, frame); ret = avcodec_send_packet(decoder, packet); }
if (ret < 0) return reportDecodeError(ret);
receiveAvailable(decoder, frame); // receive 到 EAGAIN 后再送下一包
// demux EOF 后发送空包进入 draining mode
while ((ret = avcodec_send_packet(decoder, nullptr)) == AVERROR(EAGAIN)) receiveAvailable(decoder, frame);
if (ret >= 0) receiveUntil(decoder, frame, AVERROR_EOF);
// 最终 av_packet_free、av_frame_free、avcodec_free_context
```

023 time_base 是什么？如何理解时间戳和 time_base 的关系?

答案 time_base 是有理数形式的时钟刻度，例如 1/90000 表示一个 timestamp 单位为 1/90000 秒；实际秒数为 timestamp×time_base。不同流、解码器和编码器可使用不同刻度，必须通过 av_rescale_q 转换。帧率不是时间基的同义词，B 帧或可变帧率视频的展示时刻应以 PTS 为准。

```
const AVRational tb = stream->time_base; // 每个 timestamp 的秒数
if (frame->pts != AV_NOPTS_VALUE) {
    const double seconds = frame->pts * av_q2d(tb);
    showAt(seconds); // 展示时钟使用 PTS
}
const AVRational ms{1, 1000};
const std::int64_t timestampMs = av_rescale_q(frame->pts, tb, ms);
// time_base 不是帧率；不同流刻度不可直接比较
```

024 什么是 I 帧、P 帧、B 帧？它们在视频编码中的作用和压缩率有何不同?

答案 I 帧不依赖其他图像可独立重建，P 帧主要引用先前参考图像，B 帧可依据前后参考图像作双向预测；现代标准还可使用多参考等结构。帧间预测通常省码率，却让显示顺序和解码顺序不同，并带来重排延迟。随机访问通常从 IDR/其他可恢复点进入，再解码后续依赖帧。

025 什么是 NAL Unit？H.264 码流的基本结构是怎样的？

答案 NAL Unit 是 H.264 基本码流单元，由 NAL 头和负载构成；负载可为切片、SPS/PPS 参数集、SEI 等。裸流常以 Annex B 的 00 00 01/00 00 00 01 起始码分隔，MP4 等容器通常用 AVCC 长度前缀，并将参数集另存配置记录。送入不同协议前应按需做 bitstream filter 转换。

```
const AVBitStreamFilter *filter = av_bsf_get_by_name("h264_mp4toannexb");
AVBSFContext *bsf = nullptr; check(av_bsf_alloc(filter, &bsf));
avcodec_parameters_copy(bsf->par_in, stream->codecpar); bsf->time_base_in = stream->time_base;
check(av_bsf_init(bsf)); // MP4 length-prefix 转 Annex B 起始码
check(av_bsf_send_packet(bsf, packet)); // 成功后 packet 内容所有权转移
while (av_bsf_receive_packet(bsf, filtered) == 0) writeAnnexB(filtered);
av_packet_free(&packet); av_packet_free(&filtered); av_bsf_free(&bsf);
// 参数集和 NAL 格式按目标协议/容器选择
```

026 什么是 PCM（脉冲编码调制）？采样率、位深度、声道数分别代表什么？

答案 PCM 直接表示采样后的离散幅值，未做感知有损压缩。采样率是每秒采样次数，位深描述每个样本的量化位数，声道数表示独立信号路数；未压缩数据率可估为采样率×位深×声道数，单位 bit/s。实际还要考虑 packed/planar 布局、对齐及容器头，不能仅靠文件扩展名推断参数。

```
const std::uint64_t bitsPerSecond =
    static_cast<std::uint64_t>(sampleRate) * bitsPerSample * channels;
const std::size_t bytesPerSecond = static_cast<std::size_t>((bitsPerSecond + 7) / 8);
const std::size_t blockBytes = bytesPerSecond * blockMilliseconds / 1000;
readPcmBlock(buffer, blockBytes);
playSamples(buffer, sampleRate, sampleFormat, channels);
// PCM 无自描述头：调用方须知道位深/浮点格式与 packed/planar 布局
// 容器头、对齐与设备缓冲还会增加字节数
```

027 什么是 PTS 和 DTS？为什么需要两个时间戳？

答案 DTS 表示压缩帧应送去解码的时间，PTS 表示解码后应呈现的时间；没有帧重排时二者常接近，有 B 帧时解码顺序与显示顺序不同，二者便不同。时间戳单位由流的 time_base 决定，可能缺失为 AV_NOPTS_VALUE；音视频同步应把有效 PTS 转成共同时钟再比较。

```
if (packet->dts != AV_NOPTS_VALUE)
    decoderOrderClock = av_rescale_q(packet->dts, stream->time_base, commonTb);
if (frame->pts != AV_NOPTS_VALUE)
    displayClock = av_rescale_q(frame->pts, stream->time_base, commonTb);
// 有 B 帧时解码次序和显示次序不同，DTS/PTS 不同
if (frame->pts == AV_NOPTS_VALUE) useFallbackTimestamp();
// 同步音视频前先把有效时间戳换到共同时钟
```

16.3 图像、音频与视频编码原理

028 什么是 SIMD？如何使用 SIMD 优化视频处理？

答案 SIMD 让一条向量指令同时处理多个像素或样本，例如并行做颜色转换、阈值和卷积。先用连续数据、减少分支并检查编译器自动向量化；热点明确后再考虑 SSE/AVX/NEON intrinsics 或已有优化库。需处理尾部不足一个向量的元素和对齐，且内存带宽、缓存与数据搬运常比算术指令更先成为瓶颈。

029 什么是 YUV 颜色空间？YUV 和 RGB 有什么区别？

答案 RGB 用三原色通道直接描述显示颜色；视频常用 Y′CbCr，将伽马校正后的亮度分量与色度差分开，便于降低色度采样率。工程接口常把 Y′CbCr 俗称 YUV，但模拟 YUV 与数字 YCbCr 概念不同。转换必须匹配 BT.601/709/2020 矩阵和全/限幅范围，否则会出现色偏、黑白位不正确。

```
SwsContext *convert = makeYuvToRgbContext(frame->width, frame->height);
const int *coeff = sws_getCoefficients(SWS_CS_ITU709); // 色彩矩阵
sws_setColorspaceDetails(convert, coeff, sourceIsFullRange,
                         coeff, 1, 0, 1 << 16, 1 << 16);
sws_scale(convert, frame->data, frame->linesize, 0, frame->height, rgb, stride);
showRgb(rgb); // 亮度/色度通过矩阵组合到 RGB
sws_freeContext(convert); // 同时检查 source range/full vs limited metadata
```

030 什么是丢包（Packet Loss）？如何处理丢包问题？

答案 丢包指传输中的媒体包未到达或未能按时使用，可从 RTP 序列号缺口、RTCP 统计和解码错误观测。可按剩余播放时限尝试 NACK 重传、前向纠错、PLC/帧隐藏或丢弃受损参考链并请求关键帧；实时流不能盲目无限等待。网络拥塞应结合降码率和有界缓冲处理，避免重传加剧排队。

031 什么是关键帧间隔？如何设置合适的关键帧间隔？

答案 关键帧间隔表示随机访问点之间的帧数或时间，常与 GOP 长度相关。间隔短可缩短 seek/直播切入等待并加快错误恢复，但关键帧较大，可能抬高码率；间隔长通常压缩更有效，却让客户端等待更久。直播分片常要求关键帧与分片边界对齐，帧率可变时应按时间而非只按帧数校准。

```
encoder->time_base = AVRational{1, fps};
encoder->framerate = AVRational{fps, 1};
encoder->gop_size = fps * targetKeyframeSeconds; // 近似按秒设随机访问间隔
encoder->max_b_frames = 0; // 低延迟场景可减少重排
if (avcodec_open2(encoder, codec, nullptr) < 0) reportUnsupportedSettings();
// 分片直播常要求关键帧与切片边界对齐
// 短 GOP 提高恢复频率，也可能增加码率
```

032 什么是分辨率（Resolution）？常见的视频分辨率有哪些？

答案 分辨率是编码帧的样本宽高，如 1920×1080；p 常指逐行扫描标识，而不只是清晰度等级。480p、720p、1080p、UHD 4K 3840×2160 是常见档位，但 SD/2K/4K 命名存在地区和影院标准差异。像素尺寸不等于显示宽高比，非方形像素素材还需结合 SAR/PAR。

```
encoder->width = 1920; encoder->height = 1080; // 编码样本宽高
encoder->sample_aspect_ratio = AVRational{1, 1}; // 方形像素
stream->sample_aspect_ratio = encoder->sample_aspect_ratio;
stream->avg_frame_rate = AVRational{30, 1};
const AVRational dar = av_mul_q(
    av_mul_q(AVRational{encoder->width, encoder->height}, encoder->sample_aspect_ratio),
    AVRational{1, 1});
logDisplayRatio(av_q2d(dar)); // 还需结合目标显示像素比例
// p 通常描述逐行扫描；分辨率不等于显示宽高比
```

033 什么是去块效应滤波（Deblocking Filter）？为什么需要它？

答案 去块滤波在重建块边界附近按运动、量化强度和边缘特征自适应平滑，减少低码率下明显的方块接缝。它通常嵌入编码标准解码流程，滤波后的重建帧还可能用于后续预测参考；关闭会改变解码画面并影响码流兼容预期。过度平滑也会损失细节，因此不是简单对全图做模糊。

034 什么是宏块（Macroblock）？H.264 中宏块的大小是多少？

答案 H.264 的宏块亮度基本尺寸为 16×16，色度块大小由 4:2:0、4:2:2 等抽样格式决定；宏块可分成更小分区用于运动预测，也可细分变换块。它是编码结构单位，不表示一帧被划成固定不变的方格。H.265 采用更大的 CTU 及递归编码树划分，结构更灵活。

035 什么是容器格式？容器格式和编码格式有什么关系？

答案 容器是媒体文件的组织格式，例如 MP4、MKV，负责将轨道、时间戳、索引、元数据和压缩数据封装在一起；编码格式如 H.264、AV1、AAC 决定媒体如何压缩。一个容器可以装多种编码，但容器支持不代表目标播放器支持其中每种 codec、profile 或像素格式。

036 什么是宽高比（Aspect Ratio）？像素宽高比（PAR）和显示宽高比（DAR）有什么关系？

答案 PAR 是单个像素的显示宽高比；SAR 在视频术语中常指样本宽高比，即帧的像素宽高比 W/H；DAR 是最终画面的显示宽高比，关系为 DAR = SAR×PAR。术语在不同文档中 SAR 也可能被写作 sample aspect ratio，需结合上下文。若忽略非方形像素，画面会被拉伸或压扁。

```
const AVRational sar = stream->sample_aspect_ratio.num
    ? stream->sample_aspect_ratio : AVRational{1, 1};
const AVRational display = av_mul_q(
    AVRational{stream->codecpar->width, stream->codecpar->height}, sar);
const double dar = av_q2d(display); // sample aspect × pixel aspect
setDisplayViewport(dar);
// 元数据缺失时可用 1:1 假设；术语 SAR/PAR 在文档间可能混用
// 不能只凭像素矩形推断最终显示比例
```

037 什么是封装器（Muxer）和解封装器（Demuxer）?

答案 Muxer 把编码后的 packet、时间戳和轨道元数据写入 MP4 等容器，并建立索引；demuxer 解析容器，输出各流压缩 packet 和时间信息。二者通常不做像素级解码或重编码，因而可用于无损转封装。更换不兼容编码时仍需解码、必要滤镜处理并重新编码。

```
// Demux：av_read_frame 取压缩 packet，不解码像素
while (av_read_frame(input, packet) >= 0) {
    const int i = packet->stream_index;
    packet->stream_index = mapInputToOutput(i);
    av_packet_rescale_ts(packet, input->streams[i]->time_base,
                         output->streams[packet->stream_index]->time_base);
    av_interleaved_write_frame(output, packet); // Mux 到新容器
}
av_write_trailer(output); // 写索引/尾部元数据
// stream copy 适用于目标容器支持 codec；按顺序关闭 AVIO/context
```

038 什么是帧内预测和帧间预测？分别采用什么技术实现?

答案 帧内预测只利用当前帧已重建的邻近像素，按方向或平面模式预测当前块；帧间预测从一幅或多幅参考帧中用运动矢量取样，再做运动补偿。编码器对原块和预测块之差形成残差，经过变换、量化和熵编码；解码器按相同步骤恢复，因此参考帧必须先正确重建。 帧内模式在切换场景或随机访问时有用，但一般比有效帧间预测消耗更多码率。

039 什么是帧率（Framerate）？常见的帧率有哪些?

答案 帧率是单位时间内显示或采集的帧数，常见电影 24、视频 25/30、运动内容 50/60 fps。固定帧率可由相邻帧 PTS 间隔推得；可变帧率应读取每帧时间戳，不能只拿标称平均帧率计算。提高帧率会增加处理、传输和存储压力，也要求快门与显示设备配合。

```
encoder->time_base = AVRational{1, 30}; // 每帧时钟单位
for (std::int64_t index = 0; index < frameCount; ++index) {
    frame->pts = index; // 固定帧率按编码器时间基递增
    int ret = avcodec_send_frame(encoder, frame);
    if (ret == AVERROR(EAGAIN)) {
        receivePacketsUntilEagain(encoder, packet);
        ret = avcodec_send_frame(encoder, frame); // 重送尚未接收的帧
    }
    check(ret); receivePacketsUntilEagain(encoder, packet);
}
sendNullFrameRetryingEagain(encoder, packet); // 输入结束后发送空帧
receivePacketsUntil(encoder, packet, AVERROR_EOF); // 取完延迟输出
// 可变帧率按源 PTS 换算；最后释放 packet/frame/context
```

040 什么是抖动（Jitter）？如何处理网络抖动?

答案 网络抖动是包到达时延或间隔的变化，可由 RTP 到达间隔统计并通过 RTCP 报告。接收端 jitter buffer 先缓存并按 RTP 时间戳平滑输出，太小会欠载、太大则增加端到端延迟；可自适应调整目标缓冲并丢弃已过期包。它只能吸收有限波动，无法弥补长期带宽不足或严重丢包。

041 什么是推流和拉流?

答案 推流是采集或生产端编码后向媒体服务器发送数据，拉流是客户端从服务器获取流并缓冲、解码和播放。RTMP 常用于推送，HLS 常用于大规模分发，WebRTC/SRT 可覆盖不同互动与传输需求；协议决定连接、重传和分片方式，最终延迟还受编码、GOP、服务器转发和播放器缓冲影响。

```
AVFormatContext *input = nullptr;
if (avformat_open_input(&input, pullUrl, nullptr, nullptr) < 0) return reportConnectError();
check(avformat_find_stream_info(input, nullptr)); // 拉流后探测轨道
AVFormatContext *output = createOutput(pushUrl); // 推流输出上下文
copyStreamParameters(input, output); // 协议/容器支持对应 codec 时可直通
writeHeader(output); pumpPackets(input, output); // 仍需有界缓存
writeTrailer(output); closeOutput(output);
avformat_close_input(&input); // 端到端延迟还取决于 GOP/播放缓冲
```

042 什么是比特流（Bitstream）?

答案 比特流是编码器输出的有序压缩数据及解码所需参数信息；例如 H.264 由 NAL 单元组成，包含图像切片和 SPS/PPS。裸 Annex B 数据以起始码分隔，AVCC 等形式使用长度前缀，MP4 则把样本放在容器结构中。处理时需区分 elementary stream 与容器 packet，避免把两种边界格式混用。

043 什么是码率（Bitrate）？码率和视频质量有什么关系？

答案 码率是单位时间产生或传输的比特数，常用 kbit/s 或 Mbit/s；平均码率可粗略由文件大小×8/时长估算，但容器开销会影响结果。提高码率通常降低有损压缩失真，却不保证更好画质：编码器效率、QP/质量模式、分辨率、帧率、噪声和场景复杂度都会改变同码率效果。

```
encoder->bit_rate = 2'000'000; // 目标平均码率示例
encoder->rc_max_rate = 2'500'000; // 可选峰值上限
encoder->rc_buffer_size = 1'000'000; // 缓冲预算由编码器解释
check(avcodec_open2(encoder, codec, nullptr));
// 提高码率通常减轻有损失真，但不保证质量单调
// 内容复杂度、分辨率、帧率和编码模式共同影响效果
```

044 什么是硬件编解码？和软件编解码有什么区别？

答案 硬件编解码借助 GPU 或 SoC 专用媒体单元完成压缩/解压，常降低 CPU 占用和功耗、提升并发吞吐；软件编解码易部署、算法和参数较灵活，也可在新标准上更快更新。使用硬件时要核对设备支持的 codec/profile/位深、驱动与 FFmpeg 构建，并计入硬件帧与系统内存间拷贝成本。

```
AVHWDeviceType type = AV_HWDEVICE_TYPE_NONE;
while ((type = av_hwdevice_iterate_types(type)) != AV_HWDEVICE_TYPE_NONE)
    qInfo() << av_hwdevice_get_type_name(type); // 构建包含的接口
AVBufferRef *device = nullptr;
if (av_hwdevice_ctx_create(&device, AV_HWDEVICE_TYPE_CUDA, nullptr, nullptr, 0) < 0)
    return openSoftwareDecoder(); // 设备/驱动不可用就回退
codec->hw_device_ctx = av_buffer_ref(device); // codec 持有自己的引用
av_buffer_unref(&device); // decoder 关闭时释放 codec context
// get_format 与目标硬件像素格式回调按设备配置
```

045 什么是编解码器（Codec）？

答案 Codec 是编码器/解码器及其遵循的压缩表示规则；H.264、HEVC、AV1 是视频编码标准，AAC、MP3、Opus 是音频编码。编码流通常放入 MP4、Matroska 等容器中传输和索引。名称相同也可能因 profile、level、位深、声道布局或授权支持而无法在目标设备播放。

046 什么是色度采样？为什么要进行色度采样？

答案 色度抽样减少 Cb/Cr 等色度平面的空间分辨率，以利用视觉对亮度细节更敏感的特性减少存储和带宽。4:2:0、4:2:2、4:4:4 表示相对亮度的色度样本密度。抽样会影响字幕、细线和彩色边缘清晰度；转码还须指定采样位置、矩阵与范围，不能只看像素格式名字。

047 什么是视频延迟（Latency）？如何降低延迟？

答案 端到端时延由采集、编码前缓冲、编码重排、网络排队/传输、播放器缓冲、解码和显示共同构成，应先逐段打点找主因。低时延可关闭 B 帧、缩短编码队列和 GOP、缩小抖动缓冲并丢弃过期帧；缓冲过小会更易卡顿，需在时延、画质和抗抖动间实测取舍。 可记录采集时间戳、解码完成和实际显示时刻，量化各段排队时间。

048 什么是视频滤镜？常见的视频滤镜有哪些？

答案 滤镜是解码后的帧处理节点，可缩放、裁剪、旋转、去噪、调色、叠加图像或文字；FFmpeg 用 -vf 或 -filter_complex 把节点连接成图。滤镜可能改变像素格式、尺寸、帧率或 PTS，需保证连接格式兼容，并按预期顺序处理，例如先缩放再定位 overlay。

```
AVFilterGraph *graph = avfilter_graph_alloc();
AVFilterContext *src = nullptr, *scale = nullptr, *sink = nullptr;
const AVFilter *scaleFilter = avfilter_get_by_name("scale");
check(avfilter_graph_create_filter(&src, avfilter_get_by_name("buffer"),
                                   "in", bufferArgs, nullptr, graph));
check(avfilter_graph_create_filter(&scale, scaleFilter, "scale",
                                   "1280:720", nullptr, graph));
check(avfilter_graph_create_filter(&sink, avfilter_get_by_name("buffersink"),
                                   "out", nullptr, nullptr, graph));
check(avfilter_link(src, 0, scale, 0)); check(avfilter_link(scale, 0, sink, 0));
// bufferArgs 应含 video_size、pix_fmt、time_base 与 pixel_aspect
check(avfilter_graph_config(graph, nullptr)); // 之后送帧、取处理帧
avfilter_graph_free(&graph); // 每个负值返回都应清理 graph
```

049 什么是转封装（Transmuxing）？它和转码（Transcoding）有什么区别？

答案 转封装只从容器中取出压缩 packet 再写入另一容器，常用 ffmpeg -i in.mkv -c copy out.mp4，速度快且避免重新压缩损失；但目标容器必须支持所含 codec 和参数。转码则需解码、滤镜处理并重编码，可改尺寸、格式和码率，却更耗时且有损编码可能累积失真。

```
// 转封装：只改 packet 容器，不解码重编码
while (av_read_frame(input, packet) >= 0) {
    mapStreamAndRescaleTimestamps(packet, input, output);
    check(av_interleaved_write_frame(output, packet));
    av_packet_unref(packet);
}
av_write_trailer(output); // 源 codec 必须受目标容器支持
// 转码则是 send_packet -> receive_frame -> filter -> send_frame -> receive_packet
// 改尺寸/采样格式需要转码，重编码可能引入失真
```

050 什么是运动补偿（Motion Compensation）？运动矢量在其中起什么作用？

答案 运动补偿用参考图像按位移取出预测块，运动矢量描述该块相对当前块的位置差；编码器先做运动估计寻找候选矢量，再以运动补偿生成预测，编码矢量及原图与预测的残差。解码器按同一参考帧和矢量重建预测后加上残差；矢量不是物体真实运动轨迹。 现代编码还可插值到亚像素位置，提升预测匹配精度，但增加搜索成本。

051 什么是音频帧？一帧音频包含多少个采样点？

答案 音频帧是 codec 的编码处理单位，包含多少采样点由编码标准、帧时长及配置决定。AAC-LC 常见 1024 个采样点，采样率 48 kHz 时约 21.33 ms；Opus 常用 2.5–60 ms 帧时长。PCM 没有统一 codec 帧边界，可按设备回调块组织；帧时长=样本数/采样率。

```
if (frame->sample_rate > 0 && frame->nb_samples > 0) {
    const AVRational sampleTb{1, frame->sample_rate};
    const std::int64_t durationUs =
        av_rescale_q(frame->nb_samples, sampleTb, AVRational{1, 1'000'000});
    qInfo() << "audio frame duration us" << durationUs;
}
// nb_samples / sample_rate 才是音频帧时长
// AAC/Opus 编码块大小不同；PCM 回调块不是 codec 固定帧
```

052 什么是音频重采样？在什么情况下需要进行重采样？

答案 重采样通过带限滤波和速率变换改变每秒采样数，也可连同样本格式或声道布局转换；例如将 48 kHz 立体声 float 转为 44.1 kHz 编码器要求的格式。转换会引入滤波延迟，需追踪输入样本数、输出数及缓存尾部；FFmpeg 可用 SwrContext，不能仅改采样率字段而不转换数据。

```
SwrContext *swr = nullptr; // 改采样率/格式/声道布局
check(swr_alloc_set_opts2(&swr, &stereo, AV_SAMPLE_FMT_S16, 44100,
                          &inputLayout, AV_SAMPLE_FMT_FLTP, 48000, 0, nullptr));
check(swr_init(swr)); // 不只是改 AVFrame 的 sample_rate 字段
int capacity = swr_get_out_samples(swr, input->nb_samples);
allocateOutput(capacity);
int produced = swr_convert(swr, out, capacity, input->extended_data, input->nb_samples);
consume(out, produced); // 可能有滤波延迟
int tail = swr_convert(swr, out, capacity, nullptr, 0); // flush 尾样本
freeOutput(); av_channel_layout_uninit(&stereo); av_channel_layout_uninit(&inputLayout); swr_free(&swr);
```

16.4 FFmpeg 编解码 API 流程

053 使用 FFmpeg 编码视频的基本流程是什么?

答案 先创建输出 AVFormatContext，选择流和编码器、设定尺寸/像素格式/time_base 并打开 codec，再写容器头。逐帧填 AVFrame、按时间顺序递增 PTS，调用 send_frame 后循环 receive_packet 直到 EAGAIN，换算 packet 时间戳到输出流 time_base 并 interleaved_write_frame。结束发送空帧 flush 收完 packet，写 trailer 后释放资源。

```
// drainPackets 收到 EAGAIN 为止：换时基、mux 并 unref packet
check(avcodec_open2(encoder, codec, nullptr)); check(avformat_write_header(output, nullptr));
for (std::int64_t i = 0; i < frameCount; ++i) {
    frame->pts = i; // 按 encoder time_base 递增
    int ret = avcodec_send_frame(encoder, frame);
    if (ret == AVERROR(EAGAIN)) {
        drainPackets(encoder, output, packet);
        ret = avcodec_send_frame(encoder, frame); // 重送同一帧
    }
    check(ret); drainPackets(encoder, output, packet);
}
sendNullRetryingEagain(encoder, output, packet); // EOF 发送空帧
receiveAndMuxUntil(encoder, output, packet, AVERROR_EOF); av_write_trailer(output);
// 释放 packet/frame/codec; avio_closep 后 avformat_free_context
```

054 使用 FFmpeg 解码视频的基本流程是什么?

答案 先 avformat_open_input 并 find_stream_info，选择视频流，从 codecpar 配置并打开解码器。循环 av_read_frame，只把目标流 packet 送给解码器；send_packet 后持续 receive_frame 直至 EAGAIN，处理帧后及时 unref。读到 EOF 仍要发送空 packet 并取完延迟帧，最后释放 packet、frame、codec 和 format 上下文。

```
// 已打开输入并用 codecpar 配置 decoder
while (av_read_frame(format, packet) >= 0) {
    if (packet->stream_index == videoIndex) {
        int ret = avcodec_send_packet(decoder, packet);
        if (ret == AVERROR(EAGAIN)) {
            receiveAvailable(decoder, frame);
            ret = avcodec_send_packet(decoder, packet); // 重送此包
        }
        check(ret); receiveAvailable(decoder, frame); // 到 EAGAIN
    }
    av_packet_unref(packet);
}
sendNullPacketRetryingEagain(decoder, frame); receiveUntilEof(decoder, frame); // EOF 取完延迟帧
// av_packet_free、av_frame_free、avcodec_free_context、avformat_close_input
```

055 多线程在音视频处理中如何应用?

答案 把解复用、解码、滤镜、编码和磁盘/网络 I/O 做成流水线，可让不同帧并行通过不同阶段；编解码库也可能内部使用帧/切片线程。线程间队列要有界并配背压，避免积压导致内存和延迟不断增长；正确传递引用计数帧、时间戳及 EOF/错误状态，性能应以端到端测量验证。

```
// 每个 AVCodecContext 只由所属 stage 线程调用
AVFrame *handoff = av_frame_clone(decoded); // 为消费者增加缓冲引用
if (!handoff) reportOutOfMemory();
if (!boundedQueue.tryPush(handoff)) { av_frame_free(&handoff); dropExpiredFrame(); }
// queue 有容量上限；满时背压或丢过期实时帧
// 消费者处理后 av_frame_free(&ownedFrame)，所有权在队列转移
// EOF/错误也作为显式状态发送，时间戳随 frame 传递
```

16.5 命令行处理与播放器功能

056 如何从视频中提取音频?

答案 压缩音频与目标容器兼容时可直接抽流：ffmpeg -i input.mp4 -map 0:a:0 -vn -c:a copy audio.m4a；-map 可选指定第几条音轨，-vn 排除视频。若容器不支持源 codec 或需改采样率/质量，就指定 AAC 等编码器重编码。抽取不会把压缩音频还原成 PCM。

057 如何使用 FFmpeg 命令行合并多个视频文件?

答案 无重编码拼接应使用 concat demuxer，而非泛称 concat 协议：清单写 file 'a.mp4'、file 'b.mp4'，再运行 ffmpeg -f concat -safe 0 -i list.txt -c copy out.mp4。各片段应有一致的流结构、codec、分辨率、time_base 等参数；不一致时先统一转码，或用 concat filter 解码拼接后重新编码。

058 如何使用 FFmpeg 命令行将视频转换为 GIF?

答案 可用 ffmpeg -i input.mp4 -t 5 -vf "fps=10,scale=480:-1:flags=lanczos" output.gif 限定时长、帧率与宽度；GIF 调色板最多 256 色，直接转码易出现色带。较好做法是先 palettegen 生成调色板，再 paletteuse 复用；透明度有限，复杂视频文件也容易很大。

059 如何使用 FFmpeg 命令行工具进行视频转码?

答案 例如 ffmpeg -i in.mp4 -c:v libx264 -crf 23 -c:a aac out.mp4 会将视频用 x264 质量模式编码、音频转 AAC；CRF 是编码器特定参数，数值越低通常质量越高、文件越大，不保证固定码率。需按播放设备选择 yuv420p 等像素格式、profile 和分辨率，并可用 ffprobe 检查输出。

060 如何使用 FFmpeg 命令行提取视频的第一帧?

答案 要取文件开头解码出的第一张图，可执行 ffmpeg -i input.mp4 -frames:v 1 first.jpg，输出帧会转为 JPEG 支持的像素格式。若要取指定时刻，使用 -ss；它放在 -i 前通常由 demuxer 快速跳到附近 seek point，转码时默认再解码丢弃到目标处， stream copy 则未必精确到帧。

061 如何使用 FFmpeg 命令行给视频添加水印?

答案 图片水印可通过两路输入叠加：ffmpeg -i input.mp4 -i logo.png -filter_complex "overlay=10:10" -c:a copy out.mp4。滤镜会重编码视频，因此不能同时对视频做 -c:v copy；坐标基于输出帧左上角，可用表达式贴右下角，并确认透明通道、尺寸和输出编码器。

062 如何使用 FFmpeg 命令行调整视频的分辨率?

答案 用 -vf "scale=1280:-2" 固定宽度并按原比例计算高度；-2 常用于让另一边自动取编码器常见的偶数尺寸。若直接写宽高会拉伸画面，可使用 force_original_aspect_ratio=decrease 后补边。缩放会重新编码视频，需决定 CRF/码率，并保留正确色彩矩阵、范围与 SAR。

063 如何减少内存拷贝提升性能?

答案 复用 AVFrame/AVPacket 缓冲区、传递引用计数对象、减少不必要像素格式转换和重复 clone，可先消除常见 CPU memcpy；硬件链路可在解码、滤镜、编码间传递同一设备帧。先用 profiler 找到真实复制热点；所谓零拷贝仍需明确同步、背压、设备兼容和最后一个引用释放时机。

```
AVFrame *shared = av_frame_clone(decoded); // 新 header，共享引用计数 AVBuffer
if (!shared) return reportOutOfMemory();
enqueueOwnedFrame(shared); // 队列拿到所有权，消费后 av_frame_free(&frame)
// 修改前对共享帧调用 av_frame_make_writable()，可能触发复制
if (av_frame_make_writable(shared) < 0) reportOutOfMemory();
av_frame_free(&shared); // 释放本端引用，其他持有者仍有效
// 零拷贝仍需同步、格式兼容和最后引用释放时机
```

064 如何判断一个视频文件是否包含音频流?

答案 可用 ffprobe -v error -select_streams a -show_entries stream=index -of csv=p=0 input.mp4，只选择音频流并列出索引；有一行输出表示至少有一条音频轨。程序中以返回码和解析到的 stream 数量判断，空输出可能代表无音频，也可能是探测失败或文件损坏，应区分错误。

```
AVFormatContext *format = nullptr;
if (avformat_open_input(&format, path, nullptr, nullptr) < 0) return reportProbeError();
const int probe = avformat_find_stream_info(format, nullptr);
if (probe < 0) { avformat_close_input(&format); return reportProbeError(); }
const int audio = av_find_best_stream(format, AVMEDIA_TYPE_AUDIO, -1, -1, nullptr, 0);
if (audio == AVERROR_STREAM_NOT_FOUND) reportNoAudioTrack();
else if (audio < 0) reportProbeError();
else reportAudioStream(format->streams[audio]);
avformat_close_input(&format); // 无音轨与探测失败是不同状态
```

065 如何在视频上添加水印?

答案 图片水印用 overlay，文字可用 drawtext；例如在滤镜图里指定 x/y、透明度和 enable 时间表达式。滤镜在解码帧上工作，因此视频要重新编码，音频通常可 -c:a copy 保留；文字滤镜还需字体配置，检查中文字体路径、输出分辨率变化和字幕坐标。

066 如何处理视频文件的旋转信息?

答案 先读取容器显示矩阵或旋转元数据，确认播放器是否已自动应用；若转码时再旋转一次会导致双重旋转。可将变换通过 transpose 等滤镜烘焙到像素并删除/重置旧矩阵，此时宽高可能互换，需同步更新尺寸、SAR 和定位；无损转封装时则需目标容器和播放器都支持该显示矩阵。

```
std::size_t sideSize = 0;
const std::uint8_t *matrix = av_stream_get_side_data(stream, AV_PKT_DATA_DISPLAYMATRIX, &sideSize);
if (matrix && sideSize >= 9 * sizeof(std::int32_t)) {
    const double angle = av_display_rotation_get(
        reinterpret_cast<const std::int32_t *>(matrix));
    reportRotation(angle); // 先确认播放器是否已自动应用
}
// 转码烘焙 transpose 后清理旧显示矩阵，避免双重旋转
// side data 指针属于 stream；复制前不要跨其生命周期保存
```

067 如何实现一个简单的 PCM 音频播放器?

答案 PCM 没有自描述头，播放器必须已知采样率、位深/浮点格式、声道数和交错布局；把读到的样本块送入 SDL 或系统音频 API。用有界环形缓冲区在 I/O 线程与音频回调间解耦，回调内避免阻塞和动态分配；播放进度按已消费样本数除以采样率计算。

068 如何实现一个简单的 YUV 播放器?

答案 YUV 裸流没有头部，必须预先知道像素格式、宽高、帧率、平面排列和 stride；逐帧按各平面实际行跨度读取并送入渲染器。紧密排列且偶数宽高的 YUV420P 可按 Y=WH、U=V=WH/4 估算，每帧共 1.5WH 字节；有行对齐或奇数尺寸时不能直接用该公式。

```
// 裸 YUV420P 没有头：宽高/格式/帧率由调用方给定
const int chromaW = (width + 1) / 2, chromaH = (height + 1) / 2;
const std::size_t yBytes = static_cast<std::size_t>(width) * height;
const std::size_t cBytes = static_cast<std::size_t>(chromaW) * chromaH;
readPlane(file, y, width, height, yStride); // 每行按 stride
readPlane(file, u, chromaW, chromaH, uStride);
readPlane(file, v, chromaW, chromaH, vStride);
renderer.uploadYuv420(y, u, v, width, height);
// 紧密偶数尺寸才可简单算 1.5*W*H；处理 EOF/短读
```

069 如何实现视频截图功能?

答案 命令行可用 ffmpeg -ss 00:01:23 -i input.mp4 -frames:v 1 output.jpg 取目标附近的一帧，或用 -vf fps=1 每秒抽一张。输入侧 -ss 通常快速 seek 到前一个 seek point，转码时默认准确解码并丢弃之前帧；`-c copy` 无法保证精确到目标帧。批量截图还应控制输出命名和总帧数。

```
const std::int64_t target = av_rescale_q(targetUs, AVRational{1, 1'000'000}, stream->time_base);
decoder->pkt_timebase = stream->time_base; // 解码输出时间戳刻度
check(av_seek_frame(format, videoIndex, target, AVSEEK_FLAG_BACKWARD)); // 先到前一随机访问点
avcodec_flush_buffers(decoder); // 丢弃 seek 前解码器状态
while (av_read_frame(format, packet) >= 0) {
    if (packet->stream_index == videoIndex) decodeAndCheckPts(packet, target, frame);
    av_packet_unref(packet); if (gotFrameAtOrAfterTarget) break;
}
convertFrameToRgb(frame, image); // 保存单帧
// 输入侧 seek 后还要解码到目标；失败出口释放 packet/frame/context
```

070 如何实现视频的倍速播放?

答案 视频倍速播放需把媒体时钟按播放速率推进，并将音频、视频 PTS 映射到同一播放时钟；视频可丢帧或重复帧来适配刷新率，音频可用 atempo 做保音高变速，简单重采样虽变速也会改音调。解码不足会欠载，倍速时还需限制队列并同步 seek/暂停状态。

071 如何实现视频的快进和快退?

答案 快进通常 seek 到目标时间之前的随机访问点，再继续解码并丢弃目标之前的帧，才能得到依赖链完整且位置准确的画面；仅跳过 packet 可能造成花屏。快退可周期性 seek 到更早关键帧并正向解码目标 GOP，再缓存或倒序显示帧；长 GOP 会增加等待，音频需静音、反放或重新定位。

```
const std::int64_t target = av_rescale_q(seekUs, AVRational{1, 1'000'000}, stream->time_base);
decoder->pkt_timebase = stream->time_base; // 解码输出时间戳刻度
check(av_seek_frame(format, videoIndex, target, AVSEEK_FLAG_BACKWARD)); // seek 到关键帧附近
avcodec_flush_buffers(decoder); // 清空旧参考帧
while (readVideoPacket(packet)) {
    sendAndReceiveAll(decoder, packet, frame);
    if (frame->pts != AV_NOPTS_VALUE && frame->pts >= target) display(frame);
    av_packet_unref(packet); if (targetFrameReady) break;
}
// 快退可重复 seek 到更早 GOP 并缓存帧；直接跳 packet 会破坏依赖链
// seek 后同步音频时钟；最终 free packet/frame/codec/format
```

072 如何实现视频的拼接和合并?

答案 首尾拼接参数相同的文件可用 concat demuxer 清单加 -c copy；concat 协议只是把文件字节顺序拼在一起，仅适用于支持这种物理拼接的少数容器。编码参数不同应把各输入解码后送入 concat filter，先统一画面尺寸、帧率、像素格式及音频采样率/布局，再输出并编码，否则滤镜输入会不兼容或产生时间跳变。

073 如何实现视频的旋转?

答案 直角旋转可在转码滤镜中用 transpose=clock 或 transpose=cclock，具体方向可用测试帧确认；任意角度可用 rotate=角度表达式。旋转后画布默认策略可能裁切，应按角度计算输出宽高或显式设置 ow/oh，并处理填充颜色。若只改显示矩阵而不变换像素，则需考虑播放器兼容。

074 如何实现视频的缩放?需要注意什么?

答案 使用 scale 滤镜或 libswscale 按目标尺寸缩放，保持显示宽高比时可固定一边并让另一边按比例计算，再选择插值算法。缩放前后要考虑 SAR/PAR、色度采样对偶数尺寸的要求和 BT.601/709 等色彩矩阵；更改像素格式或尺寸通常意味着重编码，需核验输出标签和实际像素一致。

075 如何实现视频的裁剪?

答案 FFmpeg crop=w:h:x:y 从每帧左上角偏移 (x,y) 截取宽高为 w×h 的区域；例如 crop=iw/2:ih/2:iw/4:ih/4 截出原图中心半宽、半高的区域。坐标以输入帧为基准，须满足裁剪框在图像内；4:2:0 等色度抽样可能要求边界对齐到偶数，且裁切后视频需重编码。

076 如何实现音视频的分离和合并?

答案 分离时可为输出加 -vn 或 -an，并在目标容器支持时用 -c copy 保留压缩流；选择多个输入中的哪条轨需显式 -map。合并独立音视频可写 ffmpeg -i v.mp4 -i a.aac -map 0:v:0 -map 1:a:0 -c copy out.mp4；若 codec、时间基或容器不兼容需转码，并确认起始时间偏移。

077 如何提取视频的关键帧?

答案 ffprobe -select_streams v:0 -show_frames -show_entries frame=key_frame,best_effort_timestamp_time 可列出第一视频流各解码帧的关键帧标志与时间。提取时还需 seek 并解码到该时刻，而非直接把压缩 packet 当图像；编码类型的 key_frame 标志不总等同于 IDR，开放 GOP 还可能有前置依赖。

```
// receive_frame 成功后处理，返回 EAGAIN 时送下一 packet
while (decodeNextFrame(decoder, format, packet, frame)) {
    if (frame->flags & AV_FRAME_FLAG_KEY) { // 帧标记不一定等于 H.264 IDR
        convertToImage(frame, image);
        saveImage(image, makeTimestampedPath(frame->pts));
    }
    av_frame_unref(frame); // 接收后复用前释放引用
}
// 真正随机访问还需 seek 到关键帧并解码依赖帧
// 结束时 free frame/packet/codec/format
```

078 如何计算一帧 YUV420P 图像占用的内存大小?

答案 紧密排列、偶数宽高、8 位 planar YUV420P 的可见数据量为 WH + 2×(W/2)(H/2)=1.5WH 字节，例如 1920×1080 为 3,110,400 字节。AVFrame 实际分配按各 plane 的 linesize×plane 高度计算，stride 对齐会增加空间；奇数宽高时色度平面向上取整，须按格式规则计算。

```
const std::size_t visibleY = static_cast<std::size_t>(width) * height;
const std::size_t chromaW = (width + 1) / 2, chromaH = (height + 1) / 2;
const std::size_t visible = visibleY + 2 * chromaW * chromaH; // Y/U/V
const std::size_t allocated =
    static_cast<std::size_t>(std::abs(frame->linesize[0])) * height
  + static_cast<std::size_t>(std::abs(frame->linesize[1])) * chromaH
  + static_cast<std::size_t>(std::abs(frame->linesize[2])) * chromaH;
// linesize 含对齐；这里只是可见容量 vs 常规 planar 分配估算
```

079 如何读取一个视频文件的基本信息（时长、分辨率、码率等)?

答案 ffprobe -v error -show_entries format=duration,bit_rate:stream=codec_type,width,height,r_frame_rate -of json input.mp4 可查看容器时长/码率和逐流尺寸、帧率等信息。分辨率只适用于视频流，bit_rate 或 duration 可能缺失、估算或属于全容器值；可追加 codec_name、sample_rate、channels、avg_frame_rate，并检查实际流而非假定只有一条。

```
AVFormatContext *format = nullptr;
if (avformat_open_input(&format, path, nullptr, nullptr) < 0) return reportOpenError();
if (avformat_find_stream_info(format, nullptr) < 0) {
    avformat_close_input(&format); return reportProbeError();
}
qInfo() << "duration us" << format->duration; // 单位 AV_TIME_BASE
for (unsigned i = 0; i < format->nb_streams; ++i) {
    const AVStream *s = format->streams[i];
    qInfo() << s->codecpar->codec_type << s->codecpar->width << s->codecpar->height
            << s->avg_frame_rate.num << s->avg_frame_rate.den;
}
avformat_close_input(&format); // duration/bitrate 可能缺失或估算
```

080 常见的硬件加速方案有哪些?

答案 常见硬件接口包括 NVIDIA NVENC/NVDEC、Intel Quick Sync、AMD AMF、Apple VideoToolbox、Linux VA-API 和 Windows DXVA/D3D11VA。它们是平台接口或厂商方案，具体可编解码标准、profile、位深、表面格式和并发数受芯片、驱动及 FFmpeg 构建影响；用 ffmpeg -encoders/-hwaccels 并实际打开设备验证。

```
AVHWDeviceType type = AV_HWDEVICE_TYPE_NONE;
while ((type = av_hwdevice_iterate_types(type)) != AV_HWDEVICE_TYPE_NONE) {
    qInfo() << av_hwdevice_get_type_name(type); // 当前构建可识别的后端
}
// 再按 codec 的 AVCodecHWConfig 选择像素格式和设备类型
// NVENC/VideoToolbox/VA-API 等能力由硬件、驱动、构建共同决定
// 实际创建设备并解码验证；失败时回退软件路径
```

16.6 格式、性能与播放质量

081 常见的视频封装格式有哪些? MP4、FLV、MKV、AVI 各有什么特点?

答案 MP4 通常兼容性好，适合点播与下载；Matroska/MKV 支持多轨、字幕和附件，封装灵活；FLV 曾广泛用于 RTMP 直播工作流；AVI 是较早容器，时序和新 codec 支持能力有限。容器本身不决定编码格式，也不保证所有播放器兼容其中的 codec、字幕类型和时间戳特性。

082 常见的音频编码格式有哪些? AAC、MP3、Opus 各有什么特点?

答案 MP3 兼容面很广，AAC 常见于移动设备和网络视频，Opus 面向低延迟交互并覆盖语音与音乐，三者都是有损编码。质量取决于编码器实现、码率、采样率、声道和输入内容，不存在各码率都成立的固定优劣排序；还要确认播放器和容器对目标 codec 的支持。

083 直播推流时如何处理网络波动?

答案 推流端持续观测发送队列、丢包、RTT 和可用带宽，通过自适应码率/分辨率控制负载，并给队列设置上限以避免延迟堆积。可按场景用重传、FEC 或丢弃过期帧；关键帧间隔影响恢复等待。TCP 重传可能造成队头阻塞，UDP 则需应用协议处理丢失，策略要符合实时性目标。

084 码率控制有哪些方式？CBR、VBR、ABR、CRF 分别适用于什么场景？

答案 CBR 让目标码率较稳定，便于受限链路和广播预算，但瞬时质量会随内容变化；VBR 按复杂度分配码率，常见于质量优先文件。ABR 通常把长时间平均码率作为目标；CRF 是部分编码器的恒定质量控制，码率随内容变化且不等于固定大小保证。模式名称和参数含义因编码器而异。

```
encoder->bit_rate = targetBps; // 目标平均速率
encoder->rc_max_rate = peakBps; encoder->rc_buffer_size = bufferBits; // CBR/VBV 类约束
av_opt_set(encoder->priv_data, "crf", "23", 0); // 某些编码器的质量模式
if (avcodec_open2(encoder, codec, nullptr) < 0) return reportOptionError();
writePacketsUntilFlush(encoder);
// CRF 不保证固定文件大小；选项名称/范围依具体编码器
// 同一模式还受分辨率、场景复杂度和编码器实现影响
```

085 视频播放出现花屏的可能原因有哪些？

答案 花屏可来自源 packet 损坏/丢失、参考帧缺失、NAL 格式与解码器不匹配、硬解驱动问题，也可能是 plane 指针、stride、像素格式或色彩范围传错。先用 ffprobe/软件解码确认码流，再对比硬件解码输出；检查解码日志、关键帧恢复点和渲染上传尺寸，可区分传输、解码和显示阶段问题。

```
if (frame->format == AV_PIX_FMT_NONE) reportMissingPixelFormat();
for (int plane = 0; plane < AV_NUM_DATA_POINTERS; ++plane) {
    if (frame->data[plane]) qInfo() << plane << frame->linesize[plane];
}
if (frame->hw_frames_ctx) {
    check(av_hwframe_transfer_data(cpuFrame, frame, 0)); // 对比软件可读画面
}
// 检查 packet 丢失、参考帧、plane/stride、range 与矩阵
// 用关键帧恢复并比较软件/硬件解码区分输入与渲染问题
```

086 视频播放卡顿的可能原因有哪些？如何优化？

答案 先分别测网络等待/解复用、解码、滤镜、渲染和音频回调耗时，观察队列深度与 PTS；卡顿也可能是时钟漂移或丢帧策略不当。按瓶颈优化：启用兼容的硬解、异步 I/O、预读和有界队列，或降低输出尺寸/滤镜成本。增加缓冲可掩盖短抖动，却会增加延迟，不能替代性能修复。

087 解释 GOP（Group of Pictures）的概念，GOP 大小对视频质量和码率有什么影响？

答案 GOP 是从随机访问图像到后续预测图像的编码结构，长度常用帧数或秒数表示；长 GOP 少用关键帧，往往提高压缩效率，但切入/seek 和丢包后恢复更慢。短 GOP 增加随机访问点并改善直播切片对齐，通常需要更多码率。开放/闭合 GOP、B 帧与 IDR 设置也会影响依赖和恢复边界。

```
encoder->gop_size = fps * gopSeconds; // 约每 gopSeconds 秒一次随机访问点
encoder->max_b_frames = 2; // 预测效率/重排延迟的取舍
encoder->keyint_min = encoder->gop_size / 2; // 如编码器支持
check(avcodec_open2(encoder, codec, nullptr));
for (AVFrame *f : frames) sendFrameAndDrainPackets(encoder, f);
// 切片边界通常要对齐 IDR/随机访问帧
// 长 GOP 省关键帧开销但切入/恢复更慢；最后 free codec context
```

088 解释熵编码的概念，CAVLC 和 CABAC 有什么区别？

答案 熵编码把量化后符号按统计概率映射为紧凑码字，本身无损；有损信息主要在量化阶段丢弃。H.264 CAVLC 依据邻近上下文做可变长编码，结构较简单；CABAC 使用上下文建模与算术编码，通常压缩更有效但解码计算较多。H.264 Baseline 不支持 CABAC，Main/High 可支持，设备 profile 必须匹配。

089 运动估计（Motion Estimation）的原理是什么？常用的运动估计算法有哪些？

答案 编码器在参考帧的搜索区域中寻找与当前块最相似的候选位置，得到整数像素或亚像素运动矢量，再编码预测残差。全搜索结果通常好但计算量大；菱形、六边形、分层金字塔等搜索用较少候选加快估计，x264 等还会做亚像素细化、多参考帧搜索及率失真选择。估计最优不等于真实物体运动。

090 量化（Quantization）在视频编码中的作用是什么？QP 值如何影响视频质量？

答案 量化把变换系数按量化步长压缩，是视频有损信息损失的主要来源之一。H.264 的 QP 通常为 0–51，QP 越高步长越大，细节保留减少；降低 QP 通常提高质量并增加码率。QP 每升 6，量化步长约翻倍，但码率变化受内容和模式控制，不保证正好减半；固定 QP 下场景码率也会变。

```
// 仅当所选编码器支持其私有参数时设置
if (av_opt_set(encoder->priv_data, "qp", "23", 0) < 0)
    reportUnsupportedQpMode();
check(avcodec_open2(encoder, codec, nullptr));
sendFrameAndDrainPackets(encoder, frame);
// H.264 QP 越高量化步长越大；较低通常保留细节也增码率
// QP 每升 6 步长约翻倍，但码率不保证减半
// CRF 与固定 QP 是不同控制模式
```

091 零拷贝技术在音视频处理中如何应用?

答案 零拷贝常通过引用计数 AVBufferRef、共享 DMA-BUF/硬件帧、内存映射或设备间表面传递，减少 CPU 将整帧 memcpy 到新缓冲区的次数。只有解码器、滤镜和编码器支持兼容格式及同一设备上下文时才能贯通；同步栅栏、所有权和生命周期仍需管理，格式转换或跨设备读回常会重新引入拷贝。

```
AVFrame *handoff = av_frame_clone(decoded); // 共享 AVBufferRef 帧数据
if (!handoff) return reportOutOfMemory();
queue.push(handoff); // 队列持有引用，消费者处理完 av_frame_free
if (needsWrite) check(av_frame_make_writable(handoff)); // 共享时可能复制
// 硬件帧贯通还要求相同设备/格式并处理同步 fence
// 跨设备读回或像素格式转换会重新产生拷贝
```

092 音频采样格式有哪些？Planar 和 Packed 格式有什么区别?

答案 采样格式规定单个样本是有符号整数还是浮点以及位宽，如 S16、S32、FLT；声道布局另行描述包含哪些声道。Packed 把同一采样时刻的各声道交错存放，Planar 则每个声道有独立 plane，例如立体声 FLTP 有两路 float 数组。读取 AVFrame 时应检查 format、ch_layout、nb_samples 和 linesize，不能按 channels 简单推断指针布局。

```
const auto format = static_cast<AVSampleFormat>(frame->format);
qInfo() << av_get_sample_fmt_name(format) << frame->nb_samples << frame->sample_rate;
if (av_sample_fmt_is_planar(format)) {
    for (int ch = 0; ch < frame->ch_layout.nb_channels; ++ch)
        consumeChannel(frame->extended_data[ch], frame->nb_samples); // 每声道 plane
} else {
    consumePacked(frame->data[0], frame->nb_samples * frame->ch_layout.nb_channels);
}
// 读取前检查 ch_layout/linesize; planar 和 packed 指针布局不同
```

第 17 章 Python

17.1 Python 对象、类型与容器

001 Python 的动态类型和强类型分别约束什么?

答案 动态类型表示名字在运行时可以绑定不同类型的对象，不要求变量具有一个固定的静态类型；强类型通常表示运算不会任意把互不兼容的类型悄悄转换为可运算类型，例如字符串与整数不能直接相加。这两个分类维度并不矛盾。对象本身仍有确定的类型，类型决定支持的协议和操作；需要转换时显式调用 int、str 等接口，并处理转换失败。Python 的数值类型之间存在按规则进行的混合运算，不能把“强类型”理解成完全没有隐式转换。〔N161、N162〕

002 Python 常见类型、对象可变性和函数传参是什么关系?

答案 Python 中常见对象类型包括 int、float、complex、bool、str、list、tuple、dict、set、frozenset、range、bytes、bytearray 和 NoneType。list、dict、set、bytearray 通常可变；数字、str、tuple、frozenset、range、bytes 通常不可变，tuple 内部仍可能引用可变对象。名字保存对对象的绑定，赋值通常不会复制对象。调用函数时，形参获得同一对象的引用；重新给形参赋值只改变局部绑定，原地修改可变对象则能被调用者观察到。+= 会优先尝试原地操作，因此列表可能保留身份并修改内容，而整数通常产生新对象；判断副作用要同时看别名关系与具体类型的协议。〔N161、N162〕

```
def change(items, number):
    items.append(1)  # 修改双方共同引用的列表
    number += 1      # 仅重新绑定局部名字
xs, n = [], 0
change(xs, n)
assert xs == [1] and n == 0
```

003 Python 中如何区分对象身份、值相等和真假判断?

答案 is 判断是否为同一个对象，== 调用类型定义的相等比较；除 None 等约定的单例外，比较业务值通常使用 ==。None 表示没有值，False、0、空容器则是不同的对象或类型，只是在条件判断中可能都为假。判断可选参数用 is None，避免把合法的 0 或空列表当成缺失。CPython 的小整数缓存和字符串驻留会让某些相等值偶然共享身份，但范围、编译方式和实现可能变化，不能依赖 is 比较数字或字符串。对象还可通过 __bool__ 或 __len__ 自定义真假。〔N161、N163〕

004 and、or 为什么不一定返回 bool?

答案 and、or 会短路并返回被选中的操作数本身：x and y 在 x 为假时返回 x，否则求值并返回 y；x or y 在 x 为真时返回 x，否则求值并返回 y。not 才总是产生布尔结果。短路可以避免不必要的访问或计算，但也意味着右侧的函数调用可能根本不发生。用 value or default 提供默认值时，0、空字符串、空容器也会被替换；若只允许 None 触发默认值，应使用显式条件表达式。〔N162、N163〕

005 type 与 isinstance 如何处理继承和抽象接口?

答案 type(obj) 返回对象的实际类型，用 type(obj) is T 判断的是精确类型，派生类不会匹配。isinstance(obj, T) 按实例关系判断，通常接受 T 的子类，也支持抽象基类注册等协议；例如 bool 是 int 的子类，因此 isinstance(True, int) 为真。如果接口只依赖某种能力，优先清楚地调用协议或检查适当的抽象基类，避免大量分支判断具体类型。需要拒绝布尔值等特殊子类时，可以明确检查精确类型或单独排除。〔N164、N161〕

006 Python 的 list、tuple、dict、set 和 deque 如何选择?

答案 list 适合可变顺序序列、按下标访问，也可用 append/pop 作为栈；tuple 适合固定位置含义的不可变组合，但其中元素仍可引用可变对象。dict 保存键值关联并保持插入顺序，set 适合成员判断与集合运算；两者的键或元素需要可哈希。collections.deque 适合两端频繁入队出队。优先队列可用 heapq 管理列表形式的最小堆：堆顶读取为 O(1)，push/pop 为 O(log n)；需要线程安全队列可考虑 queue.PriorityQueue。Python 没有内建链表或通用树/图容器，通常用自定义节点类、嵌套对象或 dict/list 邻接表表达。应按操作语义、规模和实际测量选型。〔N163、N165〕

007 CPython 的列表如何保存异构元素，常见操作复杂度是什么?

答案 CPython 的 list 以可调整容量的对象指针数组保存元素，异构来自指针可引用不同类型的对象，并不是把所有对象本体紧挨着存储。按下标读取通常 O(1)，无索引的值查找 O(n)；中间插入、删除需移动后续引用，通常 O(n)。append 添加一个对象，extend 逐个加入可迭代对象的元素，insert 在指定位置加入对象。扩容预留容量使连续尾部追加具有均摊 O(1) 成本，但单次扩容仍可能 O(n)。具体增长公式属于 CPython 版本实现，不是 Python 语言承诺。〔N166、N163〕

008 Python 容器复制、切片和重复为什么会产生共享嵌套对象?

答案 赋值只建立别名；list 切片、list.copy 和 copy.copy 通常创建新外层容器，但内部元素仍引用同一对象。copy.deepcopy 递归复制并借助备忘录处理循环和重复引用，具体行为还能由类型定制，不能理解成复制一切外部资源。[[0] * 3] * 3 重复的是同一个内层列表的引用，修改一个位置会影响多行，应用推导式逐行新建。tuple 不可变仅意味着它保存的元素引用不能被替换，引用的列表仍可修改。〔N167、N163〕

```
rows = [[0] * 3 for _ in range(3)]  # 每一行单独创建
rows[0][0] = 1
assert rows[1][0] == 0
# rows[:] 仍会共享各行；需要独立嵌套状态时再评估 deepcopy
```

009 sort 与 sorted 的修改行为及稳定排序规则是什么?

答案 list.sort 原地修改列表并返回 None，sorted 消费可迭代对象并返回新的列表，适合不改变输入的场景。两者支持 key 与 reverse，并保证稳定：比较键相等的元素维持输入中的相对顺序。多个字段可用元组作为 key，按元组的字典序比较；不同字段需要不同方向时，也可从次要键到主要键依次进行稳定排序。排序前确认键值之间可以比较，不要把来自不同业务语义的 None、数字、字符串混合交给默认比较。〔N163、N164〕

```
ordered = sorted(users, key=lambda u: (u["age"], u["name"]))
# 原 users 不变；key 只负责提取可比较的业务字段
```

010 str、bytes 和字符串拼接要注意哪些编码边界?

答案 str 表示 Unicode 文本，bytes 表示字节序列；通过 encode 把文本编码为字节，通过 decode 按约定编码还原文本。网络和文件边界需要统一编码，遇到错误应确定协议或数据来源，而不是随意忽略解码失败。字符串不可变，循环反复相加可能反复分配和复制；拼接已有片段通常用分隔符.join，流式写文件则避免先组装整个巨串。文本长度计数不必等于屏幕字符数或 UTF-8 字节数，处理截断、协议长度和用户显示时应分清单位。〔N163、N168〕

011 推导式与生成器表达式在执行时机和内存上有什么差别?

答案 列表、集合和字典推导式会立即遍历并构建结果，适合结果要复用或确实需要随机访问的场景；生成器表达式按需产生元素，通常只保留迭代状态，适合流水线和大数据输入。惰性不会自动减少计算总量，也不能保证整条处理链都不缓存。生成器表达式的最左侧可迭代表达式会立即求值，后续迭代与条件通常在消费时执行，因此异常和外部状态变化的观察时机也不同。可读性优先，复杂嵌套或副作用通常改成普通循环更清楚。〔N162、N163〕

012 CPython 字典如何查找、处理冲突并扩容?

答案 CPython 的 dict 使用哈希表索引键值条目，先计算哈希，再定位槽位并按探测序列处理冲突，必要时比较键是否相等。删除会留下与未用槽不同的标记，避免破坏后续探测链；表容量和可用槽不足时会分配并重建索引。查找、插入通常具有平均 O(1) 成本，冲突、重建或恶意输入可能增加开销。现代实现把索引与条目存储分开，并存在实例属性等专门布局。负载阈值、增长策略和自由线程构建细节属于版本实现，不能直接套用所有哈希表的规则。〔N169、N163〕

013 dict 保持的是插入顺序还是键的排序?

答案 Python 3.7 起，字典保持插入顺序是语言保证；CPython 3.6 已有这一实现行为。它并不按键大小排序，更新已有键的值通常不会改变位置，删除后重新插入则会移到末尾。需要按键或值排序时，应显式使用 sorted 生成所需的迭代顺序。字典相等比较主要按键值映射判断，不能据此判断插入顺序一致；有顺序语义的专门数据结构要按其文档核对。顺序保证也不意味着允许边迭代边任意改变字典大小。〔N163〕

014 可哈希对象需要满足什么约束，为什么 1、True 和 1.0 会共用字典键?

答案 可哈希对象要在参与哈希集合期间保持哈希值稳定，并满足两个对象相等时哈希值也相等；哈希相同并不要求对象相等。list、dict 通常不可哈希，tuple 只有所有元素也可哈希时才可作键。自定义 __eq__ 时须同步考虑 __hash__，不要用会变化的字段计算在表中对象的哈希，否则可能再也查不到它。1、True、1.0 在数值比较中相等且哈希一致，因此作为 dict 键会对应同一个映射条目；想区分类型时应使用包含显式类型标记的复合键。〔N161、N163〕

015 为什么遍历 dict 时不应直接增删键?

答案 dict 的视图和迭代器通常反映底层字典，迭代期间改变大小可能抛出 RuntimeError，也可能破坏期望的遍历结果；不能依赖“这次没有报错”作为正确性依据。只改已有键对应的值，与改变键集合不同，但仍需避免使业务逻辑依赖遍历中的副作用。需要删除一批键时，先生成待删除键列表，或遍历 list(d) 的快照；也可以用字典推导式构建新映射。并发线程修改时，快照与锁的使用应按共享状态协议设计，单个操作安全不能替代整体同步。〔N163〕

016 dict、defaultdict、Counter 及缺失键 API 如何选择?

答案 d[key] 缺失时抛 KeyError，get 返回默认值但不插入键，setdefault 在缺失时插入并返回默认值；默认表达式在调用前就会求值，并不一定惰性。defaultdict 的工厂主要由下标访问触发，适合分组累积，但 get 不会自动创建默认值。Counter 表达元素计数，缺失计数通常返回 0，适合统计频次。不要为了读操作方便而让容器意外增长；若默认对象昂贵或需要按键计算，显式判断或封装创建逻辑往往更清楚。〔N163、N165〕

017 set 与 frozenset 如何去重，为什么不能承诺元素顺序？

答案 set 与 frozenset 都按哈希和相等关系判定成员身份，支持交、并、差等集合运算，平均成员查询通常较快。set 可增删元素且不可哈希，frozenset 不可变并可作为字典键或其他集合的元素；其中成员仍须可哈希。集合不承诺插入顺序，也不能把一次运行观察到的遍历结果当作稳定协议。若去重后要保持原输入顺序，可利用字典的插入顺序，例如 list(dict.fromkeys(values))，并确认相等和哈希规则符合业务需求。〔N163〕

17.2 Python 函数、迭代与对象协议

018 Python 的函数对象和 lambda 适合怎样的接口？

答案 函数是对象，可以赋值、放入容器、作为参数或返回值，这使回调、策略和高阶函数很自然。lambda 只包含一个表达式，隐式返回该表达式的结果，不是独立的闭包机制，也不会让计算自动并行。简单映射和筛选可用 map、filter，它们通常返回惰性迭代器；需要清晰表达元素和过滤条件时，推导式往往更直观。逻辑复杂、需要文档和明确签名或多条语句时，应定义普通函数，避免用层层 lambda 降低调试和追踪能力。〔N161、N162、N164〕

019 Python 参数的 /、*、*args 和 **kwargs 如何约束调用？

答案 普通形参通常既可按位置也可按名字传递；/ 前是仅限位置参数，裸 * 或 *args 之后是仅限关键字参数。*args 收集多余位置参数形成元组，**kwargs 收集多余关键字参数形成字典；调用侧的 * 与 ** 则把已有容器展开。重复为同一参数赋值或传入不支持的参数会抛 TypeError。设计公共接口时，明确哪些名字属于稳定 API，必要时用仅限位置参数避免绑定内部命名，用仅限关键字参数让易混淆选项的含义更明显。〔N170、N171〕

020 可变默认参数为什么会被多次函数调用共享？

答案 默认参数表达式在执行 def 定义函数时求值，结果保存在函数对象中，不会在每次调用时重新创建。因此 def f(items=[]) 中的列表可跨调用累积修改，造成隐蔽状态共享。常见做法用 None 或专门的哨兵对象作为默认值，在函数内部按需创建新容器；如果 None 本身是合法业务值，应使用独立哨兵区分“未传参数”。共享缓存有时是刻意设计，但应明确生命周期、并发保护和清理策略，不要让可变默认值偶然承担这些责任。〔N170〕

```
def collect(value, items=None):
    if items is None:
        items = []  # 每次缺省调用都创建自己的容器
    items.append(value)
    return items
```

021 LEGB、global 和 nonlocal 如何影响名字绑定？

答案 函数通常按局部、外层函数、模块全局、内置名字的顺序查找名字，即 LEGB，但类体、推导式和注解作用域等存在专门规则。函数内若有赋值，该名字通常在整个函数中被判定为局部，先读后赋可能产生 UnboundLocalError。global 让该作用域的绑定落到模块全局，nonlocal 让绑定指向最近的、已经存在的外层函数变量。两者作用于名字绑定，不是修改可变对象内容的必要条件；能通过参数与返回值表达依赖时，通常比隐藏的共享状态更清晰。〔N172〕

022 闭包如何保存外层变量，循环 lambda 的晚绑定问题怎么解决？

答案 闭包保留对外层函数变量单元的访问，而不是自动保存定义时的值快照。循环中创建的多个 lambda 若读取同一个外层名字，在调用时常会看到循环结束后的同一绑定，这称为晚绑定。可以在每次迭代中用工厂函数建立新的局部单元，或用默认参数固定当前对象引用；后者固定的是引用，若对象之后被原地修改，仍能观察变化。需要修改外层名字的绑定时使用 nonlocal，并留意闭包延长大型对象的存活时间。〔N172、N170〕

```
funcs = [lambda i=i: i for i in range(3)]
assert [f() for f in funcs] == [0, 1, 2]
# 默认参数在函数定义时求值，避免所有闭包读同一个循环变量
```

023 装饰器怎样替换函数，带参装饰器和异步包装有哪些边界？

答案 @d 定义后相当于把函数对象传给 d，再把返回值绑定回函数名；多个装饰器按从内到外组合，调用时由最外层包装进入。带参数的装饰器通常先由工厂接收配置，再返回实际装饰器。functools.wraps 复制名称、文档等元数据并设置 __wrapped__，便于检查与调试，但不自动保证所有运行时行为一致。包装 async def 时通常也应提供异步包装并 await 原函数，否则计时、异常捕获和资源清理可能只覆盖“创建协程对象”而未覆盖实际执行。〔N171、N173、N174〕

024 可迭代对象、迭代器与 for 循环的协议是什么？

答案 iter(obj) 获取迭代器，next(iterator) 推进到下一个元素，结束时抛 StopIteration；for 循环围绕这一协议获取元素并处理正常结束。迭代器自身的 __iter__ 通常返回 self，因此从当前状态继续消费，耗尽后一般不会自动恢复。列表等容器通常每次 iter 都能创建一个新的独立迭代器，生成器本身则是一种一次性的迭代器。接口若需要多次遍历，应接收可重复迭代的容器、传入创建迭代器的工厂，或显式缓存数据，不能盲目把输入迭代器重复使用。〔N161、N164〕

025 yield、return 和 yield from 怎样控制生成器执行？

答案 含 yield 的函数被调用时返回生成器，消费时才执行到下一个暂停点，局部变量和执行位置会被保存。yield 交出一个值后可继续恢复，return 则结束生成器，其返回值可成为 StopIteration 的 value。yield from 不只是简写循环，还把发送值、异常和关闭等协议委托给子迭代器，并接收子生成器的返回值。生成器可减少一次性结果的内存，但会保持所引用的局部对象和资源；提前停止消费时，应明确关闭或把资源边界放进可靠的上下文管理。〔N162、N161〕

026 实例方法、classmethod 和 staticmethod 怎样绑定参数?

答案 普通函数放入类后具有描述符行为，经实例取出时通常得到绑定方法，调用时自动传入该实例；self 只是约定名字，并非关键字。classmethod 绑定类而非实例，常用于多态工厂方法，继承调用时会收到实际的派生类。staticmethod 不自动传 self 或 cls，适合与类关联但不依赖对象状态的辅助操作。不要仅按“是否写了 self”推断绑定规则，装饰器与描述符共同决定实际行为；需要访问实例状态时应使用普通实例方法。〔N161、N175、N176〕

027 __new__ 和 __init__ 在对象创建中分别负责什么?

答案 一般调用类时，先由 __new__ 创建并返回对象，再对符合条件的返回实例调用 __init__ 初始化状态。__new__ 是特殊的静态创建入口，通常调用 super().__new__(cls)，定制不可变类型子类时尤其有用；__init__ 不能返回非 None 的值。若 __new__ 返回的对象不是该类实例，通常不会按这一流程调用该类的 __init__。两者不能简单对应 C++ 的构造与内存分配，元类还可能改变类调用过程。避免在 __init__ 中执行难以回滚的大量外部副作用。〔N161〕

028 类属性和实例属性如何查找，可变类属性为什么会共享?

答案 类体中创建的属性属于类，经实例读取时，在没有更优先的描述符或实例属性时可回退到类属性。若类属性是列表，多个实例可能同时引用同一个列表；调用 append 会修改共享对象，而 self.items = [] 通常是在实例上建立新绑定。需要每个对象独立的状态时，应在 __init__ 中创建；dataclass 可通过 default_factory 表达。访问规则还受数据描述符与 __slots__ 影响，不能把“实例字典总是最优先”当成完整结论。〔N175、N176、N177〕

029 Python 的鸭子类型如何表达多态，何时需要 Protocol 或抽象基类?

答案 鸭子类型关注对象是否提供所需行为，而非是否继承某个指定基类；只要支持 read、迭代等协议，同一个函数就能处理不同实现。这种多态不依赖 C++ 式虚函数表声明，但仍有明确的调用约定和失败方式。需要静态工具检查结构化接口时可使用 typing.Protocol；需要约定抽象方法或运行时分类时可采用 abc.ABC。运行时 Protocol 检查存在限制，不能代替完整的签名和语义验证。接口应记录返回值、异常、资源所有权和并发约束。〔N161、N178、N175〕

030 Python 多继承的 MRO 与 super 为什么不是“直接调用父类”?

答案 类的方法解析顺序通常由 C3 线性化确定，保留局部继承顺序和单调性；顺序不一致时，类定义可能直接失败。super 返回一个代理，在当前类之后沿实际对象类型的 MRO 继续查找，因此下一站未必是源码中肉眼看到的某个直接父类。协作式多继承要求各参与类采用兼容签名，并按协议继续调用 super，避免同一初始化被重复执行或被跳过。可以查看 Class.__mro__ 来验证顺序，不应把其他语言的菱形继承布局规则直接套到 Python。〔N175、N161〕

031 描述符、property 与属性访问钩子如何协作?

答案 定义 __get__、__set__ 或 __delete__ 的类可以提供描述符协议，函数绑定、property 等都建立在这一机制上。常规实例查找中，数据描述符优先于实例字典，非数据描述符通常在实例字典之后，再按类的 MRO 查找。__getattribute__ 参与几乎所有常规属性读取，覆盖时应委托给基类实现以免递归；__getattr__ 主要在正常查找以 AttributeError 失败后提供后备行为。property 用方法实现受控读写，可在保持 obj.name 调用形式的同时校验状态。〔N176、N161〕

032 __repr__、__str__ 等特殊方法怎样让对象支持内置操作?

答案 __repr__ 面向开发和调试，理想情况下尽量准确表达对象；__str__ 面向用户展示，未定义时通常回退到 repr 相关行为。__len__、__iter__、__contains__、算术运算等特殊方法让对象参与 Python 的内置协议。解释器进行隐式特殊方法调用时，通常从类型上查找，而不是简单读取实例同名属性，因此在实例上临时赋 __len__ 不一定改变 len(obj)。实现协议要遵守返回类型和语义，尤其是相等比较与哈希的一致性，避免把调试字符串当成稳定序列化格式。〔N161〕

033 __slots__ 能节省哪些内存，为什么它不是完整的封装工具?

答案 __slots__ 声明通常让实例以固定槽位保存指定属性，避免每个实例都分配常规 __dict__，适合数量较多、属性集合稳定的对象。未声明的属性通常不能动态新增，但基类或派生类若引入实例字典，这一限制和节省效果会变化。需要弱引用时通常要显式保留 __weakref__ 槽位，继承中应避免重复槽名和不兼容布局。它不保证所有对象都更省内存，也不提供私有访问控制；是否值得使用应通过真实对象规模测量。〔N161〕

034 下划线命名与双下划线名字改写能提供真正的私有成员吗?

答案 单下划线常表示内部实现约定，并影响某些星号导入行为，不会禁止外部访问。类体中的双前导下划线名字通常改写为带类名的形式，主要减少继承中的意外命名冲突，仍可通过改写后的名字访问，因此不是安全边界。首尾双下划线名字属于 Python 特殊协议命名空间，不应随意发明。接口的可用范围应通过清晰文档、模块边界和 API 设计表达，不要依赖名字改写阻止恶意代码访问。〔N175、N161〕

17.3 Python 异常、模块与工程接口

035 try、except、else、finally 如何影响异常和返回值?

答案 try 正常结束时执行 else，发生匹配异常时进入 except；随后 finally 用于清理。返回语句通常先求出待返回值，再执行 finally，但 finally 中新的 return 或异常会覆盖原先待完成的返回或异常，这容易把故障悄悄吞掉。应尽量让 finally 只做清理，不改变控制流，资源可以优先交给 with 管理。仅把真正需要捕获的操作放进 try，避免顺便捕获后续业务代码的无关错误。进程被强制终止等情况下，不能依赖 finally 一定执行。〔N171〕

036 Python 应如何选择异常基类、保留回溯并建立异常链?

答案 普通业务异常通常继承 Exception；BaseException 还包含 KeyboardInterrupt、SystemExit 等控制类异常，宽泛捕获可能阻止取消或退出。except 内裸 raise 重新抛出当前异常并保留原回溯；raise e 会引入当前抛出位置，raise NewError(...) from e 则明确记录原因链。自定义异常应围绕调用者需要区分的失败类型设计，保留必要上下文，避免把敏感数据写入消息。只有本层能恢复、补偿或转换为稳定接口错误时才捕获，否则让异常继续向上传递，通常不要反复记录同一故障。〔N179、N171〕

037 with 和 contextmanager 怎样确保资源清理，什么时候会吞掉异常?

答案 with 先调用上下文管理器的 __enter__，成功后执行语句体，退出时调用 __exit__；后者收到异常信息，返回真值会抑制该异常。如果 __enter__ 自身失败，不能指望本对象的 __exit__ 自动完成回滚。contextlib.contextmanager 允许用只 yield 一次的生成器表达这一协议：yield 前获取资源，finally 中释放。捕获异常后若不重新抛出，生成器包装也可能把异常视为已处理。锁、文件、事务应明确成功提交和失败回滚边界，不依赖垃圾回收决定释放时机。〔N180、N171〕

```
from contextlib import contextmanager

@contextmanager
def opened(path):
    file = open(path, encoding="utf-8")
    try:
        yield file
    finally:
        file.close()  # 正常返回或异常退出都释放文件
```

038 模块、包与 import 缓存如何影响循环导入和程序入口?

答案 模块提供独立的名字空间，包用于组织模块，普通包通常带 __init__.py，命名空间包有不同规则。首次导入通常先把模块放入 sys.modules，再执行顶层代码；后续导入主要复用缓存。循环依赖可能读到尚未初始化完整的模块，应拆分公共依赖或延后必要导入，而不是盲目重试。import m 绑定模块，from m import x 绑定当时取出的对象，之后 m.x 重新赋值不会自动重绑本地 x。脚本入口中的 __name__ 为 __main__，可用入口判断隔离启动副作用，方便作为模块导入和进程池加载。〔N181、N182〕

039 虚拟环境解决什么隔离问题，怎样让依赖安装可复现?

答案 venv 为项目提供独立的 Python 包安装位置和解释器入口，降低不同项目的依赖冲突，但不是容器级操作系统隔离，也不自动固定 Python 或系统库版本。安装时可用 python -m pip 确认工具对应当前解释器，记录支持的 Python 版本及依赖版本约束，必要时使用锁定文件和制品校验。环境目录通常不提交仓库，应提交可重建的依赖描述。包含本地扩展时，还要记录平台、编译工具链和外部动态库，不能把“开发机装成功”当作所有机器可复现的证据。〔N183〕

040 类型注解、TypedDict、dataclass 和校验模型分别提供什么保证?

答案 Python 类型注解主要供静态检查和工具使用，默认不会让解释器强制检查每次赋值；Any 放宽静态约束，T | None 表示可缺失值类型，不等于参数自动有默认值。TypedDict 描述普通字典的键结构，本身不做运行时数据校验；dataclass 自动生成初始化、比较等方法，也不默认校验所有字段类型。Pydantic 等模型可在运行时校验并按配置转换输入，但必须明确严格模式、额外字段和错误处理。外部工具或 API 参数应在边界做实际校验，不能只写注解就信任输入。〔N178、N177、N184〕

041 读取大文件怎样避免一次性占满内存，文本流还要注意什么?

答案 read() 不指定大小时通常读取剩余全部内容，readline() 读取一行，readlines() 通常创建行列表；直接遍历文本文件更适合按行流式处理。行可能本身很大，二进制数据则可按固定大小分块读取，解析器也必须避免把所有块重新累积到内存。通过 with 明确关闭文件，文本模式显式指定编码，并考虑换行转换、解码错误和缓存行为。需要原始字节偏移或协议分块时应使用二进制模式，不能把字符数量直接当成磁盘字节位置。〔N168、N164〕

042 JSON 的 dump/load 与 dumps/loads 有什么区别和限制?

答案 dump、load 面向文件类对象，dumps、loads 面向字符串或兼容输入；编码结果是 JSON 文本，不是任意 Python 对象的完整快照。JSON 的对象键是字符串，元组、集合、自定义对象、循环引用和精确数值类型都需要明确转换策略。不要把不可信内容交给 eval，也不要以为多个连续 dump 就自然形成有效的单一 JSON 文档；流式场景可设计 NDJSON 等有边界的协议。对外接口应限制输入体积和嵌套深度、校验结构，并为日期、浮点特殊值和字段兼容性建立约定。〔N185〕

17.4 Python 并发与 asyncio

043 GIL 如何影响 Python 并行，为什么仍然需要业务锁?

答案 传统启用 GIL 的 CPython 通常同一时刻仅允许一个线程执行 Python 字节码。GIL 是 CPython 实现层的全局同步机制，用来保护解释器管理的对象状态并简化引用计数等内部操作；这属于实现取舍，不是 Python 语言要求。阻塞 I/O 和部分扩展会释放它，因此线程仍适合许多 I/O 任务；CPU 密集的纯 Python 计算通常评估多进程、原生扩展或自由线程构建。CPython 3.13 起提供可选自由线程构建，具体依赖和扩展是否兼容、是否重新启用 GIL 要核对运行环境。GIL 也不是业务事务锁：检查再写入、跨对象不变量以及可能调用外部代码的复合操作都应显式同步；关闭 GIL 后同样需要设计共享状态保护。〔N186、N187、N188〕

044 Python 的 Lock 和 RLock 怎样选择，重入锁能消除死锁吗?

答案 Lock 是普通互斥原语，适合简单的共享临界区；RLock 记录持有线程及递归层数，允许同一线程再次获取，但每次成功获取都必须对应释放。函数相互调用且可能重复进入同一受保护资源时，RLock 可避免本线程自锁，却不能解决不同锁获取顺序不一致造成的环路，也不能修复把慢 I/O 放进锁内的问题。通常用 with 保证异常路径释放，保持统一锁序并缩小临界区。asyncio 协程之间应使用适当的异步同步原语，不要用线程锁阻塞事件循环。〔N187、N189〕

045 queue.Queue 如何协调线程任务，task_done 和 join 表示什么?

答案 queue.Queue 为线程提供带同步的任务传递，maxsize 可形成背压，put、get 可以阻塞或带超时。每次取出的任务都应在处理结束后对应 task_done，join 等待的是已入队任务的未完成计数归零，而不是某个线程退出。异常路径也要更新完成状态，否则 join 可能永久等待；失败后是否重试则是另外的业务协议。empty 和 qsize 只是观测结果，不能保证下一次 get 不阻塞。停止信号、版本支持的 shutdown 或专门退出协议应与生产者、消费者生命周期统一设计。〔N190〕

046 Python 线程池、进程池和 Future 的结果传播有什么区别?

答案 ThreadPoolExecutor 在同一进程的线程中执行任务，适合共享对象和阻塞 I/O；ProcessPoolExecutor 使用独立进程，适合可并行的 CPU 计算，但跨进程数据传输、启动和序列化成本更高。Future 表示尚未完成的结果，result 可等待并重新抛出任务异常，取消也不意味着可以强制杀死已运行任务。进程间可用 multiprocessing.Queue 或 Pipe 传递消息；Value、Array 与 shared_memory 适合共享数据但需要同步，Manager 提供代理对象、使用方便但有额外通信开销。spawn/forkserver 等启动方式要求入口受 if __name__ == "__main__" 保护，任务和参数通常须可 pickle，优先使用模块顶层函数并显式管理上下文、超时与关闭。〔N191、N188〕

047 协程对象、Task、Future 与事件循环怎样完成异步调度?

答案 调用 async def 通常只创建协程对象，不会立即运行函数体；await 驱动或等待可等待对象，Task 则把协程交给事件循环调度，Future 代表将来可完成的结果。事件循环在一个线程中协作运行多个任务，任务在真正等待未完成操作时才让出执行权，因此每个 await 都切换任务并不成立。连续 await 两个协程通常顺序完成，先创建两个 Task 才能让它们重叠等待。异步擅长隐藏 I/O 等待，不会自动把 CPU 计算分摊到多核；必须持有并等待任务，妥善传播异常和关闭资源。〔N174〕

048 asyncio.gather 与 TaskGroup 遇到子任务异常时怎样处理?

答案 gather 汇集一组结果并保持输入顺序，默认会把遇到的第一个异常传播给等待者，但其他子任务通常不会因此自动全部取消；return_exceptions=True 则把异常作为结果元素，需要调用者逐项处理。TaskGroup 从 Python 3.11 提供结构化并发：离开上下文前等待组内任务，某个任务发生非取消异常时通常取消其余任务，清理后以异常组等规则报告。两者取消传播仍有细节，不能把“一个任务失败”简单理解成所有副作用已回滚，补偿与幂等必须由业务负责。〔N174〕

049 异步函数里怎样处理同步 I/O 和 CPU 密集工作?

答案 在事件循环线程中执行 time.sleep、同步网络调用或长时间计算，会让同一循环上的其他任务一并停顿。能用异步库时直接 await；只能调用同步阻塞 I/O 时，可用 asyncio.to_thread 或 run_in_executor 移交线程。传统启用 GIL 的环境中，纯 Python CPU 计算通常应评估进程池或释放 GIL 的扩展，不能期待线程池自动多核加速。卸载任务仍有排队和取消边界，停止 await 通常不代表底层线程函数已终止，资源超时也应传给真正执行 I/O 的库。〔N174、N191〕

050 asyncio 的超时和任务取消为什么是协作式的?

答案 Task.cancel 请求在后续合适的暂停点向协程注入 CancelledError，不是立即终止任意指令；被取消代码仍需在 finally 或异步上下文管理器中释放资源。通常完成清理后重新传播取消，随意吞掉会破坏 TaskGroup、超时等上层协议。asyncio.timeout 与 wait_for 提供不同形式的超时边界，取消后的清理可能使总耗时超过标称等待值。对数据库、网络和外部工具调用，还要设置它们自身的超时，并明确请求已提交但本地取消时如何查询结果、补偿或防止重复执行。〔N174〕

051 Semaphore 和有界队列为什么比一次创建海量 Task 更适合限并发?

答案 Semaphore 限制同时进入某段操作的任务数，适合保护连接池或下游限额；有界 asyncio.Queue 限制等待任务的积压量，生产者 put 在队列满时等待，从而形成背压。仅在每个任务内部加信号量，仍可能先创建几十万个 Task 并占用大量内存，因此大规模输入更适合有限工作协程从有界队列取任务。使用 async with 管理信号量，并在消费者 finally 中完成 task_done，统一处理生产终止、异常和取消，避免令牌泄漏或队列 join 一直等待。〔N189、N174〕

052 contextvars 如何隔离异步请求上下文，它和线程局部存储有什么不同?

答案 线程局部状态按线程区分，而多个 asyncio 任务常共享同一线程，直接使用 threading.local 可能把请求信息串在一起。ContextVar 绑定当前上下文，asyncio 会为任务维护相应上下文视图，适合请求 ID、追踪信息等；创建任务时通常继承当前上下文的副本，不能把它当成任意任务之间的共享消息通道。set 返回的 token 可在 finally 中 reset，避免同一执行路径后续代码读到残留状态。上下文复制通常复制绑定，不会深复制其中的可变对象，因此仍应避免把可变共享状态藏进上下文。〔N192、N174〕

17.5 CPython 内存与执行机制

053 CPython 对象为什么比其业务数值占用更多内存?

答案 CPython 对象通常需要类型信息、引用管理信息以及对象自身的数据，容器还需存储元素引用、容量和索引结构，因此一个整数对象不只是一个机器整数的几字节。小对象分配器、对齐和内存池也会带来额外占用。sys.getsizeof 一般只反映对象自身的直接大小，不会自动把引用到的整个对象图全部加上；同一个对象被多个容器引用时，简单求和又可能重复计算。对象布局、自由线程构建和 Python 实现不同都会改变结果，应结合分配追踪与进程 RSS 观测，而不是硬背一个固定字节数。〔N193、N161〕

054 引用计数、循环垃圾回收和 del 分别改变什么?

答案 传统 CPython 主要用引用计数管理对象，新增或移除强引用会影响计数，计数归零通常可触发回收，但不能据此给所有构建和对象承诺精确销毁时机。循环中的对象可能彼此持有强引用，即使外部不可达也不能单靠计数发现，因此还有面向容器对象的循环垃圾回收。del name 首先删除名字绑定，并不等于强制销毁对象；其他引用、缓存或环仍可保留它。代际策略和阈值随版本及构建调整，不应死记固定“三代”参数；文件、锁、连接等仍应显式管理生命周期。〔N194、N161、N186〕

055 Python 对象删除后 RSS 为什么不降，怎样定位真实内存增长?

答案 对象释放不等于分配器立即把内存页归还操作系统，空闲池、碎片和本地扩展缓冲都可能让 RSS 保持较高水平；因此不能只看任务管理器就断言泄漏。先观察稳定负载下对象数量、分配趋势和缓存大小，用 tracemalloc 在可疑阶段前后拍快照比较分配调用栈。它主要追踪受支持的 Python 分配，GPU、本地库等外部分配可能需要其他工具。全局容器、无界缓存、闭包和未结束任务若继续持有引用，即使有 GC 也不会释放，应先修复保留链和生命周期，再讨论手动 collect。〔N195、N193、N194〕

056 CPython 的字节码、pyc 缓存与 dis 分析分别解决什么问题?

答案 CPython 通常先解析源文件并编译成代码对象和字节码，再由解释器执行；导入模块时可缓存 pyc，并依据版本与源文件校验信息决定能否复用。缓存主要省去后续导入时的编译成本，并不意味着循环或函数本体天然执行更快，也不是可跨任意 Python 版本运行的稳定机器码。dis 可以显示反汇编结果，帮助理解取值、调用、闭包和控制流，但字节码与专门化优化随版本变化，不能把某个操作码序列当成原子性或语言语义的长期保证。性能优化仍需实际测量。〔N182、N196〕

057 定位 Python 慢请求时如何区分算法、CPU、I/O 和等待?

答案 先固定输入和负载，记录端到端耗时、吞吐和尾延迟，再拆分计算、外部请求、锁与排队等阶段。cProfile 可统计调用次数和函数累计耗时，timeit 适合隔离的小片段比较，异步服务还应检查事件循环是否被同步操作阻塞。总耗时高但 CPU 不高通常要检查 I/O、下游和等待；CPU 热点则优先审视复杂度、重复计算与数据布局，再评估本地扩展或并行。分析器会引入开销，某些本地函数细节还需专用采样工具；优化后必须在同一工作负载上复测。〔N197、N174〕

058 functools.lru_cache 为什么可能造成内存增长和过期结果?

答案 lru_cache 按函数调用参数建立缓存，参数需要可哈希，有限 maxsize 按最近使用策略逐出；缓存会持有参数和返回值，包装实例方法时也可能延长 self 的生命周期。它适合输入决定输出、可接受复用的计算，不适合直接缓存依赖时间或外部状态却没有失效设计的结果。线程安全指内部缓存结构保持一致，不保证同一缺失键只计算一次。应使用 cache_info 观测命中和规模，在必要时 cache_clear，注意关键字参数组合以及可变返回值被调用者修改带来的共享副作用。〔N173〕

第 18 章 工具命令与容器

18.1 Linux 命令与现场排查

001 ps 的进程快照和 top 的采样信息应怎样解读?

答案 ps 适合一次性输出和脚本处理，如 -e 查看全部进程、-p 选 PID、-L 查看线程、-o 指定字段；BSD 风格 aux 与 UNIX 风格 -ef 不能逐字混解。top 持续采样，展示系统摘要与进程或线程变化；%CPU 的统计口径不同于 ps 的生命周期平均值，多线程进程在某些显示模式中可超过 100%。VSZ/VIRT 是虚拟地址空间，RSS/RES 是驻留内存，不能直接相减得到泄漏。top 的 1 键常用于切换各 CPU 显示，但容器实际可用 CPU 还受配额与亲和性限制，不能只数界面行数。〔N198、N199〕

```
ps -p 1234 -L -o pid,tid,stat,pcpu,pmem,rss,vsz,comm
top -H -p 1234  # 查看指定进程的线程采样
# 文件可用 ls/stat，权限用 chmod/chown，网络用 ip/ss 分别核对
```

002 按进程名结束程序时怎样避免匹配错误并保留清理机会?

答案 先用 pgrep -a 或 ps 确认 PID、命令行和所有者，再向准确目标发送 SIGTERM，让程序处理退出、刷盘和关闭连接。pkill 的名字模式、完整命令行选项和匹配范围必须先核对，不能把宽泛 grep 输出直接批量送给 kill。确实不响应且确认后果时再考虑 SIGKILL；它不能被程序捕获，不能保证业务状态清理，也不会让不可中断等待立即消失。PID 可能被复用，长期自动化应结合进程身份检查或服务管理器操作；僵尸需要父进程回收，向它继续发信号无效。〔N200、N198〕

```
pgrep -a -x worker  # 先核对精确进程名及对应命令行
# 确认目标后，kill -TERM PID 请求正常终止；不要直接使用宽泛批量匹配
```

003 如何用 ss 和进程信息定位监听端口与连接异常?

答案 ss -lntp 查看 TCP 监听套接字及相关进程，-u 用于 UDP，-a 查看更多套接字，-n 禁止名字解析；按端口或状态过滤比全量输出更容易定位。查看其他用户的进程关联可能需要相应权限。监听地址要分清 127.0.0.1、具体网卡地址和通配地址，还要注意 IPv4/IPv6、网络命名空间以及容器端口发布。连接数量、Recv-Q/Send-Q 的含义随监听或已连接套接字而不同，应结合应用、内核队列与请求量解释。看到 LISTEN 仅说明本机有监听，不能证明客户端端到端可达。〔N201〕

```
ss -lntp "sport = :8080"  # 查谁在监听 TCP 8080
ss -ant state established
# 在容器/其他 network namespace 中排查时，应在相应命名空间检查
```

004 ping、nc 和 curl 分别能验证哪一层的连通性?

答案 ping 主要验证 ICMP 回应，失败可能是策略丢弃，成功也不代表业务 TCP 端口开放。nc -vz host port 可尝试 TCP 连接，用于区分拒绝、超时等症状，但并不校验应用协议；UDP 没有同样的握手，不能仅凭探测命令未报错判定服务可用。curl 可继续验证 HTTP 状态、TLS、证书与响应耗时。排查应逐层检查解析结果、路由、监听、防火墙、连接及协议，给探测设置超时；不要用忽略证书校验把 TLS 配置问题掩盖成正常。〔N202、N203、N201〕

```
nc -vz -w 3 example.com 443
curl --connect-timeout 3 --max-time 10 -I https://example.com/
# ping 仅是 ICMP 观测，不能替代端口和应用协议检查
```

005 free、top 和 vmstat 怎样区分内存不足、缓存与实际换页?

答案 free 和 top 的内存摘要多来自 /proc/meminfo，系统 free 很低不一定不足，MemAvailable 更接近无需大量换页即可供应用使用的估计值，页缓存有一部分可回收。进程 RSS 不等于独占内存，多个进程简单相加会重复计算共享页；容器还要单独看 cgroup 限制与当前使用量。vmstat 1 的 si/so 反映换入换出速率，第一行通常是自启动以来的统计，后续才是采样区间；sysstat 的 sar -B 1 可观察页面扫描和回收，pgscank/s 表示后台扫描，pgscand/s 表示直接回收扫描，pgsteal/s 表示扫描中成功回收的页数。应结合持续 available、换页、major fault、PSI 和 OOM 记录判断压力，不能只凭 swap 已用量或一个内存百分比下结论。〔N204、N205、N206〕

```
free -h
vmstat 1 5
# 连续观察 si/so 和可用内存；容器另查自身 memory.current、memory.max
```

006 sed、awk、grep 和 sort 如何组合处理日志而不混淆统计口径?

答案 sed 擅长逐行编辑和替换，awk 擅长按字段解析、条件筛选与累加；替换前先输出或保留备份，注意正则、分隔符与 shell 引号的不同解释。grep -c 统计匹配的行数，不是出现次数，若一行可多次出现，应采用逐匹配输出再计数等明确办法。字段清晰的空白分隔日志，可用 sort -k3,3nr 按第三列数值降序，再取前十；包含带引号字段、空格或跨行记录时，应使用真正的格式解析器。超大日志优先流式处理，别把整个文件装进 shell 变量。〔N207、N208、N209〕

```
sed 's/old/new/g' input.log > replaced.log
awk '{ print length($0) }' input.log  # 每行长度，字符/字节含义受工具及 locale 影响
grep -F -o 'needle' input.log | wc -l  # 固定字符串出现次数
sort -k3,3nr access.log | head -n 10   # 假设第3列为数值耗时
```

007 Linux load average 和 CPU 使用率为什么可能完全不同?

答案 load average 表示一段时间内处于可运行及不可中断等待等状态的任务数量的平滑平均，通常展示 1、5、15 分钟视窗，不是 CPU 利用率百分比。高 CPU 计算会抬高负载，磁盘或其他不可中断等待也可能造成“CPU 不忙但负载很高”。先用 uptime/top 看整体，再用 mpstat -P ALL 1 对照各 CPU 的 %usr、%sys、%iowait、%idle，结合核数、运行队列、D 态任务与 I/O 延迟判断；容器内看到的全机平均负载未必对应自身配额。用 pidstat -u 1 可定位进程 CPU 消耗。不要把 load 除以逻辑核数当成通用精确利用率，更不能只凭一个阈值判断服务健康。〔N210、N199、N206〕

```
uptime
mpstat -P ALL 1
pidstat -u 1
```

18.2 Git 版本控制与协作

008 Git 工作区、暂存区和提交对象怎样流转?

答案 工作区是当前检出的文件，暂存区保存下一次提交的候选快照，提交对象记录树、父提交与元数据；本地仓库的对象和引用与远端仓库分开。git add 把当前内容写入暂存区，不意味着后续继续编辑也会自动进入同一次提交；git commit 主要记录暂存区快照。fetch、push 负责与远端交换对象并更新相应引用，不能把“本地提交成功”理解成已上传。Git 采用按内容寻址的对象与提交关系，分支是可移动引用，审查时应分别检查工作区、暂存区和准备推送的提交。〔N211、N212〕

```
git status --short
git diff          # 工作区相对暂存区
git diff --cached # 暂存区相对 HEAD
# 提交前确认待记录的快照，而不只查看编辑器里的文件
```

009 merge、rebase 与快进合并怎样改变提交历史，冲突如何处理?

答案 merge 保留既有提交关系，必要时创建有多个父节点的合并提交；目标是当前分支祖先时，可以只移动引用完成 fast-forward。rebase 把一组变更重放到新的基线上，通常生成新的提交 ID，历史更线性但会改写这组提交的身份。冲突时先理解两侧意图，修改文件、检查并暂存，再 continue 或完成合并；无法继续时可用对应 abort 返回操作前状态。共享分支上的历史改写需协调，不能把无条件强推当作冲突解决。pull --rebase 是获取后重放本地提交，不等于普通 merge 型拉取。〔N213、N211〕

010 fetch 和 pull 对本地分支有什么不同影响?

答案 fetch 下载缺少的对象并按配置更新远程跟踪引用，通常不会自动把远端变更合入当前本地分支，因此适合先审查再决定集成。pull 在获取之后还会按选项和配置执行 merge、rebase 或仅允许快进，可能改变当前分支与工作区，并可能遇到冲突。origin/main 是本地保存的远程跟踪引用，不是服务器分支的实时视图。操作前确认当前分支、upstream 和未提交内容，必要时先 fetch，再比较日志与差异，避免以为拉取只是刷新远端信息。〔N214、N211〕

011 git diff 的常见比较对象是什么，A..B 与 A...B 如何理解?

答案 不带提交参数的 git diff 主要比较工作区与暂存区，--cached 比较暂存区与指定提交，通常默认 HEAD；指定两个提交则比较它们对应的快照。对 diff 来说 A..B 基本等价于直接比较 A 与 B，并不是列出这段历史；A...B 则从 A 与 B 的合并基点比较到 B，常用于查看分支引入的改动。还可按路径限定范围，用 --stat 快速看规模，再看上下文。评审未追踪文件时要另用 status 或显式比较，因为普通 diff 不会自动展示它们。〔N212〕

012 HEAD 和 detached HEAD 分别指向什么，怎样保留临时提交?

答案 正常情况下 HEAD 是指向当前分支的符号引用，提交后分支引用向前移动；检出某个具体提交时 HEAD 可直接指向该提交，称为 detached HEAD。此时仍能编辑和提交，但切换走后若没有分支或标签保留，新提交可能只在 reflog 中暂时可找。需要继续保存工作时，应在当前提交创建分支再切换，或记录提交 ID 后建立引用；不要把游离状态误认为内容不能修改。reflog 有保留期限，不能代替长期备份和明确命名的分支。〔N215〕

013 reset 的三种模式与 revert 适用于哪些撤销场景?

答案 reset 移动当前分支或 HEAD 到目标提交；--soft 保留暂存区和工作区，--mixed 通常重置暂存区但保留工作区，--hard 还会重置相关工作区文件并可能覆盖阻碍检出的未追踪内容。它适合整理尚未共享的本地历史。revert 根据旧提交生成反向变更的新提交，保留原历史，更适合共享分支撤销，遇到后续依赖仍可能冲突。两者都不是万能恢复键，操作前确认目标与差异；合并提交的 revert 还需正确选择主线父节点并理解再次合并的后果。〔N216、N217〕

014 commit --amend 为什么会更换提交 ID?

答案 --amend 使用当前暂存区及指定元数据替换最近一次提交，通常生成新的提交对象；即使只是改提交说明，提交 ID 也会变化。适合修正尚未共享的小错误，但它不会自动把所有工作区修改都放进去，仍需先审查暂存区。若旧提交已被他人获取，直接替换远端历史会影响他们的基线，应按团队策略采用新的修正提交或协调改写。作者、提交者和签名也可能受影响，不能把 amend 当成原提交不变的原地编辑。〔N211〕

015 switch、restore 和 checkout 怎样划分切分支与恢复文件的职责?

答案 checkout 兼有切换分支、检出提交和恢复路径等历史功能，参数形态不同可能产生完全不同的副作用。switch 主要处理分支切换与创建，restore 主要从选定来源恢复工作区或暂存区内容，使意图更明确。restore 默认来源与 --staged 时不同，恢复文件前要确认是从 HEAD、暂存区还是其他提交取内容；它不会自动创建一条保留旧内容的提交。旧版本 Git 可能没有这些独立命令，使用时核对版本，并先用 status、diff 审查可能被覆盖的修改。〔N211、N215〕

016 stash 能暂存哪些修改，apply 和 pop 发生冲突时怎么办?

答案 stash 通常保存已追踪文件的工作区与暂存区状态，-u 可包含未追踪文件，-a 还会包含忽略文件；它保存在本地仓库，不会随普通 push 自动上传。apply 尝试应用而保留记录，pop 在成功应用后移除相应记录，出现冲突时通常需要手动解决，不能假定已回到完整原状。恢复后若要还原暂存状态需留意 --index 及冲突限制。stash 适合短时切换上下文，重要工作应建立明确分支和提交；带敏感或巨大忽略文件时，不应随意使用 -a。〔N218〕

017 密钥进入 Git 历史后，为什么只删除文件或加 gitignore 不够?

答案 提交中的凭据应先撤销或轮换，因为旧提交、克隆、缓存和日志都可能已有副本；删除当前文件不能让泄露失效。随后界定受影响引用，按平台指南使用专用历史重写工具清除相关内容，并协调协作者、分支保护和重新获取基线。历史改写会改变提交 ID，不能保证收回所有外部副本，也不能把扫描器未命中当成从未泄露。最后分别扫描待上传文件、暂存区及所有相关历史，并配置安全的本地秘密管理与忽略规则。公开反馈只使用脱敏信息，避免在工单中再次传播凭据。〔N219、N220〕

018 cherry-pick 怎样选择单个变更，有哪些重复和依赖风险?

答案 cherry-pick 把选定提交引入的变更应用到当前分支，并通常生成一个新的提交，适合把修复移植到维护分支。它并不自动搬运整个分支及其上下文；提交若依赖此前重构或其他修改，可能冲突、构建失败或行为不完整。重复挑选以及后续再次合并原分支也会使历史更复杂，应记录来源并检查实际差异。可用 -x 在说明中保留来源 ID，冲突后按流程继续或中止；挑选合并提交需明确主线父节点，不能仅凭提交标题判断影响。〔N211〕

019 删除本地和远端分支后，提交是否立即消失?

答案 git branch -d 在满足合并检查时删除本地引用，-D 强制绕过该检查；删除远端分支通常使用 git push remote --delete name，操作的是服务器引用。远程跟踪引用的过期清理又是另一件事，可按配置执行 fetch --prune。删除引用不等于立即抹去对象，其他分支、标签、reflog 或克隆可能仍保留提交，之后对象才可能被垃圾回收。因此删除前确认分支拥有者、合并状态和备份需要，不能用删分支来处理已经泄露的敏感内容。〔N211、N214〕

020 主干开发、功能分支与 Fork/PR 工作流怎样选择?

答案 工作流应围绕发布频率、维护版本和协作权限选取。Git Flow 以 develop、release 等分支组织版本发布，适合有明确版本周期但会增加长期分支维护；GitHub Flow 和 GitLab Flow 倾向短期分支与评审合并，主干开发要求小步集成和可靠自动验证。Fork 是托管平台上的独立仓库副本，Pull Request/Merge Request 是请求评审和集成变更的协作对象，并非 Git 核心命令。无论采用哪种模式，都应明确基线、合入策略、检查和负责人；分支越多并不自然越安全，长期偏离会放大冲突和测试矩阵。〔N221、N211〕

021 git log 怎样查看分支关系并限制到相关变更?

答案 git log 展示提交历史，可用 --oneline 简化摘要、--graph 显示关系、--decorate 显示引用，用 --all 纳入更多引用；这些选项只是展示，不会修改仓库。可按作者、时间、提交消息或路径筛选，并用 -p 查看具体补丁。注意日志中的 A..B 表示可从 B 到达但不能从 A 到达的提交集合，与 diff 的点号语义不同；--follow 的单文件重命名追踪也有启发式限制。定位问题时应结合父子关系和实际改动，不凭日期或最后提交者直接推断原因。〔N222〕

022 轻量标签、附注标签与可验证发布有什么关系?

答案 轻量标签基本是直接指向对象的引用，附注标签拥有独立标签对象，可记录打标签者、消息和可选签名。两者都可标识版本，但标签名并不自动保证内容可信或永远不变；发布流程应明确签名验证、提交来源和是否允许移动标签。git push 默认行为不一定上传所有标签，应显式选择要发布的引用，避免把本地临时标签一并推出。已有用户依赖的版本标签通常保持不变，修复应发布新版本；签名验证仍需可信公钥和身份管理。〔N223〕

023 blame 和 bisect 如何帮助定位回归，分别有什么局限?

答案 blame 按行展示当前内容最近关联的提交，适合找到上下文与设计说明，但重构、格式化和代码搬移会影响结果，不能把它等同于责任归属。bisect 在已知好坏提交之间按提交图缩小范围，配合可重复的验证命令可自动寻找引入回归的提交。验证应区分失败、成功和无法测试的版本，环境不稳定或中间提交不可构建会误导二分。定位到提交后还要阅读补丁、复现并验证因果关系，因为提交可能只是触发潜伏问题，而不是唯一根因。〔N224、N222〕

024 gitignore 为什么忽略不了已经追踪的文件?

答案 忽略规则主要决定哪些未追踪路径不进入常规添加流程，不会把已提交的文件从索引和历史中移除。需要保留本地文件但停止追踪时，可先审查影响，再对指定路径使用 git rm --cached 并提交变更，同时加入适当忽略规则。用 git check-ignore -v 查看是哪条规则生效，留意目录规则、否定规则和路径作用域。不要把配置文件中的秘密寄希望于 ignore：已经存在的旧提交仍需单独处理。团队共享的忽略与个人编辑器、系统缓存规则可按项目策略放在不同配置位置。〔N220〕

18.3 Docker 容器与部署工具

025 Linux 容器依靠什么隔离和限额，cgroup v2 与虚拟机有何不同?

答案 Linux 容器通常通过 namespaces 隔离进程、挂载、网络等可见视图，cgroups 统计并约束资源，配合 capabilities、seccomp 等机制收缩权限。cgroup v2 使用统一层级并按规则启用控制器，CPU、内存等限额不是另起一套客体内核；cpu.max、memory.max 等文件含义与统计需按内核文档核对。虚拟机通常拥有独立客体内核，容器则共享宿主内核，因此隔离和兼容性边界不同，不能认为容器等于轻量虚拟机。Docker Desktop 运行 Linux 容器时通常还位于虚拟机中，性能、启动时间和内存开销取决于实际部署，不能用固定数量级概括。〔N225、N226〕

026 Docker 客户端、Engine、镜像、容器与 Registry 怎样协作?

答案 客户端通过 API 向 Docker Engine 的守护进程发出构建、运行和管理请求，Engine 再协调底层容器运行组件。镜像是运行所需文件系统层与配置的不可变模板，容器是基于它创建的运行实例，通常有自己的可写层和隔离配置；同一镜像可创建多个容器。Registry 提供镜像分发，仓库与标签用于组织引用，标签可移动而 digest 对应具体内容。停止容器不等于删除镜像，删除容器也不等于删除其卷。远程管理 API、Registry 权限与镜像来源都是独立边界，不能把客户端所在机器默认当成实际宿主机。〔N225〕

027 镜像分层和容器可写层如何影响缓存与磁盘占用?

答案 镜像通常由可复用的只读文件系统层和配置组成，容器叠加自己的可写层；修改底层文件时，存储实现可能进行 copy-up 等操作。Dockerfile 的 RUN、COPY 等会改变文件系统，而 CMD、ENV 等主要改变镜像配置，不能说每条指令都新增同样的数据层。后续层删除文件通常不能抹去先前层已保存的内容，因此大文件或秘密即使随后删除仍可能留在镜像中。构建缓存复用受指令、上下文与依赖影响，合理组织稳定依赖和常变源码，并用实际镜像历史和磁盘统计检查效果。〔N227、N228、N229〕

028 Dockerfile 的构建指令与运行配置如何区分，COPY 与 ADD 怎样选择?

答案 FROM 选择阶段基础，WORKDIR 设工作目录，RUN 在构建时执行，COPY 把上下文或指定阶段的文件加入镜像；USER、ENV、EXPOSE、VOLUME、CMD 等设置后续构建或运行配置。EXPOSE 只是声明，不自动发布宿主端口；VOLUME 声明容器内的挂载点，实际挂接何种卷或主机路径由运行时配置决定。ADD 除复制外还支持按条件解包本地归档、远程 URL 或 Git 来源等功能，具体依 Dockerfile 版本而定；普通本地复制优先使用更明确的 COPY。ARG 是构建参数，不等于安全秘密通道，可能出现在元数据或历史中，应使用 BuildKit secrets 等专用机制。构建上下文还应通过 .dockerignore 排除无关文件。〔N227、N228〕

029 CMD 与 ENTRYPOINT 如何组合，为什么 exec 形式影响退出信号?

答案 ENTRYPOINT 通常指定容器主程序，CMD 提供缺省命令或默认参数；使用 exec 数组形式的 ENTRYPOINT 时，docker run 后附加的参数通常替代 CMD 并传给主程序，而 --entrypoint 可以显式覆盖入口。shell 形式会经 shell 执行，其参数展开与信号传递不同，包装脚本通常需要 exec 把主进程替换为应用。容器主进程收到停止信号后还要处理清理和子进程回收，exec 形式本身不保证业务优雅退出。面试时应写出准确组合后的 argv，避免笼统说入口永远不能被覆盖。〔N227、N230〕

```
ENTRYPOINT ["/app/server"]
CMD ["--port", "8080"]
# docker run image --port 9090 将默认参数改为 --port 9090
```

030 多阶段构建怎样缩小运行镜像，为什么不应盲目选择最小基础镜像?

答案 在一个 Dockerfile 中使用多个 FROM，可让构建阶段包含编译器和完整依赖，运行阶段只 COPY --from 所需产物；构建也可以用 --target 选择目标阶段，不能说永远只有最后一个阶段有意义。配合小而合适的运行基础、.dockerignore 和在同一层清理临时缓存，可减少体积。精简不能破坏动态库、证书、时区和诊断需求，Alpine 的 musl 与 glibc 兼容性也需核对。可复现还要求固定依赖、基础镜像及下载校验；不要把静态编译假设直接套给所有 C++ 可执行文件。〔N231、N228、N227〕

031 volume、bind mount、tmpfs 与容器可写层各适合保存什么?

答案 容器可写层随容器生命周期管理，不适合把关键持久数据只放在其中。volume 由 Docker 管理，适合跨容器重建保留数据；bind mount 映射宿主路径，便于开发或与宿主共享文件，但更依赖宿主布局且可能覆盖容器目标目录的原有视图。tmpfs 适合临时内存数据，通常随容器停止消失，仍需考虑主机换页等边界。持久化并不等于备份或高可用，数据库卷还需要一致性备份。挂载权限、只读选项、用户 UID/GID 与宿主访问范围必须明确。〔N229〕

032 Docker bridge、host、none、container 和 overlay 网络怎样影响容器通信?

答案 用户自定义 bridge 适合单机容器通信，通常提供按容器名解析；默认 bridge 的自动服务发现行为不同。host 模式共享宿主网络命名空间，减少端口隔离，-p 发布端口通常无意义；none 通常只保留回环环境。container:<name|id> 模式共享指定容器的网络命名空间，因此两者共享网络接口和 localhost，通常不能再单独发布端口。overlay 用于支持该驱动的多主机容器网络，不等于 Kubernetes 所有网络实现都采用 Docker overlay。应使用明确的网络与服务名，区分容器目标端口和宿主发布端口；EXPOSE 不会单独建立访问路径。〔N232、N230〕

033 Docker Compose 描述了什么，服务依赖为什么不等于业务就绪?

答案 Compose 用声明式配置组织多个服务及其镜像或构建、网络、卷、变量和运行参数，适合统一启动、更新和销毁一套应用，常用 docker compose up、ps、logs、down 操作。服务名通常可在项目网络中解析。启动顺序本身不保证数据库已经可接受请求，需要 healthcheck、适当依赖条件，以及应用自身的超时和重试。Compose 可用于单机生产场景，但不会自动获得 Kubernetes 式跨节点调度和集群控制能力。down 与 down --volumes 对持久数据的影响不同，配置中也不应明文提交真实凭据。〔N233、N232〕

034 docker run 的参数怎样决定运行实例，日常管理命令如何分工?

答案 docker run 创建并启动新容器，start 启动已有容器；-d 后台运行，-it 保持交互终端，-p 发布端口，--mount 指定存储，--network 选网络，--memory 与 --cpus 设置相应资源约束，-e/--env 设置环境变量，-u/--user 指定容器内用户。--restart 是重启策略，不等于应用健康检查，--rm 适合一次性容器。ps -a、logs、inspect、stats 分别查看生命周期、输出、详细配置与资源观测；stop/restart 管理运行状态，rm 删除容器，rmi 删除镜像，build/pull/push 处理镜像。默认端口发布可能暴露到更多宿主接口，需要时明确绑定地址；环境变量也不天然是保密存储，不要把真实令牌直接写入可共享命令。〔N230、N225〕

```
docker run --name demo --rm -p 127.0.0.1:8080:8080 \
  --memory 512m --cpus 1 --user 1000 -e APP_ENV=prod example/app:version
# 镜像名为示例；真实部署还要明确网络、挂载、用户及配置
```

035 docker exec 与 attach 对容器进程有什么不同影响?

答案 exec 在运行中的容器里启动一个新进程，可用 -it 运行镜像已有的 shell，也可以直接执行诊断命令；镜像可能没有 bash，极简镜像甚至没有 shell。attach 则连接主进程现有标准流，输入和信号可能直接影响应用，退出方式要按终端设置处理，不能简单当作另开一个会话。exec 需要主容器仍在运行，不是让已退出容器复活。调试应优先读取 logs 与 inspect，避免临时修改运行容器后误以为变更会进入下一次镜像部署。〔N230、N225〕

036 容器启动后立即退出，应从哪些证据判断原因?

答案 先用 docker ps -a、logs 和 inspect 查看退出码、启动命令、错误字段及 OOMKilled，再确认主进程是否完成正常一次性任务、启动失败或自行后台化。容器主进程退出后实例通常停止，给 docker run 加 -d 不能把已结束的进程维持运行。137 常对应 SIGKILL，但不独占表示 OOM，需要结合 OOM 标记和宿主事件判断；重启策略也不能修复错误配置。核对工作目录、挂载遮挡、权限、动态库、端口和依赖就绪，必要时覆盖入口进行受控复现，并把修复写回可重建配置。〔N230、N225〕

037 docker commit 为什么不能替代可审查的镜像构建流程?

答案 commit 基于容器当前可写状态和部分配置生成镜像，适合临时保留调查结果，但挂载卷中的数据通常不包含在提交内，运行进程内存状态也不是容器检查点。build 按 Dockerfile、构建上下文和依赖执行，可纳入版本控制与审查，更适合持续集成。两者都可能意外带入秘密；构建过程“写在文件里”也不自动保证完全可复现，还需固定基础、依赖和外部输入。临时容器中验证过的修复应最终落到源码及构建步骤，避免不可说明的手工镜像成为唯一产物。〔N228、N227、N229〕

038 Docker 构建的镜像与 Kubernetes 的容器运行时是什么关系?

答案 Docker 提供构建、分发和运行容器的一套工具，Kubernetes 主要负责声明式集群编排、调度、扩缩容和恢复。Kubernetes 通过 CRI 与支持的运行时通信，例如 containerd、CRI-O；内置 dockershim 在 Kubernetes 1.24 被移除，不意味着 Docker 构建的标准镜像不能运行。镜像格式、运行时接口和编排能力是不同层次，外部兼容适配也应按部署文档判断。应用迁移时还需匹配 CPU 架构、操作系统、权限、卷和网络配置，不能仅凭镜像能拉取就认定可在任何节点正常执行。〔N234、N225〕

039 Docker prune 的清理范围为什么必须逐项确认?

答案 先用 docker system df、容器和卷列表确认磁盘占用及引用，再选择 image、container、network、builder 或 volume 对应清理命令。image prune 默认主要清悬空镜像，-a 扩大到未被容器使用的镜像；system prune 会同时影响多类未使用资源，但不默认删除所有卷。当前文档中 volume prune 默认针对未被容器引用的匿名卷，命名卷还需相应选项；版本与参数应核对 --help。停止的容器仍可能关联需要恢复的数据，未使用不代表业务无用，因此不能把 prune -a --volumes 当成无风险的通用维护命令。〔N235、N229〕

第 19 章 后端进阶与平台工程

19.1 存储选型与代码组织

001 MongoDB 的文档模型、副本集与分片分别解决什么问题?

答案 MongoDB 以 BSON 文档表达聚合数据，嵌入相关字段可减少跨表连接，但文档无边界增长和大量重复字段同样会带来更新与容量问题。副本集用主节点接收写入并复制到其他节点，读写关注级别影响确认与读取语义；分片按分片键把数据分布到多台机器，mongos 负责路由。它支持索引、模式校验与事务，不能等同于“没有结构、没有事务”。选择 MongoDB 或关系数据库要从访问模式、连接查询、约束以及运维能力出发，而非仅因数据量大就更换。〔N236、N237〕

002 HDFS 如何把大文件存储和故障恢复分布到集群?

答案 HDFS 把文件切成较大的块，NameNode 管理命名空间与块位置等元数据，DataNode 保存和提供块内容。读时客户端先向 NameNode 查询副本位置，再选择 DataNode 直接读块并校验 checksum；写时 NameNode 返回目标节点链路，客户端将数据包送到首个 DataNode，由副本管线逐台转发，确认沿链路返回后才确认写入。NameNode 通过 fsimage 快照和 edit log 记录元数据变化，高可用部署中的 Standby 跟随共享日志并准备接管。心跳和块报告支持发现失效及恢复；副本或纠删码容忍节点故障。它偏向大文件顺序、高吞吐访问，海量小文件会增加元数据压力，低延迟随机改写也不是主要目标；副本不能代替异地备份、权限控制或误删恢复方案。〔N238〕

003 业务分支变多时，怎样运用策略、工厂和观察者模式?

答案 先区分业务条件本身与可替换的变化点。固定少量条件用清晰的 if 或 switch 即可；当多种算法共享接口时，策略模式把选择与执行分开，工厂集中管理创建与所有权，观察者把事件产生者与多个订阅者解耦。表驱动适合规则能用数据表达的情况，不能把有顺序和副作用的业务机械改成哈希表。模式需要说明解决的变化、额外间接层的成本、生命周期和测试方式，不能只罗列名字，更不能把设计模式数量当成代码质量指标。〔N239〕

```
// 示意：创建、选择、执行分别承担明确责任
auto strategy = factory.create(request.type);
Result result = strategy->execute(request);
events.publish(result); // 订阅者失败策略与资源生命周期另行约定
```

19.2 订单、认证与接口边界

004 订单创建、支付回调与库存释放如何组成一致的状态机?

答案 先定义订单主状态及合法迁移，例如待支付、已支付、待发货、已发货、已完成、已取消；售后可单独建申请中、退款处理中、售后完成等逆向状态，并保存退款流水。以条件更新或版本号原子地竞争迁移权，订单创建与本库库存预留尽量放在同一事务，跨服务则用明确补偿和可恢复消息。支付回调按渠道交易号唯一去重并核对金额、币种与订单绑定；超时任务仅能取消仍待支付的订单。取消与支付并发时须规定晚到款处理。退款请求超时可能是渠道已成功但响应丢失，应先用退款流水号查询渠道状态，再幂等重试并由对账修复，不能直接重复扣款或伪报失败。状态变更后经 Outbox 驱动下游。〔N127〕

```
BEGIN;
UPDATE orders SET state = :next
WHERE id = :id AND state = :expected;
-- 只有影响一行才写库存变更及 outbox，同库事务一起提交
COMMIT;
```

005 用户密码应如何保存、校验和渐进升级?

答案 保存密码专用的慢哈希结果及每条记录的随机盐、算法标识和成本参数，使用成熟库实现 Argon2id、scrypt 或适用的 bcrypt/PBKDF2，成本结合部署资源选择。盐可与摘要一起存库；可选 pepper 必须独立于数据库保管并规划轮换。不能存明文、可逆加密后的密码，或仅用快速 SHA-256/MD5 再加盐。登录时由库验证并在旧参数落后时重新散列，配合在线限速、泄漏密码检查和安全的重置流程。日志、异常和监控也不应记录输入的密码或完整认证凭据。〔N240〕

006 单点登录如何用统一身份提供方建立各业务会话?

答案 用户在身份提供方完成认证后，各业务系统通过约定协议获得可验证的身份结果，并建立自己的会话；不能让任意不同站点共享一个裸 Cookie。以 OIDC 授权码流程为例，业务方验证重定向 URI、state 及 PKCE，后端交换授权码后校验 ID Token 的签名、issuer、audience、有效期及需要的 nonce。OAuth 主要解决授权，OIDC 在其上增加身份层。退出和撤销需设计中央会话与各业务会话的关系；短期令牌、密钥轮换、跨站 Cookie 规则和登录 CSRF 都应纳入边界。〔N241〕

007 对外 API 如何同时处理身份、权限、重放和重复请求?

答案 用 TLS 保护传输，认证确定调用者身份，授权在每次资源操作中校验对象归属和动作权限，不能只依赖前端隐藏按钮。接口约束字段、长度、速率和成本，避免日志泄漏凭据。需要应用层签名时，明确被签名的请求方法、路径、正文摘要及时间窗口，并结合 nonce 或服务端状态拒绝重放；签名本身不会自动证明业务权限。幂等键另用于识别重试业务，需绑定调用者、请求内容与既有结果。统一版本、错误码、审计和密钥轮换，避免客户端与服务端各自猜测规则。〔N242、N243〕

19.3 容量、隔离与运维

008 如何把“一秒内响应”转成可测量、可执行的延迟目标?

答案 先用 SLI/SLO 指定成功率、观测窗口和延迟分位数，例如一定比例的成功请求在目标时间内完成，而不是承诺所有请求永远低于一秒。把端到端预算分给排队、网关、应用、数据库和外部调用，沿调用链传播截止时间。客户端超时通常不会自动停止服务端已经开始的业务，服务端须检查取消/截止时间并尽早停止无用工作；否则客户端重试可能与仍在执行的原请求重复副作用。为连接池、线程池和请求队列设置上限及背压，重试必须受剩余预算和幂等性限制。用分布式追踪与压测观察尾延迟和饱和点，按瓶颈优化；超载时快速拒绝或降级，避免无界等待。〔N244〕

009 Java、Go 与 CPython 的垃圾回收机制为什么不能混为一谈?

答案 Java 的具体行为取决于所选收集器，常见设计结合可达性分析、分代、并发标记及整理，不能声称所有收集器都使用同一复制算法。Go 使用并发标记清扫和写屏障等机制，在暂停、GC CPU 与堆空间之间权衡，可通过运行时指标和内存限制调优。CPython 常规构建主要用引用计数回收，同时检测部分容器形成的不可达引用环，细节随版本与构建方式变化。三者都无法回收仍被业务错误引用的数据，也不保证及时释放文件或连接，因此资源仍应通过结构化清理管理。〔N245、N246、N194〕

010 四层与七层负载均衡按什么粒度转发?

答案 四层负载均衡围绕 IP、端口和传输层连接选择后端，通常不解析业务报文；七层代理终止并理解 HTTP 等应用协议，可按域名、路径、请求头或租户路由，并实施应用级超时和策略。七层有额外解析及连接管理成本，但不能简单断言必然慢于某个四层方案。TLS 在何处终止、长连接复用、源地址传递、健康检查及跨可用区流量费用都会影响选型。层次决定可见信息，轮询、最少连接或一致性哈希则是选择算法，两者不是同一维度。〔N247〕

011 突发流量或慢依赖出现时，怎样用资源隔离限制故障范围?

答案 舱壁隔离把有限资源按依赖、租户或关键路径划分，使用独立并发配额、连接池、线程池或进程边界，避免一个慢服务占满所有执行资源。若各业务仍共用同一 Redis、数据库或 MQ 集群，单独隔离线程池仍可能被大键、热点分区或连接耗尽拖垮；核心依赖可按业务和故障域拆分实例或集群。每个隔离区还要配有限队列、超时与拒绝策略，否则只是把无限积压转移到别处。限流约束进入速率，熔断暂时停止访问故障依赖，隔离控制故障影响面，三者可协作但不能互相替代。过细切分会降低空闲资源利用率，应按真实饱和指标分配保底与弹性额度，并通过故障注入验证关键业务仍可服务。〔N248〕

012 监控指标用推还是拉？怎样避免指标系统本身失控?

答案 拉模式由采集端定期抓取目标，便于统一间隔、超时和 up 健康信号；推模式由应用发送，适合某些短任务或网络边界，但须解决接收端背压、重试和身份鉴别。Prometheus 主要使用拉取，Pushgateway 适用范围较窄，不应当作所有常驻服务的默认中转。指标用低基数标签汇总状态，不能把用户 ID、完整 URL 或请求 ID 无界加入标签。需要事件明细用日志，需要跨服务路径用追踪。告警应围绕用户影响、错误率、尾延迟和资源饱和配置，而非仅展示平均值。〔N249〕

013 旧库怎样迁到新库，同时保持增量追平和可回滚?

答案 先定义兼容模式和数据转换规则，取得一致快照并记录与之对应的日志位置，再用 CDC 或等价增量通道重放后续变化；简单全量复制后直接切流会丢失复制期间的更新。增量消费必须可幂等恢复，并处理删除、更新顺序和表结构变化。迁移中按主键范围做行数、校验和及业务约束核对，追平后用写入栅栏、短暂冻结或受控路由切换确定最终边界。分阶段放量并监测延迟与错误，回滚必须考虑新库已接受的写入，不能只把读开关拨回旧库。〔N250〕

19.4 容器编排与平台边界

014 容器运行时和 Kubernetes 分别负责什么?

答案 容器运行时负责按镜像创建、运行和隔离进程，Kubernetes 则在集群上以声明式对象协调期望状态。Pod 是调度与共享部分资源的基本单元，Deployment 管理副本和滚动更新，Service 提供稳定访问与后端选择；调度器、控制器和节点组件共同推动实际状态向声明靠拢。Kubernetes 通过 CRI 使用相容运行时，不要求集群节点安装 Docker Engine。它不会自动替应用完成数据一致性、备份或容量规划，仍需资源 requests/limits、探针、持久卷及安全策略配合。〔N251〕

参考文献

S1 仅用于面经考点调研；S2–S22 为语言标准、官方手册、RFC、原始论文与 API 文档，用于核对答案中的规则、行为和适用边界。

- [S1] 面经与题目主题调研（仅用于发现考点）：C++ 面试问题集锦 · C++ 面试题卡片集
- [S2] C++ Working Draft：语言规则、库设施与并发规范：在线工作草案
- [S3] GCC 优化选项：LTO 与 profile guided optimization：GCC 手册
- [S4] Itanium C++ ABI：对象布局、调用接口和运行时约定示例：ABI 规范

- [S5] Linux man-pages：fork、listen、epoll、io_uring 与 socket 选项：fork(2) · listen(2) · epoll(7) · io_uring(7) · socket(7)
- [S6] Boost.Asio：Proactor 异步操作模型：设计说明
- [S7] IETF TCP 与 HTTP 规范：TCP RFC 9293 · HTTP/2 RFC 9113 · HTTP/3 RFC 9114 · HTTP 语义 RFC 9110 · TLS 1.3 RFC 8446
- [S8] IETF DNS 概念与设施 RFC 1034：RFC 1034
- [S9] MySQL Reference Manual：InnoDB redo、MVCC、锁和 binlog：Redo log · MVCC · 锁 · Binlog 与两阶段提交 · undo · 索引 · 隔离级别
- [S10] Redis 文档：热 Key、大 Key 与分布式锁：热点与大 Key 观测 · 分布式锁和 fencing token
- [S11] Raft 原始论文：In Search of an Understandable Consensus Algorithm：扩展版论文
- [S12] Apache Kafka 设计：幂等生产者与事务语义：设计文档
- [S13] Debezium 文档：Outbox Event Router：Transactional Outbox
- [S14] OpenCV 4.10 文档：cv::Mat 与相机标定：cv::Mat · 相机标定
- [S15] Microsoft Azure Architecture Center：Circuit Breaker Pattern：熔断器模式
- [S16] Qt 6 官方文档：QObject 生命周期、线程与信号槽：QObject · Threads and QObjects · QThread · Signals & Slots
- [S17] Qt 6 官方文档：容器、模型/视图与数据库：容器 · QAbstractItemModel · QSqlQuery · QSqlDatabase
- [S18] Qt 6 官方文档：QML、插件与版本迁移：QML/C++ 集成 · QPluginLoader · Qt 6 迁移
- [S19] Qt 6 官方文档：图表与多媒体：Qt Charts · Qt Graphs · Qt Multimedia
- [S20] FFmpeg 官方文档：命令行、编解码 API、滤镜与拼接：文档入口 · 解码 API · 滤镜手册 · FAQ
- [S21] OpenCV 4.x 官方文档：图像处理、矩阵与标定：文档入口 · cv::Mat · 相机标定
- [S22] ROS 2 QoS 官方设计文档：QoS 策略 · Deadline、Liveliness 与 Lifespan

S23 为本次增补的选题与去重来源；N 系列为新增答案核对的官方资料。新增题目按考点独立整理，近义问法统一收录，逐项比对清单见仓库《增补与去重记录》。

- [S23] 小林 coding 面试栏目（2026-10-29 读取）：C++ · Python · MySQL · Redis · 计算机网络 · 操作系统 · 数据结构与算法 · 消息队列 · 分布式 · 系统设计 · Linux 命令 · Git · Docker · 后端进阶路线图
- [N1] C++ 工作草案：复合类型与指针转换
- [N2] C++ 工作草案：指针转换
- [N3] C++ 工作草案：函数调用
- [N4] C++ 工作草案：相等运算
- [N5] C11 公开工作草案 N1570：6.5.9
- [N6] C++ 工作草案：sizeof
- [N7] C++ 工作草案：类与访问控制
- [N8] C++ 工作草案：联合体
- [N9] C++ 工作草案：构造与析构期间的对象
- [N10] C++ 工作草案：虚函数覆盖
- [N11] Itanium C++ ABI：对象与虚表布局
- [N12] C++ 工作草案：对象生命周期
- [N13] Linux man-pages：C 字符串拷贝
- [N14] Linux man-pages：strlen
- [N15] Linux man-pages：memcpy
- [N16] C++ 工作草案：对象表示
- [N17] C++ 工作草案：new 表达式
- [N18] Linux man-pages：malloc 与分配机制
- [N19] C++ 工作草案：显式默认化函数
- [N20] C++ 工作草案：删除函数
- [N21] C++ 工作草案：范围 for 语句
- [N22] C++ 工作草案：枚举声明
- [N23] C++ 工作草案：变长参数模板
- [N24] GCC 预处理器：头文件搜索
- [N25] Linux man-pages：dlopen 与动态装载
- [N26] x86-64 System V ABI 规范

- [N27] System V ABI：ELF 对象文件格式
- [N28] Linux 内核 x86 启动协议
- [N29] Linux man-pages：系统调用与库函数包装
- [N30] Linux man-pages：POSIX 线程共享与独立资源
- [N31] Linux man-pages：调度策略
- [N32] Linux 内核文档：页表层次和地址转换
- [N33] Linux man-pages：多线程进程 fork 的约束
- [N34] Linux man-pages：信号机制
- [N35] Linux man-pages：POSIX 信号量
- [N36] Linux 内核文档：EEVDF 调度
- [N37] Linux man-pages：pthread 自旋锁
- [N38] Linux 内核文档：锁类型与 PREEMPT_RT 差异
- [N39] E. W. Dijkstra：银行家算法原稿 EWD108
- [N40] Linux 内核文档：多代 LRU 页面回收
- [N41] Linux man-pages：文件描述符与打开文件描述
- [N42] Linux man-pages：unlink 后文件存续规则
- [N43] Linux 内核文档：Page Cache
- [N44] Linux man-pages：mmap(2)
- [N45] Linux man-pages：fsync/fdatasync 的持久化边界
- [N46] Linux 内核文档：通用 IRQ 处理
- [N47] Linux 内核文档：工作队列
- [N48] Linux man-pages：read(2) 返回值与错误
- [N49] Linux man-pages：epoll(7) 就绪通知
- [N50] IETF RFC 1122：Internet 主机分层要求
- [N51] IETF RFC 9110：HTTP 方法、范围请求与代理语义
- [N52] IETF RFC 9112：HTTP/1.1 报文边界与连接管理
- [N53] IETF RFC 9111：HTTP 缓存及验证
- [N54] Linux man-pages：hosts(5) 与本地名称映射
- [N55] IETF RFC 4033：DNSSEC 安全性与边界
- [N56] IETF RFC 7766：DNS over TCP 实施要求
- [N57] IETF RFC 6265：Cookie 状态管理
- [N58] Mozilla MDN：浏览器 Set-Cookie 属性
- [N59] WHATWG HTML：Web Storage
- [N60] IETF RFC 7519：JWT 结构与声明
- [N61] IETF RFC 8725：JWT 验证最佳实践
- [N62] gRPC 官方文档：RPC 核心概念与调用生命周期
- [N63] IETF RFC 6455：WebSocket 协议
- [N64] WHATWG Fetch：CORS 协议
- [N65] Amazon CloudFront：边缘分发工作流程
- [N66] IETF RFC 9293：TCP 报文和连接状态机
- [N67] Linux man-pages：accept(2)
- [N68] CISA/FBI/MS-ISAC：DDoS 响应指南
- [N69] Linux 内核文档：TCP SYN 队列与 syncookies 参数
- [N70] Linux man-pages：tcp(7) 与半关闭状态参数
- [N71] IETF RFC 826：ARP 地址解析
- [N72] IETF RFC 792：ICMP 差错与回显消息
- [N73] IETF RFC 3022：传统 NAT/NAPT
- [N74] Linux man-pages：send(2) 的返回值与错误
- [N75] OWASP：XSS 防护速查
- [N76] OWASP：CSRF 防护速查
- [N77] Redis 官方文档：运行与开发概览
- [N78] MongoDB：关系与非关系数据模型

- [N79] Microsoft：数据库规范化
- [N80] MySQL JOIN 语法
- [N81] MySQL 子查询优化
- [N82] MySQL CHAR 与 VARCHAR
- [N83] MySQL BLOB 与 TEXT
- [N84] MySQL 整数类型
- [N85] MySQL 精确定点运算
- [N86] MySQL IP 地址转换函数
- [N87] MySQL 外键约束
- [N88] MySQL 函数与运算符
- [N89] MySQL 聚合函数
- [N90] MySQL TRUNCATE TABLE
- [N91] MySQL SELECT 语法
- [N92] MySQL 窗口函数
- [N93] MySQL 存储引擎
- [N94] MySQL Buffer Pool
- [N95] MySQL Binary Log 与组提交
- [N96] MySQL 移除基于文件的元数据存储
- [N97] MySQL 独立表空间
- [N98] MySQL InnoDB AUTO_INCREMENT
- [N99] MySQL 索引条件下推
- [N100] MySQL 列索引与前缀索引
- [N101] MySQL 死锁处理
- [N102] MySQL 元数据锁
- [N103] MySQL InnoDB 参数
- [N104] MySQL Checkpoint
- [N105] MySQL Doublewrite Buffer
- [N106] MySQL LIMIT 优化
- [N107] MySQL 复制实现
- [N108] MySQL 半同步复制
- [N109] MySQL GTID 概念
- [N110] Azure：分片模式
- [N111] Redis 7.2 源码：ZSet、渐进 rehash 与 listpack
- [N112] Redis 7.2 listpack source
- [N113] Redis 7.2 dictionary source
- [N114] Redis 7.2 SDS source
- [N115] Redis 7.2 event loop source
- [N116] Redis 延迟排查
- [N117] Redis 事务
- [N118] Redis Pipeline
- [N119] Redis EXPIRE
- [N120] Redis 复制
- [N121] Redis Sentinel
- [N122] Redis Cluster 规范
- [N123] Redis 客户端缓存
- [N124] Azure：Cache Aside 模式
- [N125] Redis SCAN
- [N126] Redis Streams
- [N127] MySQL InnoDB 锁定读
- [N128] MySQL 命名锁函数
- [N129] ZooKeeper Recipes
- [N130] Google SRE：处理过载

- [N131] RocketMQ 5.0 架构
- [N132] RabbitMQ 交换机
- [N133] Kafka 4.1 设计
- [N134] RocketMQ 5.0 顺序消息
- [N135] Kafka 4.1 监控
- [N136] RocketMQ 5.0 消费重试
- [N137] RabbitMQ Dead Letter Exchanges
- [N138] RocketMQ 5.0 定时消息
- [N139] RocketMQ 5.0 事务消息
- [N140] RocketMQ 4.x 存储架构
- [N141] Kafka 4.1 Consumer 配置
- [N142] RabbitMQ 消息确认
- [N143] RabbitMQ Consumer Prefetch
- [N144] RabbitMQ Quorum Queues
- [N145] Jepsen：一致性模型
- [N146] gRPC Deadlines
- [N147] ZooKeeper Internals：ZAB
- [N148] OpenTelemetry Traces
- [N149] Apache Seata AT 模式
- [N150] C++ 工作草案：优先队列适配器
- [N151] C++ 工作草案：sort 的要求
- [N152] GCC libstdc++：排序算法实现
- [N153] Sedgewick/Wayne Algorithms：红黑树参考实现
- [N154] Sedgewick/Wayne Algorithms：AVL 树参考实现
- [N155] Redis 8.0 源码：有序集合跳表实现
- [N156] Sedgewick/Wayne Algorithms：Trie 参考实现
- [N157] OpenDSA 教材：外部排序与多路归并
- [N158] Sedgewick/Wayne Algorithms：邻接表图实现
- [N159] Jeff Erickson Algorithms：Greedy Algorithms
- [N160] Jeff Erickson Algorithms：Dynamic Programming
- [N161] Python 数据模型
- [N162] Python 表达式
- [N163] Python 内置类型
- [N164] Python 内置函数
- [N165] Python collections 模块
- [N166] CPython listobject.c 实现
- [N167] Python copy 模块
- [N168] Python io
- [N169] CPython dictobject.c 实现
- [N170] Python 函数与参数
- [N171] Python 复合语句
- [N172] Python 执行模型与名字绑定
- [N173] Python functools 模块
- [N174] Python asyncio 任务
- [N175] Python 类教程
- [N176] Python 描述符指南
- [N177] Python dataclasses
- [N178] Python typing
- [N179] Python 异常
- [N180] Python contextlib 模块
- [N181] Python 导入系统
- [N182] Python 编译与字节码缓存

- [N183] Python venv
- [N184] Pydantic 模型
- [N185] Python JSON 编解码
- [N186] Python 自由线程指南
- [N187] Python threading
- [N188] Python multiprocessing
- [N189] Python asyncio 同步原语
- [N190] Python queue
- [N191] Python concurrent.futures
- [N192] Python contextvars
- [N193] CPython 内存管理
- [N194] Python gc
- [N195] Python tracemalloc
- [N196] Python dis
- [N197] Python 性能分析器
- [N198] procps-ng ps 手册
- [N199] procps-ng ps、top、free 与 vmstat 手册
- [N200] Linux kill(2) 信号语义
- [N201] iproute2 ss 手册
- [N202] OpenBSD nc 手册
- [N203] curl 官方手册
- [N204] procps-ng free 手册
- [N205] Linux /proc/loadavg 与 /proc/meminfo
- [N206] procps-ng vmstat 手册
- [N207] GNU sed 手册
- [N208] GNU awk 手册
- [N209] GNU coreutils 手册
- [N210] Linux /proc/loadavg
- [N211] Git 官方命令手册
- [N212] Git diff
- [N213] Git rebase
- [N214] Git fetch
- [N215] Git checkout 与 detached HEAD
- [N216] Git reset
- [N217] Git revert
- [N218] Git stash
- [N219] GitHub 移除敏感数据指南
- [N220] Git gitignore
- [N221] Pro Git 分布式工作流
- [N222] Git log
- [N223] Git tag
- [N224] Git bisect
- [N225] Docker 架构与对象
- [N226] Linux cgroup v2 文档
- [N227] Dockerfile 参考
- [N228] Docker 构建最佳实践
- [N229] Docker 存储概览
- [N230] Docker run 参考
- [N231] Docker 多阶段构建
- [N232] Docker 网络概览
- [N233] Docker Compose 文档
- [N234] Kubernetes 容器运行时

- [N235] Docker 资源清理
- [N236] MongoDB：文档建模
- [N237] MongoDB：副本集
- [N238] Apache Hadoop：HDFS 架构
- [N239] C++ Core Guidelines：接口、工厂与类设计
- [N240] OWASP：密码存储
- [N241] OpenID Connect Core 1.0
- [N242] OWASP：REST 接口安全
- [N243] RFC 9421：HTTP 消息签名
- [N244] Google SRE：服务级别目标
- [N245] Java 25：垃圾回收调优导论
- [N246] Go：垃圾回收指南
- [N247] NGINX：TCP/UDP 负载均衡
- [N248] Azure Architecture：舱壁隔离模式
- [N249] Prometheus：架构与抓取
- [N250] AWS DMS：持续变更捕获
- [N251] Kubernetes：声明式编排